Getting Started

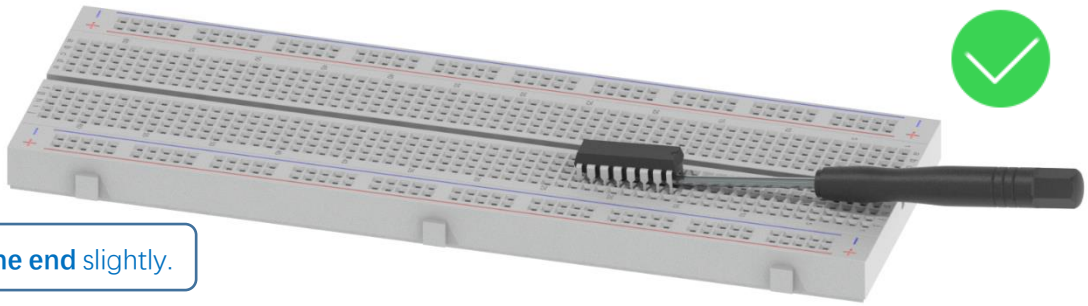
Thank you for choosing Freenove products!
After you download the ZIP file we provide. Unzip it and you will get a folder contains several files and folders. There are three PDF files:

- **Tutorial.pdf**
  It contains basic operations such as installing system for Raspberry Pi.
  The code in this PDF is in C.
- **Tutorial_GPIOZero.pdf**
  It contains basic operations such as installing system for Raspberry Pi.
  The code in this PDF is in Python.
- **Processing.pdf** in Freenove_Complete_Starter_Kit_for_Raspberry_Pi\Processing
  The code in this PDF is in Java.

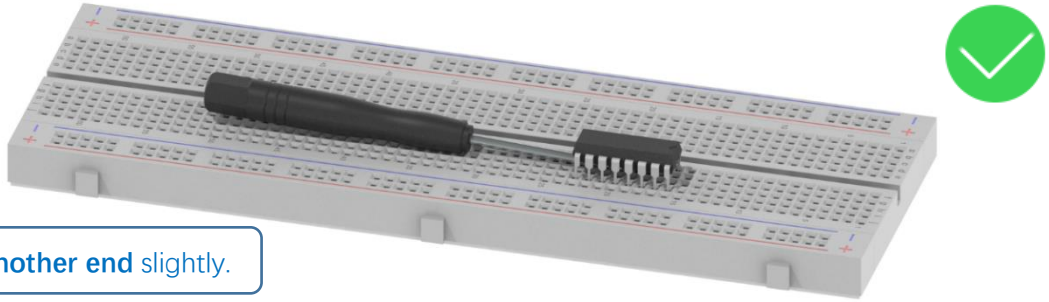
We recommend you to start with **Tutorial.pdf** or **Tutorial_GPIOZero.pdf** first.
If you want to start with Processing.pdf or skip some chapters of Tutorial.pdf, you need to finish necessary steps in **Chapter 7 AD/DA** of **Tutorial.pdf** first.

Remove the Chips

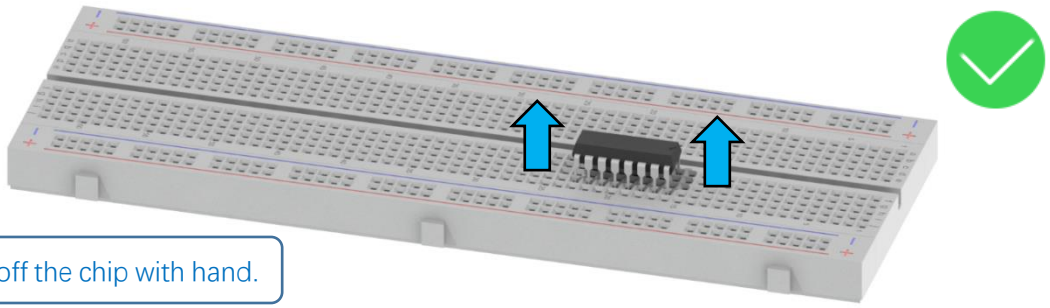
Some chips and modules are inserted into the breadboard to protect their pins.
You need to remove them from breadboard before use. (There is no need to remove GPIO Extension Board.)
Please find a tool (like a little screw driver) to handle them like below:



Step 1, lift **one end** slightly.

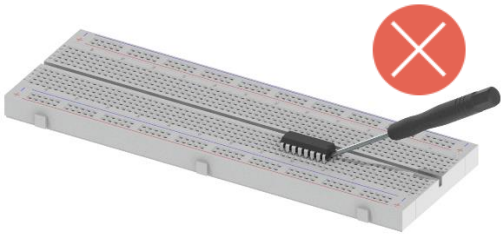


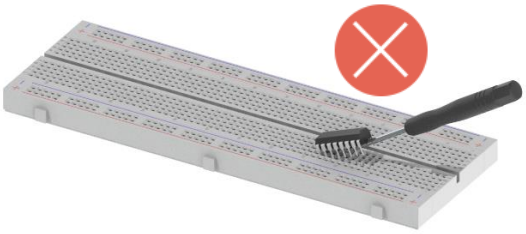
Step 2, lift **another end** slightly.



Step 3, take off the chip with hand.

Avoid lifting one end with big angle directly.





Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it

cools down! When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

Copyright

All the files, materials and instructional guides provided are released under Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License. A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.

This means you can use these resource in your own derived works, in part or completely, but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.

Raspberry Pi® is a trademark of Raspberry Pi Foundation (https://www.raspberrypi.org/).

Contents

Getting Started I
Remove the Chips I
Safety and Precautions II
About Freenove III
Copyright III
Contents IV
Preface 1
Raspberry Pi 2
Chapter 0 Preparation 9
Linux Command 9
Install GPIO Zero Python library 12
Obtain the Project Code 13
Python2 & Python3 14
Chapter 1 LED 17
Project 1.1 Blink 17
Freenove Car, Robot and other products for Raspberry Pi 32
Chapter 2 Buttons & LEDs 33
Project 2.1 Push Button Switch & LED 33
Project 2.2 MINI Table Lamp 37
Chapter 3 LED Bar Graph 40
Project 3.1 Flowing Water Light 40
Chapter 4 Analog & PWM 44
Project 4.1 Breathing LED 44
Chapter 5 RGB LED 49
Project 5.1 Multicolored LED 50
Chapter 6 Buzzer 53
Project 6.1 Doorbell 53
Project 6.2 Alertor 59
(Important) Chapter 7 ADC 61
Project 7.1 Read the Voltage of Potentiometer 61
Chapter 8 Potentiometer & LED 73
Project 8.1 Soft Light 73
Chapter 9 Potentiometer & RGBLED 78
Project 9.1 Colorful Light 78
Chapter 10 Photoresistor & LED 83
Project 10.1 NightLamp 83
Chapter 11 Thermistor 89
Project 11.1 Thermometer 89
Chapter 12 Joystick 95
Project 12.1 Joystick 95

Chapter 13 Motor & Driver 101
Project 13.1 Control a DC Motor with a Potentiometer 101
Chapter 14 Relay & Motor 112
Project 14.1.1 Relay & Motor 112
Chapter 15 Servo 118
Project 15.1 Servo Sweep 118
Chapter 16 Stepper Motor 125
Project 16.1 Stepper Motor 125
Chapter 17 74HC595 & Bar Graph LED 133
Project 17.1 Flowing Water Light 133
Chapter 18 74HC595 & 7-Segment Display 139
Project 18.1 7-Segment Display 139
Project 18.2 4-Digit 7-Segment Display 144
Chapter 19 74HC595 & LED Matrix 152
Project 19.1 LED Matrix 152
Chapter 20 LCD1602 160
Project 20.1 I2C LCD1602 160
Chapter 21 Hygrothermograph DHT11 165
Project 21.1 Hygrothermograph 165
Chapter 22 Matrix Keypad 170
Project 22.1 Matrix Keypad 170
Chapter 23 Infrared Motion Sensor 177
Project 23.1 PIR Infrared Motion Detector with LED Indicator 177
Chapter 24 Ultrasonic Ranging 183
Project 24.1 Ultrasonic Ranging 183
Chapter 25 Attitude Sensor MPU6050 189
Project 25.1 Read a MPU6050 Sensor Module 189
Chapter 26 High-sensitivity microphone sensor 194
Project 26.1 High-sensitivity microphone sensor and LED 194
Chapter 27 Touch Sensor 199
Project 27.1 Touch Sensor and LED 199
Project 27.2 Touch Sensor and RGB LED 204
Chapter 28 U-shaped photoelectric sensor 208
Project 28.1 U-shaped photoelectric sensor and LED 208
Project 28.2 U-shaped photoelectric sensor and buzzer 213
Chapter 29 Hall sensor 217
Project 29.1 Hall sensor and LED 217
Project 29.2 Hall Sensor and Buzzer 222
Chapter 30 Infrared Obstacle Avoidance Sensor 226
Project 30.1 Infrared obstacle avoidance sensor and LED 226
Project 30.2 Infrared obstacle avoidance sensor and buzzer 230
Chapter 31 Rotary Encoder 234
Project 31.1 Rotary Encoder 234
Project 31.2 Rotary Encoder and LED 240

Chapter 32 LEDpixel **245**
Project 32.1 LEDpixel 245
Project 32.2 RainbowLight 252
Project 32.3 SPI_LEDpixel 257
Chapter 33 BMP180 Barometric Pressure Sensor **263**
Project 33.1 Barometer 263
Chapter 34 RFID **272**
Project 34.1 RFID 272
Chapter 35 Speaker amplifier module PAM8403 ..... **283**
Project 35.1 Play local music 283
Project 35.2 TTS reminder 292
Chapter 36 Camera **296**
Project 36.1 Photograph 296
Project 36.2 Video Recording 302
Chapter 37 Web IoT **306**
Project 37.1 Remote LED 306
Chapter 38 Soldering a Circuit Board **312**
Project 38.1 Soldering a Buzzer 312
Project 38.2 Soldering a Flowing Water Light 316
Other Components **322**
What's Next? **324**

Preface

Raspberry Pi is a low cost, **credit card sized computer** that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is an incredibly capable little device that enables people of all ages to explore computing, and to learn how to program in a variety of computer languages like Scratch and Python. It is capable of doing everything you would expect from a desktop computer, such as browsing the internet, playing high-definition video content, creating spreadsheets, performing word-processing, and playing video games. For more information, you can refer to Raspberry Pi official website. For clarification, this tutorial will also reference Raspberry Pi as RPi, RPI and RasPi.

In this tutorial, most chapters consist of **Components List**, **Component Knowledge**, **Circuit**, and **Code** (**Python** code). We provide Python code for each project in this tutorial. After completing this tutorial, you can learn Java by reading Processing.pdf.

This kit does not contain **Raspberry and its accessories**. You can also use the components and modules in this kit to create projects of your own design.

Additionally, if you encounter any issues or have questions about this tutorial or the contents of kit, you can always contact us for free technical support at:

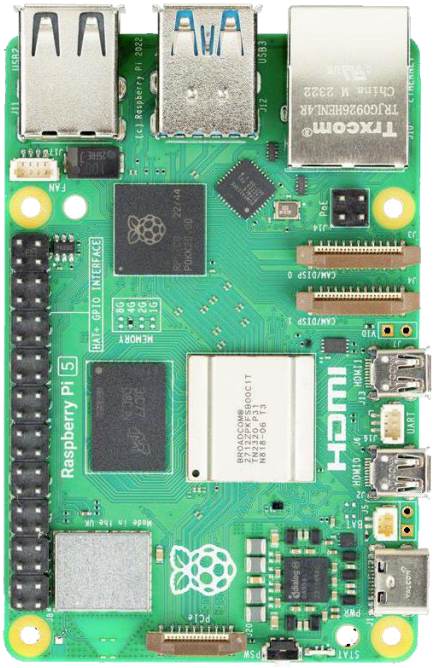
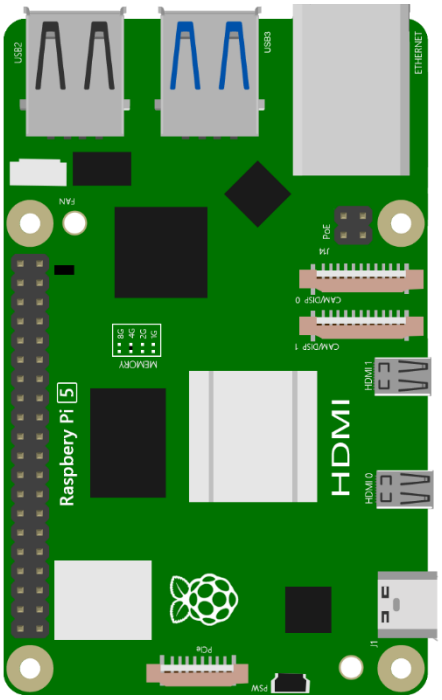
support@freenove.com

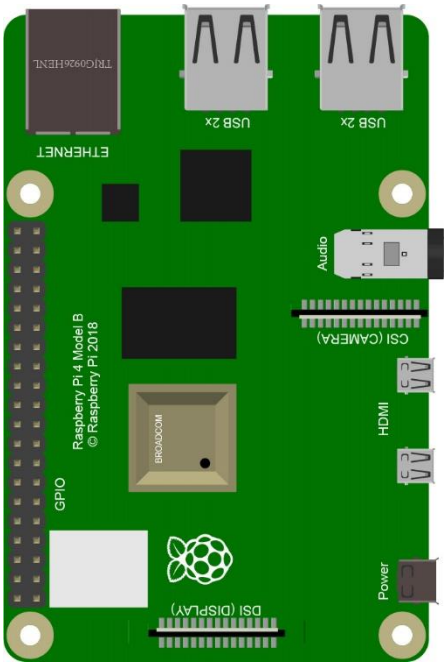
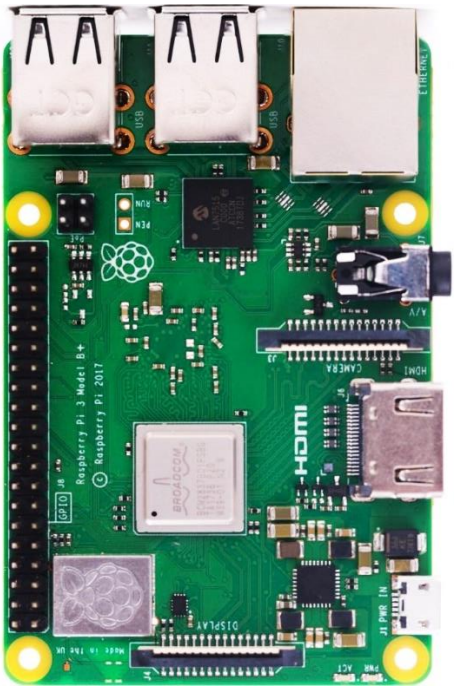
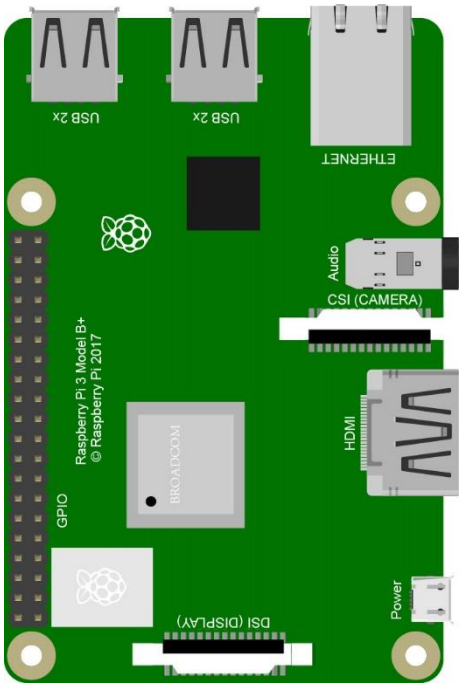
Raspberry Pi

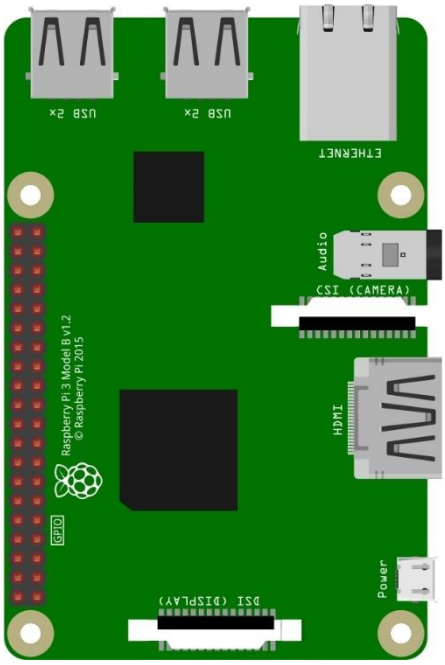
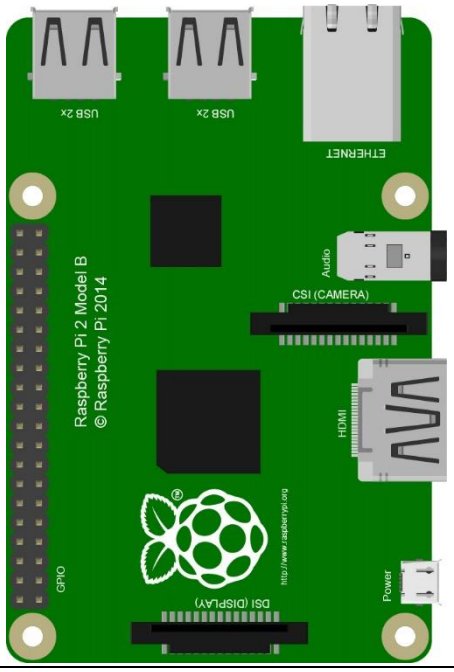
So far, at this writing, Raspberry Pi has advanced to its fifth generation product offering. Version changes are accompanied by increases in upgrades in hardware and capabilities.

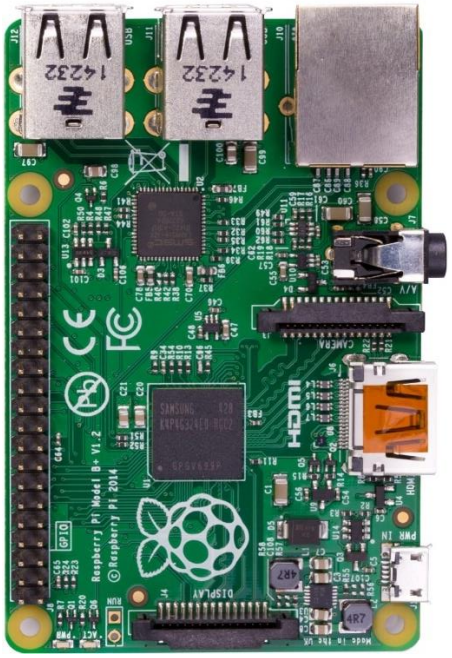
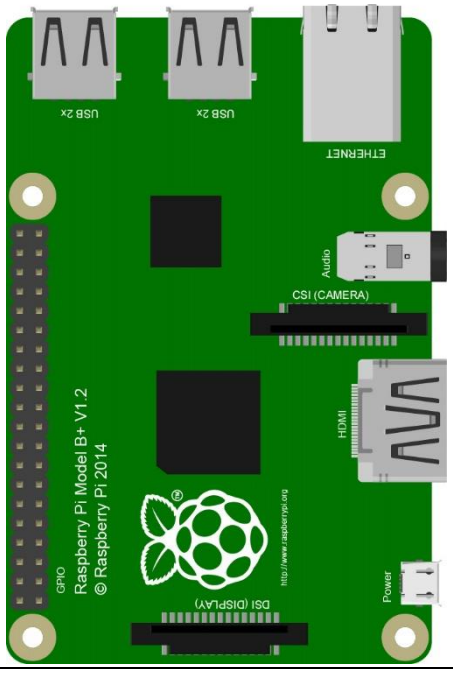
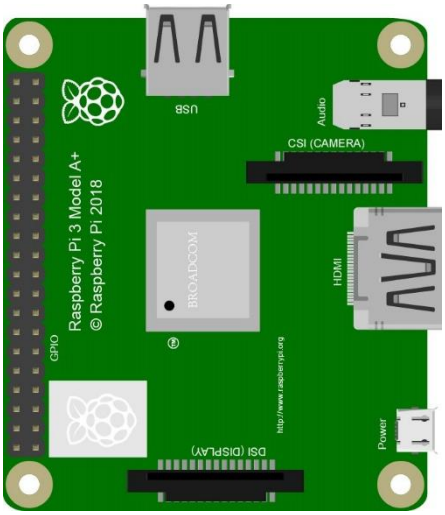
The A type and B type versions of the first generation products have been discontinued due to various reasons. What is most important is that other popular and currently available versions are consistent in the order and number of pins and their assigned designation of function, making compatibility of peripheral devices greatly enhanced between versions.

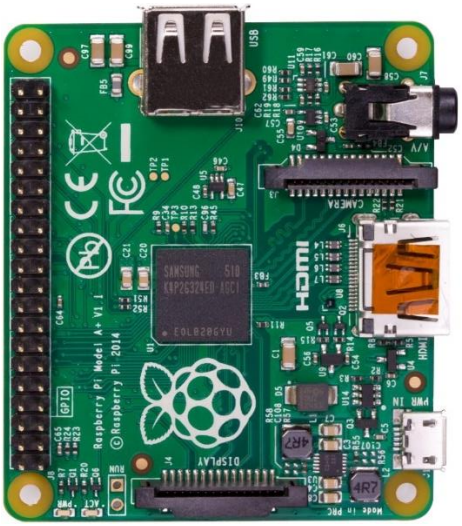
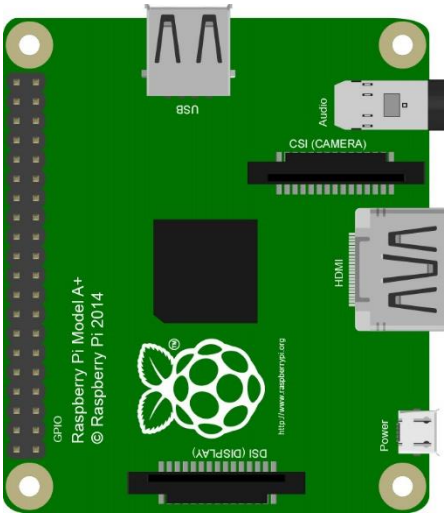
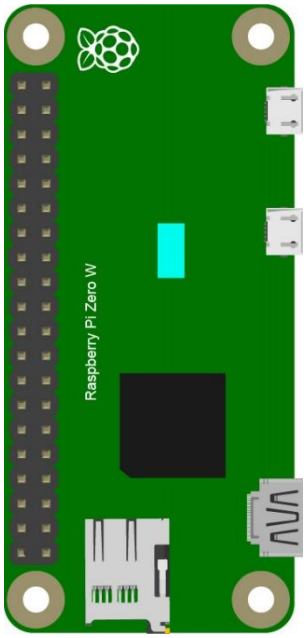
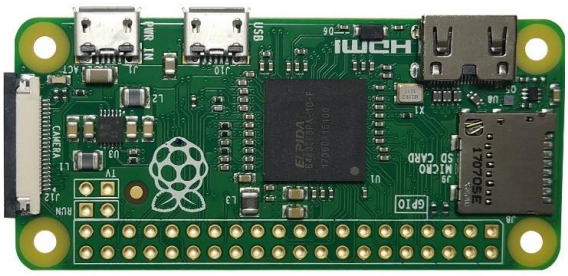
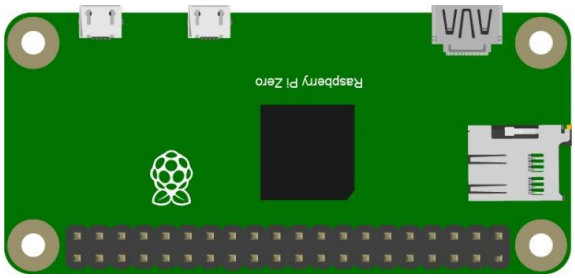
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.

Practicality picture of Raspberry Pi 5:	Model diagram of Raspberry Pi 5:
	

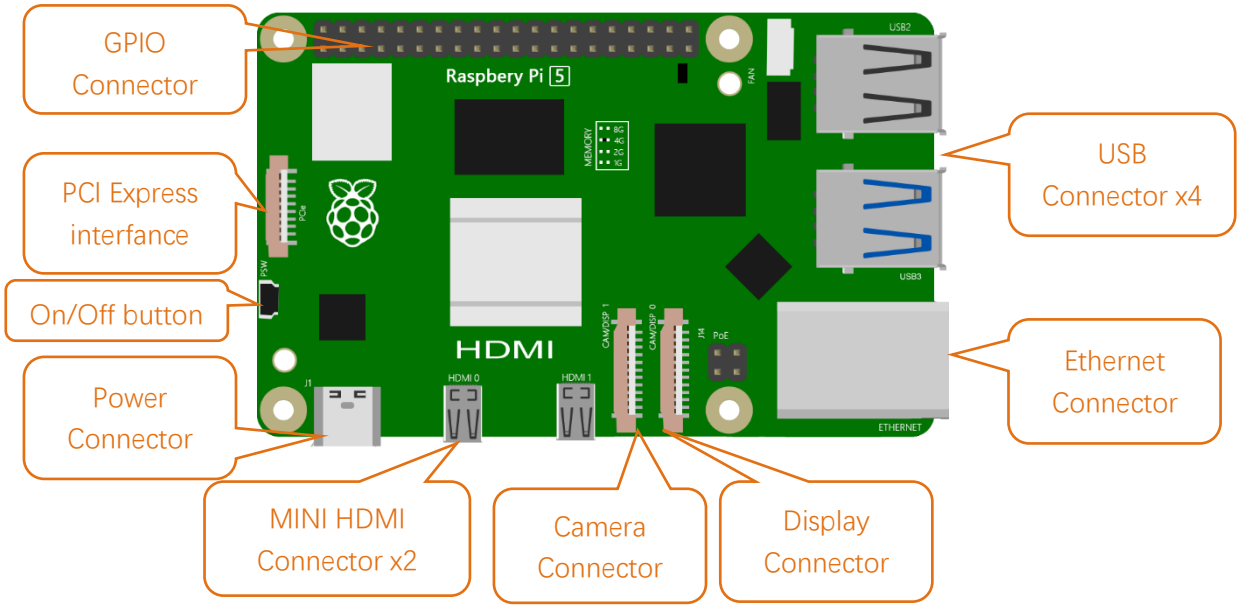
Actual image of Raspberry Pi 4 Model B:	CAD image of Raspberry Pi 4 Model B: 
Actual image of Raspberry Pi 3 Model B+: 	CAD image of Raspberry Pi 3 Model B+: 

Actual image of Raspberry Pi 3 Model B: 	CAD image of Raspberry Pi 3 Model B: 
Actual image of Raspberry Pi 2 Model B: 	CAD image of Raspberry Pi 2 Model B: 

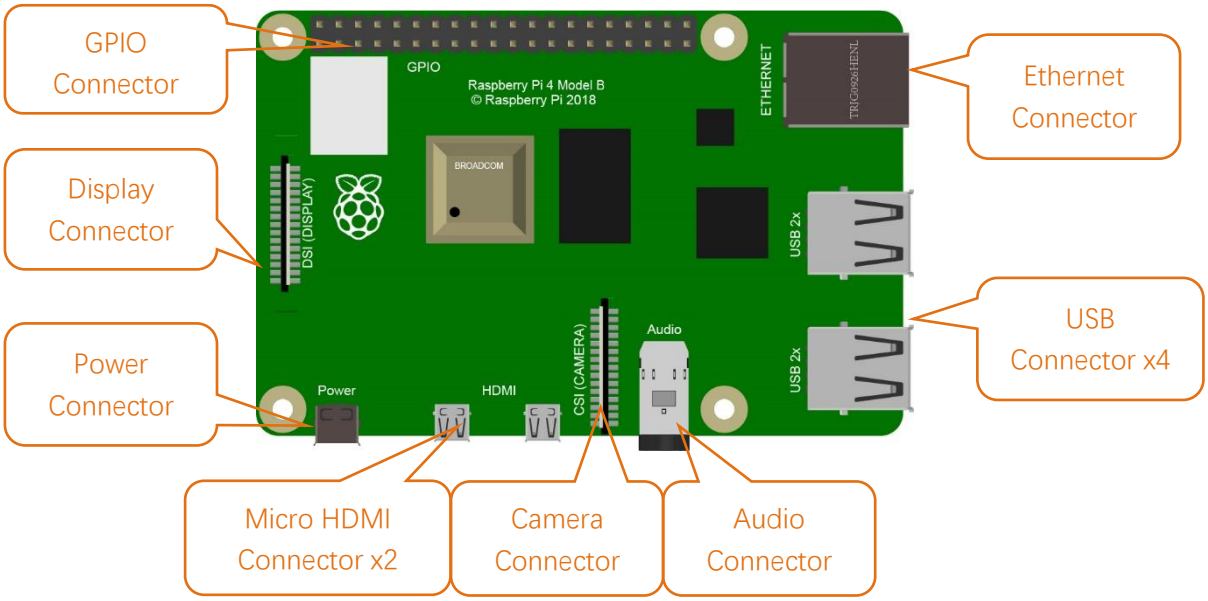
Actual image of Raspberry Pi 1 Model B+: 	CAD image of Raspberry Pi 1 Model B+: 
Actual image of Raspberry Pi 3 Model A+:	CAD image of Raspberry Pi 3 Model A+: 

Actual image of Raspberry Pi 1 Model A+:	CAD image of Raspberry Pi 1 Model A+:
	
Actual image of Raspberry Pi Zero W:	CAD image of Raspberry Pi Zero W:
	
Actual image of Raspberry Pi Zero:	CAD image of Raspberry Pi Zero:
	

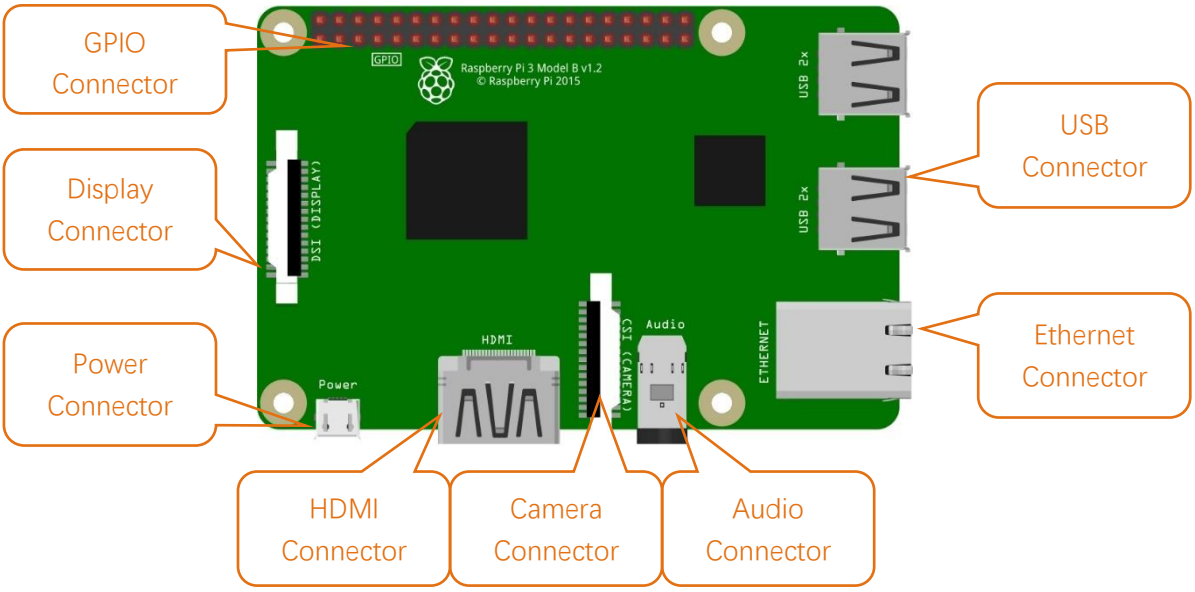
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.
Hardware interface diagram of RPi 5 is shown below:



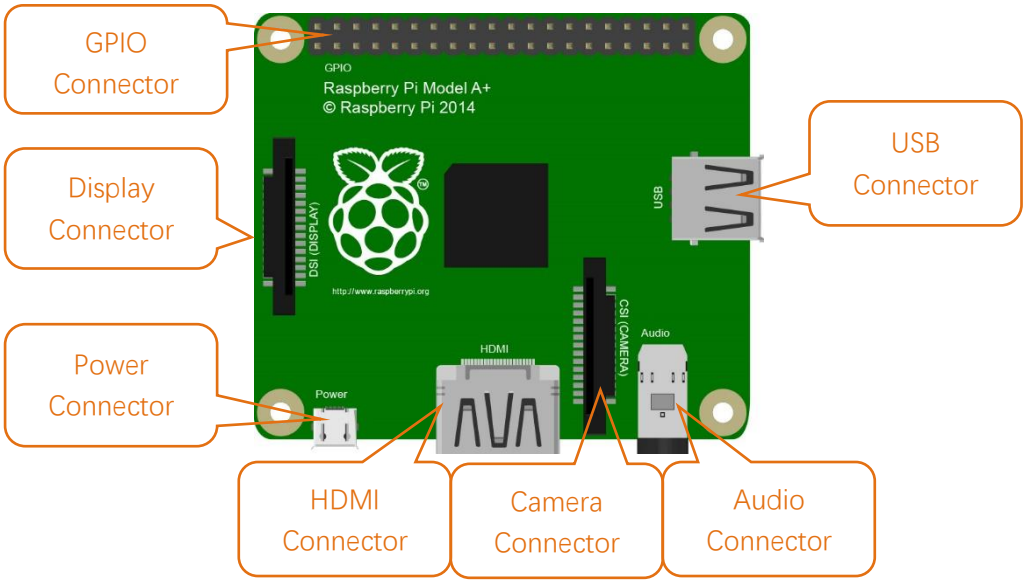
Hardware interface diagram of RPi 4B is shown below:



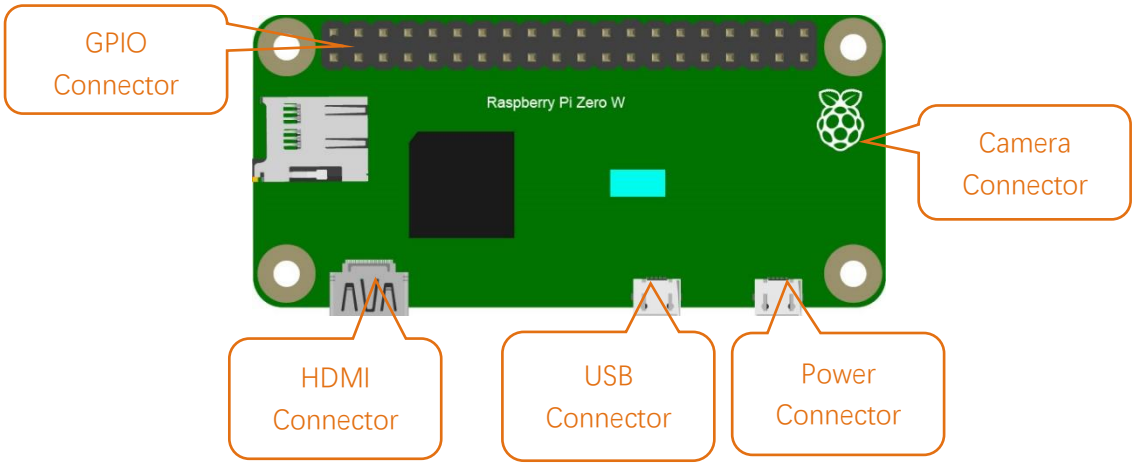
Hardware interface diagram of RPi 3B+/3B/2B/1B+:



Hardware interface diagram of RPi 3A+/A+:



Hardware interface diagram of RPi Zero/Zero W:



Chapter 0 Preparation

Why "Chapter 0"? Because in program code the first number is 0. We choose to follow this rule. In this chapter, we will do some necessary foundational preparation work: Start your Raspberry Pi and install some necessary libraries.

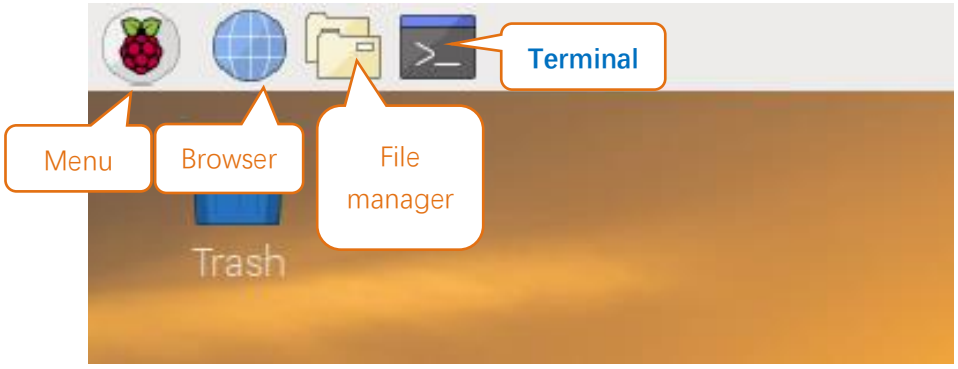
If you haven't installed the system for Raspberry Pi yet, please refer to:

Freenove_Complete_Starter_Kit_for_Raspberry_Pi/Installing Raspberry Pi OS.pdf

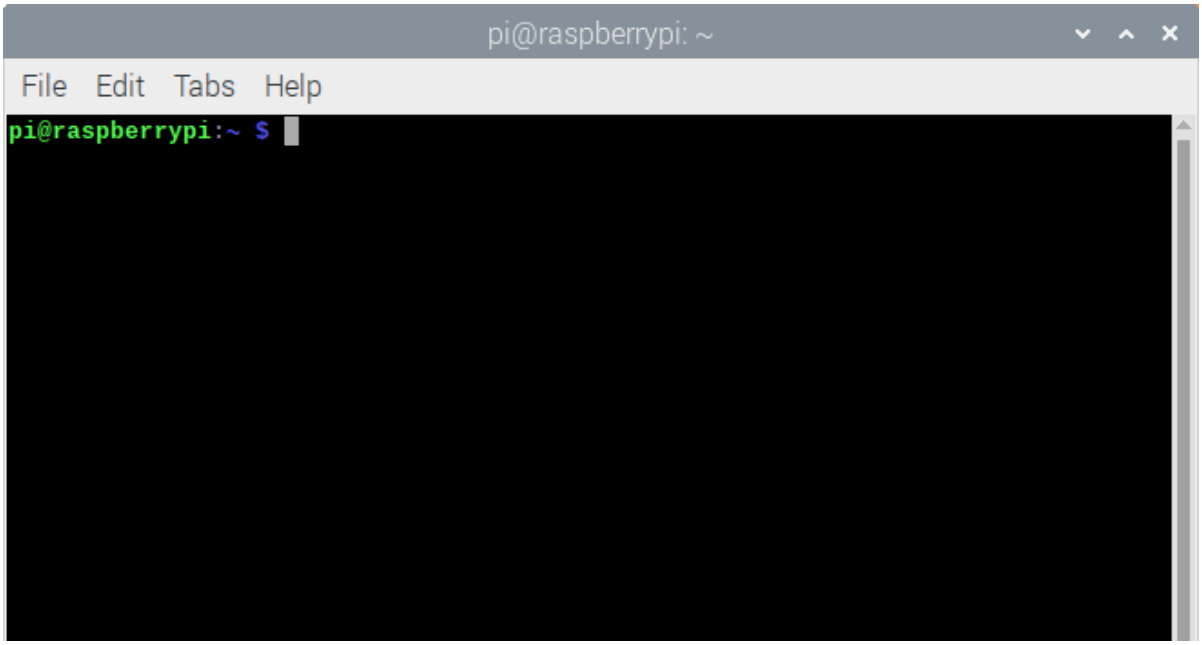
Linux Command

Raspberry Pi OS is based on the Linux Operation System. Now we will introduce you to some frequently used Linux commands and rules.

First, open the Terminal. All commands are executed in Terminal.

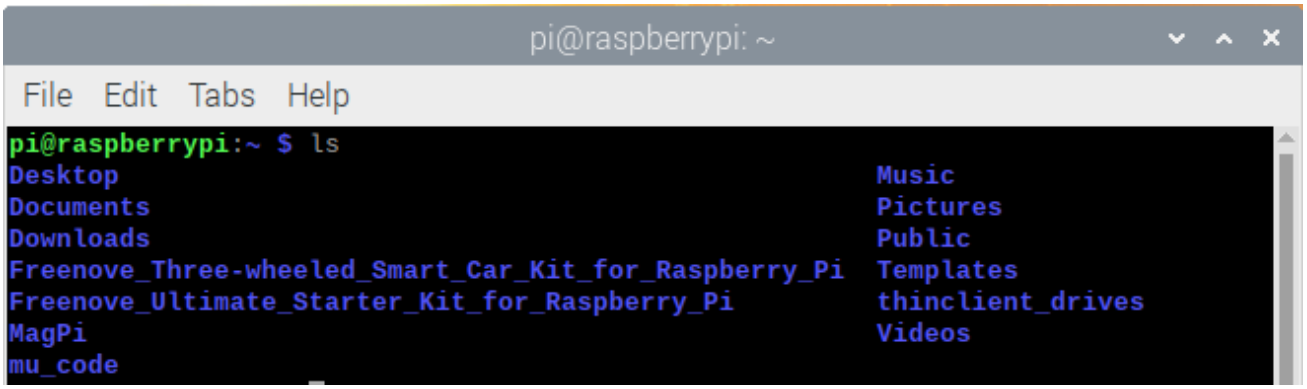


When you click the Terminal icon, following interface appears.



Note: The Linux is case sensitive.

First, type "ls" into the Terminal and press the "Enter" key. The result is shown below:



The "ls" command lists information about the files (the current directory by default).

Content between "$" and "pi@raspberrypi:" is the current working path. "~" represents the user directory, which refers to "/home/pi" here.

```
pi@raspberrypi:~ $ pwd
/home/pi
```

"cd" is used to change directory. "/" represents the root directory.

```
pi@raspberrypi:~ $ cd /usr
pi@raspberrypi:/usr $ ls
bin  games  include  lib  local  man  sbin  share  src
pi@raspberrypi:/usr $ cd ~
pi@raspberrypi:~ $
```

Later in this Tutorial, we will often change the working path. Typing commands under the wrong directory may cause errors and break the execution of further commands.

Many frequently used commands and instructions can be found in the following reference table.

Command	instruction
ls	Lists information about the FILEs (the current directory by default) and entries alphabetically.
cd	Changes directory
sudo + cmd	Executes cmd under root authority
./	Under current directory
gcc	GNU Compiler Collection
git clone URL	Use git tool to clone the contents of specified repository, and URL in the repository address.

There are many commands, which will come later. For more details about commands. You can refer to: http://www.linux-commands-examples.com

Shortcut Key

Now, we will introduce several commonly used shortcuts that are very useful in Terminal.

1. **Up and Down Arrow Keys**: Pressing "↑" (the Up key) will go backwards through the command history and pressing "↓" (the Down Key) will go forwards through the command history.

2. **Tab Key**: The Tab key can automatically complete the command/path you want to type. When there is only one eligible option, the command/path will be completely typed as soon as you press the Tab key even you only type one character of the command/path.

As shown below, under the '~' directory, you enter the Documents directory with the "cd" command. After typing "cd D", pressing the Tab key (there is no response), pressing the Tab key again then all the files/folders that begin with "D" will be listed. Continue to type the letters "oc" and then pressing the Tab key, the "Documents" is typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Install GPIO Zero Python library

GPIO Zero is a simple interface to GPIO devices with Raspberry Pi. GPIO Zero is installed by default in the Raspberry Pi OS desktop image, and the Raspberry Pi Desktop image for PC/Mac, both available from raspberrypi.org. Follow these guides to installing on Raspberry Pi OS Lite and other operating systems.

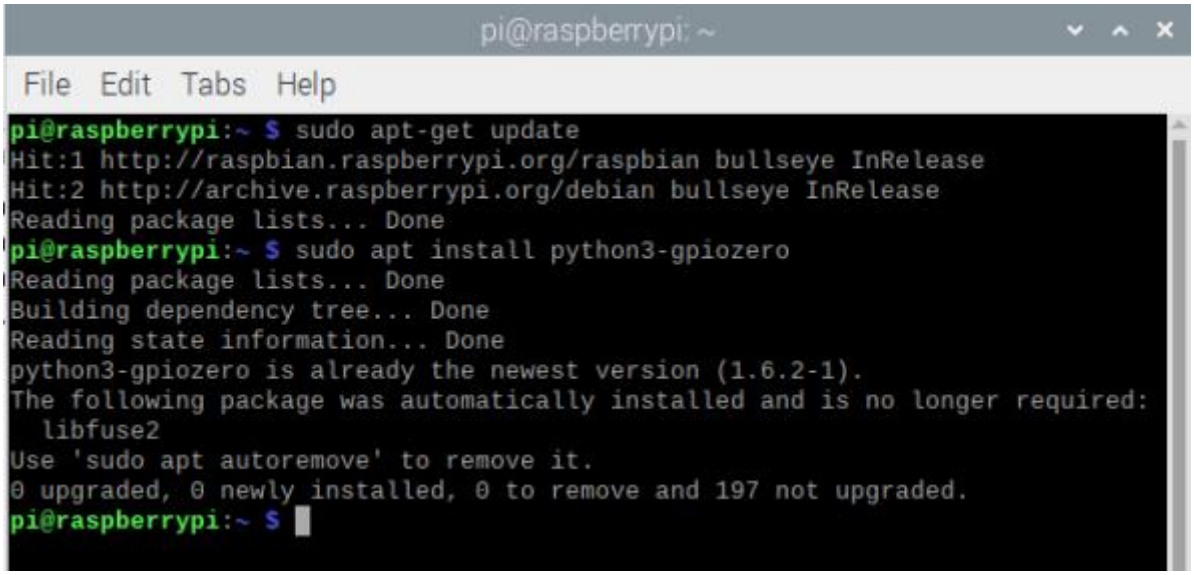
GPIO Zero Python library Installation Steps

To install the GPIO Zero Python library, please open the Terminal and then follow the steps and commands below.

Note: For a command containing many lines, execute them one line at a time.

Enter the following commands **one by one** in the **terminal** to install GPIO Zero:

```
sudo apt-get update
sudo apt install python3-gpiozero
```



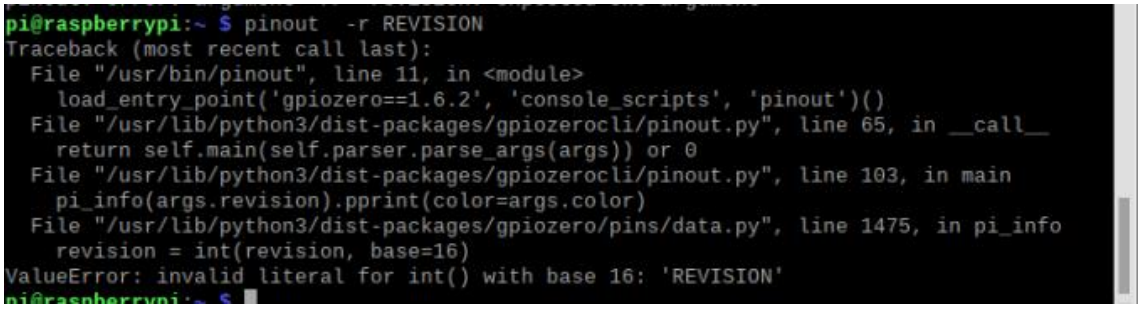
If you're using another operating system on your Raspberry Pi, you may need to use pip to install GPIO Zero instead. Install pip using get-pip and then type:

```
sudo pip3 install gpiozero
```

Run the gpiozero command to check the installation:

```
pinout  -r REVISION
```

That should give you some confidence that the installation was a success.



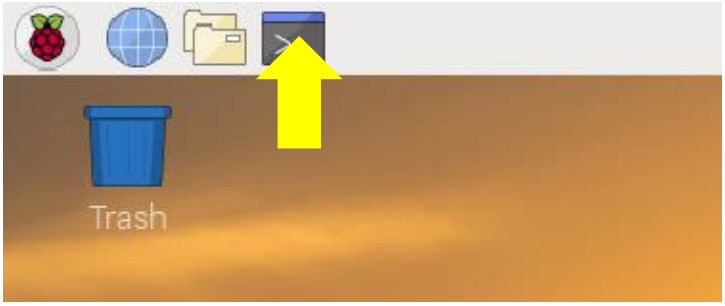
Obtain the Project Code

After the above installation is completed, you can visit our official website (http://www.freenove.com) or our GitHub resources at (https://github.com/freenove) to download the latest available project code.
In this tutorial, we provide Python language code for each project.

This is the method for obtaining the code:
In the pi directory of the RPi terminal, enter the following command.

```
cd
git clone --depth 1 https://github.com/freenove/Freenove_Complete_Starter_Kit_for_Raspberry_Pi
```

(There is no need for a password. If you get some errors, please check your commands.)

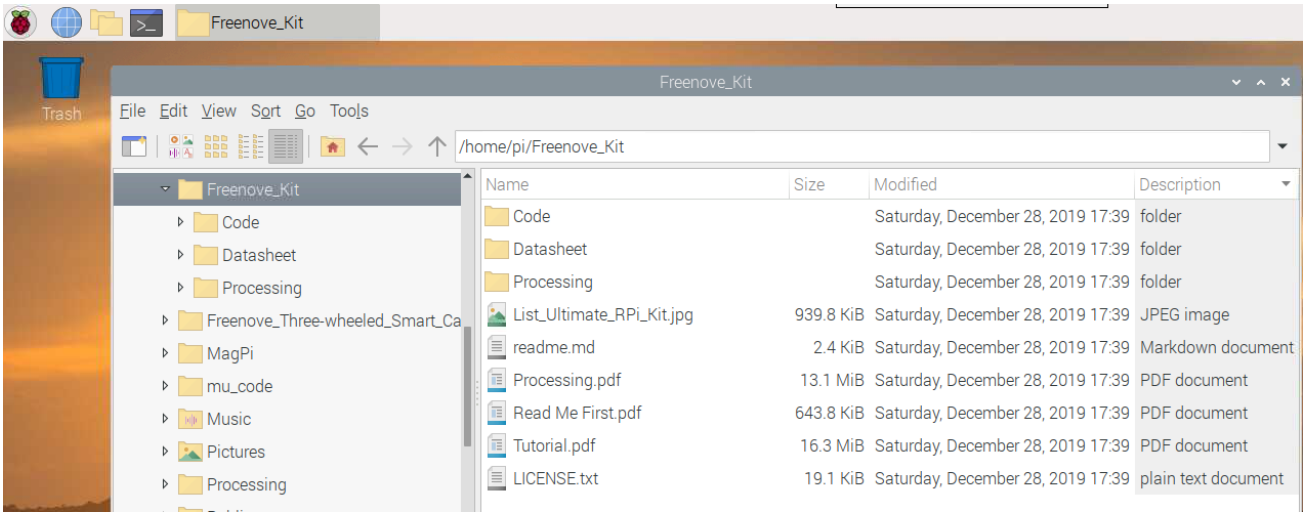


After the download is completed, a new folder "Freenove_Complete_Starter_Kit_for_Raspberry_Pi" is generated, which contains all of the tutorials and required code.

This folder name seems a little too long. We can simply rename it by using the following command.

```
mv Freenove_Complete_Starter_Kit_for_Raspberry_Pi Freenove_Kit
```

"Freenove_Kit" is now the new and much shorter folder name.



If you have no experience with Python, we suggest that you refer to this website for basic information and knowledge.
https://python.swaroopch.com/basics.html

Python2 & Python3

Python code, used in our kits, can now run on Python2 and Python3. **Python3 is recommend**. If you want to use Python2, please make sure your Python version is 2.7 or above. Python2 and Python3 are not fully compatible. However, Python2.6 and Python2.7 are transitional versions to python3, therefore you can also use Python2.6 and 2.7 to execute some Python3 code.

You can type "python2" or "python3" respectively into Terminal to check if python has been installed. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python2
Python 2.7.18 (default, Jul 14 2021, 08:11:37)
[GCC 10.2.1 20210110] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
[1]+  Stopped                 python2
pi@raspberrypi:~ $ python3
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[2]+  Stopped                 python3
pi@raspberrypi:~ $
```

Type "python", and Terminal shows that it links to python3.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                 python
^Xpi@raspberrypi:~ $
```

Set Python3 as default python

First, execute python to check the default python on your raspberry Pi. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                 python
^Xpi@raspberrypi:~ $
```

If it is python3, you can skip this section.

If it is python2, you need execute the following commands to set default python to python3.

1. Enter directory /usr/bin

```
cd /usr/bin
```

2. Delete the originalpython link.

```
sudo rm python
```

3. Create new python links to python.

```
sudo ln -s python3 python
```

4. Check python. Press Ctrl-Z to exit.

```
python
```

```
pi@raspberrypi:/usr/bin $ sudo rm python
pi@raspberrypi:/usr/bin $ sudo ln -s python3 python
pi@raspberrypi:/usr/bin $ python
Python 3.5.3 (default, Jan 19 2017, 14:11:04)
[GCC 6.3.0 20170124] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If you want to set python2 as default python in **other projects**, just repeat the commands above and change python3 to python2.

Shortcut Key

Now, we will introduce several shortcuts that are very **useful** and **commonly used** in terminal.

1. **up and down arrow keys**. History commands can be quickly brought back by using up and down arrow keys, which are very useful when you need to reuse certain commands.

When you need to type commands, pressing "↑" will go backwards through the history of typed commands, and pressing "↓" will go forwards through the history of typed command.

2. **Tab key**. The Tab key can automatically complete the command/path you want to type. When there are multiple commands/paths conforming to the already typed letter, pressing Tab key once won't have any result. And pressing Tab key again will list all the eligible options. This command/path will be completely typed as soon as you press the Tab key when there is only one eligible option.

As shown below, under the '~'directory, enter the Documents directory with the "cd" command. After typing "cd D", press Tab key, then there is no response. Press Tab key again, then all the files/folders that begin with "D" is listed. Continue to type the character "oc", then press the Tab key, and then "Documents" is completely typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Chapter 1 LED

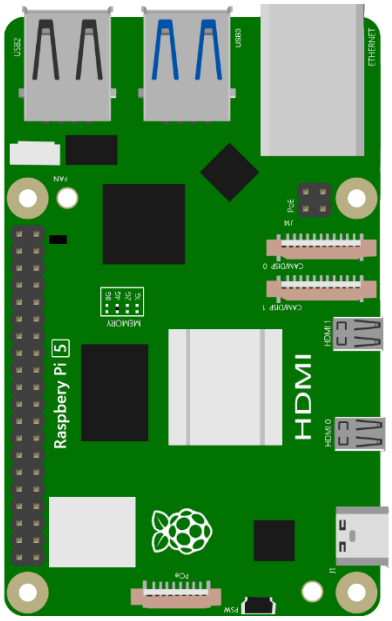
This chapter is the Start Point in the journey to build and explore RPi electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

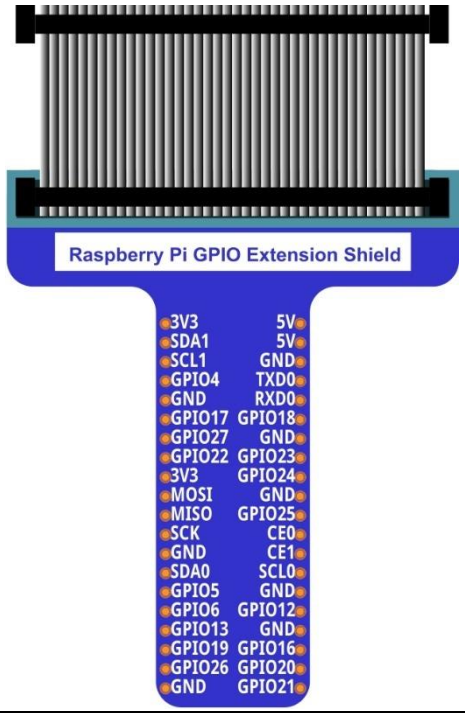
In this project, we will use RPi to control blinking a common LED.

Component List

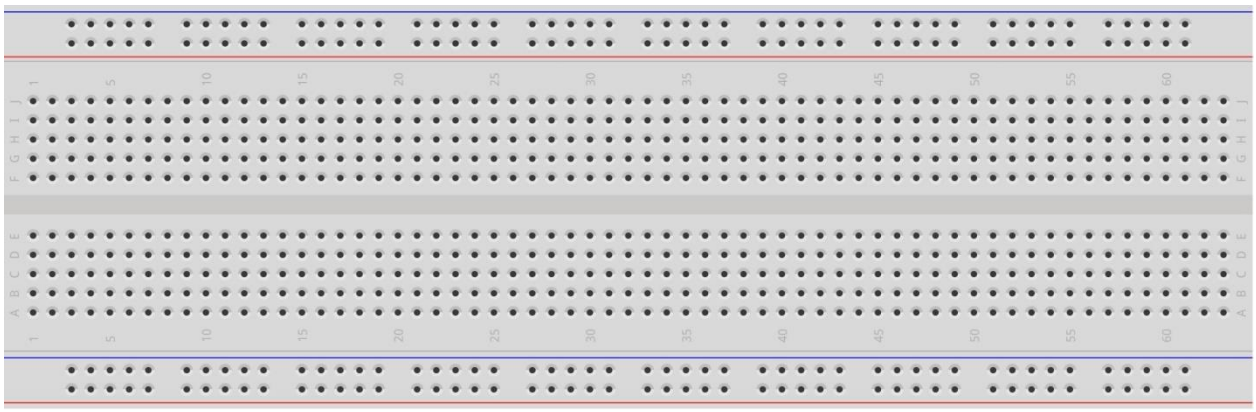
Raspberry Pi
(Recommended: Raspberry Pi 4B / 3B+ / 3B
Compatible: 3A+ / 2B / 1B+ / 1A+ / Zero W / Zero)



GPIO Extension Board & Ribbon Cable



Breadboard x1



LED x1	Resistor 220Ω x1	Jumper **Specific quantity depends on the circuit.**

In the components list, 3B GPIO, Extension Shield Raspberry and Breadboard are necessary for each project. Later, they will be reference by text only (no images as in above).

GPIO

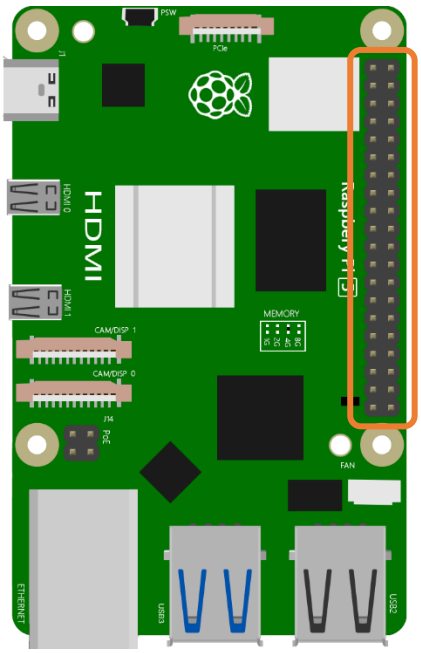
GPIO: General Purpose Input/Output. Here we will introduce the specific function of the pins on the Raspberry Pi and how you can utilize them in all sorts of ways in your projects. Most RPi Module pins can be used as either an input or output, depending on your program and its functions.

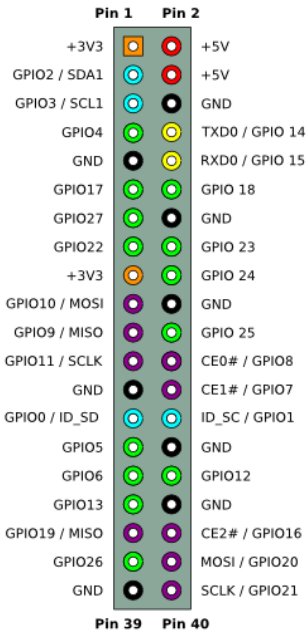
When programming GPIO pins there are 3 different ways to reference them: GPIO Numbering, Physical Numbering and WiringPi GPIO Numbering.

BCM GPIO Numbering

The Raspberry Pi CPU uses Broadcom (BCM) processing chips BCM2835, BCM2836 or BCM2837. GPIO pin numbers are assigned by the processing chip manufacturer and are how the computer recognizes each pin. The pin numbers themselves do not make sense or have meaning as they are only a form of identification. Since their numeric values and physical locations have no specific order, there is no way to remember them so you will need to have a printed reference or a reference board that fits over the pins.

Each pin's functional assignment is defined in the image below:

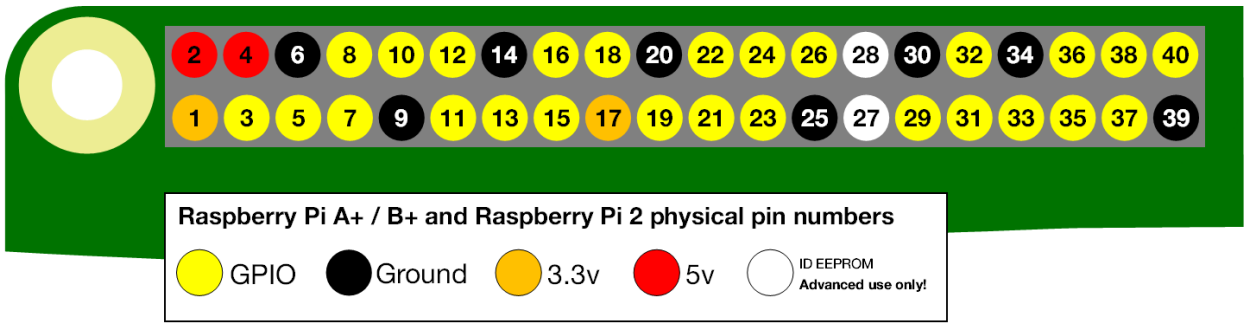




For more details about pin definition of GPIO, please refer to http://pinout.xyz/

PHYSICAL Numbering

Another way to refer to the pins is by simply counting across and down from pin 1 at the top left (nearest to the SD card). This is 'Physical Numbering', as shown below:



GPIO Numbering

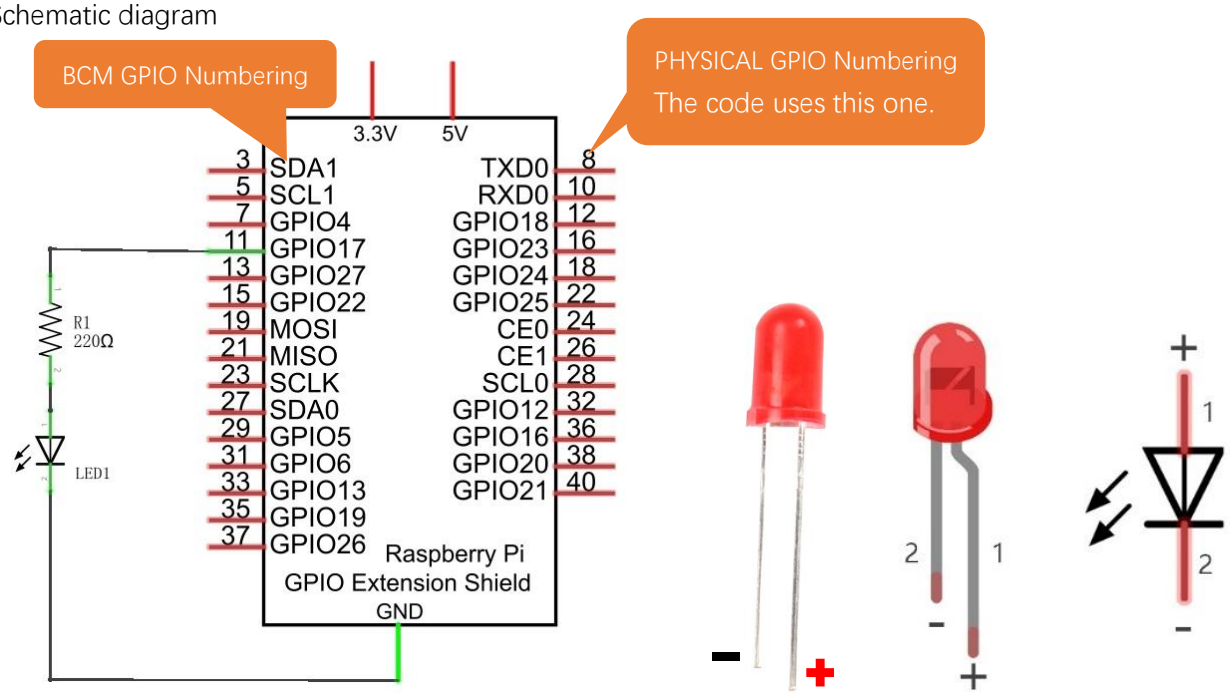
You can use the following command to view their correlation.

```
Pinout
```

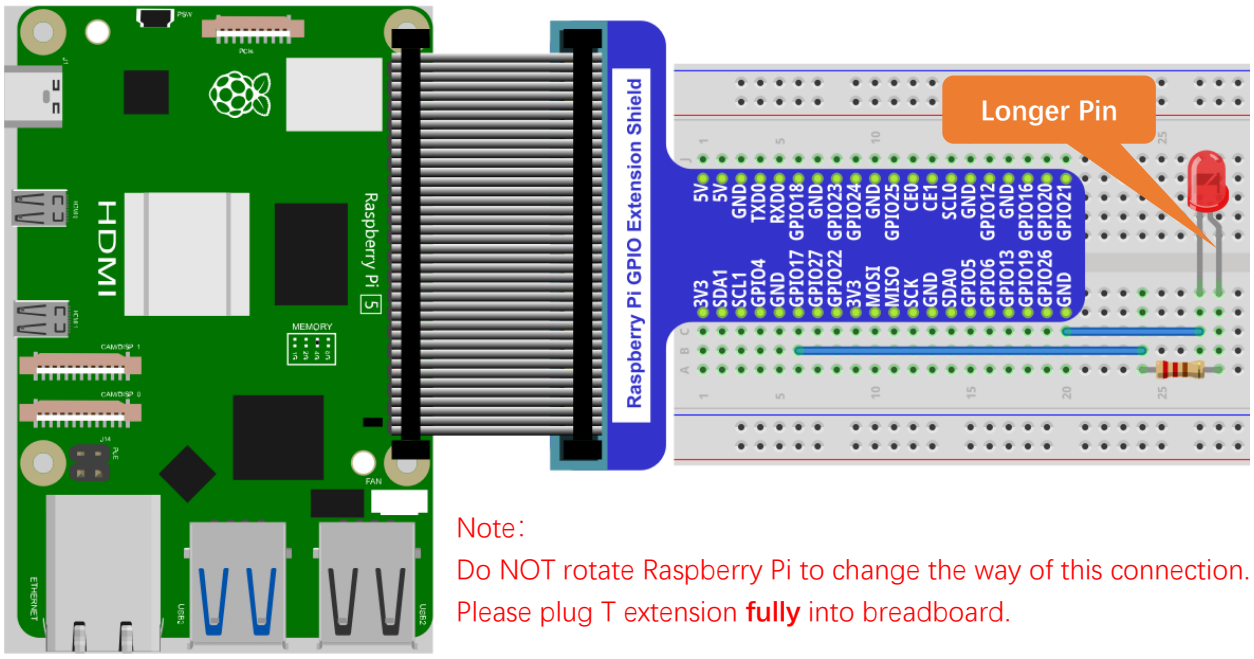
Circuit

First, disconnect your RPi from the GPIO Extension Shield. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the RPi to GPIO Extension Shield.
CAUTION: Avoid any possible short circuits (especially connecting 5V or GND, 3.3V and GND)!
WARNING: A short circuit can cause high current in your circuit, create excessive component heat and cause permanent damage to your RPi!

Schematic diagram

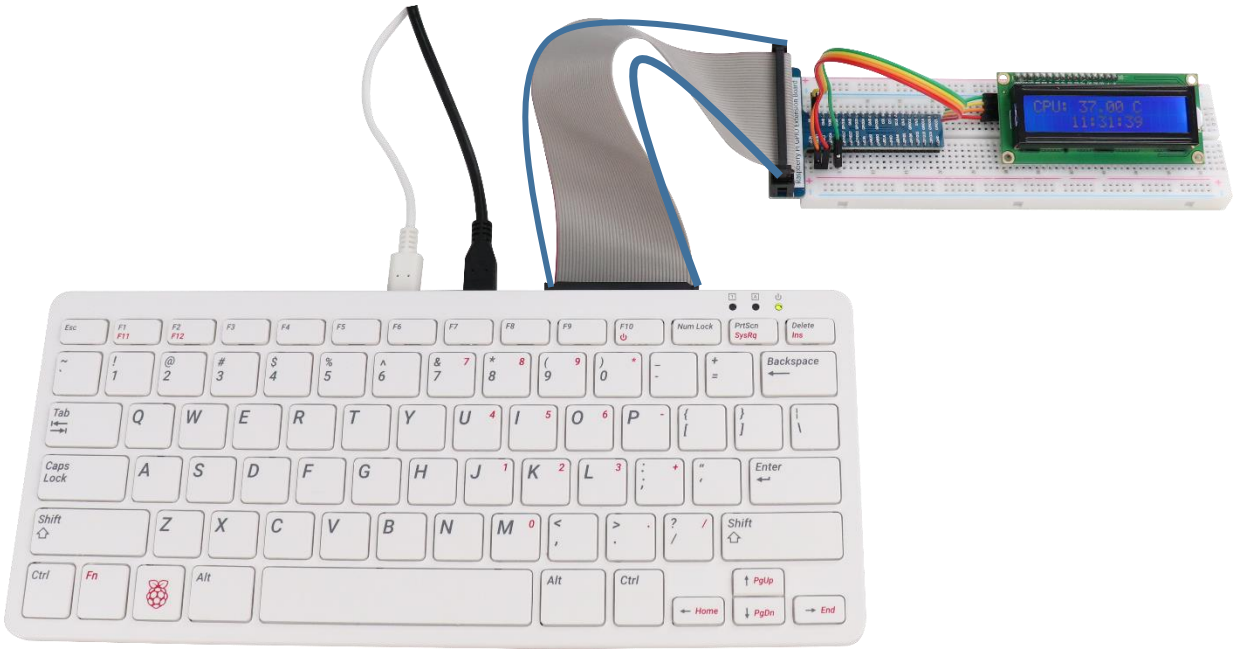


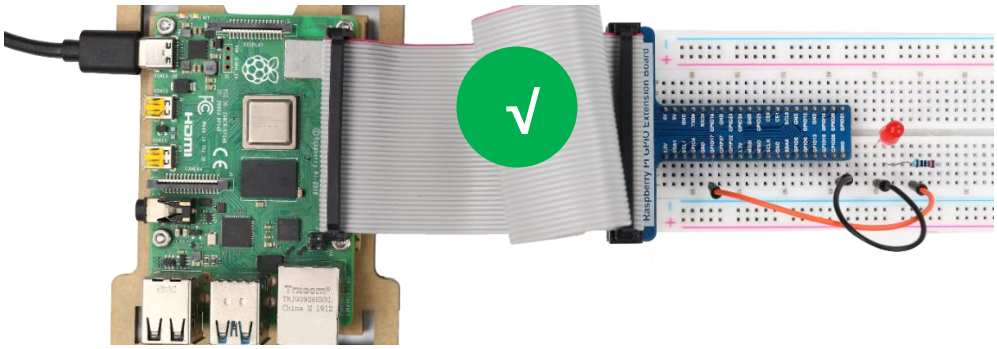
Hardware connection. **If you need any support, please contact us via: support@freenove.com**

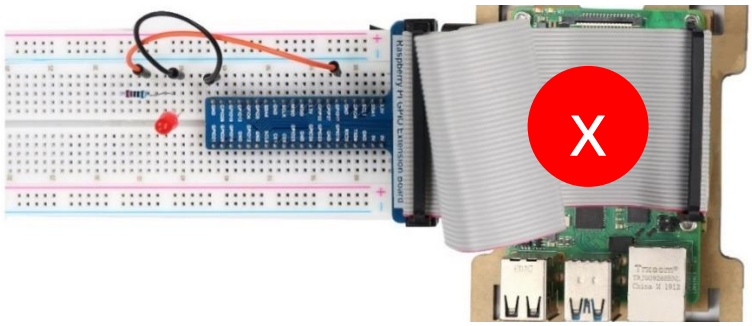


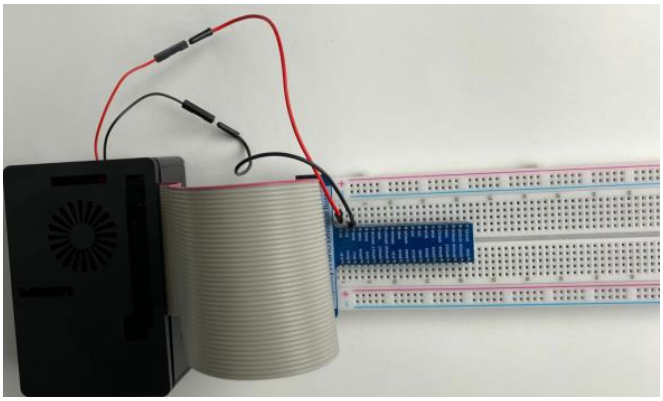
Note:
Do NOT rotate Raspberry Pi to change the way of this connection.
Please plug T extension **fully** into breadboard.

The connection of Raspberry Pi T extension board is as below. Don't reverse the ribbon.









If you have a fan, you can connect it to 5V GND of breadboard via jumper wires.

How to distinguish resistors?

There are only three kind of resistors in this kit.

The one with 1 red ring is 10KΩ

The one with 2 red rings is 220Ω

The one with 0 red ring is 1KΩ

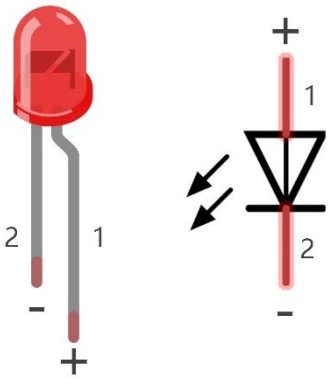
Future hardware connection diagrams will only show that part of breadboard and GPIO Extension Shield.

Component knowledge

LED

An LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two Poles. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-) output, which is also referred to as Ground (GND). This type of component is known as “Polar” (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burnt out.



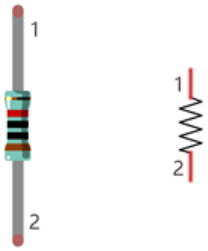
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). 1MΩ=1000kΩ, 1kΩ=1000Ω.

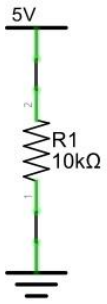
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the card in the kit package.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: I=V/R known as Ohm’s Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: I=U/R=5V/10kΩ=0.0005A=0.5mA.

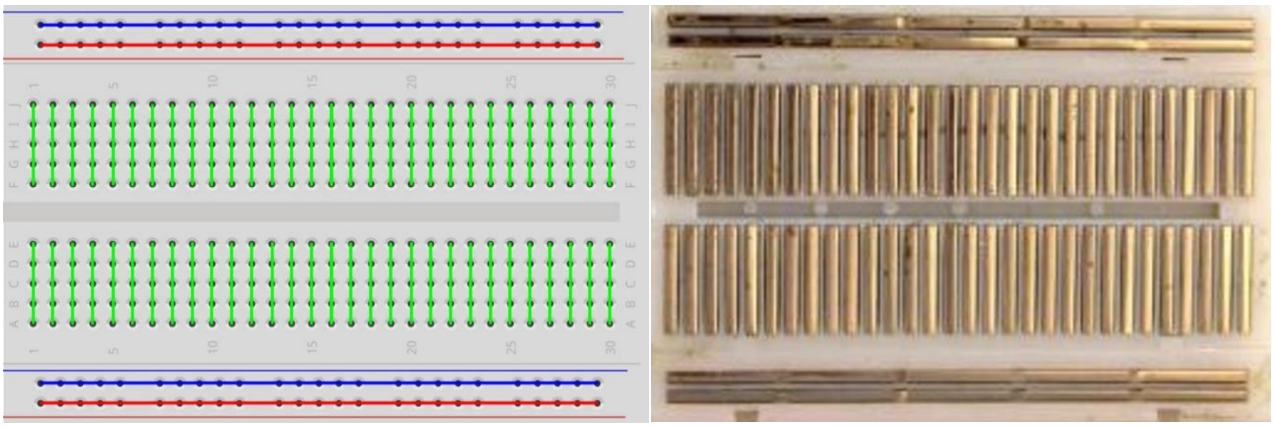


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and re non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

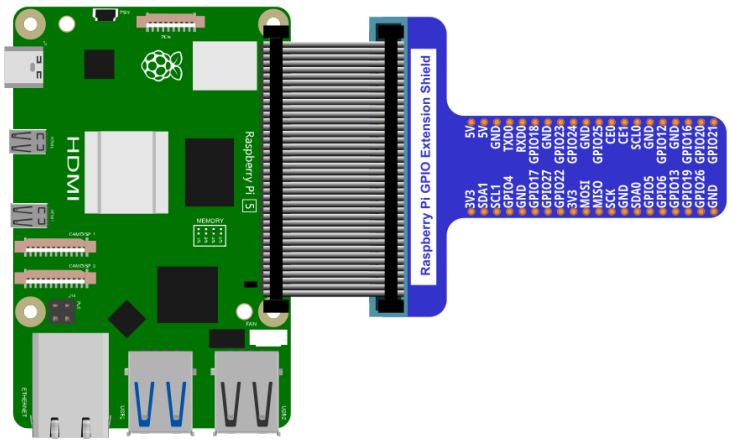
Breadboard

Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the ways the pins have shared electrical connection and the right picture shows the actual internal metal, which connect these rows electrically.



GPIO Extension Board

GPIO board is a convenient way to connect the RPi I/O ports to the breadboard directly. The GPIO pin sequence on Extension Board is identical to the GPIO pin sequence of RPi.



Code

According to the circuit, when the GPIO17 of RPi output level is high, the LED turns ON. Conversely, when the GPIO17 RPi output level is low, the LED turns OFF. Therefore, we can let GPIO17 cycle output high and output low level to make the LED blink. We will use Python code to achieve the target.

Python Code 1.1.1 Blink

Now, we will use Python language to make a LED blink.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

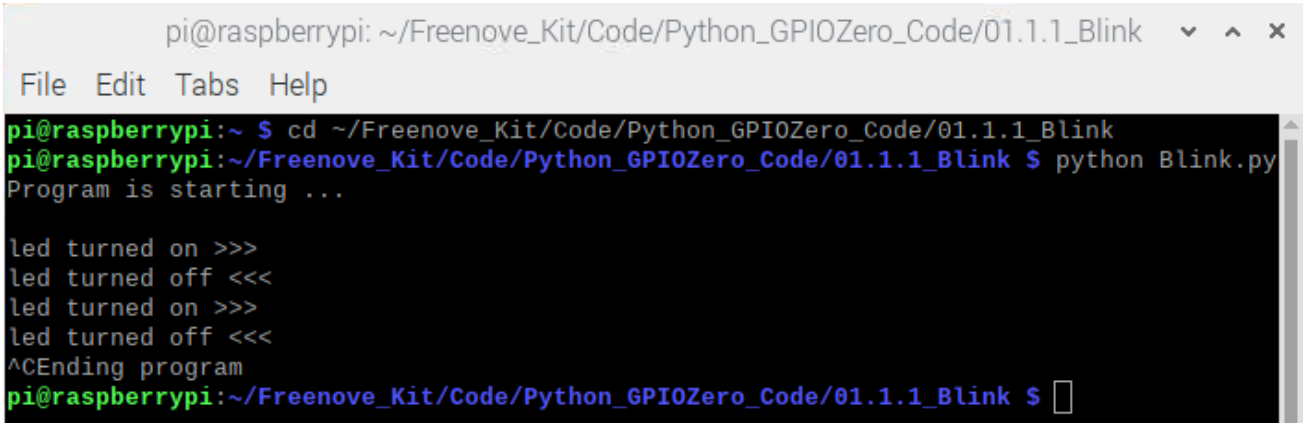
1. Use cd command to enter 01.1.1_Blink directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink
```

2. Use python command to execute python code blink.py.

```
python Blink.py
```

The LED starts blinking.



You can press "Ctrl+C" to end the program. The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3
4  led = LED(17)           # define LED pin according to BCM Numbering
5  #led = LED("J8:11")     # BOARD Numbering
6  '''
7  # pins numbering, the following lines are all equivalent
8  led = LED(17)           # BCM
9  led = LED("GPIO17")     # BCM
10 led = LED("BCM17")      # BCM
11 led = LED("BOARD11")    # BOARD
12 led = LED("WPI0")       # WiringPi
13 led = LED("J8:11")      # BOARD
14 '''
15 def loop():
16     while True:
```

```
17          led.on()     # turn on LED
18          print ('led turned on >>>')  # print message on terminal
19          sleep(1)     # wait 1 second
20          led.off()    # turn off LED
21          print ('led turned off <<<') # print message on terminal
22          sleep(1)     # wait 1 second
23
24  if __name__ == '__main__':     # Program entrance
25      print ('Program is starting ... \n')
26      try:
27          loop()
28      except KeyboardInterrupt:  # Press ctrl-c to end the program.
29          print("Ending program")
```

Import the LED class from the gpiozero library.

```
from gpiozero import LED
```

Create an LED assembly for controlling the LED.

```
led = LED(17)           # define LED pin according to BCM Numbering
```

Turn on LED device.

```
        led.on()     # turn on LED
```

Turn off LED devices.

```
        led.off()    # turn off LED
```

The main function turns on the LED for one second and then turns it off for one second, which repeats endless.

```
def loop():
    while True:
        led.on()     # turn on LED
        print ('led turned on >>>')  # print message on terminal
        sleep(1)     # wait 1 second
        led.off()    # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)     # wait 1 second
```

Reference

About GPIO Zero:

GPIO Zero

A simple interface to GPIO devices with Raspberry Pi, Using the GPIO Zero library makes it easy to get started with controlling GPIO devices with Python. The library is comprehensively documented at https://gpiozero.readthedocs.io/en/stable/
https://github.com/gpiozero/gpiozero

For more information about the methods used by the LED class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_output.html#led

For more information about the methods used by the DigitalOutputDevice class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_output.html#digitaloutputdevice

“import time” time is a module of python.

https://docs.python.org/2/library/time.html?highlight=time%20time#module-time

In Python, libraries and functions used in a script must be imported by name at the top of the file, with the exception of the functions built into Python by default.

For example, to use the LED interface from GPIO Zero, it should be explicitly imported:

```
from gpiozero import LED
```

Now LEDis available directly in your script:

```
led = LED(17)            # define LED pin according to BCM Numbering
#led = LED("J8:11")      # BOARD Numbering
```

Alternatively, the whole GPIO Zero library can be imported:

```
import gpiozero
```

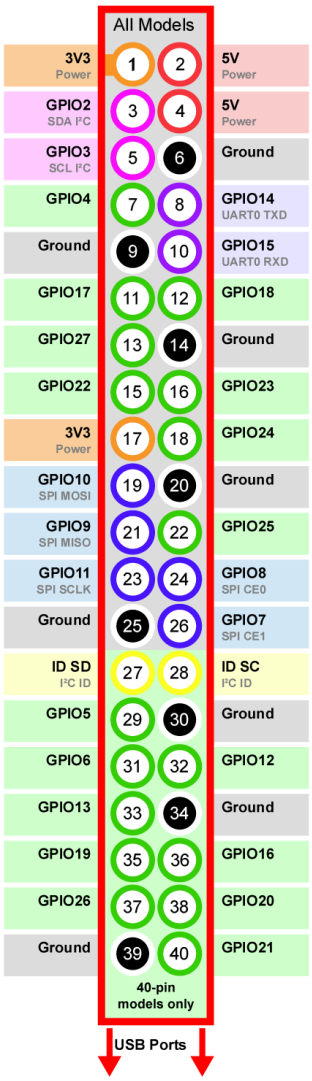
In this case, all references to items within GPIO Zero must be prefixed:

```
led = gpiozero.LED(17)            # define LED pin according to BCM Numbering
#led = gpiozero.LED("J8:11")      # BOARD Numbering
```

Pin Numbering

This library uses Broadcom (BCM) pin numbering for the GPIO pins, as opposed to physical (BOARD) numbering. Unlike in the RPi.GPIO library, this is not configurable. However, translation from other schemes can be used by providing prefixes to pin numbers (see below).

Any pin marked “GPIO” in the diagram below can be used as a pin number. For example, if an LED was attached to “GPIO17” you would specify the pin number as 17 rather than 11:



If you wish to use physical (BOARD) numbering you can specify the pin number as “BOARD11”. If you are familiar with the wiringPi pin numbers (another physical layout) you could use “WPI0” instead.

Finally, you can specify pins as “header:number”, e.g. “J8:11” meaning physical pin 11 on header J8 (the GPIO header on modern Pis). Hence, the following lines are all equivalent:

```
led = LED(17)
led = LED("GPIO17")
led = LED("BCM17")
led = LED("BOARD11")
led = LED("WPI0")
led = LED("J8:11")
```

Note that these alternate schemes are merely translations. If you request the state of a device on the command line, the associated pin number will always be reported in the Broadcom (BCM) scheme:

```
    led = LED("BOARD11")
    led
<gpiozero.LED object on pin GPIO17, active_high=True, is_active=False>
```

In this tutorial, we will use the default integer pin number in the Broadcom (BCM) layout.

GPIO Numbering Relationship

WingPi	BCM(Extension)	Physical		BCM(Extension)	WingPi
3.3V	3.3V	1	2	5V	5V
8	GPIO2/SDA1	3	4	5V	5V
9	GPIO3/SCL1	5	6	GND	GND
7	GPIO4	7	8	GPIO14/TXD0	15
GND	GND	9	10	GPIO15/RXD0	16
0	GPIO17	11	12	GPIO18	1
2	GPIO27	13	14	GND	GND
3	GPIO22	15	16	GPIO23	4
3.3V	3.3V	17	18	GPIO24	5
12	GPIO10/MOSI	19	20	GND	GND
13	GPIO9/MOIS	21	22	GPIO25	6
14	GPIO11/SCLK	23	24	GPIO8/CE0	10
GND	GND	25	26	GPIO7/CE1	11
30	GPIO0/SDA0	27	28	GPIO1/SCL0	31
21	GPIO5	29	30	GND	GND
22	GPIO6	31	32	GPIO12	26
23	GPIO13	33	34	GND	GND
24	GPIO19	35	36	GPIO16	27
25	GPIO26	37	38	GPIO20	28
GND	GND	39	40	GPIO21	29

In loop(), there is a while loop, which is an endless loop (a while loop). That is, the program will always be executed in this loop, unless it is ended because of external factors. In this loop, set LED output high level, then the LED turns ON. After a period of time delay, set LED output low level, then the LED turns OFF, which is followed by a delay. Repeat the loop, then LED will start blinking.

```
def loop():
    while True:
        led.on()     # turn on LED
        print ('led turned on >>>')  # print message on terminal
        sleep(1)     # wait 1 second
        led.off()    # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)     # wait 1 second
```

In gpiozero, at the end of your script, cleanup is run automatically, restoring your GPIO pins to the state they were found.To explicitly close a connection to a pin, you can manually call the close() method on a device object:

```
    led = LED(17)
```

```
    led.on()
    led
<gpiozero.LED object on pin GPIO17, active_high=True, is_active=True>
    led.close()
    led
<gpiozero.LED object closed>
```

This means that you can reuse the pin for another device, and that despite turning the LED on (and hence, the pin high), after calling close() it is restored to its previous state (LED off, pin low).

In this tutorial, most projects have added an active run cleanup program to restore the GPIO pin to the found default state.

Freenove Car, Robot and other products for Raspberry Pi

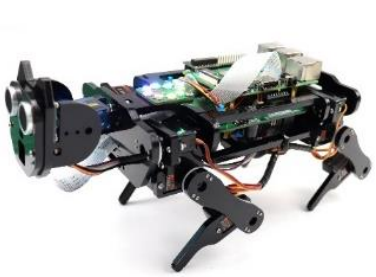
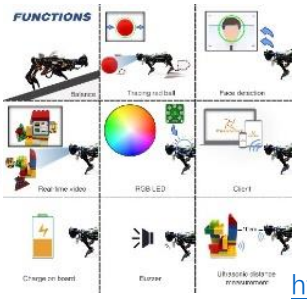
We also have car and robot kits for Raspberry Pi. You can visit our website for details.

https://www.amazon.com/freenove

FNK0043 **Freenove 4WD Smart Car Kit for Raspberry Pi**

https://www.youtube.com/watch?v=4Zv0GZUQjZc

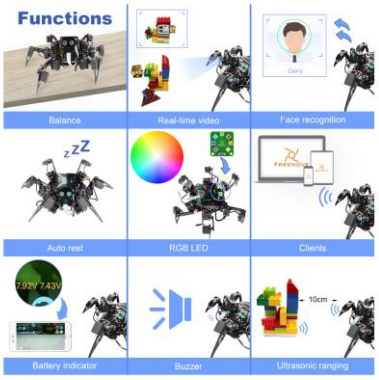
FNK0050 **Freenove Robot Dog Kit for Raspberry Pi**

https://www.youtube.com/watch?v=7BmlZ8_R9d4

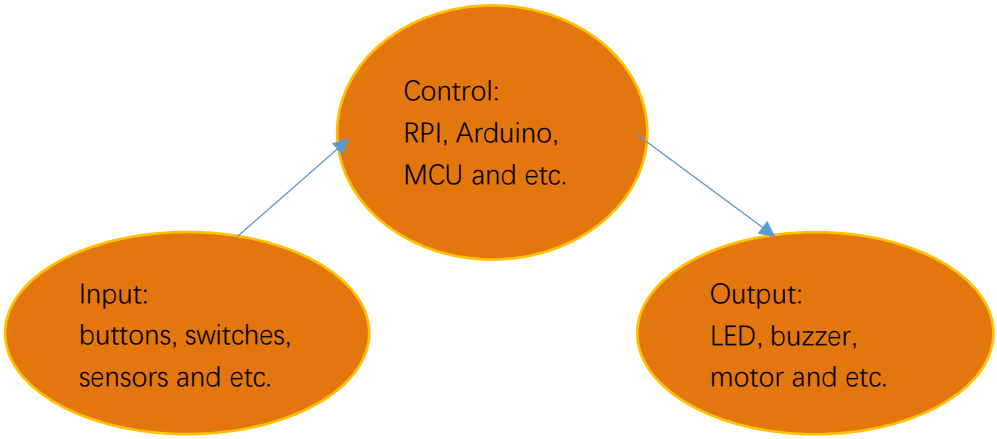
FNK0052 **Freenove_Big_Hexapod_Robot_Kit_for_Raspberry_Pi**

https://youtu.be/LvghnJ2DNZ0



Chapter 2 Buttons & LEDs

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and RPI was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as turn on LEDs, make a buzzer beep and so on.



Next, we will build a simple control system to control an LED through a push button switch.

Project 2.1 Push Button Switch & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

Component List

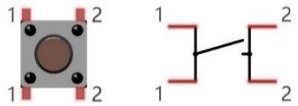
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	LED x1	Resistor 220Ω x1	Resistor 10kΩ x2	Push Button Switch x1
Jumper Wire				

Please Note: In the code “button” represents switch action.

Component knowledge

Push Button Switch

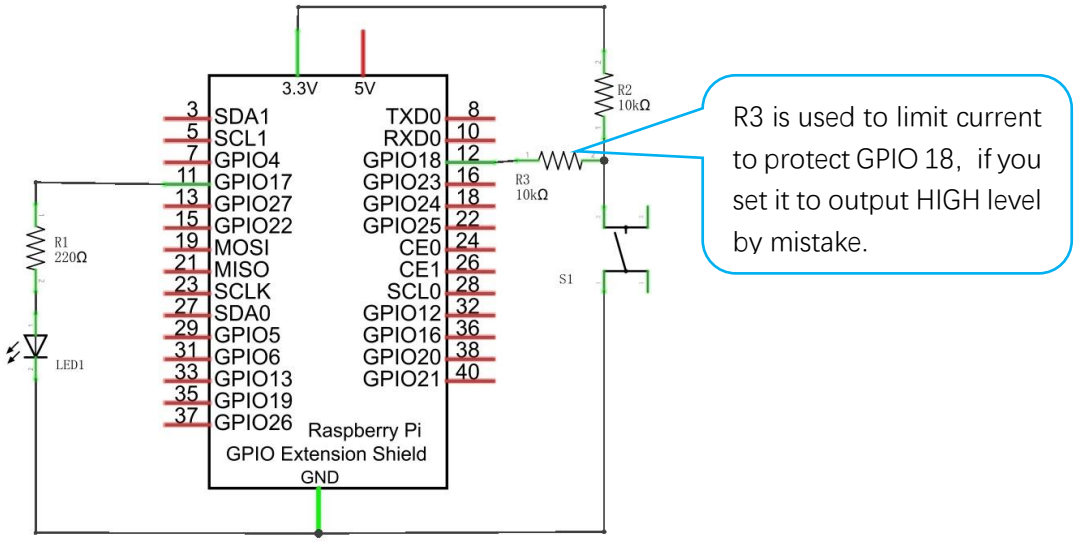
This type of Push Button Switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



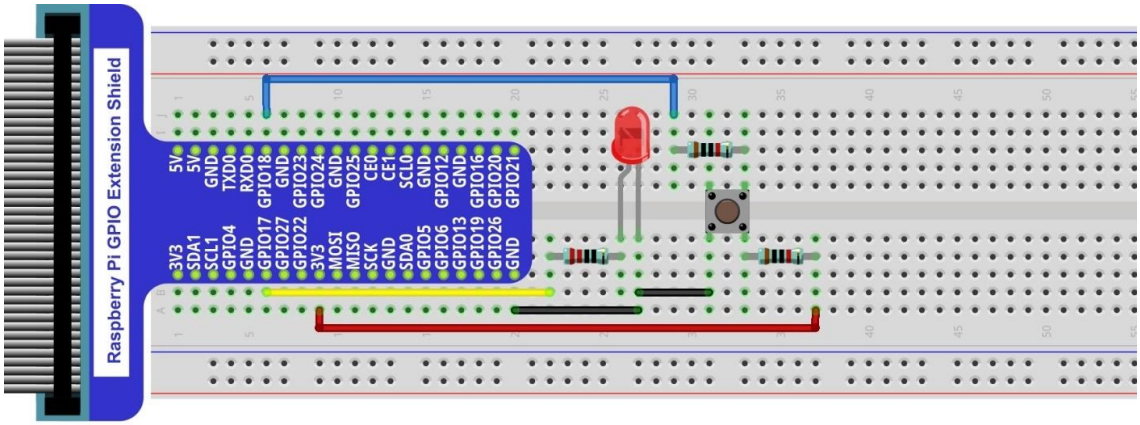
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:support@freenove.com



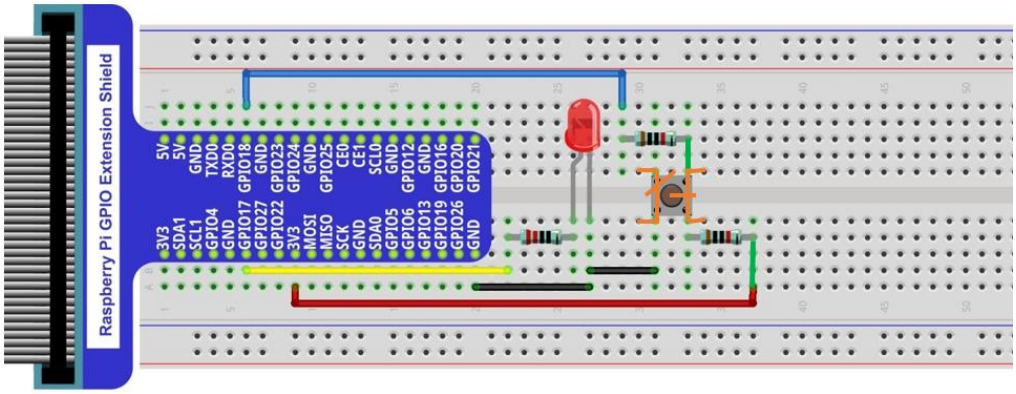
There are two kinds of push button switch in this kit.

The smaller push button switches are contained in a plastic bag.

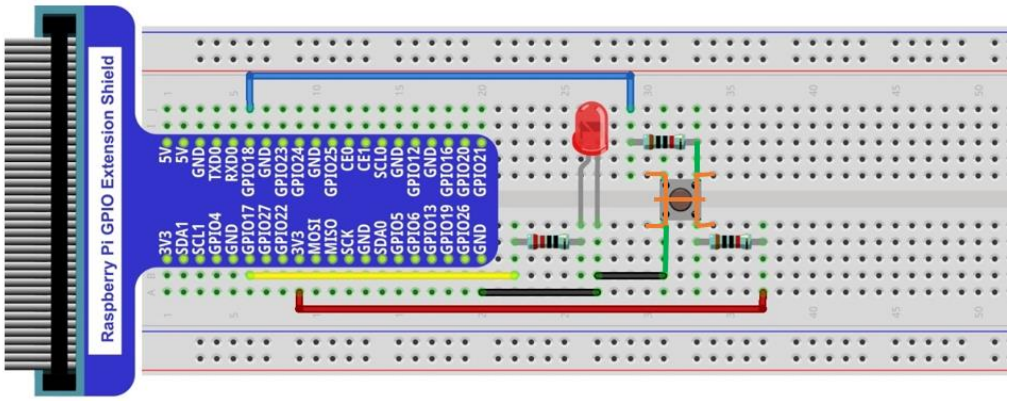
Youtube video: https://youtu.be/_5ge1d6f1nM

This is how it works.

When button switch is released:



When button switch is pressed:



Code

This project is designed for learning how to use Push Button Switch to control an LED. We first need to read the state of switch, and then determine whether to turn the LED ON in accordance to the state of the switch.

Python Code 2.1.1 ButtonLED

First, observe the project result, then learn about the code in detail. Remember in code “button” = switch function

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.1.1_ButtonLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/02.1.1_ButtonLED
```

2. Use Python command to execute btnLED.py.

```
python ButtonLED.py
```

Then the Terminal window continues to show the characters “led off…”, press the switch button and the LED turns ON and then Terminal window shows "led on…". Release the button, then LED turns OFF and then the terminal window text "led off…" appears. You can press "Ctrl+C" at any time to terminate the program.

The following is the program code:

```
1  from gpiozero import LED, Button
2
```

```
led = LED(17)        # define LED pin according to BCM Numbering
button = Button(18) # define Button pin according to BCM Numbering

def loop():
    while True:
        if button.is_pressed:  # if button is pressed
            led.on()         # turn on led
            print("Button is pressed, led turned on >>>") # print information on terminal
        else : # if button is relessed
            led.off() # turn off led
            print("Button is released, led turned off <<<")

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
```

Import the Button class that controls Button from the gpiozero library.

```
from gpiozero import LED, Button
```

Define GPIO17 as the LED control pin and GPIO18 as the button control pin. The button is set to the input mode with a pull-up resistor by default.

```
led = LED(17)        # define LED pin according to BCM Numbering
button = Button(18) # define Button pin according to BCM Numbering
```

The loop continuously determines whether the key is pressed. When the button is pressed, the variable button.is_pressed has a value of 1 and the LED lights up. Otherwise, the LED will be off.

```
def loop():
    while True:
        if button.is_pressed:  # if button is pressed
            led.on()         # turn on led
            print("Button is pressed, led turned on >>>") # print information on terminal
        else : # if button is relessed
            led.off() # turn off led
            print("Button is released, led turned on <<<")
```

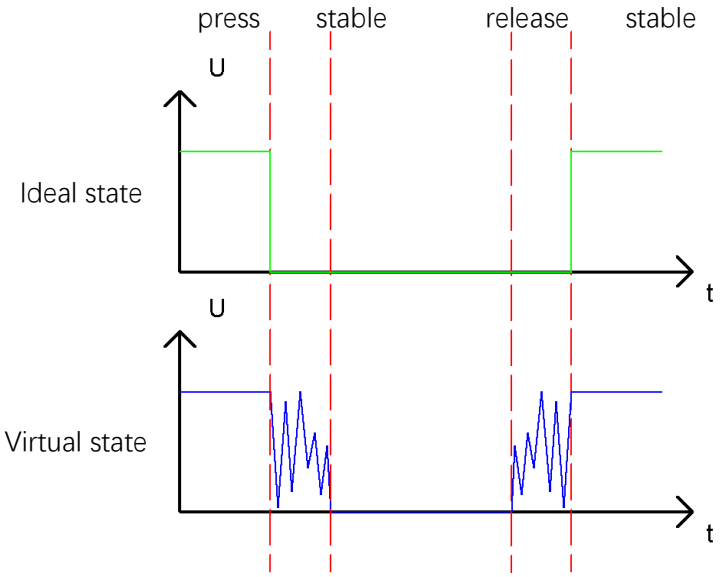
For more information about GPIOZero, please refer to the link below:
https://gpiozero.readthedocs.io/en/stable/
For more information about the methods used by the Button class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_input.html#button

Project 2.2 MINI Table Lamp

We will also use a Push Button Switch, LED and RPi to make a MINI Table Lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.
First, let us learn something about the push button switch.

Debounce a Push Button Switch

When a Momentary Push Button Switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it stabilizes in a new state too fast for Humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



Therefore, if we can directly detect the state of the Push Button Switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).
This project needs the same components and circuits as we used in the previous section.

Code

In this project, we still detect the state of Push Button Switch to control an LED. Here we need to define a variable to define the state of LED. When the button switch is pressed once, the state of LED will be changed once. This will allow the circuit to act as a virtual table lamp.

Python Code 2.2.1 Tablelamp

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.2.1_Tablelamp directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/02.2.1_Tablelamp
```

2. Use python command to execute python code "Tablelamp.py".

```
python Tablelamp.py
```

When the program is executed, pressing the Button Switch once turns the LED ON. Pressing the Button Switch again turns the LED OFF.

```
1  from gpiozero import LED, Button
2  import time
3
4  led = LED(17) # define LED pin according to BCM Numbering
5  button = Button(18) # define Button pin according to BCM Numbering
6
7  def onButtonPressed():  # When button is pressed, this function will be executed
8      led.toggle()
9      if led.is_lit :
10         print("Led turned on >>>")
       else :
11         print("Led turned off <<<")
12 def loop():
13     #Button detect
14     button.when_pressed = onButtonPressed
15     while True:
16         time.sleep(1)
17 def destroy():
18     led.close()
19     button.close()
20 if __name__ == '__main__':     # Program entrance
21     print ('Program is starting...')
22     try:
23         loop()
24     except KeyboardInterrupt:  # Press ctrl-c to end the program.
25         destroy()
26         print("Ending program")
```

In GPIO Zero, you assign the when_pressed and when_released properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function onButtonPressed().In the onButtonPressed function, the led. toggle () function reverses the state of the LED device.If it's on, turn it off; If it's off, turn it on.Each time the key is pressed, the state of the LED will change once.

```
def onButtonPressed():  # When button is pressed, this function will be executed
    led.toggle()
    if led.is_lit :
        print("Led turned on >>>")
    else :
        print("Led turned off <<<")

def loop():
    #Button detect
    button.when_pressed = onButtonPressed
    while True:
        time.sleep(1)
```

To explicitly close a connection to a pin, you can manually call the close() method on a device object:

```
def destroy():
    led.close()
    button.close()

    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

For more information about the methods used by the Button class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#button

Chapter 3 LED Bar Graph

We have learned how to control one LED to blink. Next, we will learn how to control a number of LEDs.

Project 3.1 Flowing Water Light

In this project, we use a number of LEDs to make a flowing water light.

Component List

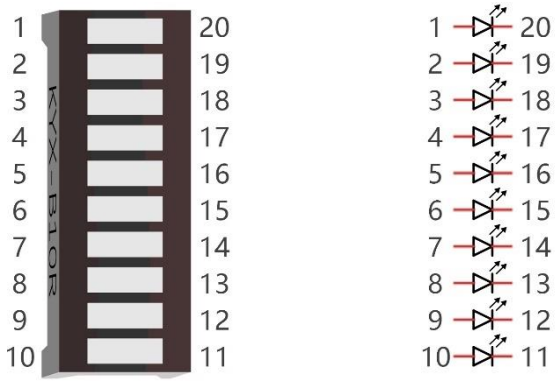
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Bar Graph LED x1	Resistor 220Ω x10
Jumper Wire x 1		

Component knowledge

Let us learn about the basic features of these components to use and understand them better.

Bar Graph LED

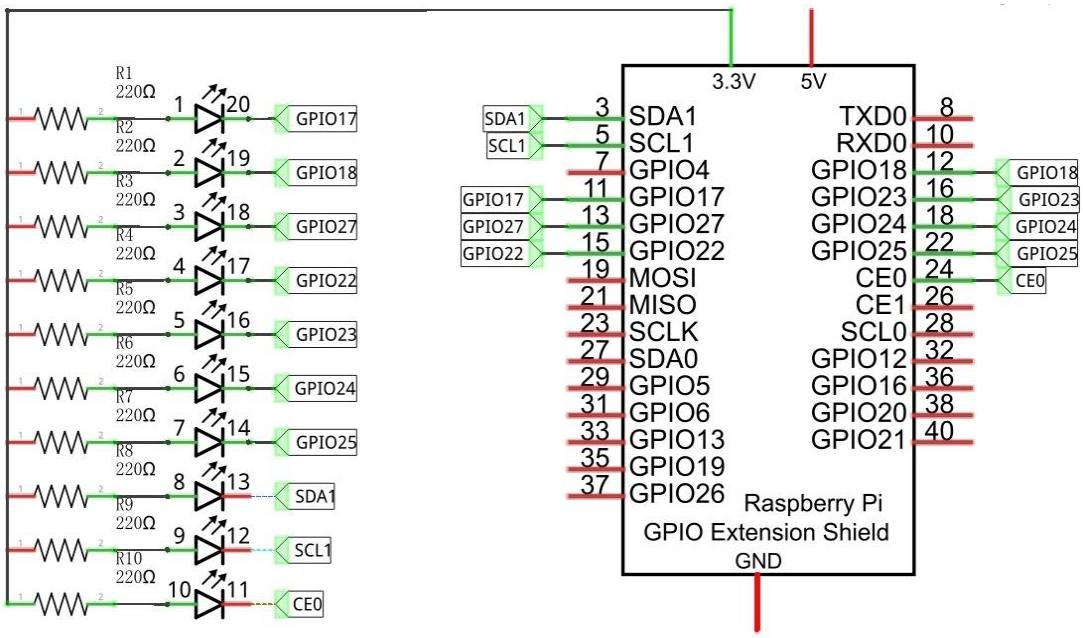
A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.



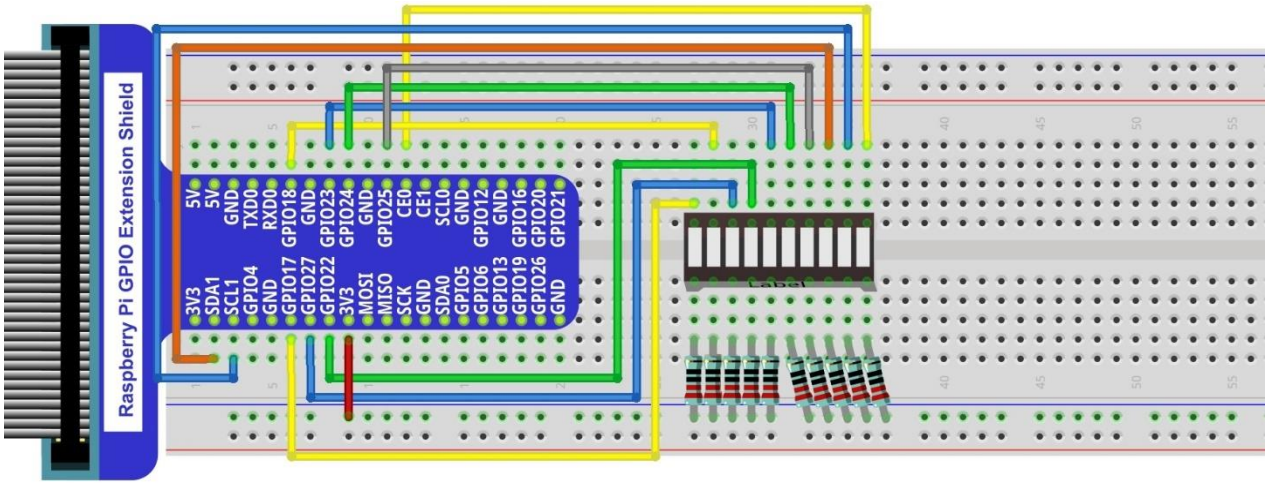
Circuit

A reference system of labels is used in the circuit diagram below. Pins with the same network label are connected together.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



If LEDbar doesn't work, rotate LEDbar 180° to try. The label is random.

Youtube video: https://youtu.be/3rh-b05VoiU

In this circuit, the cathodes of the LEDs are connected to the GPIO, which is different from the previous circuit. The LEDs turn ON when the GPIO output is low level in the program.

Code

This project is designed to make a flowing water lamp, which are these actions: First turn LED #1 ON, then

turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the "movements" of flowing water.

Python Code 3.1.1 LightWater

First observe the project result, and then view the code.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 03.1.1_LightWater directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/03.1.1_LightWater
```

2. Use Python command to execute Python code "LightWater.py".

```
python LightWater.py
```

After the program is executed, you will see that LED Bar Graph starts with the flowing water way to be turned on from left to right, and then from right to left.

The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3  ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
4  leds = [LED(pin=pin) for pin in ledPins]
5
6  def loop():
7      while True:
8          for index in range(0,len(ledPins),1):      # make led(on) move from left to right
9              leds[index].off()
10             sleep(0.1)
11             leds[index].on()
12         for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
13             leds[index].off()
14             sleep(0.1)
15             leds[index].on()
16
17 if __name__ == '__main__':     # Program entrance
18     print ('Program is starting...')
19     try:
20         loop()
21     except KeyboardInterrupt:  # Press ctrl-c to end the program.
22         print("Ending program")
23     finally:
24         for index in range(0,len(ledPins),1):
25             leds[index].close()
```

Import the LED class that controls LED Bar Graph from the gpiozero library.

```
from gpiozero import LED
```

Create the LED class for controlling the LEDBarGraph.

```
ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
leds = [LED(pin=pin) for pin in ledPins]
```

The LED is turned on or off by specifying the index of the LED, if no parameter is specified, the same Settings are applied to all leds. leds.off() means that all leds are turned off.

```
        for index in range(0,len(ledPins),1):       # make led(on) move from left to right
            leds[index].off()
            sleep(0.1)
            leds[index].on()
        for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
            leds[index].off()
            sleep(0.1)
            leds[index].on()
```

In the program, first define 10 pins connected to the LED and set them to output mode. In the loop() function, two for loops are used to make the lights flow from right to left and from left to right.

```
def loop():
    while True:
        for index in range(0,len(ledPins),1):       # make led(on) move from left to right
            leds.off(index)
            sleep(0.1)
            leds.onindex)
        for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
            leds.offndex)
            sleep(0.1)
            leds.onindex)
```

For more information about the methods used by the LEDBoard class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#ledboard
For more information about the methods used by the LEDBarGraph class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#ledbargraph

In this experiment you can use the LEDBoard and LEDBarGraph classes to control the LEDBarGraph

Chapter 4 Analog & PWM

In previous chapters, we learned that a Push Button Switch has two states: Pressed (ON) and Released (OFF), and an LED has a Light ON and OFF state. Is there a middle or intermediated state? We will next learn how to create an intermediate output state to achieve a partially bright (dim) LED.
First, let us learn how to control the brightness of an LED.

Project 4.1 Breathing LED

We describe this project as a Breathing Light. This means that an LED that is OFF will then turn ON gradually and then gradually turn OFF like "breathing". Okay, so how do we control the brightness of an LED to create a Breathing Light? We will use PWM to achieve this goal.

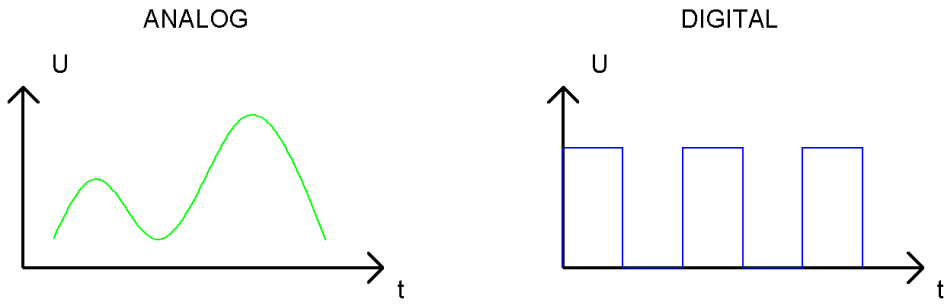
Component List

<table>
<tr><td>Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td rowspan="2">LED x1
 </td><td rowspan="2">Resistor 220Ω x1
 </td></tr>
<tr><td>Jumper Wire
 </td></tr>
</table>

Component Knowledge

Analog & Digital

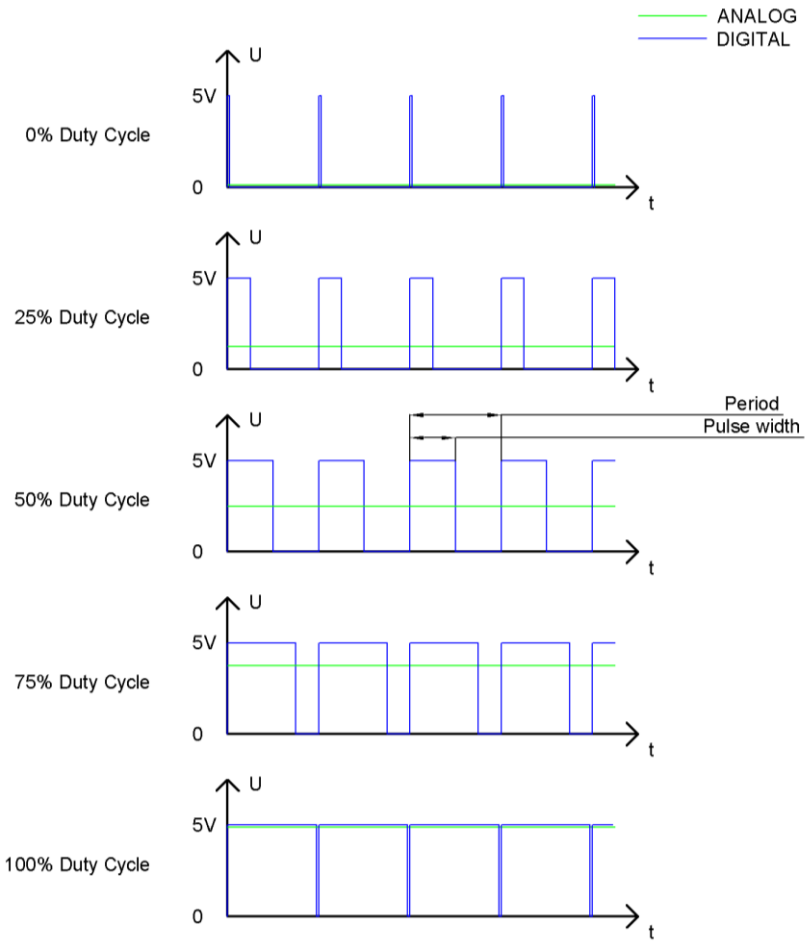
An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0℃ to 10℃. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



Note that the Analog signals are curved waves and the Digital signals are “Square Waves”.
In practical applications, we often use binary as the digital signal, that is a series of 0’s and 1’s. Since a binary signal only has two values (0 or 1) it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Digital processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals).
PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called “pulse width”, and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform. The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.

It is evident, from the above, that PWM is not actually analog but the effective value of voltage is equivalent to the corresponding analog value. Therefore, by using PWM, we can control the output power of to an LED and control other devices and modules to achieve multiple effects and actions.

In RPi, GPIO18 pin has the ability to output to hardware via PWM with a 10-bit accuracy. This means that 100% of the pulse width can be divided into 2^{10}=1024 equal parts.

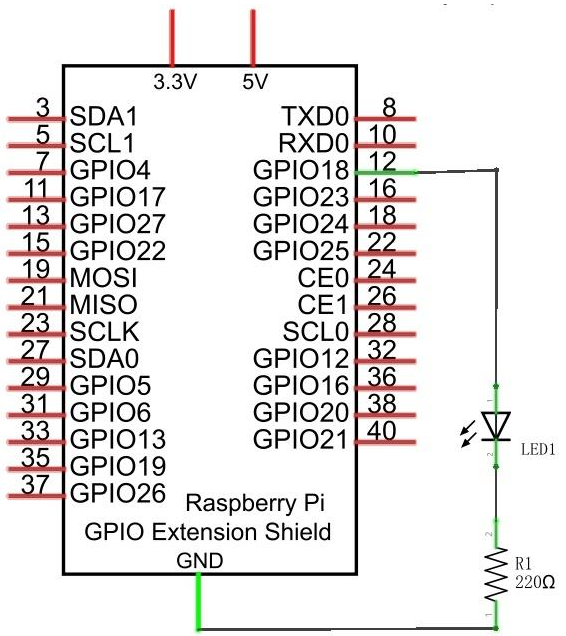
The wiringPi library of C provides both a hardware PWM and a software PWM method, while the wiringPi library of Python does not provide a hardware PWM method. There is only a software PWM option for Python.

The hardware PWM only needs to be configured, does not require CPU resources and is more precise in time control. The software PWM requires the CPU to work continuously by using code to output high level and low level. This part of the code is carried out by multi-threading, and the accuracy is relatively not high enough.

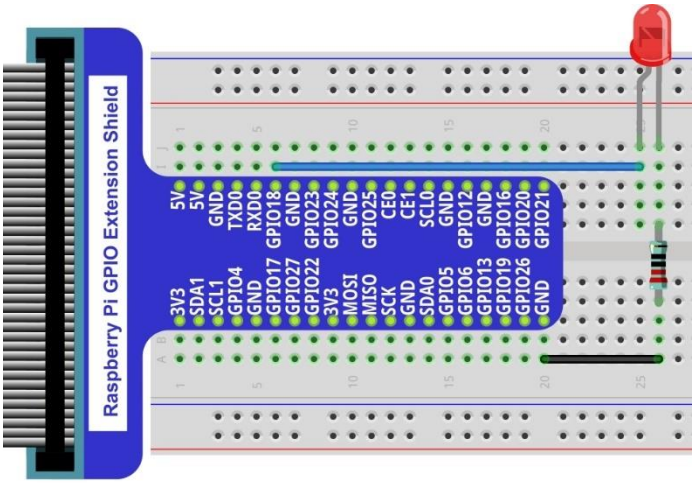
In order to keep the results running consistently, we will use software PWM.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Youtube video: https://youtu.be/rYxykuVgYtA

Code

This project uses the PWM output from the GPIO18 pin to make the pulse width gradually increase from 0% to 100% and then gradually decrease from 100% to 0% to make the LED glow brighter then dimmer.

Python Code 4.1.1 BreathingLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 04.1.1_BreathingLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/04.1.1_BreathingLED
```

2. Use the Python command to execute Python code "BreathingLED.py".

```
python BreathingLED.py
```

After the program is executed, you will see that the LED gradually turns ON and then gradually turns OFF similar to "breathing".

The following is the program code:

```
1  from gpiozero import PWMLED
2  import time
3
4  led = PWMLED(18 ,initial_value=0 ,frequency=1000)
5  def loop():
6      while True:
7          for b in range(0, 101, 1):     # make the led brighter
8              led.value = b / 100.0      # set dc value as the duty cycle
```

```
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
def destroy():
    led.close()
if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the PWMLED class that controls leds from the gpiozero library.

```
from gpiozero import PWMLED
```

Create the PWMLED class for controlling the LED.

```
led = PWMLED(18 ,initial_value=0 ,frequency=1000)
```

PWMLED is connected to GPIO18, and its PWM frequency is set to 1000HZ, and the initial duty cycle to 0%.

```
led = PWMLED(18 ,initial_value=0 ,frequency=1000) # Set the PWM frequency to 1000Hz and the initial duty cycle to 0
```

There are two “for” loops used to control the breathing LED in the next endless “while” loop. The first loop outputs a power signal to the led PWM from 0% to 100% and the second loop outputs a power signal to the led PWM from 100% to 0%.

led.value represents:The duty cycle of the PWM device. 0.0 is off, 1.0 is fully on. led.value in between may be specified for varying levels of power in the device.

```
def loop():
    while True:
        for b in range(0, 101, 1):    # make the led brighter
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
```

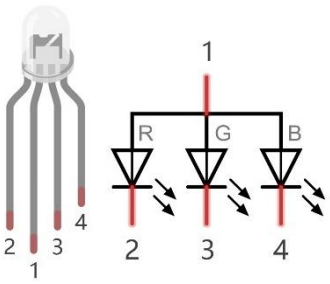
For more information about the methods used by the PWMLED class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmled

For more information about the methods used by the PWMOutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmoutputdevice

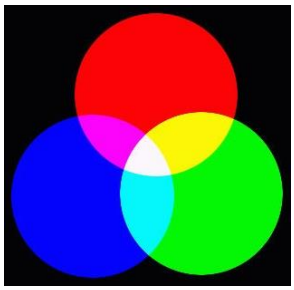
Chapter 5 RGB LED

In this chapter, we will learn how to control a RGB LED.

An RGB LED has 3 LEDs integrated into one LED component. It can respectively emit Red, Green and Blue light. In order to do this, it requires 4 pins (this is also how you identify it). The long pin (1) is the common which is the Anode (+) or positive lead, the other 3 are the Cathodes (-) or negative leads. A rendering of a RGB LED and its electronic symbol are shown below. We can make RGB LED emit various colors of light and brightness by controlling the 3 Cathodes (2, 3 & 4) of the RGB LED



Red, Green, and Blue light are called 3 Primary Colors when discussing light (Note: for pigments such as paints, the 3 Primary Colors are Red, Blue and Yellow). When you combine these three Primary Colors of light with varied brightness, they can produce almost any color of visible light. Computer screens, single pixels of cell phone screens, neon lamps, etc. can all produce millions of colors due to phenomenon.



RGB

If we use a three 8 bit PWM to control the RGB LED, in theory, we can create $2^8*2^8*2^8$=16777216 (16 million) colors through different combinations of RGB light brightness.

Next, we will use RGB LED to make a multicolored LED.

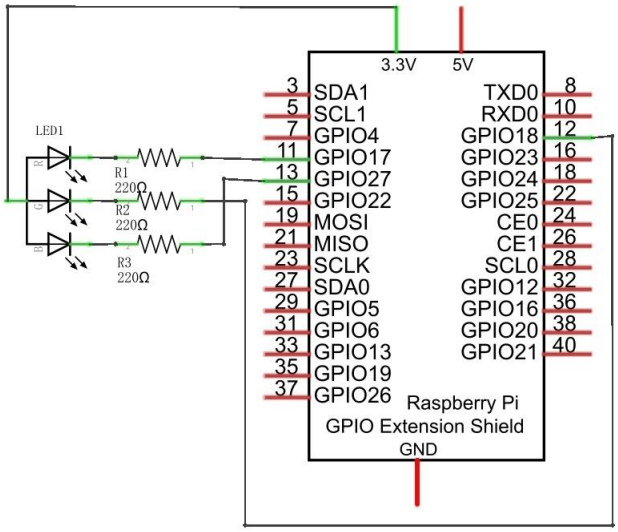
Project 5.1 Multicolored LED

Component List

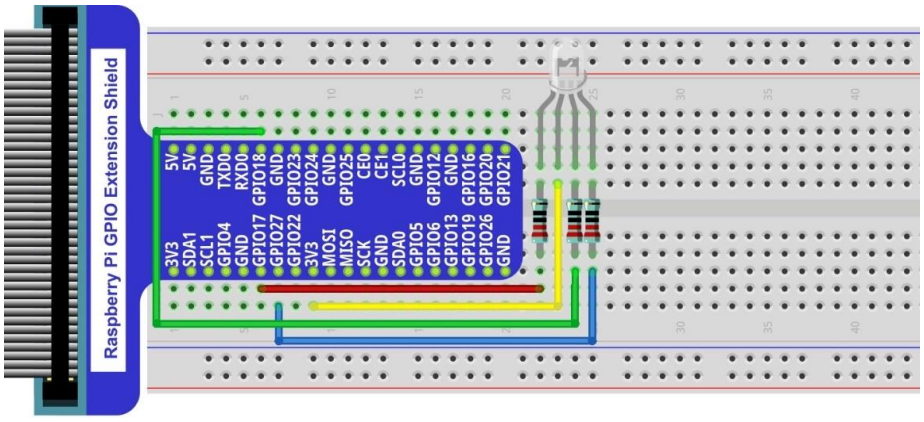
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	RGB LED x1	Resistor 220Ω x3
Jumper Wire		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

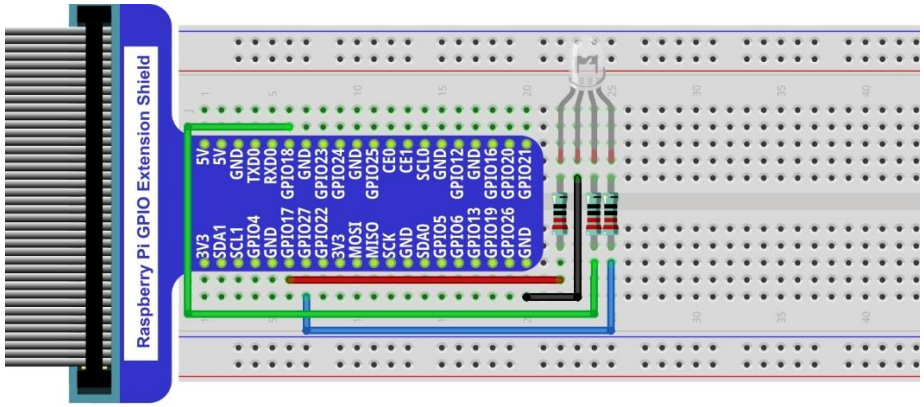


Video: https://youtu.be/tbnX2AsX2y4

In this kit, the RGB led is **Common anode**. The **voltage difference** between LED will make it work. There is no visible GND. The GPIO ports can also receive current while in output mode.

If circuit above doesn't work, the RGB LED may be common cathode. Please try following wiring.

There is no need to modify code for random color.



Code

We need to use RGBLED class to control RGBLED. The parameters for setting the RGBLED as common cathode or common anode are provided in the RGBLED class. You can set it according to the type of your RGB LED, and the default setting in our example code is based on common anode.

Python Code 5.1.1 ColorfulLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 05.1.1_ColorfulLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/05.1.1_ColorfulLED
```

2. Use python command to execute python code "ColorfulLED.py".

```
python ColorfulLED.py
```

After the program is executed, you will see that the RGB LED randomly lights up different colors.

The following is the program code:

```
1  from gpiozero import RGBLED
2  import time
3  import random
4
5  #led = RGBLED(red="J8:11", green="J8:12", blue="J8:13", active_high=False) # define the pins
   for R:11,G:12,B:13
6  led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
   R:GPIO17,G:GPIO18,B:GPIO27
7  # If your RGBLED is a common cathode LED, set active_high to True
8
9  def setColor(r_val,g_val,b_val):      # change duty cycle for three pins to r_val,g_val,b_val
10     led.red=r_val/100                 # change pwmRed duty cycle to r_val
11     led.green = g_val/100             # change pwmRed duty cycle to r_val
12     led.blue = b_val/100              # change pwmRed duty cycle to r_val
13
14 def loop():
```

```
    while True :
        r=random.randint(0,100)   #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)           #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)

def destroy():
    led.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the RGBLED class that controls RGBLED from the gpiozero library.

```
from gpiozero import RGBLED
```

Create the RGBLED class for controlling the RGBLED.

```
led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
R:GPIO17,G:GPIO18,B:GPIO27
```

In the previous chapter, we learned how to make a pin output PWM using the Python language. In this project, we output to three pins via PWM. In the "while" loop of the "loop" function, we first generate three random numbers and then assign these three random numbers to the PWM values of the three pins, which will make the RGB LED randomly produce multiple colors.

```
def loop():
    while True :
        r=random.randint(0,100)   #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)           #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)
```

For more information about the methods used by the RGBLED class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#rgbled

Chapter 6 Buzzer

In this chapter, we will learn about buzzers and the sounds they make. And in our next project, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

Project 6.1 Doorbell

We will make a doorbell with this functionality: when the Push Button Switch is pressed the buzzer sounds and when the button is released, the buzzer stops. This is a momentary switch function.

Component List

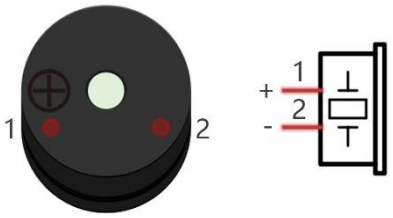
<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wire
 </td></tr>
<tr><td>NPN transistorx1
(S8050)
 </td><td>Active buzzer x1
 </td><td>Push Button Switch x1
 </td><td>Resistor 1kΩ x1
 </td><td>Resistor 10kΩ x2
 </td></tr>
</table>

Component knowledge

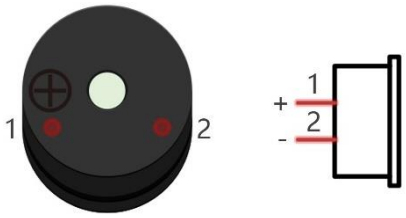
Buzzer

A buzzer is an audio component. They are widely used in electronic devices such as calculators, electronic alarm clocks, automobile fault indicators, etc. There are both active and passive types of buzzers. Active buzzers have oscillator inside, these will sound as long as power is supplied. Passive buzzers require an external oscillator signal (generally using PWM with different frequencies) to make a sound.

Active buzzer



Passive buzzer



Active buzzers are easier to use. Generally, they only make a specific sound frequency. Passive buzzers require an external circuit to make sounds, but passive buzzers can be controlled to make sounds of various frequencies. The resonant frequency of the passive buzzer in this Kit is 2kHz, which means the passive buzzer is the loudest when its resonant frequency is 2kHz.

How to identify active and passive buzzer?

1. As a rule, there is a label on an active buzzer covering the hole where sound is emitted, but there are exceptions to this rule.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes, view of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model.

Active buzzer bottom

Passive buzzer bottom

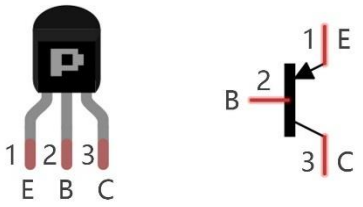
Transistors

A transistor is required in this project due to the buzzer's current being so great that GPIO of RPi's output capability cannot meet the power requirement necessary for operation. A NPN transistor is needed here to

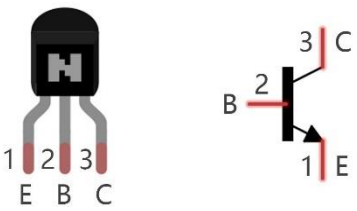
amplify the current.

Transistors, full name: semiconductor transistor, is a semiconductor device that controls current (think of a transistor as an electronic “amplifying or switching device”. Transistors can be used to amplify weak signals, or to work as a switch. Transistors have three electrodes (PINs): base (b), collector (c) and emitter (e). When there is current passing between "be" then "ce" will have a several-fold current increase (transistor magnification), in this configuration the transistor acts as an amplifier. When current produced by "be" exceeds a certain value, "ce" will limit the current output. at this point the transistor is working in its saturation region and acts like a switch. Transistors are available as two types as shown below: PNP and NPN,

PNP transistor



NPN transistor



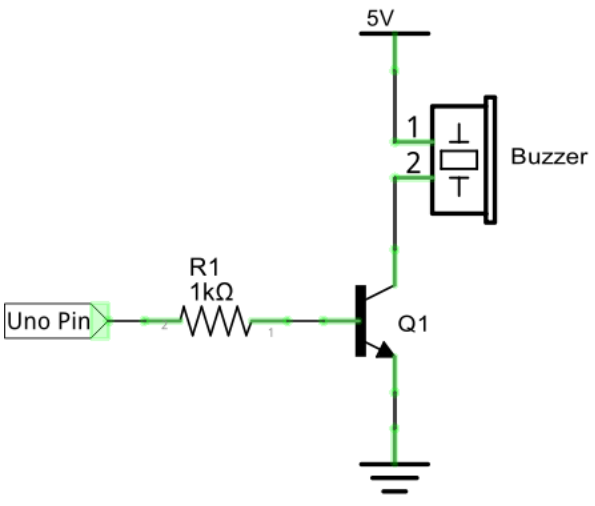
In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Thanks to the transistor's characteristics, they are often used as switches in digital circuits. As micro-controllers output current capacity is very weak, we will use a transistor to amplify its current in order to drive components requiring higher current.

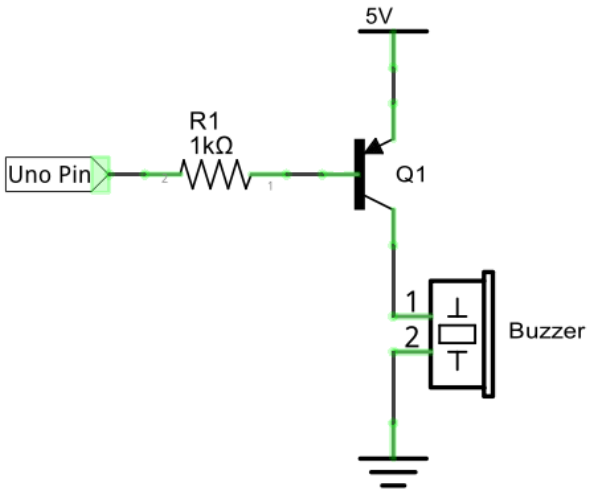
When we use a NPN transistor to drive a buzzer, we often use the following method. If GPIO outputs high level, current will flow through R1 (Resistor 1), the transistor conducts current and the buzzer will make sounds. If GPIO outputs low level, no current will flow through R1, the transistor will not conduct currentand buzzer will remain silent (no sounds).

When we use a PNP transistor to drive a buzzer, we often use the following method. If GPIO outputs low level, current will flow through R1. The transistor conducts current and the buzzer will make sounds. If GPIO outputs high level, no current flows through R1, the transistor will not conduct current and buzzer will remain silent (no sounds). Below are the circuit schematics for both a NPN and PNP transistor to power a buzzer.

NPN transistor to drive buzzer

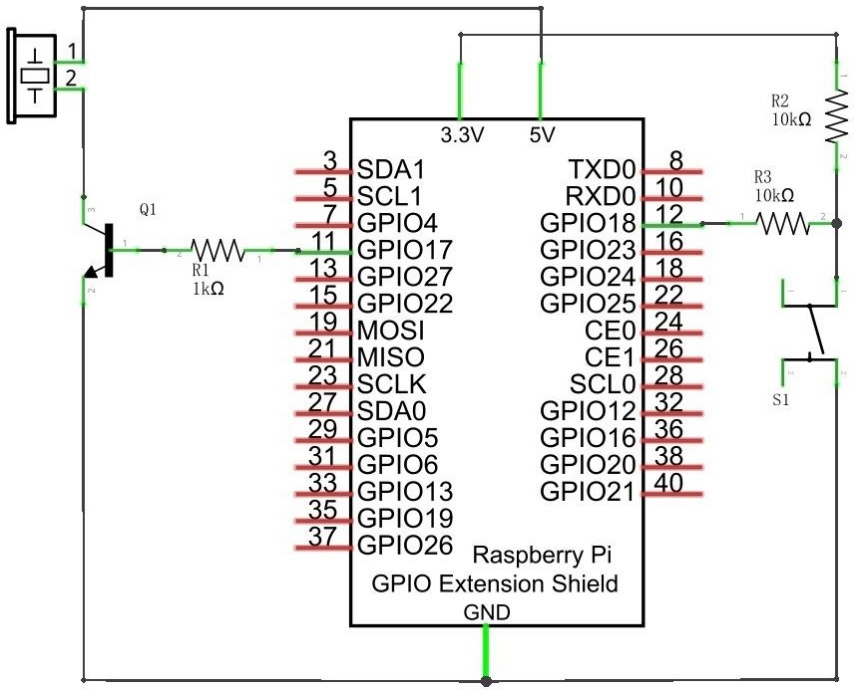


PNP transistor to drive buzzer

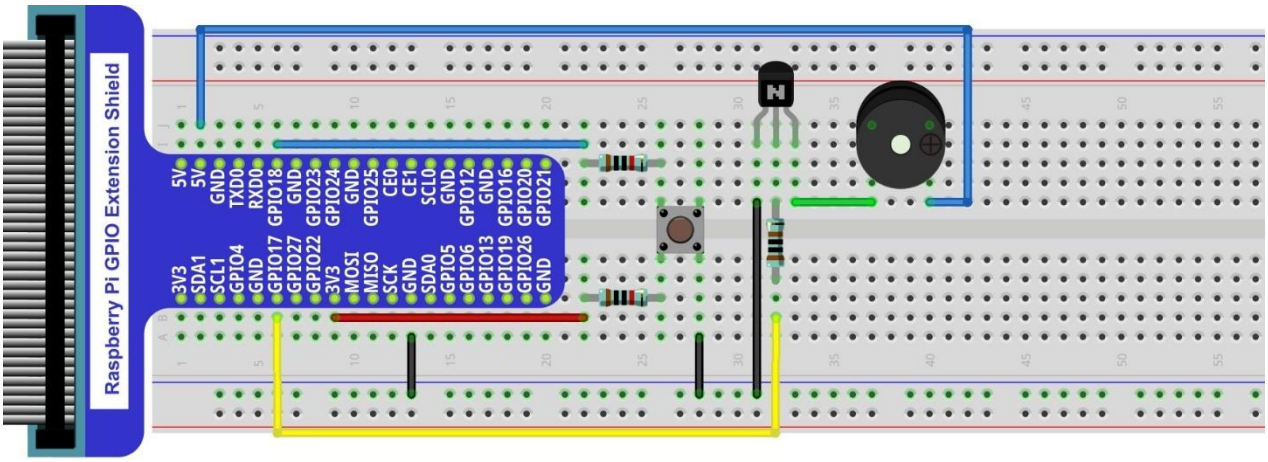


Circuit

Schematic diagram with RPi GPIO Extension Shield.



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/R_dmi3YwY-U

Note: in this circuit, the power supply for the buzzer is 5V, and pull-up resistor of the push button switch is connected to the 3.3V power feed. Actually, the buzzer can work when connected to the 3.3V power feed but this will produce a weak sound from the buzzer (not very loud).

Code

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

Python Code 6.1.1 Doorbell

First, observe the project result, then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.1.1_Doorbell directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/06.1.1_Doorbell
```

2. Use python command to execute python code "Doorbell.py".

```
python Doorbell.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
1  from gpiozero import Buzzer, Button
2  import time
3
4  buzzer = Buzzer(17)
5  button = Button(18)
6
7  def onButtonPressed():
8      buzzer.on()
9      print("Button is pressed, buzzer turned on >>>")
10
11 def onButtonReleased():
12     buzzer.off()
13     print("Button is released, buzzer turned on <<<")
14
15 def loop():
16     button.when_pressed = onButtonPressed
17     button.when_released = onButtonReleased
18     while True :
19         time.sleep(1)
20
21 def destroy():
22     buzzer.close()
23     button.close()
24
25 if __name__ == '__main__':     # Program entrance
26     print ('Program is starting ... ')
27     try:
28         loop()
29     except KeyboardInterrupt:  # Press ctrl-c to end the program.
30         destroy()
31         print("Ending program")
```

The code is exactly the same as when we used a push button switch to control an LED. You can also try using

the PNP transistor to achieve the same results.

Import the Buzzer class that controls Buzzer from the gpiozero library.

```
from gpiozero import Buzzer, Button
```

Create the Buzzer class for controlling the Buzzer.

```
buzzer = Buzzer(17)
```

In GPIO Zero, you assign the when_pressed and when_released properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function onButtonPressed(). Once it detects that the button is released, it executes the specified function onButtonReleased()

```
def loop():
    button.when_pressed = onButtonPressed
    button.when_released = onButtonReleased
```

For more information about the methods used by the Buzzer class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#buzzer

Project 6.2 Alertor

Next, we will use a passive buzzer to make an alarm.
The list of components and the circuit is similar to the doorbell project. We only need to take the Doorbell circuit and replace the active buzzer with a passive buzzer.

Code

In this project, our buzzer alarm is controlled by the push button switch. Press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
As stated before, it is analogous to our earlier project that controlled an LED ON and OFF.
To control a passive buzzer requires PWM of certain sound frequency.

Python Code 6.2.1 Alertor

First observe the project result, and then learn about the code in detail.
If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.2.1_Alertor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/06.2.1_Alertor
```

2. Use the python command to execute the Python code "Alertor.py".

```
python Alertor.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
The following is the program code:

```
1  from gpiozero import TonalBuzzer,Button
2  from gpiozero.tones import Tone
3  import time
4  import math
5
6  buzzer = TonalBuzzer(17)
7  button = Button(18) # define Button pin according to BCM Numbering
8
9  def loop():
10     while True:
11         if button.is_pressed:  # if button is pressed
12             alertor()
13             print ('alertor turned on >>> ')
14         else :
15             stopAlertor()
16             print ('alertor turned off <<<')
17 def alertor():
18     for x in range(0,361):      # Make frequency of the alertor consistent with the sine wave
19         sinVal = math.sin(x * (math.pi / 180.0))        # calculate the sine value
```

```
20          toneVal = 2000 + sinVal * 500   # Add to the resonant frequency with a Weighted
21          buzzer.play(Tone(toneVal))  # Change Frequency of PWM to toneVal
22          time.sleep(0.001)
23
24  def stopAlertor():
25      buzzer.stop()
26
27  def destroy():
28      buzzer.close()
29
30  if __name__ == '__main__':     # Program entrance
31      print ('Program is starting...')
32      try:
33          loop()
34      except KeyboardInterrupt:  # Press ctrl-c to end the program.
35          destroy()
36          print("Ending program")
```

Define GPIO17 as the buzzer control pin, and GPIO18 as the button control pin to control the passive buzzer. It requires a certain frequency of PWM to control a passive buzzer, so the TonalBuzzer class is needed.

```
buzzer = TonalBuzzer(17)
button = Button(18) # define Button pin according to BCM Numbering
```

In the while loop loop of the main function, when the push button switch is pressed the subfunction alertor() will be called and the alarm will issue a warning sound. The frequency curve of the alarm is based on a sine curve. We need to calculate the sine value from 0 to 360 degrees and multiplied by a certain value (here this value is 500) plus the resonant frequency of buzzer. We can set the PWM frequency through Tone(toneVal).

```
def alertor():
    buzzer.play(Tone(220.0))
    time.sleep(0.1)
```

When the push button switch is released, the buzzer (in this case our Alarm) will stop.

```
def stopAlertor():
    buzzer.stop()
```

For more information about the methods used by the TonalBuzzer class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#tonalbuzzer

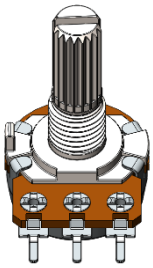
(Important) Chapter 7 ADC

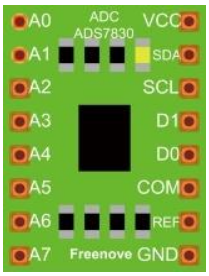
We have learned how to control the brightness of an LED through PWM and that PWM is not a real analog signal. In this chapter, we will learn how to read analog values via an ADC Module and convert these analog values into digital.

Project 7.1 Read the Voltage of Potentiometer

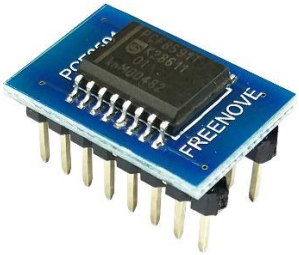
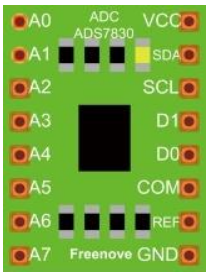
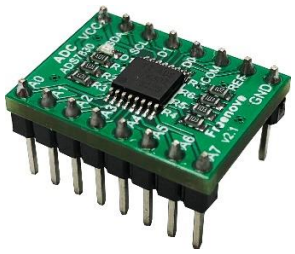
In this project, we will use the ADC function of an ADC Module to read the voltage value of a potentiometer.

Component List

<table>
<tr><td colspan="2">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper Wire M/M x1
 </td></tr>
<tr><td>Rotary potentiometer x1
  </td><td>ADC module x1

or
  </td><td>Resistor 10kΩ x2
 </td></tr>
</table>

This product contains **only one ADC module**, there are two types, PCF8591 and ADS7830. For the projects described in this tutorial, they function the same. Please build corresponding circuits according to the ADC module found in your Kit.

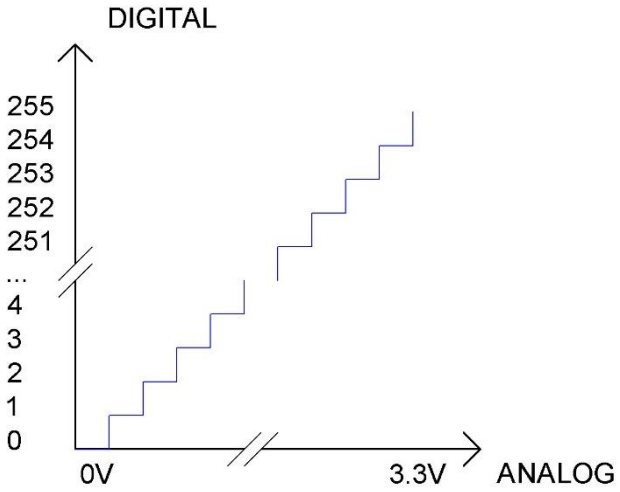
ADC module: PCF8591		**ADC module: ADS7830**	
Model diagram	Actual Picture	Model diagram	Actual Picture
			

Circuit knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC module is 8 bits, that means the resolution is 2^8=256, so that its range (at 3.3V) will be divided equally to 256 parts.

Any analog value can be mapped to one digital value using the resolution of the converter. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of 0V-3.3/256 V corresponds to digital 0;

Subsection 2: the analog in range of 3.3 /256 V-2*3.3 /256V corresponds to digital 1;

...

The resultant analog signal will be divided accordingly.

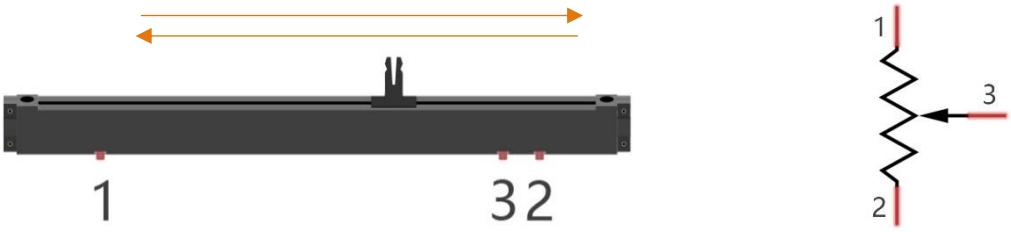
DAC

The reversing this process requires a DAC, Digital-to-Analog Converter. The digital I/O port can output high level and low level (0 or 1), but cannot output an intermediate voltage value. This is where a DAC is useful. The DAC module PCF8591 has a DAC output pin with 8-bit accuracy, which can divide VDD (here is 3.3V) into 2^8=256 parts. For example, when the digital quantity is 1, the output voltage value is 3.3/256 *1 V, and when the digital quantity is 128, the output voltage value is 3.3/256 *128=1.65V, the higher the accuracy of DAC, the higher the accuracy of output voltage value will be.

Component knowledge

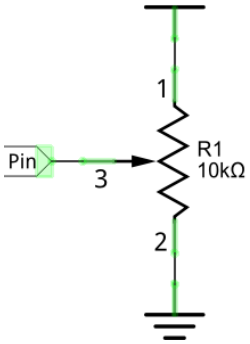
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



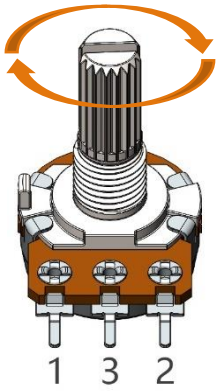
Between potentiometer pin 1 and pin 2 is the resistive element (a resistance wire or carbon) and pin 3 is connected to the brush that makes contact with the resistive element. In our illustration, when the brush moves from pin 1 to pin 2, the resistance value between pin 1 and pin 3 will increase linearly (until it reaches the highest value of the resistive element) and at the same time the resistance between pin 2 and pin 3 will decrease linearly and conversely down to zero. At the midpoint of the slider the measured resistance values between pin 1 and 3 and between pin 2 and 3 will be the same.

In a circuit, both sides of resistive element are often connected to the positive and negative electrodes of power. When you slide the brush "pin 3", you can get variable voltage within the range of the power supply.



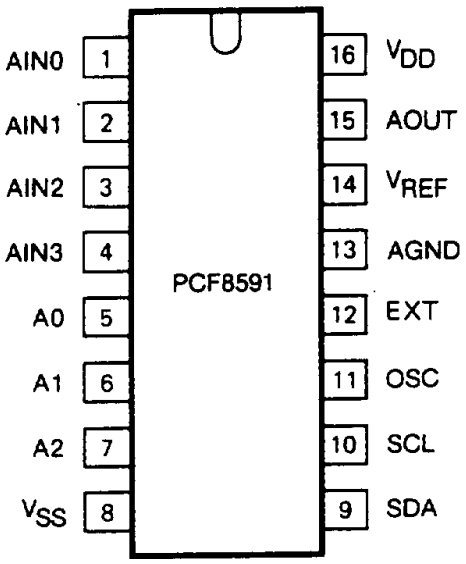
Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.



PCF8591

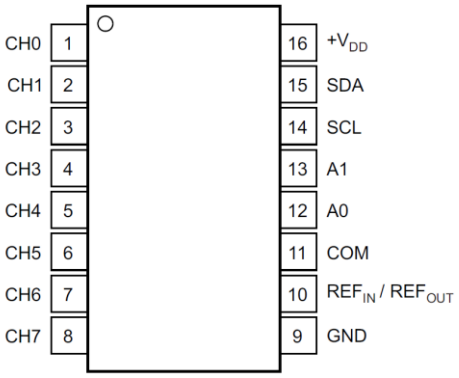
The PCF8591 is a single-chip, single-supply low power 8-bit CMOS data acquisition device with four analog inputs, one analog output and a serial I2C-bus interface. The following table is the pin definition diagram of PCF8591.

SYMBOL	PIN	DESCRIPTION	TOP VIEW
AIN0	1	Analog inputs (A/D converter)	
AIN1	2		
AIN2	3		
AIN3	4		
A0	5	Hardware address	
A1	6		
A2	7		
Vss	8	Negative supply voltage	
SDA	9	I2C-bus data input/output	
SCL	10	I2C-bus clock input	
OSC	11	Oscillator input/output	
EXT	12	external/internal switch for oscillator input	
AGND	13	Analog ground	
Vref	14	Voltage reference input	
AOUT	15	Analog output(D/A converter)	
Vdd	16	Positive supply voltage	

For more details about PCF8591, please refer to the datasheet which can be found on the Internet.

ADS7830

The ADS7830 is a single-supply, low-power, 8-bit data acquisition device that features a serial I2C interface and an 8-channel multiplexer. The following table is the pin definition diagram of ADS7830.

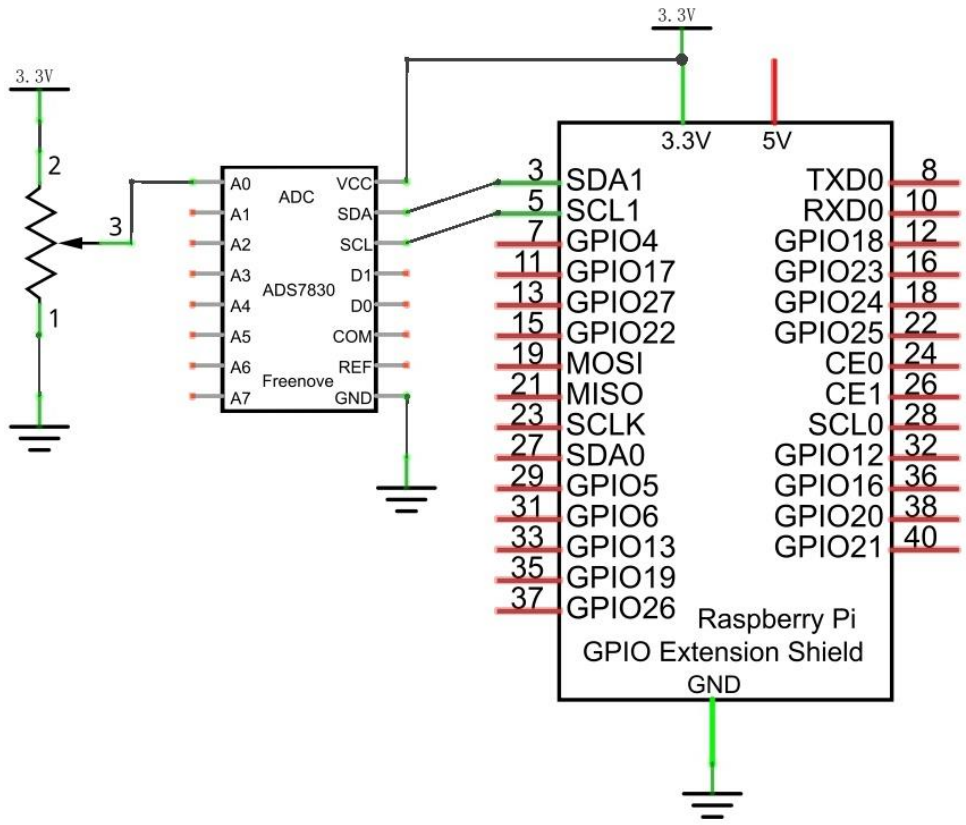
SYMBOL	PIN	DESCRIPTION	TOP VIEW
CH0	1	Analog input channels (A/D converter)	
CH1	2		
CH2	3		
CH3	4		
CH4	5		
CH5	6		
CH6	7		
CH7	8		
GND	9	Ground	
REF in/out	10	Internal +2.5V Reference, External Reference Input	
COM	11	Common to Analog Input Channel	
A0	12	Hardware address	
A1	13		
SCL	14	Serial Clock	
SDA	15	Serial Sata	
+VDD	16	Power Supply, 3.3V Nominal	

I2C communication

I2C (Inter-Integrated Circuit) has a two-wire serial communication mode, which can be used to connect a micro-controller and its peripheral equipment. Devices using I2C communications must be connected to the serial data line (SDA), and serial clock line (SCL) (called I2C bus). Each device has a unique address which can be used as a transmitter or receiver to communicate with devices connected via the bus.

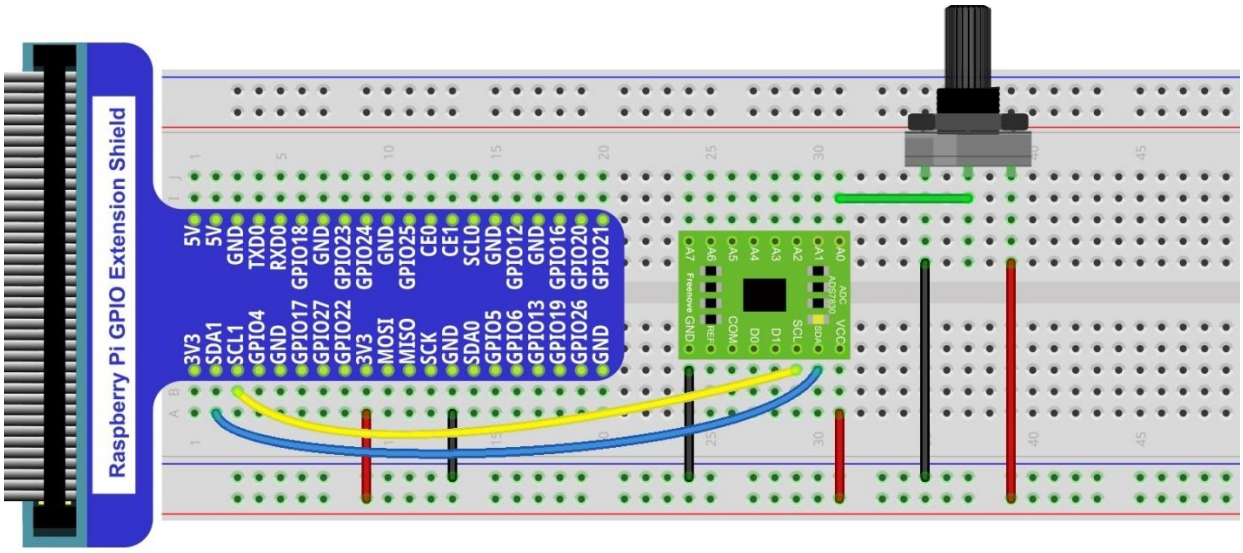
Circuit with ADS7830

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

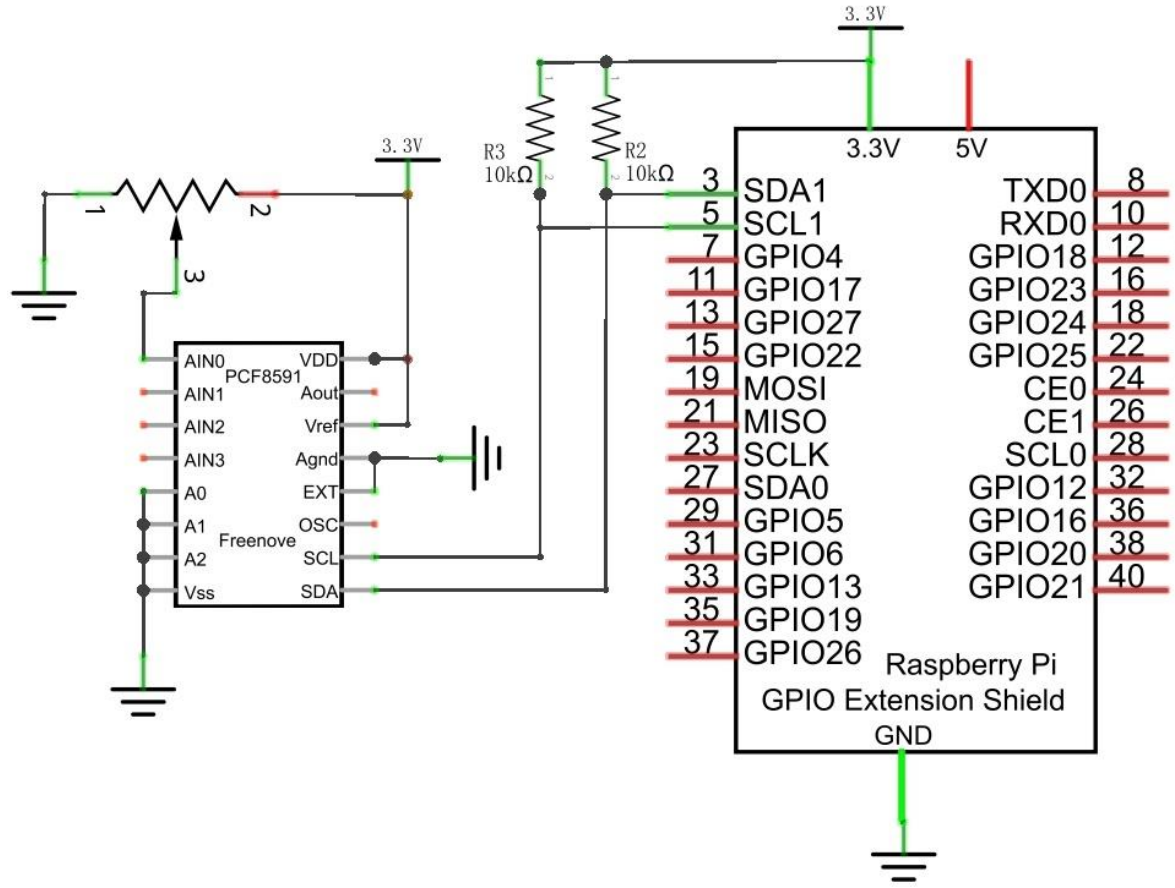
This product contains **only one ADC module.**



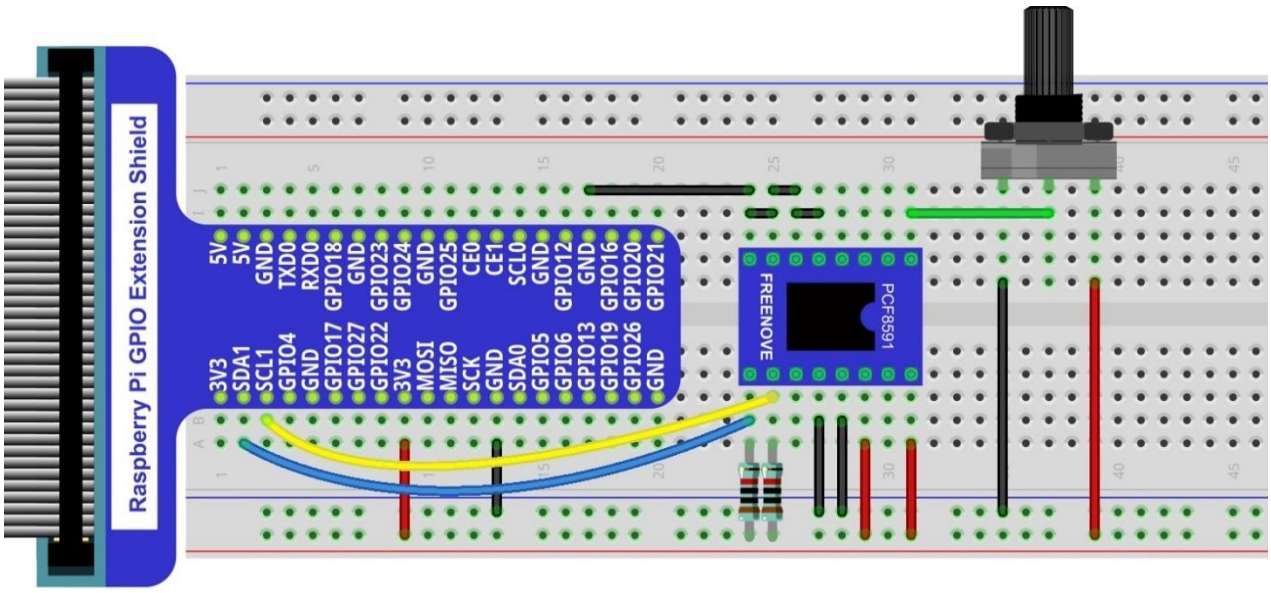
Video: https://youtu.be/PSUCctu_DqA

Circuit with PCF8591

Schematic diagram



Hardware connection



Please keep the **chip mark** consistent to make the chips under right direction and position.

Configure I2C and Install Smbus

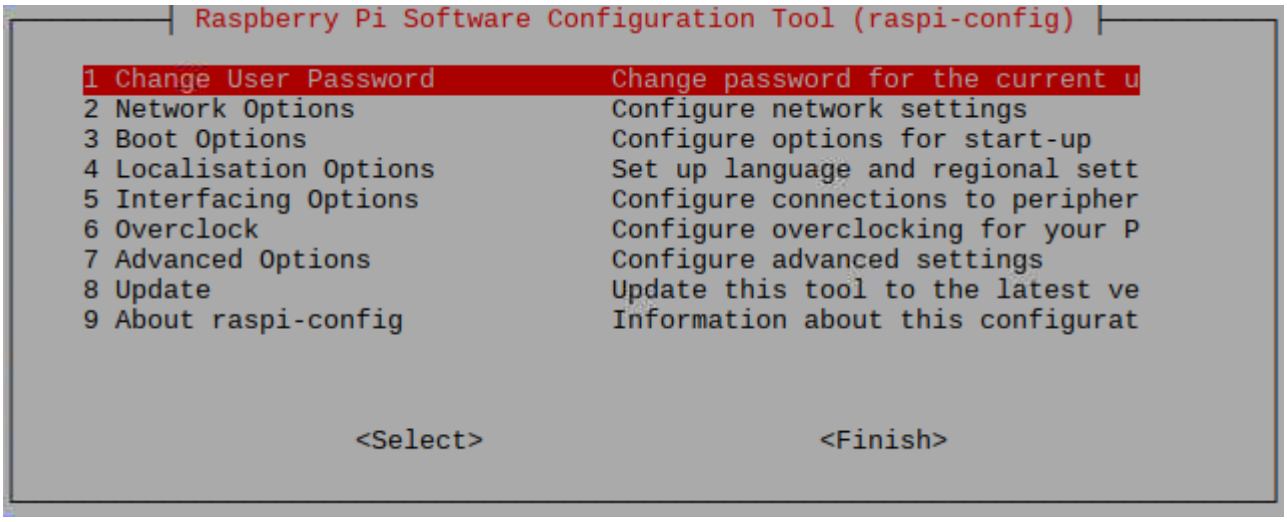
Enable I2C

The I2C interface in Raspberry Pi is disabled by default. You will need to open it manually and enable the I2C interface as follows:

Type command in the Terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose “5 Interfacing Options” then “P5 I2C” then “Yes” and then “Finish” in this order and restart your RPi. The I2C module will then be started.

Type a command to check whether the I2C module is started:

```
lsmod | grep i2c
```

If the I2C module has been started, the following content will be shown. “bcm2708" refers to the CPU model. Different models of Raspberry Pi display different contents depending on the CPU installed:

Install I2C-Tools

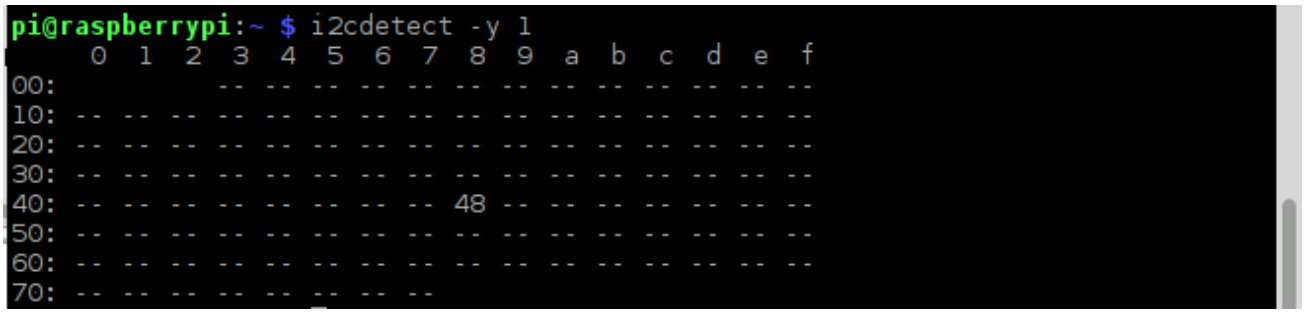
Next, type the command to install I2C-Tools. It is available with the Raspberry Pi OS by default.

```
sudo apt-get install i2c-tools
```

I2C device address detection:

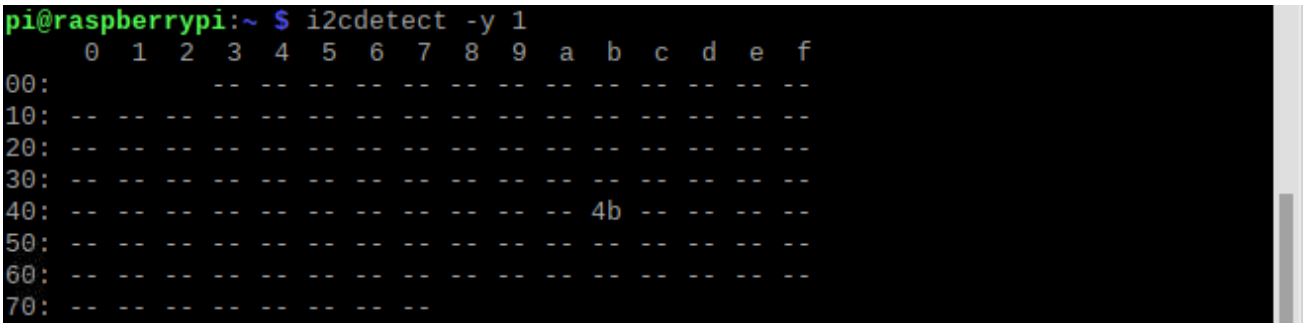
```
i2cdetect -y 1
```

When you are using the PCF8591 Module, the result should look like this:



Here, 48 (HEX) is the I2C address of ADC Module (PCF8591).

When you are using ADS, the result should look like this:



Here, 4b (HEX) is the I2C address of ADC Module (ADS7830).

Install Smbus Module

```
sudo apt-get install python-smbus
sudo apt-get install python3-smbus
```

Code

Python Code 7.1.1 ADC

For Python code, ADCDevice requires a custom module which needs to be installed.

1. Use cd command to enter folder of ADCDevice.

```
cd ~/Freenove_Kit/Libs/Python-Libs/
```

2. Unzip the file.

```
tar -zxvf ADCDevice-1.0.3.tar.gz
```

3. Open the unzipped folder.

```
cd ADCDevice-1.0.3
```

4. Install library for python3 and python2.

```
sudo python3 setup.py install
sudo python2 setup.py install
```

A successful installation, without error prompts, is shown below:

```
Installed /usr/local/lib/python3.7/dist-packages/ADCDevice-1.0.2-py3.7.egg
Processing dependencies for ADCDevice==1.0.2
Finished processing dependencies for ADCDevice==1.0.2
```

Execute the following command. Observe the project result and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 07.1.1_ADC directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/07.1.1_ADC
```

2. Use the Python command to execute the Python code "ADC.py".

```
python ADC.py
```

After the program is executed, adjusting the potentiometer will produce a readout display of the potentiometer voltage values in the Terminal and the converted digital content.

```
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 169, Voltage : 2.19
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
```

The following is the code:

```
1  import time
2  from ADCDevice import *
3
4  adc = ADCDevice() # Define an ADCDevice class object
5
6  def setup():
7      global adc
8      if(adc.detectI2C(0x48)): # Detect the pcf8591.
9          adc = PCF8591()
10     elif(adc.detectI2C(0x4b)): # Detect the ads7830
11         adc = ADS7830()
12     else:
13         print("No correct I2C address found, \n"
14         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
15         "Program Exit. \n");
```

```
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(0)    # read the ADC value of channel 0
        voltage = value / 255.0 * 3.3  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)

def destroy():
    adc.close()

if __name__ == '__main__':   # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt: # Press ctrl-c to end the program.
        destroy()
```

In this code, a custom Python module "ADCDevice" is used. It contains the method of utilizing the ADC Module in this project, through which the ADC Module can easily and quickly be used. In the code, you need to first create an ADCDevice object adc.

```
adc = ADCDevice() # Define an ADCDevice class object
```

Then in setup(), use detecticIC(addr), the member function of ADCDevice, to detect the I2C module in the circuit. Different modules have different I2C addresses. Therefore, according to the address, we can determine which ADC Module is in the circuit. When the correct module is detected, a device specific class object is created and assigned to adc. The default address of PCF8591 is 0x48, and that of ADS7830 is 0x4b.

```
def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)
```

When you have a class object of a specific device, you can get the ADC value of the specified channel by calling the member function of this class, analogRead(chn). In loop(), get the ADC value of potentiometer.

```
        value = adc.analogRead(0)    # read the ADC value of channel 0
```

Then according to the formula, the voltage value is calculated and displayed on the terminal monitor.

```
        voltage = value / 255.0 * 3.3  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)
```

Reference

About smbus Module:

smbus Module

The System Management Bus Module defines an object type that allows SMBus transactions on hosts running the Linux kernel. The host kernel must support I2C, I2C device interface support, and a bus adapter driver. All of these can be either built-in to the kernel, or loaded from modules.

In Python, you can use help(smbus) to view the relevant functions and their descriptions.

bus=smbus.SMBus(1): Create an SMBus class object.

bus.read_byte_data(address,cmd+chn): Read a byte of data from an address and return it.

bus.write_byte_data(address,cmd,value): Write a byte of data to an address.

class ADCDevice(object)

This is a base class.

```
int detectI2C(int addr);
```

This is a member function, which is used to detect whether the device with the given I2C address exists. If it exists, it returns true. Otherwise, it returns false.

```
class PCF8591(ADCDevice)
class ADS7830(ADCDevice)
```

These two classes are derived from the ADCDevice and the main function is analogRead(chn).

```
int analogRead(int chn);
```

This returns the value read on the supplied analog input pin.

Parameter chn: For PCF8591, the range of chn is 0, 1, 2, 3. For ADS7830, the range is 0, 1, 2, 3, 4, 5, 6, 7.

You can find the source file of this library in the folder below:

```
~/Freenove_Kit/Libs/Python-Libs/ADCDevice-1.0.3/src/ADCDevice/ADCdevice.py
```

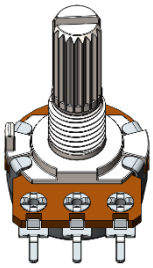
Chapter 8 Potentiometer & LED

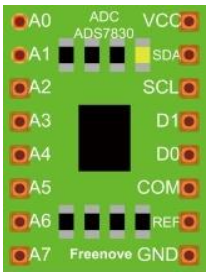
Earlier we learned how to use ADC and PWM. In this chapter, we learn to control the brightness of an LED by using a potentiometer.

Project 8.1 Soft Light

In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle ratio of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

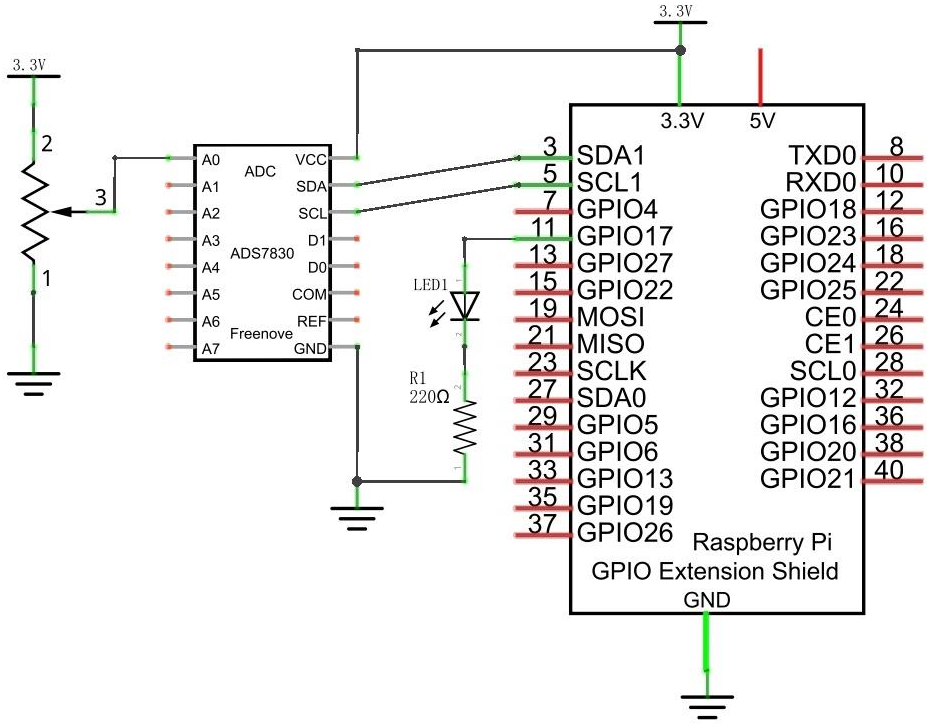
Component List

<table>
<tr><td colspan="2">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wire M/M x17
 </td></tr>
<tr><td>Rotary Potentiometer x1
  </td><td>ADC Module x1 （Only one）

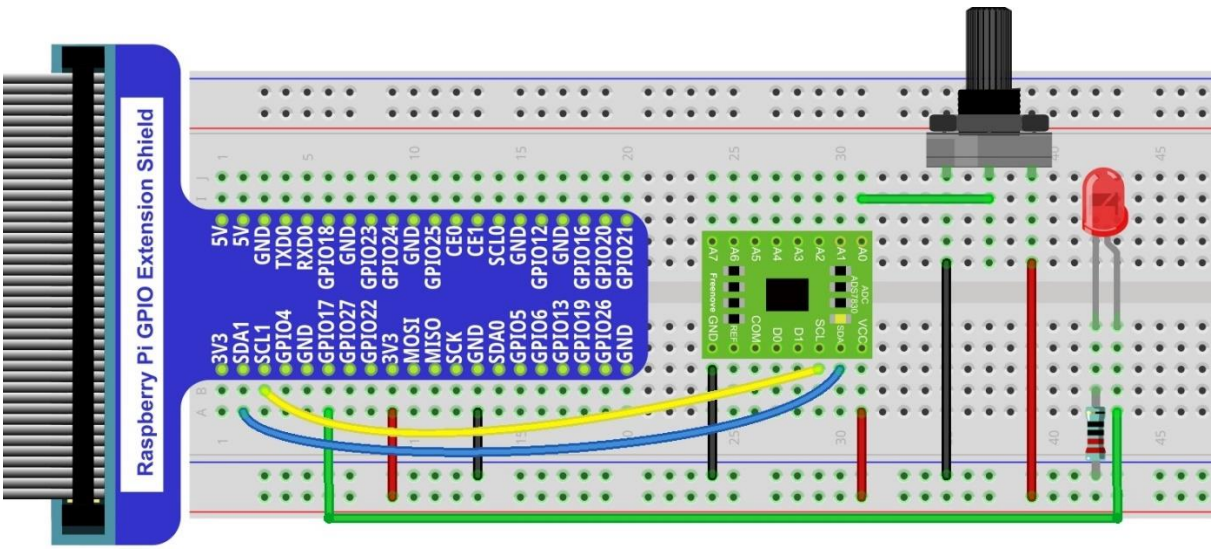
or

</td><td>10kΩ x2
 </td><td>220Ω x1
 </td><td>LED x1
 </td></tr>
</table>

Circuit with ADS7830

Schematic diagram



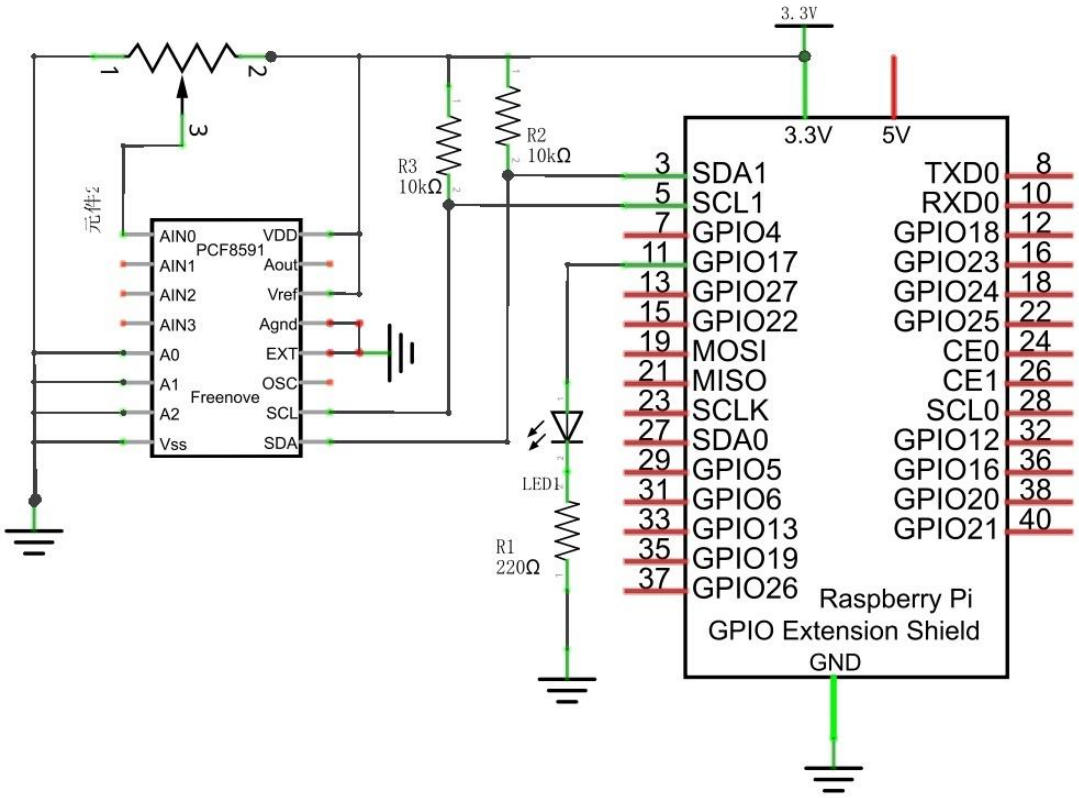
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



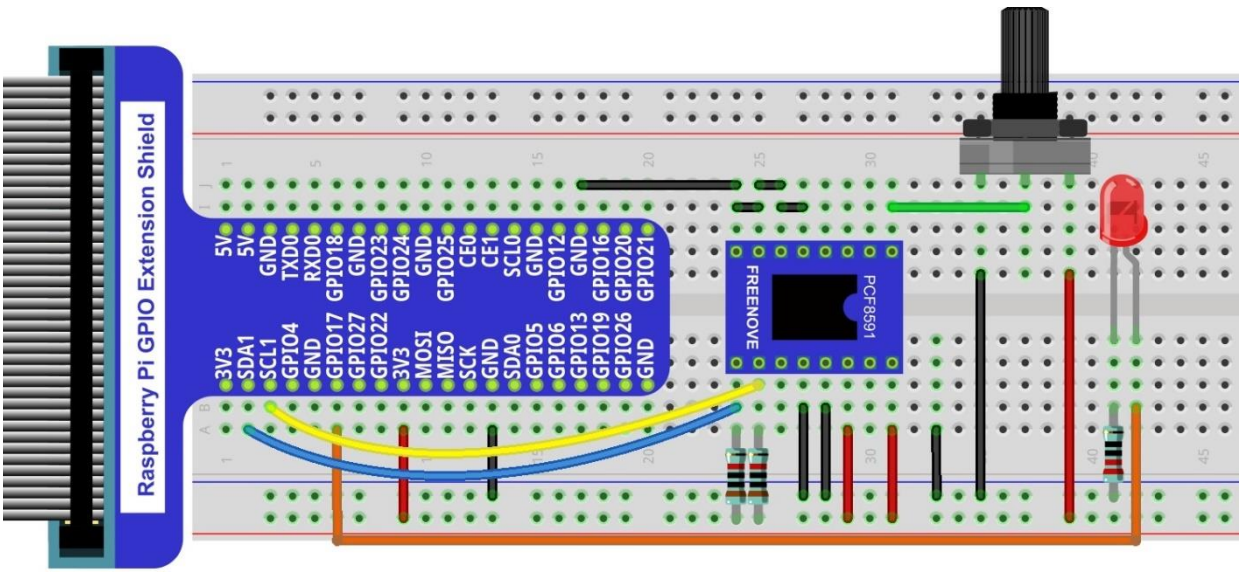
Video: https://youtu.be/YMEfe9IWU6I

Circuit with PCF8591

Schematic diagram



Hardware connection



Code

Python Code 8.1.1 Softlight

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 08.1.1_Softlight directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/08.1.1_Softlight
```

2. Use the python command to execute the Python code "Softlight.py".

```
python Softlight.py
```

After the program is executed, adjusting the potentiometer will display the voltage values of the potentiometer in the Terminal window and the converted digital quantity. As a consequence, the brightness of LED will be changed.

The following is the code:

```
1  from gpiozero import PWMLED
2  import time
3  from ADCDevice import *
4
5  led = PWMLED(17,frequency=1000)       # define LED pin according to BCM Numbering
6  adc = ADCDevice() # Define an ADCDevice class object
7
8  def setup():
9      global adc
10     if(adc.detectI2C(0x48)): # Detect the pcf8591.
11         adc = PCF8591()
12     elif(adc.detectI2C(0x4b)): # Detect the ads7830
13         adc = ADS7830()
14     else:
15         print("No correct I2C address found, \n"
16         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
17         "Program Exit. \n");
18         exit(-1)
19
20 def loop():
21     while True:
22         value = adc.analogRead(0)       # read the ADC value of channel 0
23         led.value = value / 255.0       # Mapping to PWM duty cycle
24         voltage = value / 255.0 * 3.3   # calculate the voltage value
25         print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
26         time.sleep(0.03)
```

```
27
28  def destroy():
29      led.close()
30      adc.close()
31
32  if __name__ == '__main__':   # Program entrance
33      print ('Program is starting ... ')
34      try:
35          setup()
36          loop()
37      except KeyboardInterrupt: # Press ctrl-c to end the program.
38          destroy()
39          print("Ending program")
```

In the code, read ADC value of potentiometers and map it to the duty cycle of the PWM to control LED brightness.

```
            value = adc.analogRead(0)       # read the ADC value of channel 0
            led.value = value / 255.0       # Mapping to PWM duty cycle
```

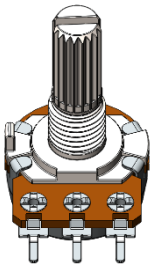
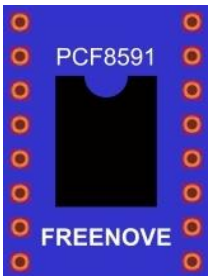
Chapter 9 Potentiometer & RGBLED

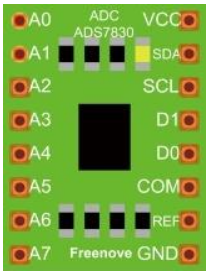
In this chapter, we will use 3 potentiometers to control the brightness of 3 LEDs of RGBLED to create multiple colors.

Project 9.1 Colorful Light

In this project, 3 potentiometers are used to control the RGB LED and in principle it is the same as with the Soft Light. project. Namely, read the voltage value of the potentiometer and then convert it to PWM used to control LED brightness. Difference is that the previous soft light project needed only one LED while this one required (1) RGB LEDs.

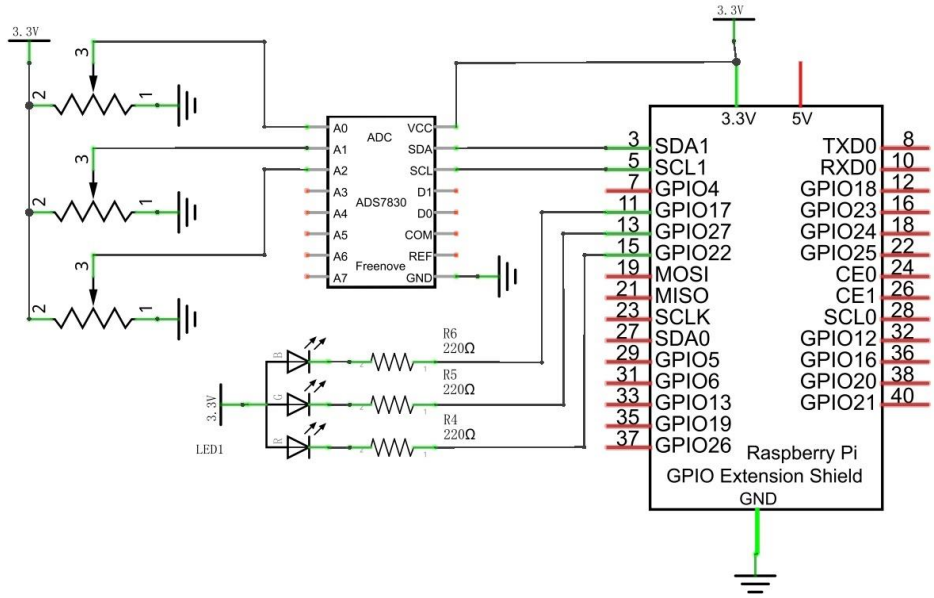
Component List

<table>
<tr><td colspan="2">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wires M/M x17
 </td></tr>
<tr><td>Rotary potentiometer x3
  </td><td colspan="2">ADC module x1
 

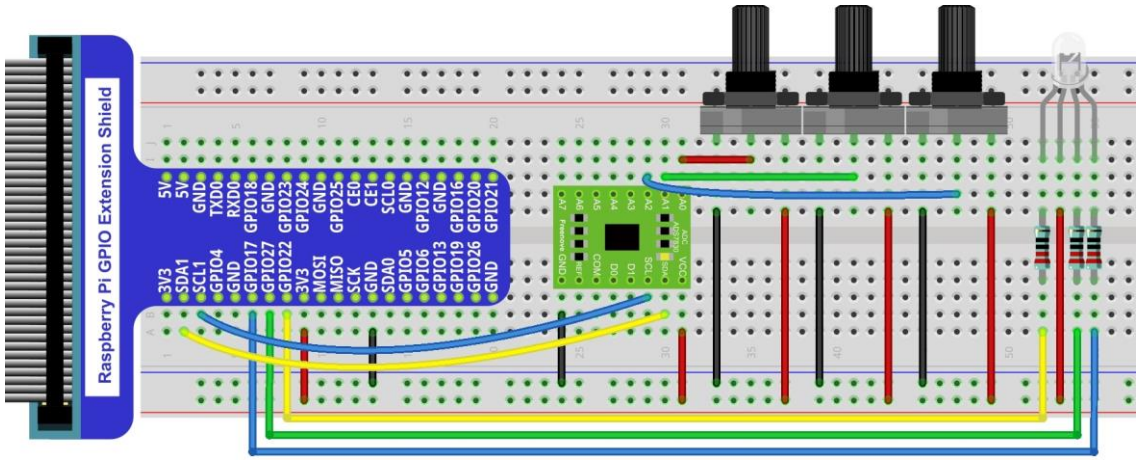
or

</td><td>10kΩ x2
 </td><td>220Ω x3
 </td><td>RGB LEDx1
 </td></tr>
</table>

Circuit with ADS7830

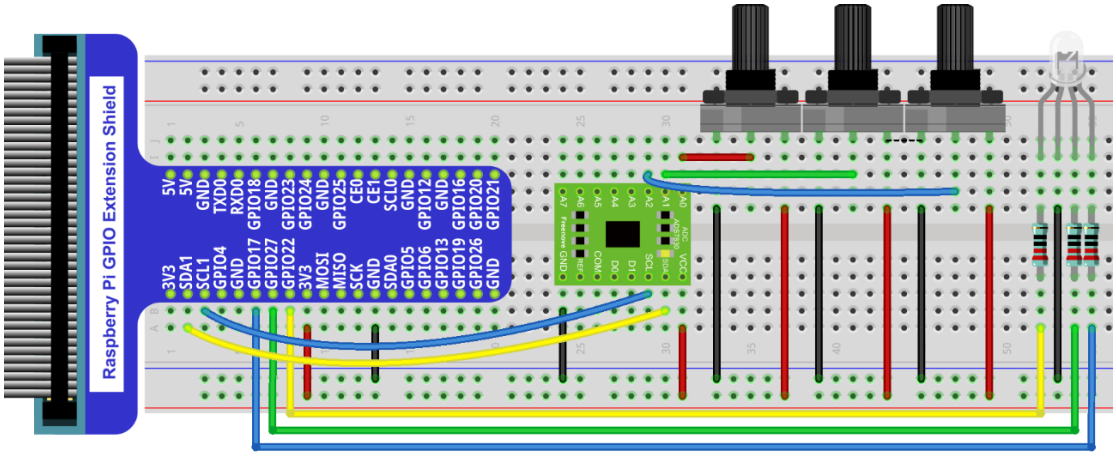
Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

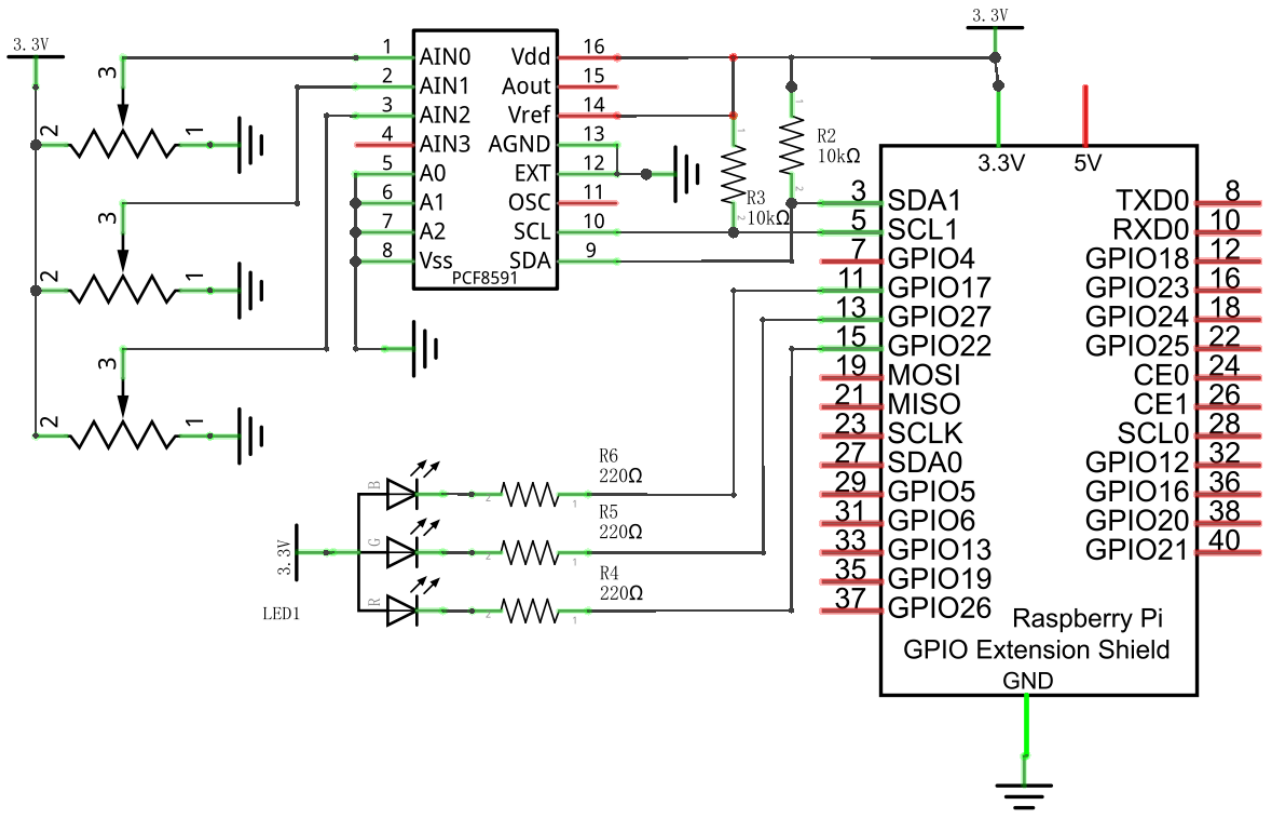


If circuit above doesn't work, please try following wiring.

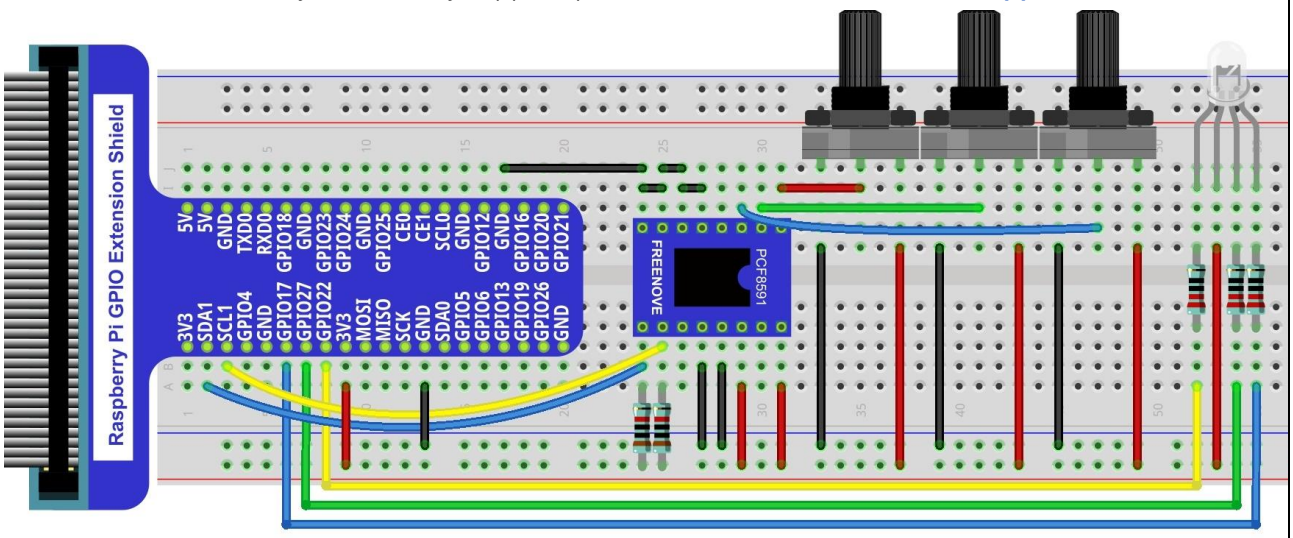


Circuit with PCF8591

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://www.youtube.com/watch?v=CmLLNsBMN2U

Code

Python Code 9.1.1 ColorfulSoftlight

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 09.1.1_ColorfulSoftlight directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/09.1.1_ColorfulSoftlight
```

2. Use python command to execute python code "ColorfulSoftlight.py".

```
python ColorfulSoftlight.py
```

After the program is executed, rotate one of the potentiometers, then the color of RGB LED will change. The Terminal window will display the ADC value of each potentiometer.
The following is the program code:

```
1  from gpiozero import RGBLED
2  import time
3  from ADCDevice import *
4
5  led = RGBLED(red=22, green=27, blue=17, active_high=False) # define the pins for
   R:GPIO22,G:GPIO27,B:GPIO17
6  #led = RGBLED(red="J8:15", green="J8:13", blue="J8:11") # according to BOARD Numbering define
   the pins for R:11,G:12,B:13
7  adc = ADCDevice() # Define an ADCDevice class object
8
9  def setup():
10     global adc
11     if(adc.detectI2C(0x48)): # Detect the pcf8591.
12         adc = PCF8591()
13     elif(adc.detectI2C(0x4b)): # Detect the ads7830
14         adc = ADS7830()
15     else:
16         print("No correct I2C address found, \n"
17         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
18         "Program Exit. \n");
19         exit(-1)
20
21 def loop():
22     while True:
23         value_Red = adc.analogRead(0)        # read ADC value of 3 potentiometers
24         value_Green = adc.analogRead(1)
25         value_Blue = adc.analogRead(2)
26
```

```
27          led.red   =value_Red/255  # map the read value of potentiometers into PWM value and
    output it
28          led.green =value_Green/255
29          led.blue  =value_Blue/255
30          # print read ADC value
31          print ('ADC Value
    value_Red: %d ,\tvlue_Green: %d ,\tvalue_Blue: %d'%(value_Red,value_Green,value_Blue))
32          time.sleep(0.01)
33
34  def destroy():
35      adc.close()
36      led.close()
37
38  if __name__ == '__main__': # Program entrance
39      print ('Program is starting ... ')
40      setup()
41      try:
42          loop()
43      except KeyboardInterrupt: # Press ctrl-c to end the program.
44          destroy()
45          print("Ending program")
```

In the code you can read the ADC values of the 3 potentiometers and map it into a PWM duty cycle to control the 3 LED elements to vary the color of their respective RGB LED.

Chapter 10 Photoresistor & LED

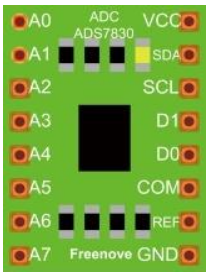
In this chapter, we will learn how to use a photoresistor to make an automatic dimming nightlight.

Project 10.1 NightLamp

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function. When the ambient light is less (darker environment), the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

Component List

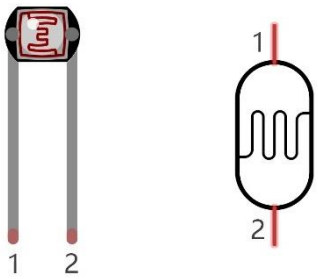
<table>
<tr><td colspan="3">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wires M/M x15
 </td></tr>
<tr><td>Photoresistor x1
 </td><td colspan="2">ADC module x1

or

</td><td>10kΩ x3
 </td><td>220Ω x1
 </td><td>LED x1
 </td></tr>
</table>

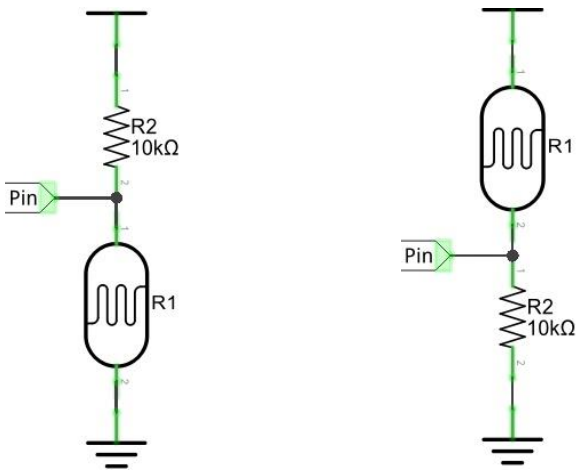
Component knowledge

Photoresistor

A Photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. A Photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a Photoresistor to detect light intensity. The Photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a Photoresistor's resistance value:

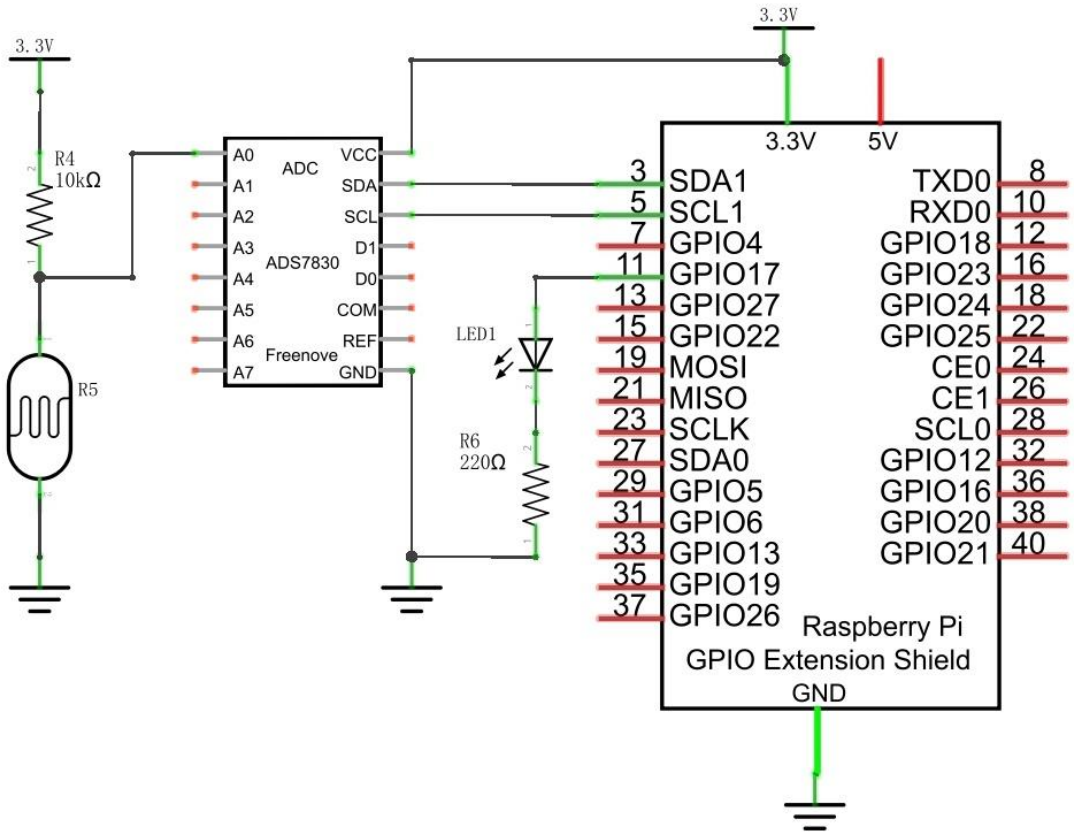


In the above circuit, when a Photoresistor's resistance vale changes due to a change in light intensity, the voltage between the Photoresistor and Resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

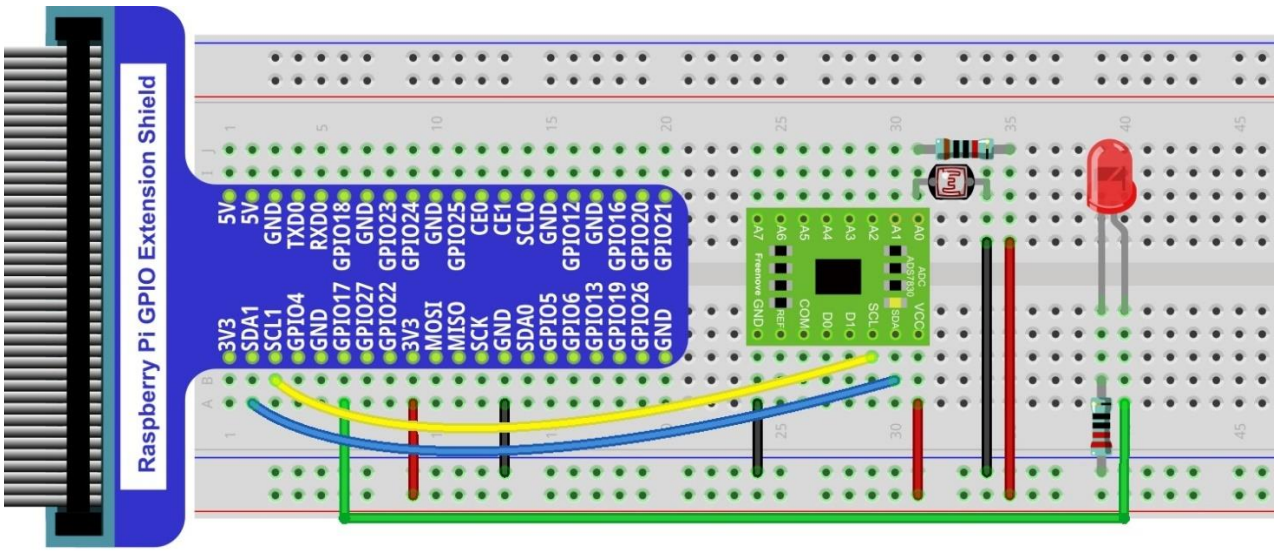
Circuit with ADS7830

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

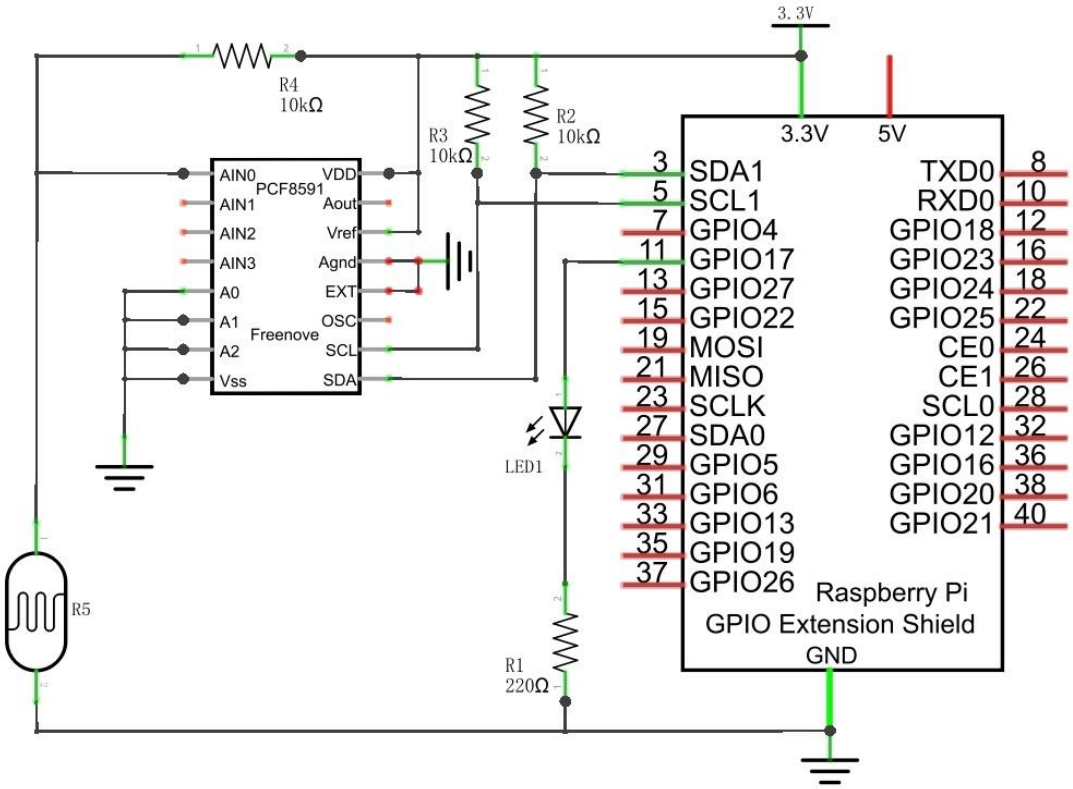


Video: https://youtu.be/r6p3zhXsyko

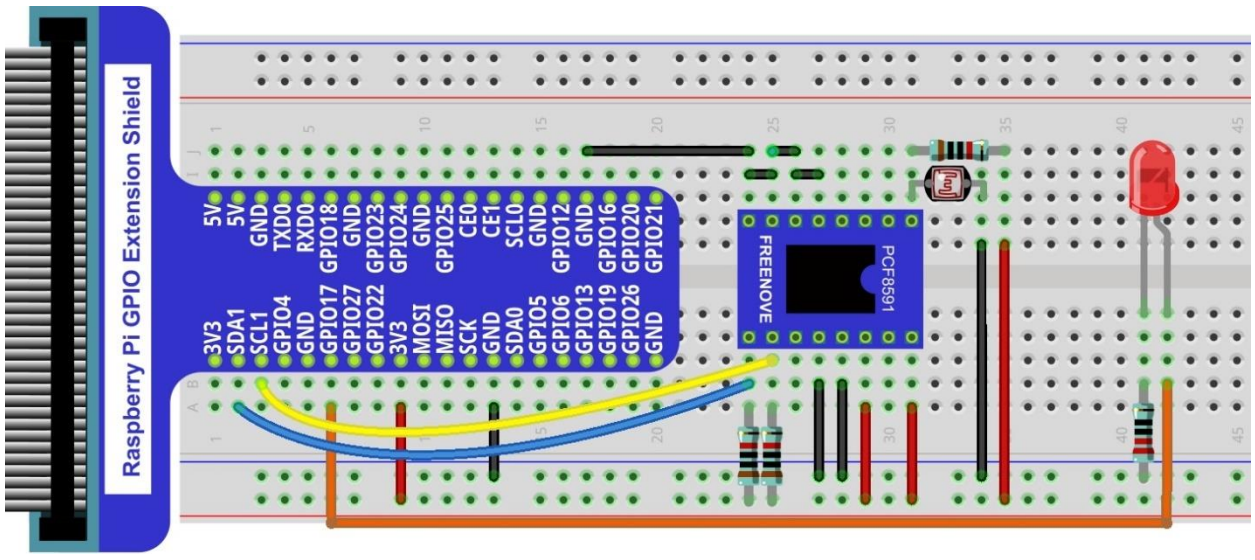
Circuit with PCF8591

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection



Code

The code used in this project is identical with what was used in the last chapter.

Python Code 10.1.1 Nightlamp

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 10.1_Nightlamp directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/10.1.1_Nightlamp
```

2. Use the python command to execute the Python code "Nightlamp.py".

```
python Nightlamp.py
```

After the program is executed, if you cover the Photoresistor or increase the light shining on it, the brightness of the LED changes accordingly. As in previous projects the Terminal window will display the current input voltage value of ADC module A0 pin and the converted digital quantity.

The following is the program code:

```
from gpiozero import PWMLED
import time
from ADCDevice import *

ledPin = 17 # define ledPin
led = PWMLED(ledPin)
adc = ADCDevice() # Define an ADCDevice class object

def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(0)     # read the ADC value of channel 0
        led.value = value / 255.0     # Mapping to PWM duty cycle
        voltage = value / 255.0 * 3.3
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.01)
```

```
28
29  def destroy():
30      led.close()
31      adc.close()
32
33  if __name__ == '__main__':   # Program entrance
34      print ('Program is starting ... ')
35      setup()
36      try:
37          loop()
38      except KeyboardInterrupt:  # Press ctrl-c to end the program.
39          destroy()
40          print("Ending program")
```

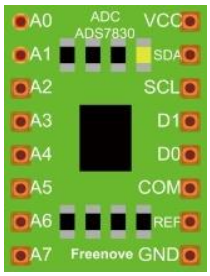
Chapter 11 Thermistor

In this chapter, we will learn about Thermistors which are another kind of Resistor.

Project 11.1 Thermometer

A Thermistoris a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

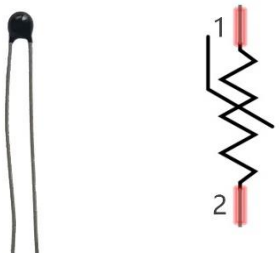
Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Jumper Wire M/M x14	
Thermistor x1	ADC module x1 or 	Resistor 10kΩ x3

Component knowledge

Thermistor

Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

Rt=R*EXP [B*(1/T2-1/T1)]

Where:

Rt is the thermistor resistance under T2 temperature;

R is in the nominal resistance of thermistor under T1 temperature;

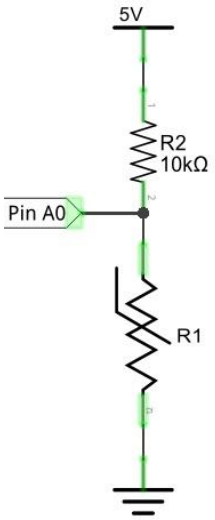
EXP[n] is nth power of e;

B is for thermal index;

T1, T2 is Kelvin temperature (absolute temperature). Kelvin temperature=273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: B=3950, R=10k, T1=25.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

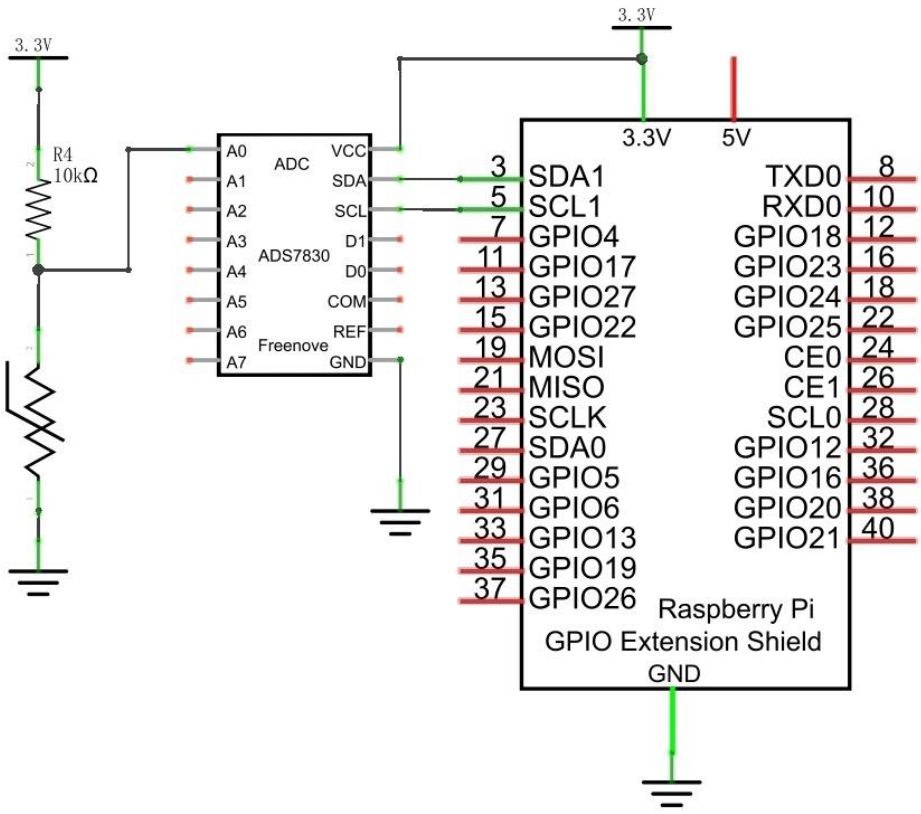
Therefore, the temperature formula can be derived as:

T2 = 1/(1/T1 + ln(Rt/R)/B)

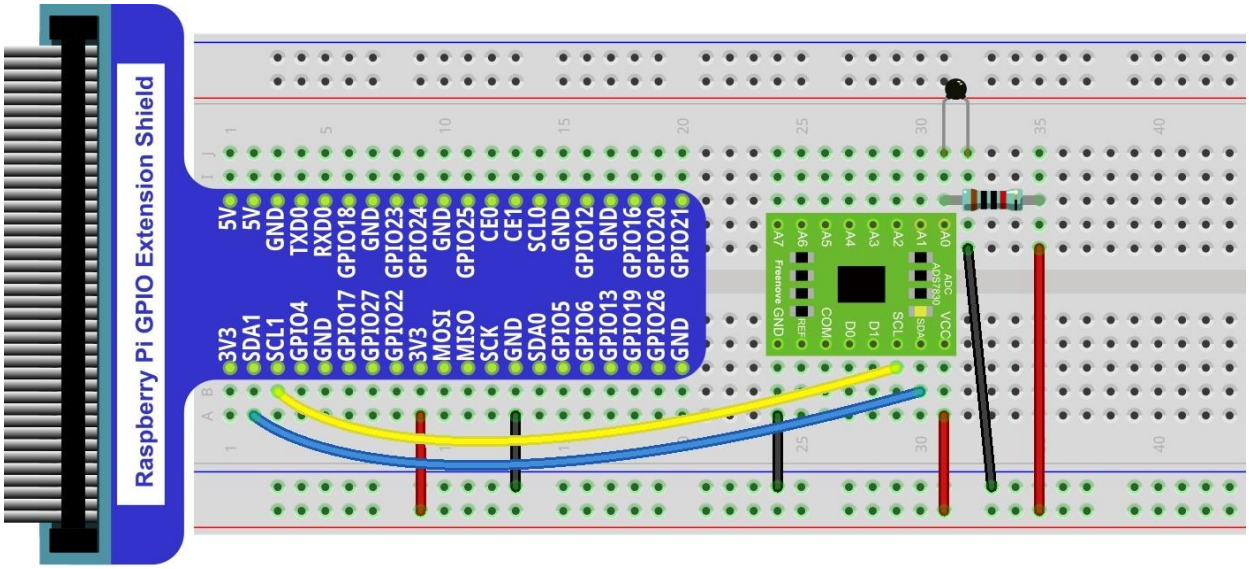
Circuit with ADS7830

The circuit of this project is similar to the one in last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



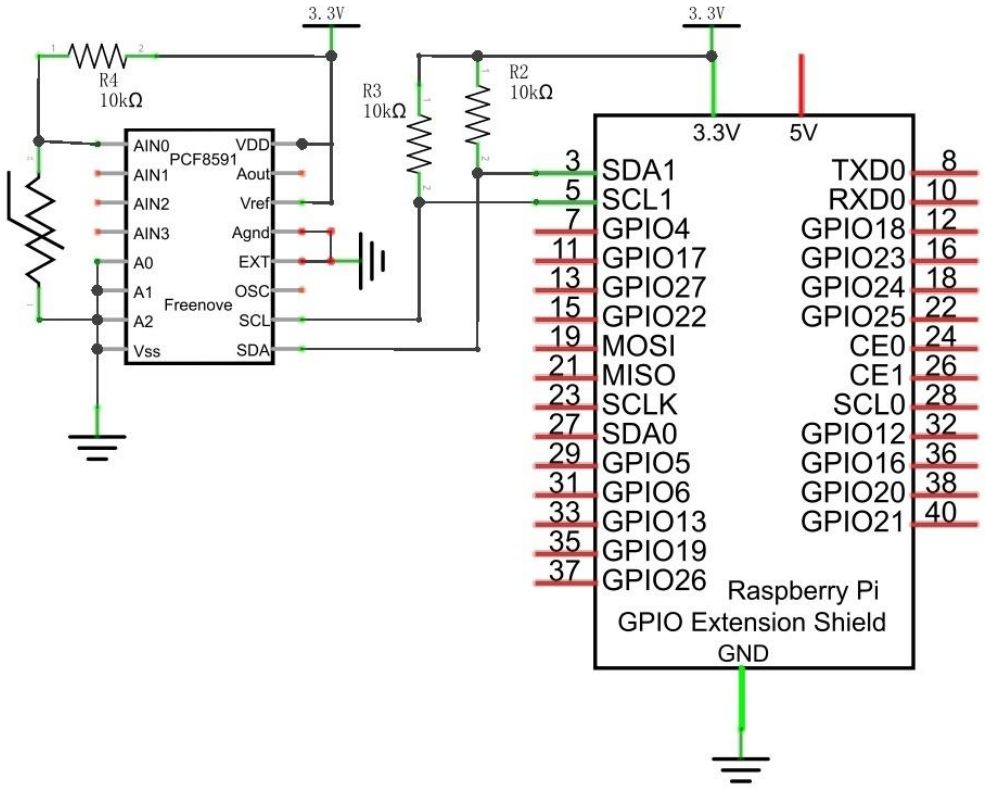
Thermistor has longer pins than the one shown in circuit.

Video: https://youtu.be/spOaIxanNMc

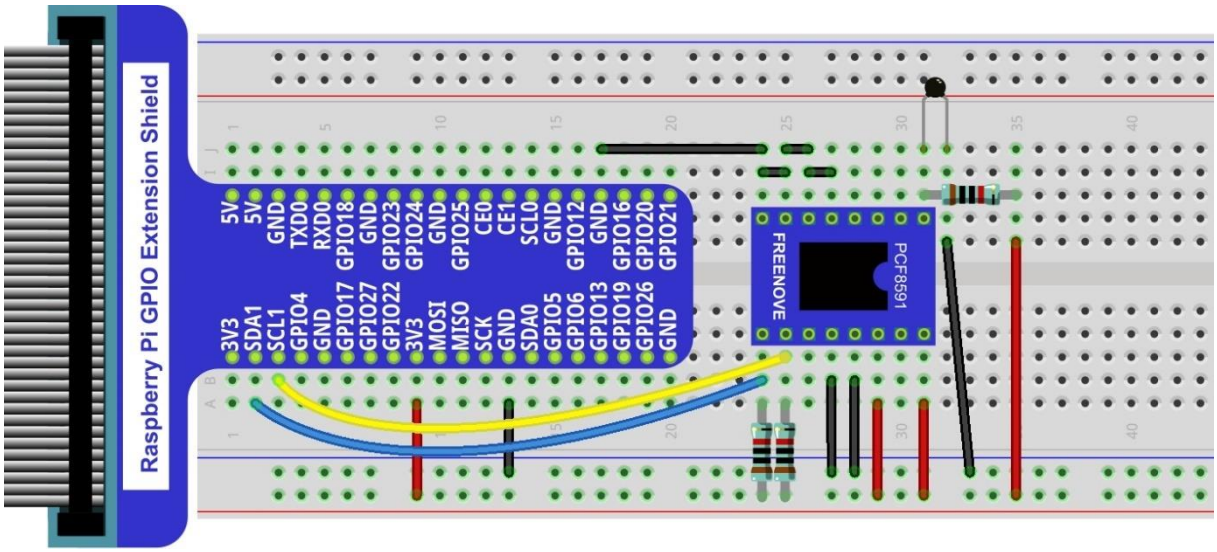
Circuit with PCF8591

The circuit of this project is similar to the one in the last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Thermistor has longer pins than the one shown in circuit.

Code

In this project code, the ADC value still needs to be read, but the difference here is that a specific formula is used to calculate the temperature value.

Python Code 11.1.1 Thermometer

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 11.1.1_Thermometer directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/11.1.1_Thermometer
```

2. Use python command to execute Python code "Thermometer.py".

```
python Thermometer.py
```

After the program is executed, the Terminal window will display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases.

```
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
```

The following is the code:

```
1  import time
2  import math
3  from ADCDevice import *
4
5  adc = ADCDevice() # Define an ADCDevice class object
6
7  def setup():
8      global adc
9      if(adc.detectI2C(0x48)): # Detect the pcf8591.
10         adc = PCF8591()
11     elif(adc.detectI2C(0x4b)): # Detect the ads7830
12         adc = ADS7830()
13     else:
```

```
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(0)         # read ADC value A0 pin
        voltage = value / 255.0 * 3.3         # calculate voltage
        Rt = 10 * voltage / (3.3 - voltage)     # calculate resistance value of thermistor
        tempK = 1/(1/(273.15 + 25) + math.log(Rt/10)/3950.0) # calculate temperature (Kelvin)
        tempC = tempK -273.15         # calculate temperature (Celsius)
        print ('ADC Value : %d, Voltage : %.2f, Temperature : %.2f'%(value,voltage,tempC))
        time.sleep(0.01)

def destroy():
    adc.close()

if __name__ == '__main__':  # Program entrance
    print ('Program is starting ... ')
    setup()
    try:
        loop()
    except KeyboardInterrupt: # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

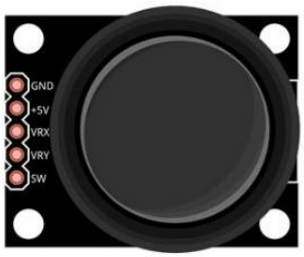
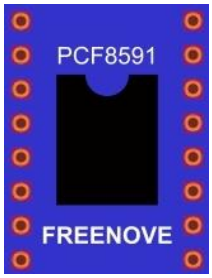
Chapter 12 Joystick

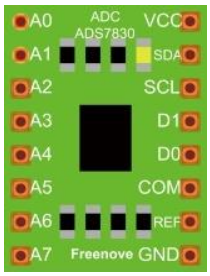
In an earlier chapter, we learned how to use Rotary Potentiometer. We will now learn about joysticks, which are electronic modules that work on the same principle as the Rotary Potentiometer.

Project 12.1 Joystick

In this project, we will read the output data of a joystick and display it to the Terminal screen.

Component List

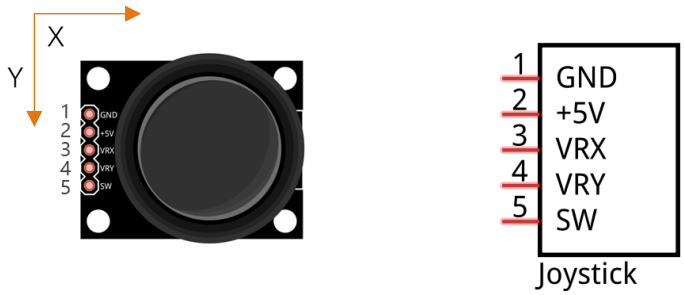
<table>
<tr><td>Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="2">Jumper x18
 </td></tr>
<tr><td>Joystick x1
 
</td><td>ADC module x1
 

or

</td><td>Resistor
10kΩ x3
 </td></tr>
</table>

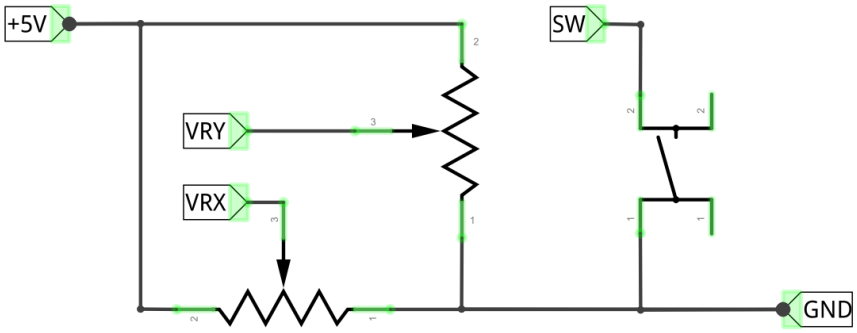
Component knowledge

Joystick

A Joystick is a kind of input sensor used with your fingers. You should be familiar with this concept already as they are widely used in gamepads and remote controls. It can receive input on two axes (Y and or X) at the same time (usually used to control direction on a two dimensional plane). And it also has a third direction capability by **pressing down (Z axis/direction)**.



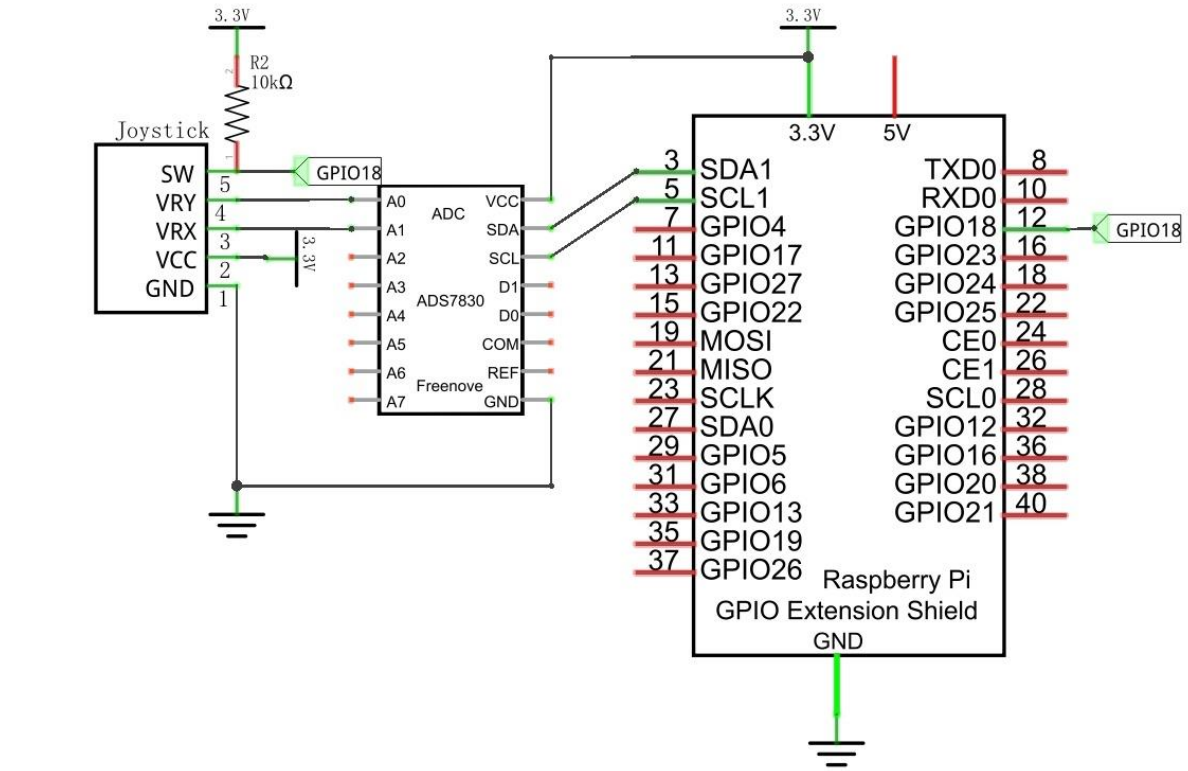
This is accomplished by incorporating two rotary potentiometers inside the Joystick Module at 90 degrees of each other, placed in such a manner as to detect shifts in direction in two directions simultaneously and with a Push Button Switch in the “vertical” axis, which can detect when a User presses on the Joystick.



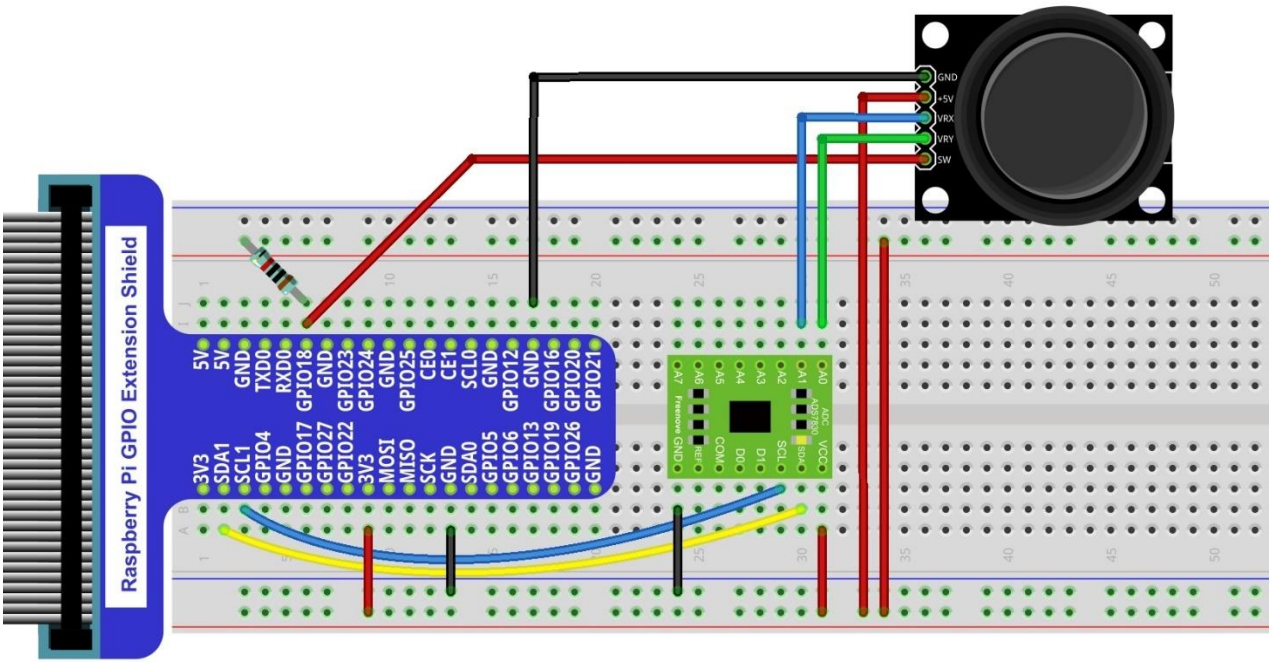
When the Joystick data is read, there are some differences between the axes: data of X and Y axes is analog, which needs to use the ADC. The data of the Z axis is digital, so you can directly use the GPIO to read this data or you have the option to use the ADC to read this.

Circuit with ADS7830

Schematic diagram



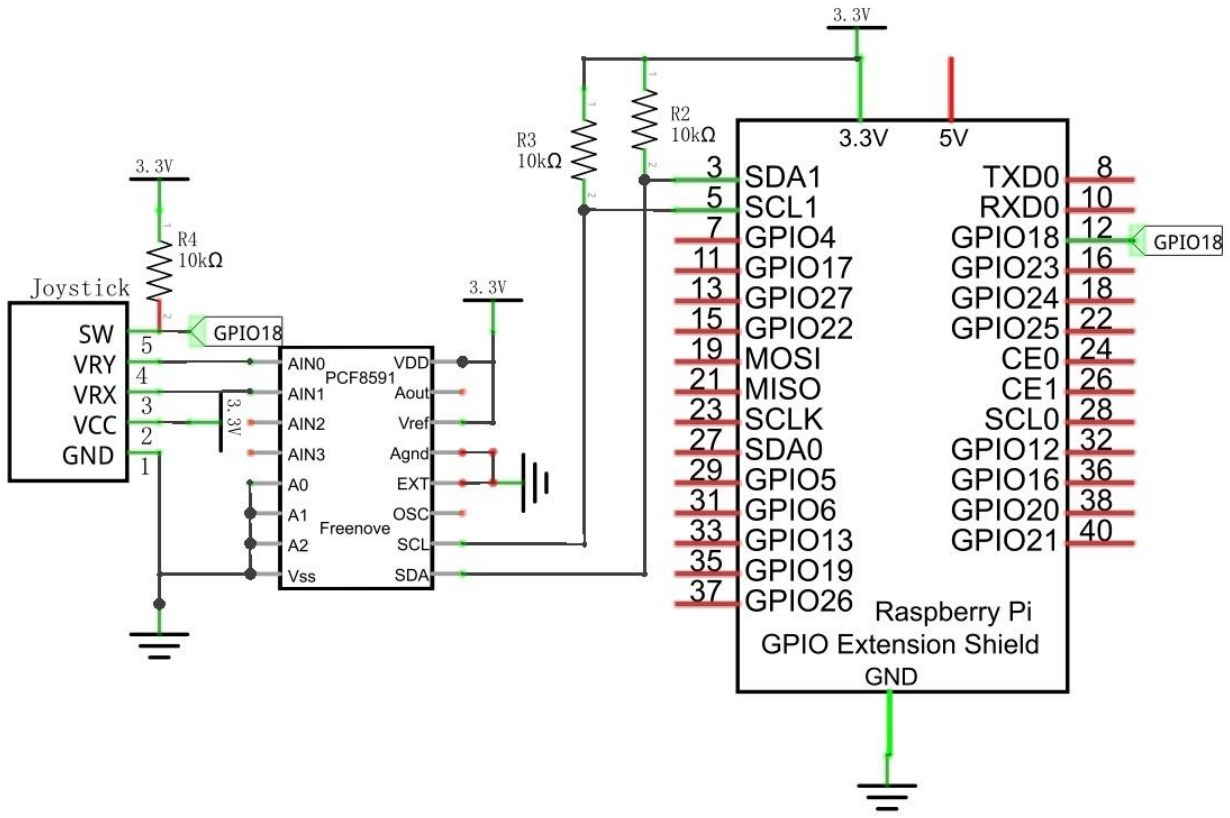
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



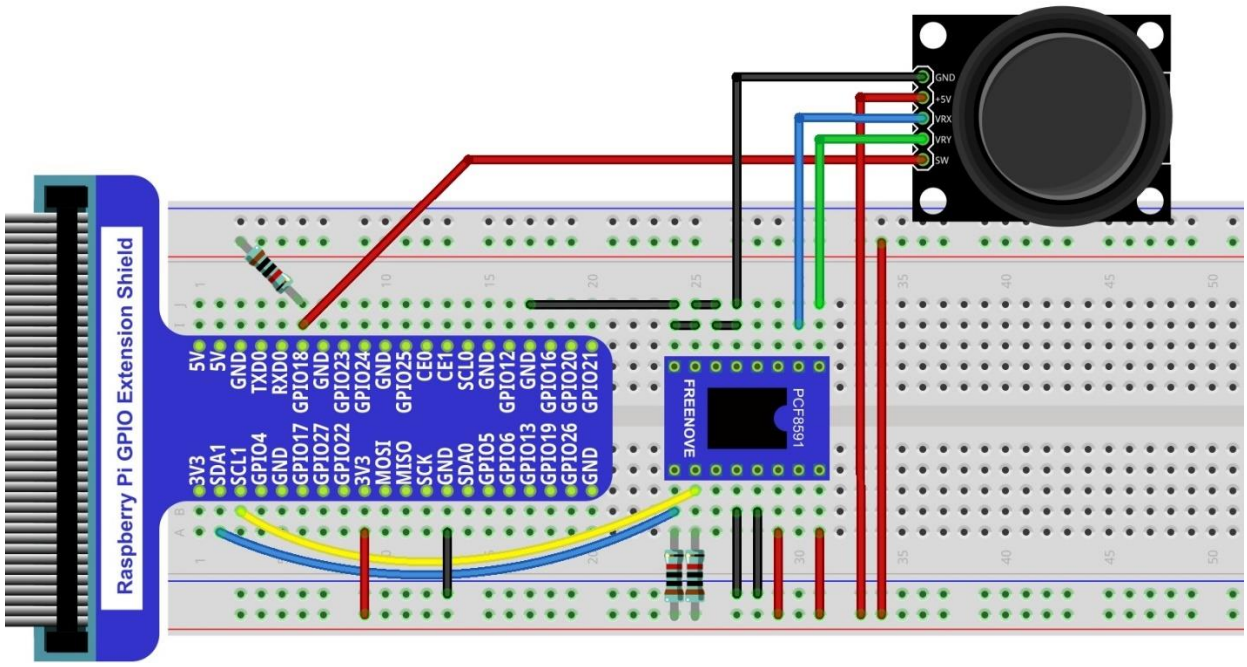
Video: https://youtu.be/qjP3HpbPJTM

Circuit with PCF8591

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

In this project's code, we will read the ADC values of X and Y axes of the Joystick, and read digital quality of the Z axis, then display these out in Terminal.

Python Code 12.1.1 Joystick

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

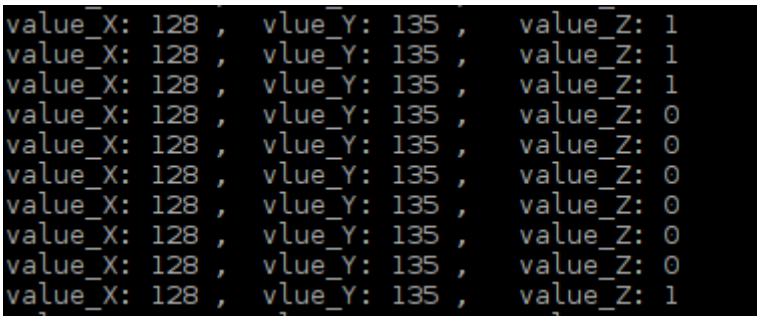
1. Use cd command to enter 12.1.1_Joystick directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/12.1.1_Joystick
```

2. Use Python command to execute Python code "Joystick.py".

```
python Joystick.py
```

After the program is executed, the Terminal window will display the data of 3 axes X, Y and Z. Shifting (moving) the joystick or pressing it down will make the data change.



The following is the program code:

```
from gpiozero import Button
import time
from ADCDevice import *

Z_Pin = 18      # define Z_Pin
button = Button(Z_Pin) # define Button pin according to BCM Numbering
adc = ADCDevice() # Define an ADCDevice class object

def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

```

```
def loop():
    while True:
        val_Z = not button.value           # read digital value of axis Z
        val_Y = adc.analogRead(0)          # read analog value of axis X and Y
        val_X = adc.analogRead(1)
        print ('value_X: %d ,\tvlue_Y: %d ,\tvalue_Z: %d'%(val_X,val_Y,val_Z))
        time.sleep(0.01)

def destroy():
    adc.close()
    button.close()

if __name__ == '__main__':
    print ('Program is starting ... ') # Program entrance
    setup()
    try:
        loop()
    except KeyboardInterrupt: # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

In the code, configure Z_Pin as pull-up input mode. In the while loop, use analogRead() to read the values of the axes X and Y and use the variable val_Z to save the value of the button.value variable for the Z axis, and then display them. When the button is pressed, the value of the variable button.value is 1, otherwise the value is 0.

```
def loop():
    while True:
        val_Z = not button.value           # read digital value of axis Z
        val_Y = adc.analogRead(0)          # read analog value of axis X and Y
        val_X = adc.analogRead(1)
        print ('value_X: %d ,\tvlue_Y: %d ,\tvalue_Z: %d'%(val_X,val_Y,val_Z))
        time.sleep(0.01)
```

For more information about the methods used by the Button class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#button

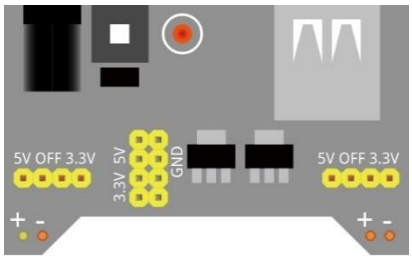
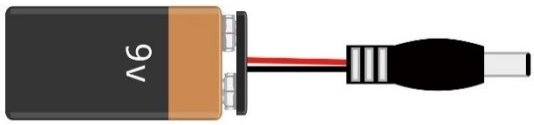
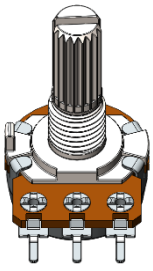
Chapter 13 Motor & Driver

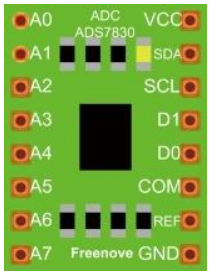
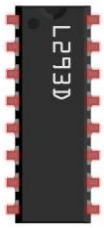
In this chapter, we will learn about DC Motors and DC Motor Drivers and how to control the speed and direction of a DC Motor.

Project 13.1 Control a DC Motor with a Potentiometer

In this project, a potentiometer will be used to control a DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reached the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction.

Component List

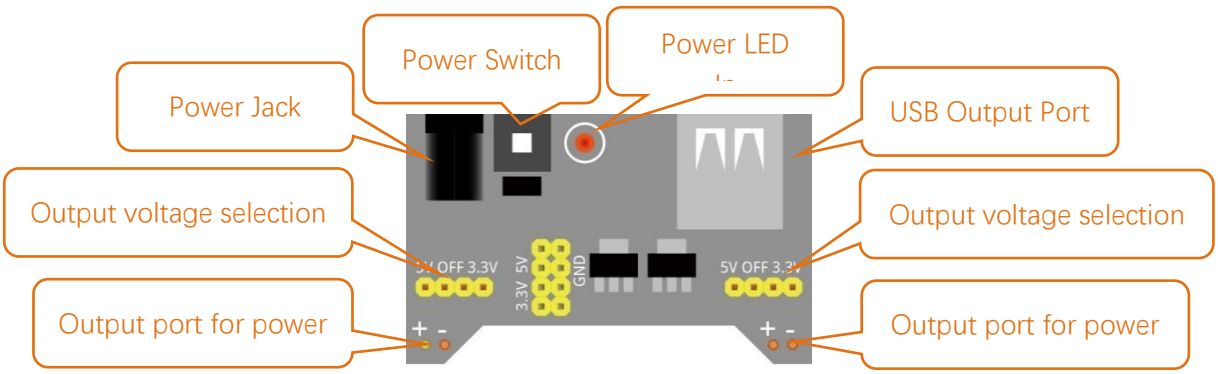
<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wires x23
 </td></tr>
<tr><td colspan="2">Breadboard Power Module x1
 
</td><td colspan="3">9V Battery (you provide) & 9V Battery Cable
 
</td></tr>
<tr><td>Rotary Potentiometer x1
  </td><td>DC Motor x1
 </td><td>10kΩ x2
 </td><td colspan="2">ADC Module x1

or

</td><td>L293D IC Chip
 
</td></tr>
</table>

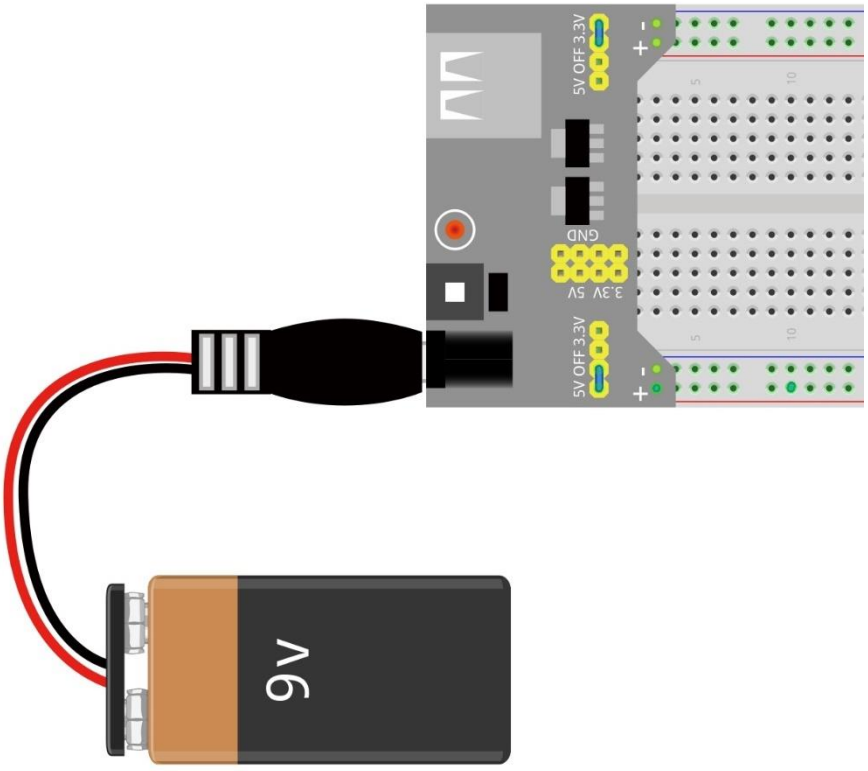
Component knowledge

Breadboard Power Module

Breadboard Power Module is an independent circuit board, which can provide independent 5V or 3.3V power to the breadboard when building circuits. It also has built-in power protection to avoid damaging your RPi module. The schematic diagram below identifies the important features of this Power Module:

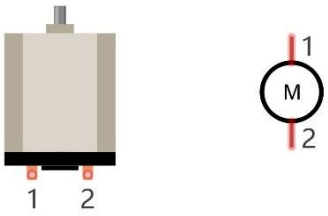


Here is an acceptable connection between Breadboard Power Module and Breadboard using a 9V battery and the provided power harness:



DC Motor

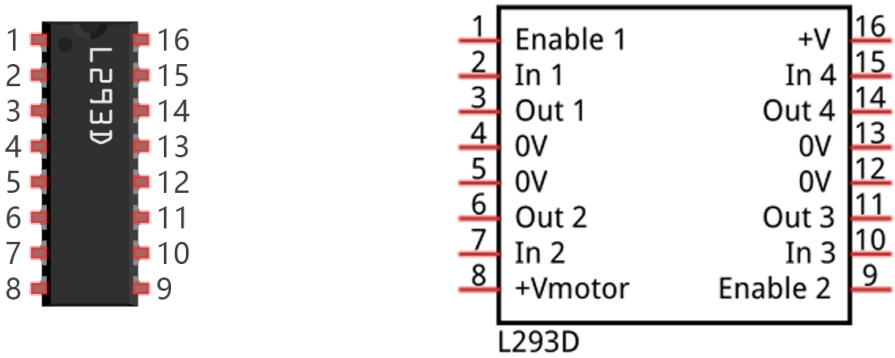
DC Motor is a device that converts electrical energy into mechanical energy. DC Motors consist of two major parts, a Stator and the Rotor. The stationary part of a DC Motor is the Stator and the part that Rotates is the Rotor. The Stator is usually part of the outer case of motor (if it is simply a pair of permanent magnets), and it has terminals to connect to the power if it is made up of electromagnet coils. Most Hobby DC Motors only use Permanent Magnets for the Stator Field. The Rotor is usually the shaft of motor with 3 or more electromagnets connected to a commutator where the brushes (via the terminals 1 & 2 below) supply electrical power, which can drive other mechanical devices. The diagram below shows a small DC Motor with two terminal pins.



When a DC Motor is connected to a power supply, it will rotate in one direction. If you reverse the polarity of the power supply, the DC Motor will rotate in opposite direction. This is important to note.

L293D

L293D is an IC Chip (Integrated Circuit Chip) with a 4-channel motor drive. You can drive a Unidirectional DC Motor with 4 ports or a Bi-Directional DC Motor with 2 ports or a Stepper Motor (Stepper Motors are covered later in this Tutorial).



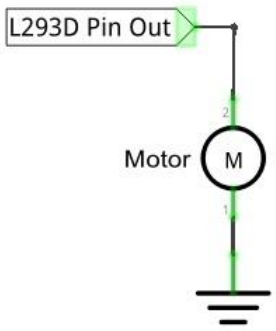
Port description of L293D module is as follows:

Pin name	Pin number	Description
In x	2, 7, 10, 15	Channel x digital signal input pin
Out x	3, 6, 11, 14	Channel x output pin, input high or low level according to In x pin, gets connected to +Vmotor or 0V
Enable1	1	Channel 1 and Channel 2 enable pin, high level enable
Enable2	9	Channel 3 and Channel 4 enable pin, high level enable
0V	4, 5, 12, 13	Power Cathode (GND)
+V	16	Positive Electrode (VCC) of power supply, supply voltage 4.5~36V
+Vmotor	8	Positive Electrode of load power supply, provide power supply for the Out pin x, the supply voltage is +V~36V

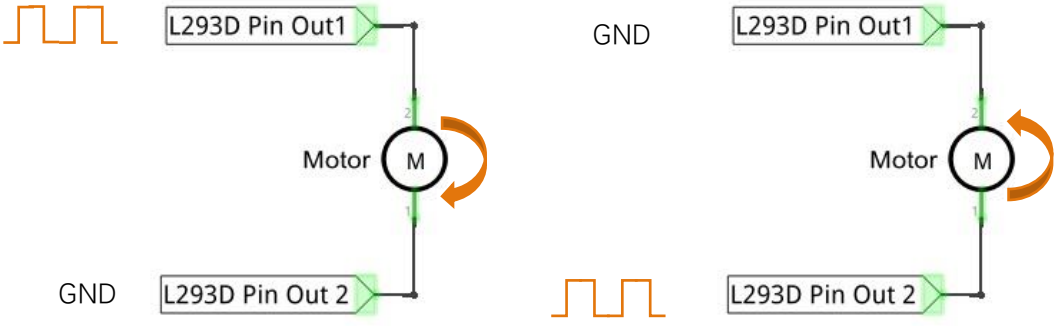
For more details, please see the datasheet for this IC Chip.

When using the L293D to drive a DC Motor, there are usually two connection options.

The following connection option uses one channel of the L239D, which can control motor speed through the PWM, However the motor then can only rotate in one direction.



The following connection uses two channels of the L239D: one channel outputs the PWM wave, and the other channel connects to GND. Therefore, you can control the speed of the motor. When these two channel signals are exchanged, not only controls the speed of motor, but also can control the direction of the motor.

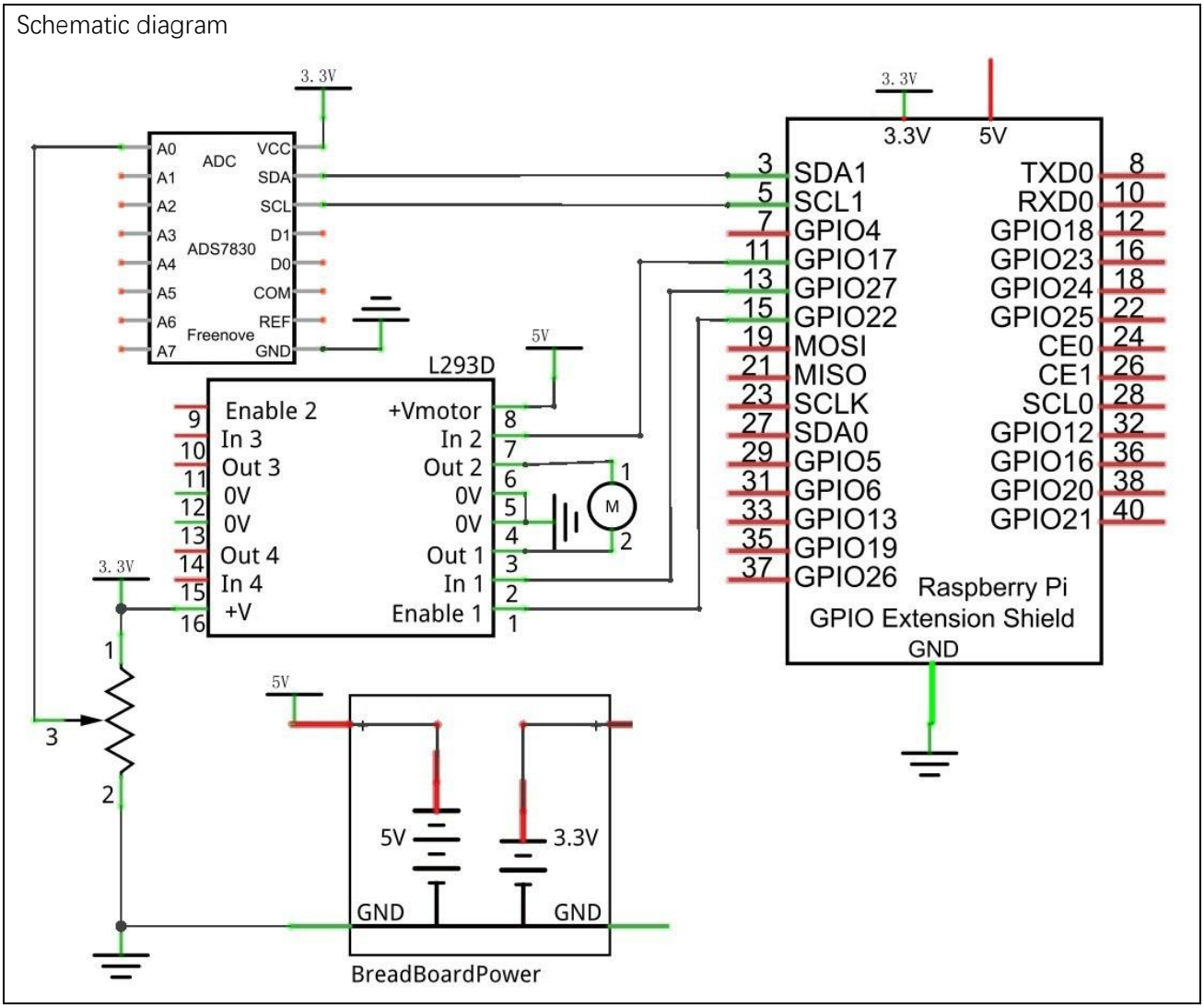


In practical use the motor is usually connected to channel 1 and by outputting different levels to in1 and in2 to control the rotational direction of the motor, and output to the PWM wave to Enable1 port to control the motor's rotational speed. If the motor is connected to channel 3 and 4 by outputting different levels to in3 and in4 to control the motor's rotation direction, and output to the PWM wave to Enable2 pin to control the motor's rotational speed.

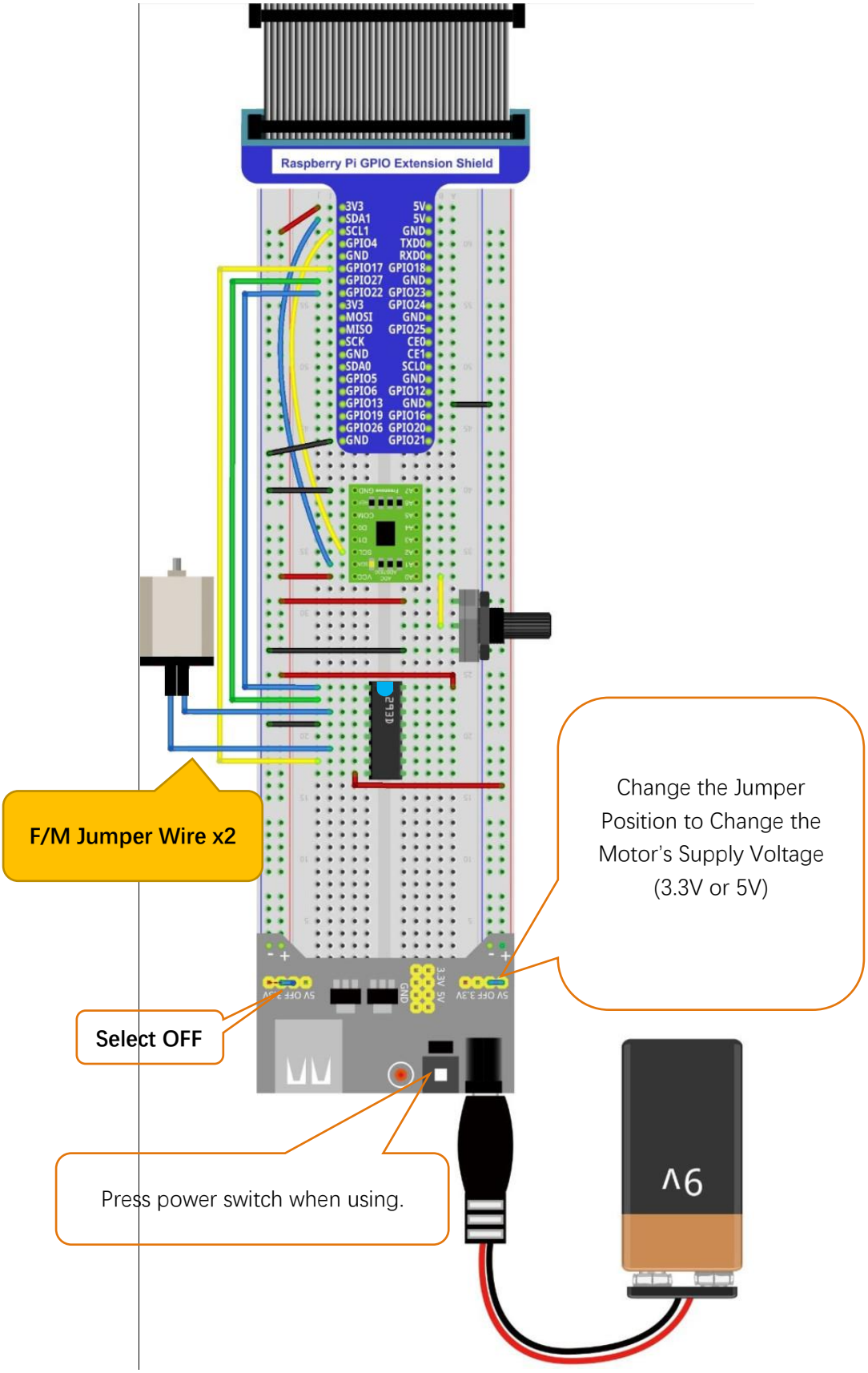
Circuit with ADS7830

Use caution when connecting this circuit because the DC Motor is a high-power component. **Do not use the power provided by the RPi to power the motor directly, as this may cause permanent damage to your RPi!** The logic circuit can be powered by the RPi's power or an external power supply, which should share a common ground with RPi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

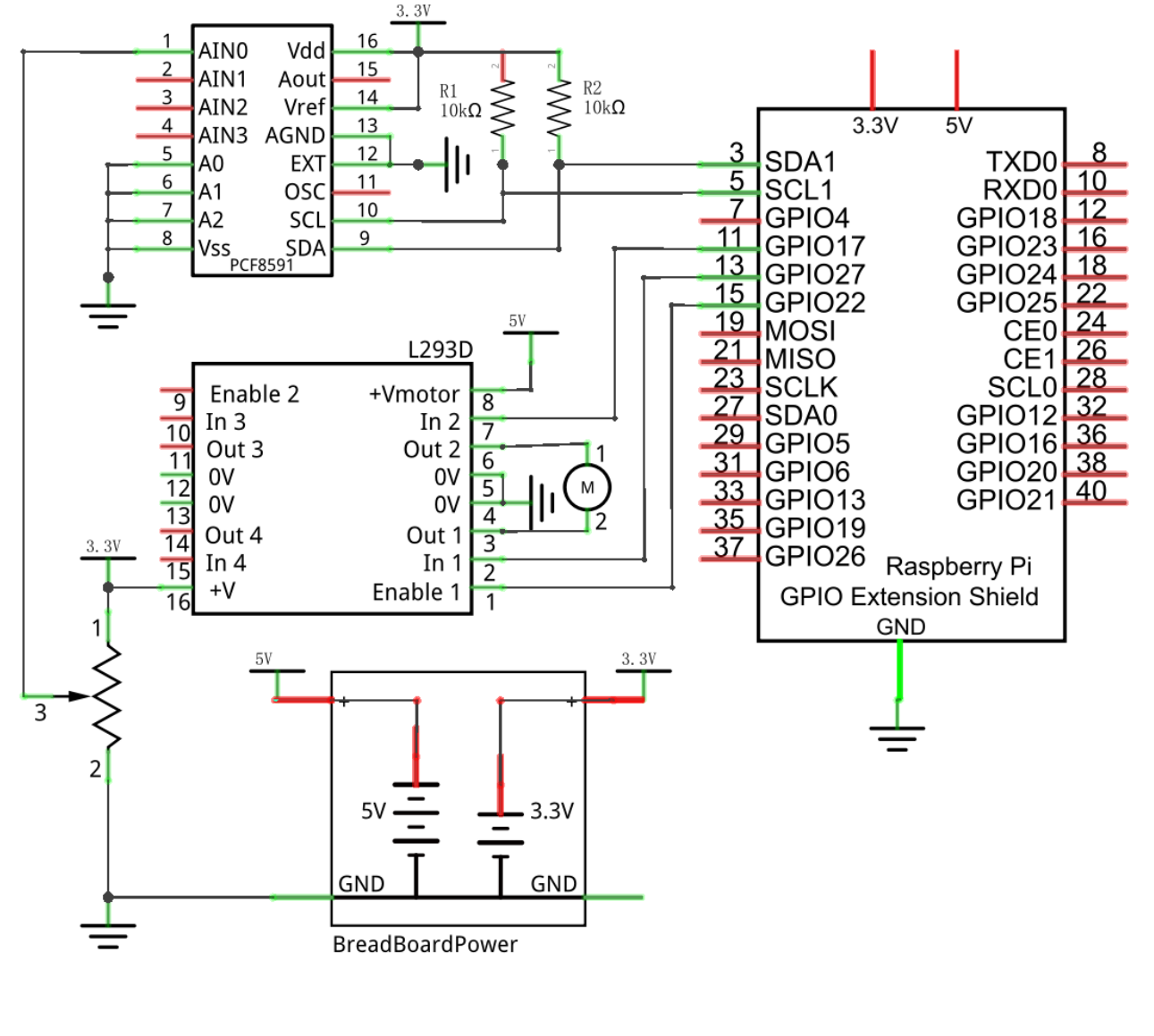


Video: https://youtu.be/d5lRMTDK-wg

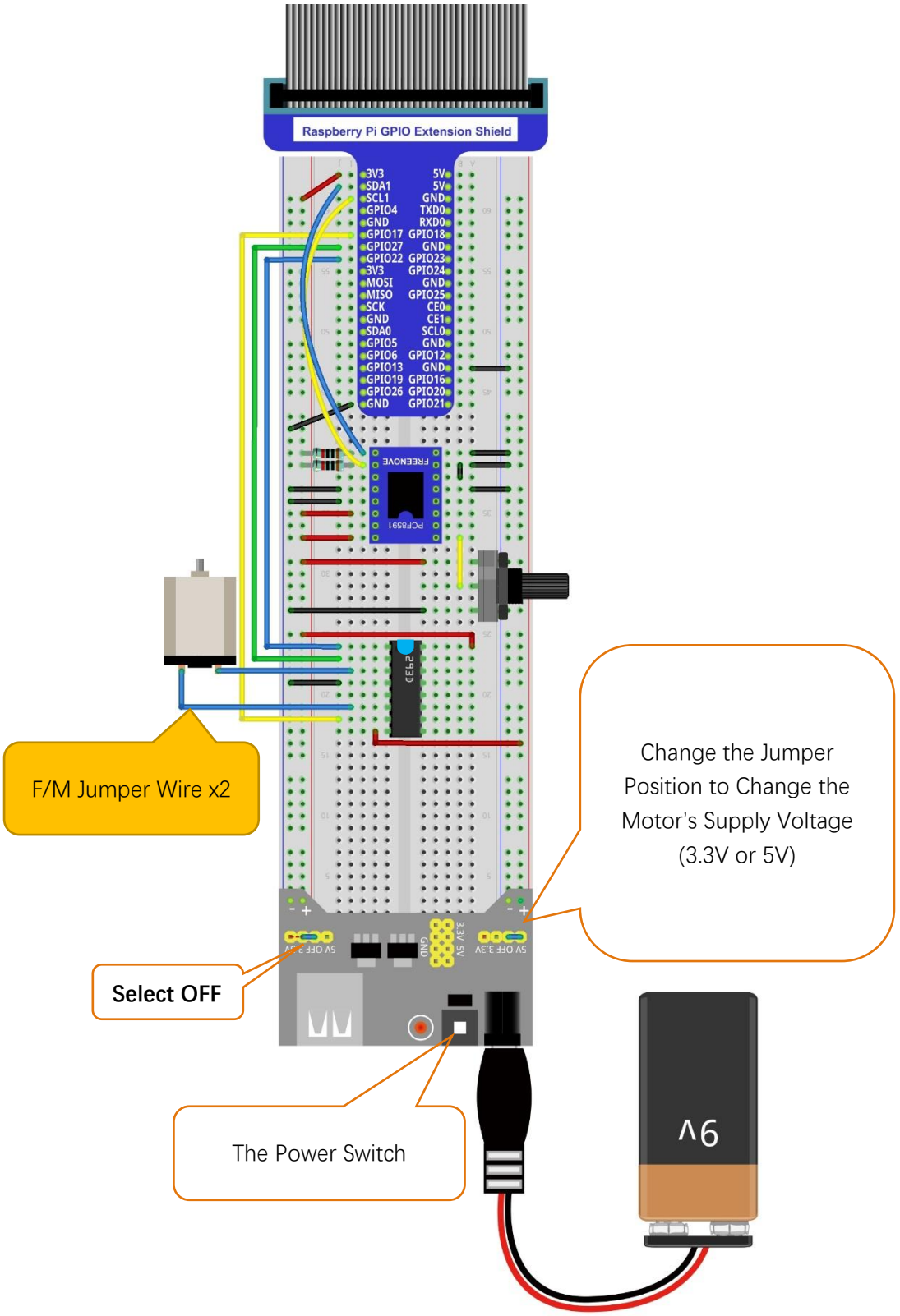
Circuit with PCF8591

Use caution when connecting this circuit because the DC Motor is a high-power component. **Do not use the power provided by the RPi to power the motor directly, as this may cause permanent damage to your RPi!** The logic circuit can be powered by the RPi's power or an external power supply, which should share a common ground with RPi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

In code for this project, first read the ADC value and then control the rotation direction and speed of the DC Motor according to the value of the ADC.

Python Code 13.1.1 Motor

If you did not configure I2C and install Smbus, please refer to Chapter 7. If you did, please Continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 13.1.1_Motor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/13.1.1_Motor
```

2. Use python command to execute the Python code "Motor.py".

```
python Motor.py
```

After the program is executed, you can use the Potentiometer to control the DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reaches the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction. You will also see the ADC value of the potentiometer displayed in the Terminal with the motor direction and the PWM duty cycle used to control the DC Motor's speed.

```
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%
```

The following is the code:

```
1  from gpiozero import DigitalOutputDevice,PWMOutputDevice
2  import time
3  from ADCDevice import *
4
5  # define the pins connected to L293D
6  motoRPin1 = DigitalOutputDevice(27)          # define L293D pin according to BCM Numbering
7  motoRPin2 = DigitalOutputDevice(17)          # define L293D pin according to BCM Numbering
8  enablePin = PWMOutputDevice(22,frequency=1000)
9  adc = ADCDevice() # Define an ADCDevice class object
```

```
10
11  def setup():
12      global adc
13      if(adc.detectI2C(0x48)): # Detect the pcf8591.
14          adc = PCF8591()
15      elif(adc.detectI2C(0x4b)): # Detect the ads7830
16          adc = ADS7830()
17      else:
18          print("No correct I2C address found, \n"
19          "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
20          "Program Exit. \n");
21          exit(-1)
22  # mapNUM function: map the value from a range of mapping to another range.
23  def mapNUM(value,fromLow,fromHigh,toLow,toHigh):
24      return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow
25
26  # motor function: determine the direction and speed of the motor according to the input ADC
    value input
27  def motor(ADC):
28      value = ADC -128
29      if (value > 0):  # make motor turn forward
30          motoRPin1.on()         # motoRPin1 output HIHG level
31          motoRPin2.off()        # motoRPin2 output LOW level
32          print ('Turn Forward...')
33      elif (value < 0): # make motor turn backward
34          motoRPin1.off()
35          motoRPin2.on()
            print ('Turn Backward...')
36      else :
37          motoRPin1.off()
38          motoRPin2.off()
39          print ('Motor Stop...')
40      b=mapNUM(abs(value),0,128,0,100)
41      enablePin.value = b / 100.0      # set dc value as the duty cycle
42      print ('The PWM duty cycle is %d%%\n'%(abs(value)*100/127))    # print PMW duty cycle.
43
44  def loop():
45      while True:
46          value = adc.analogRead(0) # read ADC value of channel 0
47          print ('ADC Value : %d'%(value))
48          motor(value)
49          time.sleep(0.2)
50
51  def destroy():
```

```
52      motoRPin1.close()
53      motoRPin2.close()
54      enablePin.close()
55      adc.close()
56
57  if __name__ == '__main__':  # Program entrance
58      print ('Program is starting ... ')
59      setup()
60      try:
61          loop()
62      except KeyboardInterrupt: # Press ctrl-c to end the program.
63          destroy()
64          print("Ending program")
```

Now that we have familiarity with reading ADC values, let's learn the subfunction void motor (int ADC): first, compare the ADC value with 128 (value corresponding to midpoint). When the current ADC value is higher, motoRPin1 outputs high level and motoRPin2 outputs low level to control the DC Motor to run in the "Forward" Rotational Direction. When the current ADC value is lower, motoRPin1 outputs low level and motoRPin2 outputs high level to control the DC Motor to run in the "Reverse" Rotational Direction. When the ADC value is equal to 128, motoRPin1 and motoRPin2 output low level, the motor STOPS. Then determine the PWM duty cycle according to the difference (delta) between ADC value and 128. Because the absolute delta value stays within 0-128. We need to use the map() subfunction mapping the delta value to a range of 0-255. Finally, we see a display of the duty cycle in Terminal.

```
def motor(ADC):
    value = ADC -128
    if (value > 0):  # make motor turn forward
        motoRPin1.on()         # motoRPin1 output HIHG level
        motoRPin2.off()        # motoRPin2 output LOW level
        print ('Turn Forward...')
    elif (value < 0): # make motor turn backward
        motoRPin1.off()
        motoRPin2.on()
        print ('Turn Backward...')
    else :
        motoRPin1.off()
        motoRPin2.off()
        print ('Motor Stop...')
    b=mapNUM(abs(value),0,128,0,100)
    enablePin.value = b / 100.0      # set dc value as the duty cycle
    print ('The PWM duty cycle is %d%%\n'%(abs(value)*100/127))    # print PMW duty cycle.
```

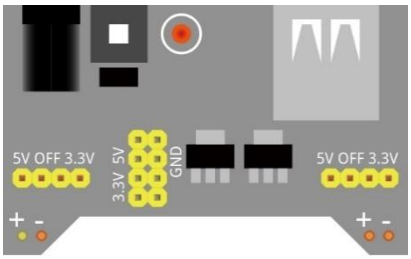
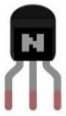
Chapter 14 Relay & Motor

In this chapter, we will learn a kind of special switch module, Relay Module.

Project 14.1.1 Relay & Motor

In this project, we will use a Push Button Switch indirectly to control the DC Motor via a Relay.

Component List

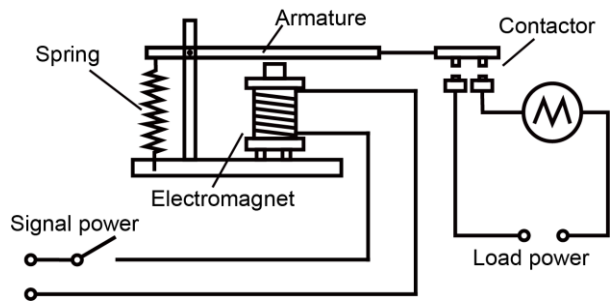
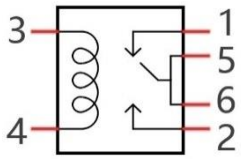
<table>
<tr><td colspan="3">Raspberry Pi (with 40 GPIO) x1
GPIO Expansion Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wire x11
 </td></tr>
<tr><td colspan="6">9V battery (prepared by yourself) & battery line
 </td></tr>
<tr><td colspan="3">Breadboard Power module x1
  </td><td>Resistor 10kΩ x2
 </td><td>Resistor 1kΩ x1
 </td><td>Resistor 220Ω x1
 </td></tr>
<tr><td>NPN transistor x1
  </td><td>Relay x1
 </td><td>Motor x1
 </td><td>Push button x1
 </td><td>LED x1
 </td><td>Diode x1
 </td></tr>
</table>

Component knowledge

Relay

Relays are a type of Switch that open and close circuits electromechanically or electronically. Relays control one electrical circuit by opening and closing contacts in another circuit using an electromagnet to initiate the Switch action. When the electromagnet is energized (powered), it will attract internal contacts completing a circuit, which act as a Switch. Many times Relays are used to allow a low powered circuit (and a small low amperage switch) to safely turn ON a larger more powerful circuit. They are commonly found in automobiles, especially from the ignition to the starter motor.

The following is a basic diagram of a common Relay and the image and circuit symbol diagram of the 5V relay used in this project:

Diagram	Feature:	Symbol
		

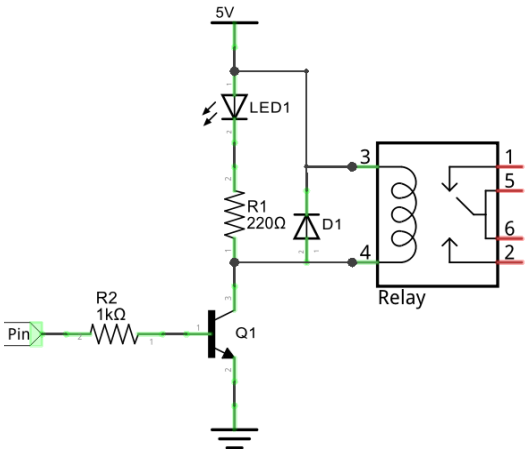
Pin 5 and pin 6 are internally connected to each other. When the coil pin3 and pin 4 are connected to a 5V power supply, pin 1 will be disconnected from pins 5 & 6 and pin 2 will be connected to pins 5 & 6. Pin 1 is called Closed End and pin 2 is called the Open End.

Inductor

The symbol of Inductance is “L” and the unit of inductance is the “Henry” (H). Here is an example of how this can be encountered: 1H=1000mH, 1mH=1000μH.

An Inductor is a passive device that stores energy in its Magnetic Field and returns energy to the circuit whenever required. An Inductor is formed by a Cylindrical Core with many Turns of conducting wire (usually copper wire). Inductors will hinder the changing current passing through it. When the current passing through the Inductor increases, it will attempt to hinder the increasing movement of current; and when the current passing through the inductor decreases, it will attempt to hinder the decreasing movement of current. So the current passing through an Inductor is not transient.

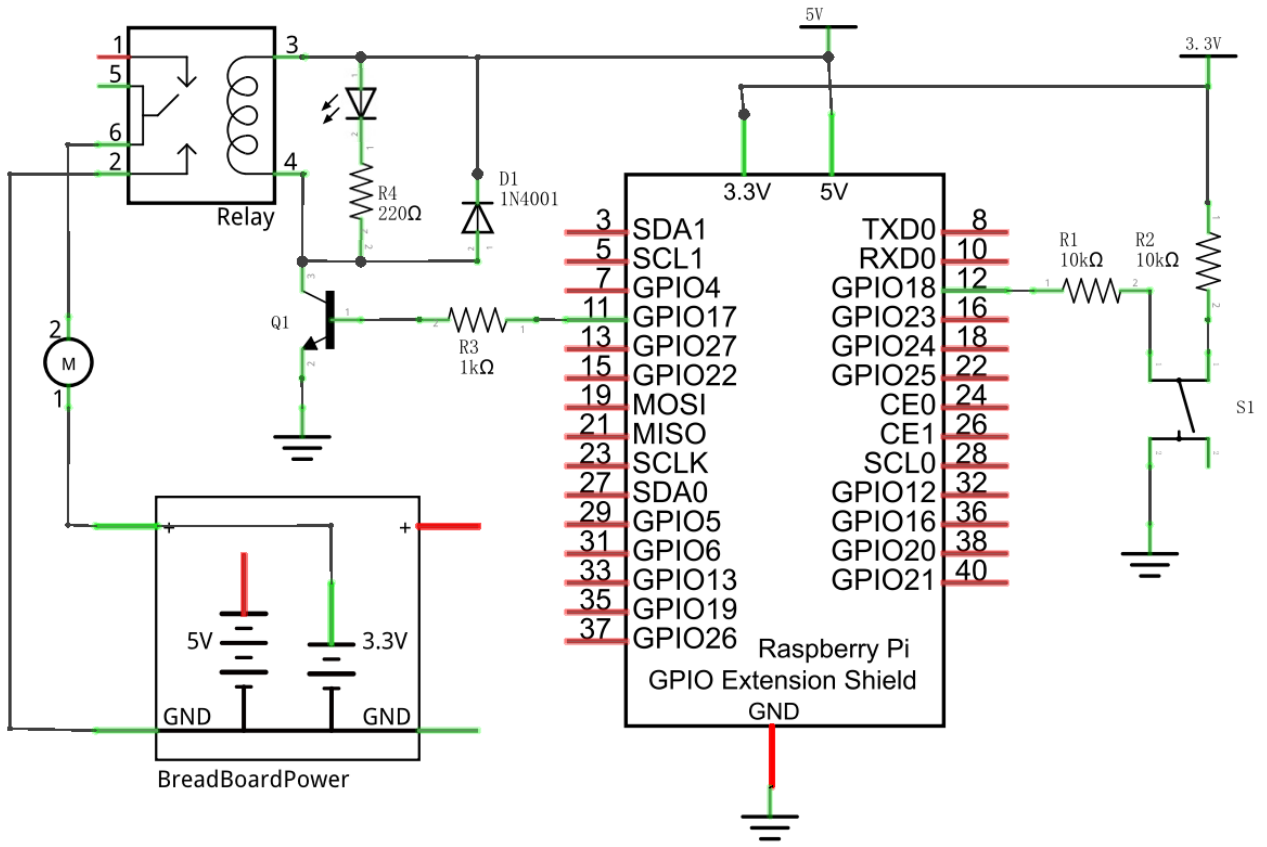
The circuit for a Relay is as follows: The coil of Relay can be equivalent to an Inductor, when a Transistor is present in this coil circuit it can disconnect the power to the relay, the current in the Relay’s coil does not stop immediately, which affects the power supply adversely. To remedy this, diodes in parallel are placed on both ends of the Relay coil pins in opposite polar direction. Having the current pass through the diodes will avoid any adverse effect on the power supply.



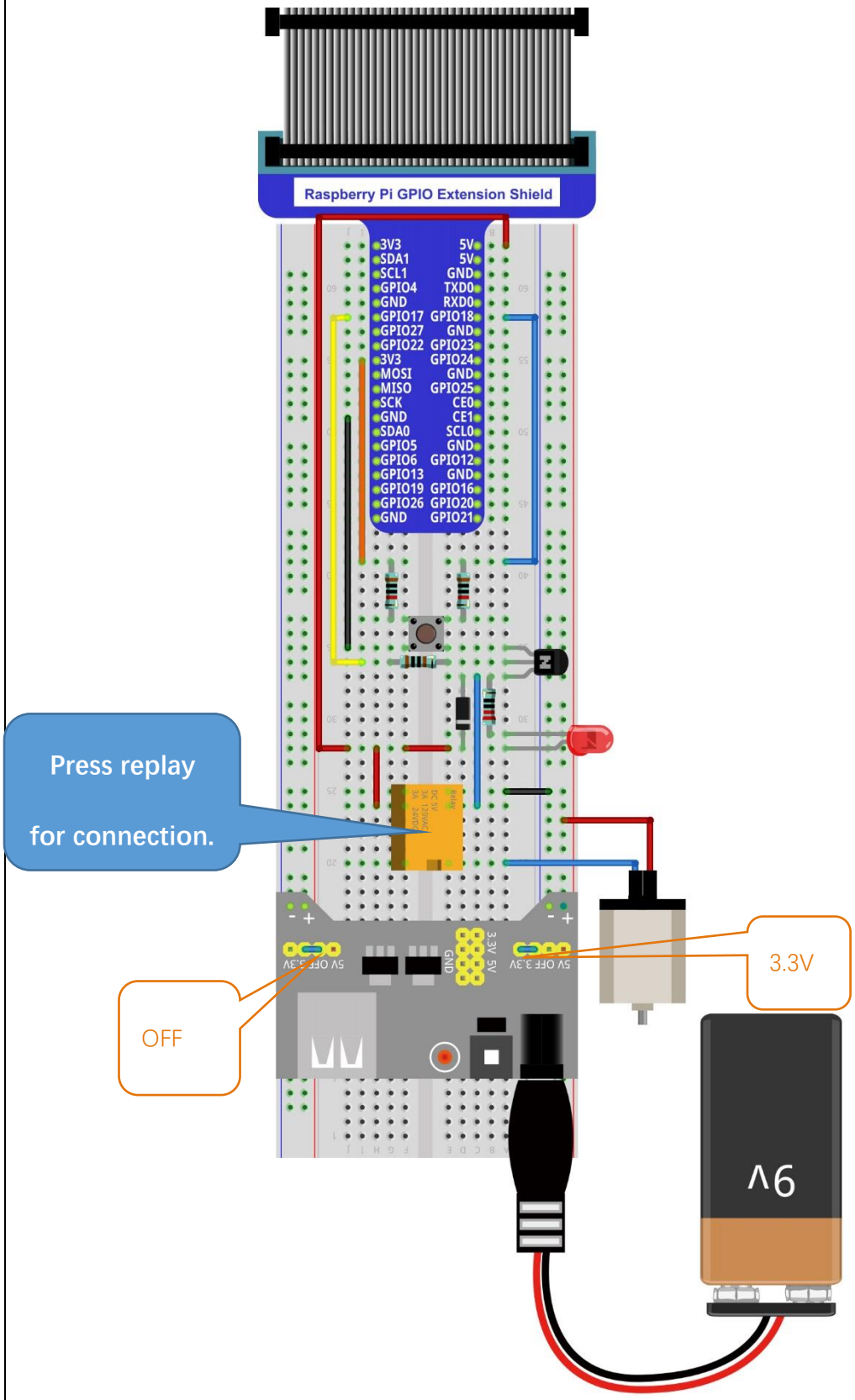
Circuit

Use caution with the power supply voltage needed for the components in this circuit. The Relay requires a power supply voltage of 5V, and the DC Motor only requires 3.3V. Additionally, there is an LED present, which acts as an indicator (ON or OFF) for the status of the Relay's active status.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/CUpPpWq8YI8

Code

The project code is in the same as we used earlier in the Table Lamp project. Pressing the Push Button Switch activates the transistor. Because the Relay and the LED are connected in parallel, they will be powered ON at the same time. Press the Push Button Switch again will turn them both OFF.

Python Code 14.1.1 Relay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 14.1.1_Relay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/14.1.1_Relay
```

2. Use Python command to execute code "Relay.py".

```
python Relay.py
```

After the program is executed, pressing the Push Button Switch activates the Relay (the internal switch is closed), which powers the DC Motor to rotate and simultaneously powers the LED to turn ON. If you press the Push Button Switch again, the Relay is deactivated (the internal switch opens), the Motor STOPS and the LED turns OFF.

The following is the program code:

```
1  from gpiozero import DigitalOutputDevice, Button
2  import time
3
4  relayPin = 17     # define the relayPin
5  buttonPin = 18    # define the buttonPin
6  relay = DigitalOutputDevice(relayPin)     # define LED pin according to BCM Numbering
7  button = Button(buttonPin) # define Button pin according to BCM Numbering
8
9  def onButtonPressed():  # When button is pressed, this function will be executed
10     relay.toggle()
11     if relay.value :
12         print("Turn on relay ...")
13     else :
14         print("Turn off relay ... ")
15
16 def loop():
17     button.when_pressed = onButtonPressed
18     while True:
19         time.sleep(1)
20
21 def destroy():
22     relay.close()
23     button.close()
24
25 if __name__ == '__main__':     # Program entrance
26     print ('Program is starting...')
```

```
27      try:
28          loop()
29      except KeyboardInterrupt:   # Press ctrl-c to end the program.
30          destroy()
31          print("Ending program")
```

The project code is in the same as we used earlier in the Table Lamp project.

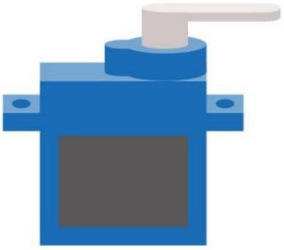
Chapter 15 Servo

Previously, we learned how to control the speed and rotational direction of a DC Motor. In this chapter, we will learn about Servos which are a rotary actuator type motor that can be controlled rotate to specific angles.

Project 15.1 Servo Sweep

First, we need to learn how to make a Servo rotate.

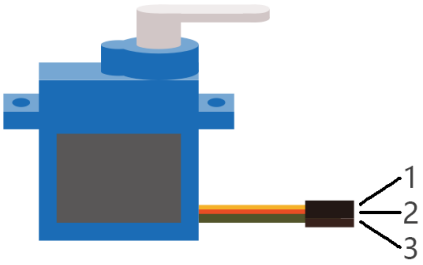
Component List

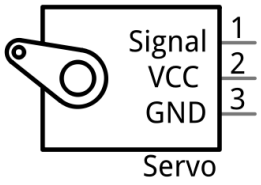
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x3
Servo x1 	

Component knowledge

Servo

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their "horn". Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.





We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The lasting time 0.5ms-2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degree linearly. Part of the corresponding values are as follows:

Note: the lasting time of high level corresponding to the servo angle is absolute instead of accumulating. For example, the high level time lasting for 0.5ms correspond to the 0 degree of the servo. If the high level time lasts for another 1ms, the servo rotates to 45 degrees.

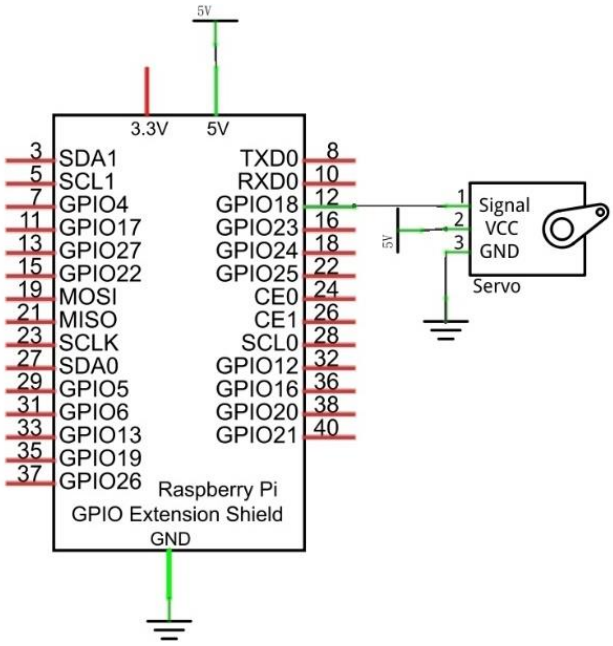
High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	90 degree
2ms	135 degree
2.5ms	180 degree

When you change the Servo signal value, the Servo will rotate to the designated angle.

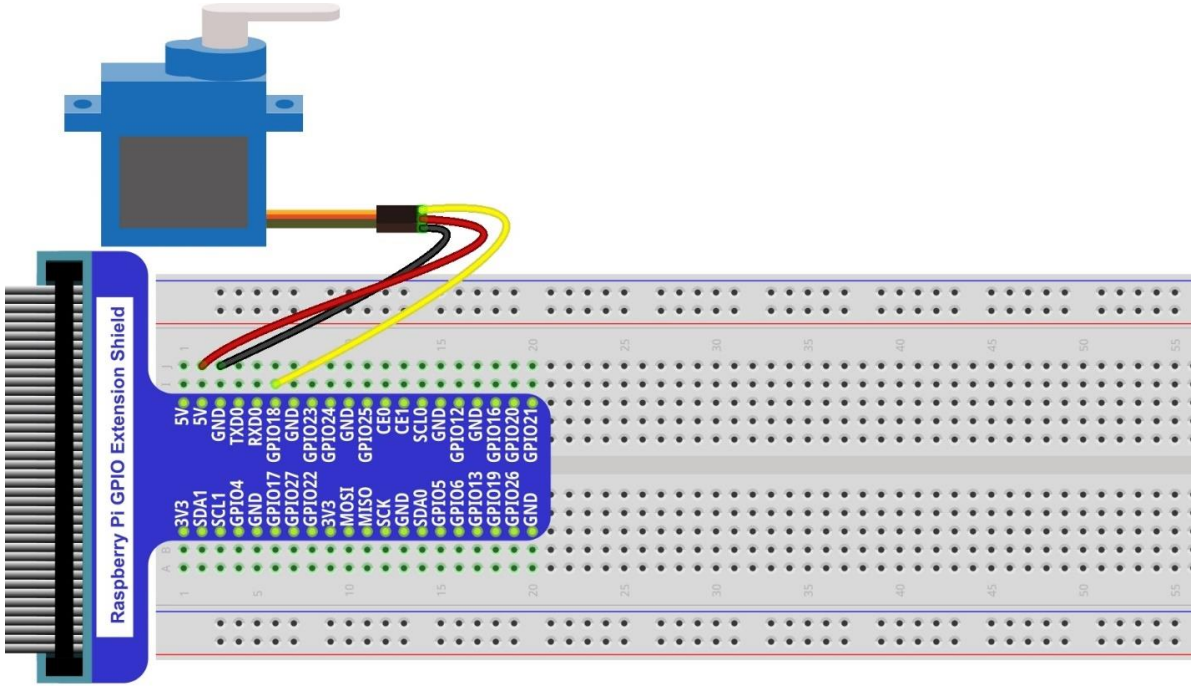
Circuit

Use caution when supplying power to the Servo it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/leptbJh32ZI

Code

In this project, we will make a Servo rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

Python Code 15.1.1 Sweep

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15.1.1_Sweep
```

2. Use python command to execute code "Sweep.py".

```
python Sweep.py
```

After the program is executed, the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

The following is the program code:

```
1  from gpiozero import AngularServo
2  import time
3
4  myGPIO=18
5  SERVO_DELAY_SEC = 0.001
6  myCorrection=0.0
7  maxPW=(2.5+myCorrection)/1000
8  minPW=(0.5-myCorrection)/1000
9  servo =  AngularServo(myGPIO,initial_angle=0,min_angle=0,
   max_angle=180,min_pulse_width=minPW,max_pulse_width=maxPW)
10
11 def loop():
12     while True:
13         for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
14             servo.angle = angle
15             time.sleep(SERVO_DELAY_SEC)
16         time.sleep(0.5)
17         for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
18             servo.angle = angle
19             time.sleep(SERVO_DELAY_SEC)
20         time.sleep(0.5)
21
22 if __name__ == '__main__':     # Program entrance
23     print ('Program is starting...')
24     try:
25         loop()
26     except KeyboardInterrupt:  # Press ctrl-c to end the program.
27         print("Ending program")
```

Import the AngularServo class that controls Servo from the gpiozero library.

```
from gpiozero import AngularServo
```

A 50 Hz pulse for a 20ms cycle is required to control the Servo. By default, the AngularServo class has set the control period to 20 milliseconds.

```
servo =  AngularServo(myGPIO,initial_angle=0,min_angle=0,
max_angle=180,min_pulse_width=minPW,max_pulse_width=maxPW)
```

The 0-180 degree rotation of the servo corresponds to a PWM pulse width of 0.5-2.5ms at a period of 20ms and a duty cycle of 2.5%-12.5%. After setting the AngularServo class and passing in the corresponding angle parameters, the servo will turn to the corresponding position. However, in actual operation, as there is a deviation in the width of the servo pulse, we need to define minimum and maximum pulse width and error offset (this is essential in robotics).

```
myCorrection=0.0 #define pulse offset of servo
maxPW=(2.5+myCorrection)/1000     #define pulse duty cycle for minimum angle of servo
minPW=(0.5-myCorrection)/1000     #define pulse duty cycle for maximum angle of s
servo =  AngularServo(myGPIO,initial_angle=0,min_angle=0,
max_angle=180,min_pulse_width=minPW,max_pulse_width=maxPW)
......
OFFSE_DUTY = 0.5        #define pulse offset of servo
SERVO_MIN_DUTY = 2.5+OFFSE_DUTY     #define pulse duty cycle for minimum angle of servo
SERVO_MAX_DUTY = 12.5+OFFSE_DUTY    #define pulse duty cycle for maximum angle of s ervo
......
        for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)
```

Finally, in the "while" cycle of main function, we need to use two separate cycles to make servo rotate from 0 degrees to 180 degrees and then from 180 degrees to 0 degrees.

```
def loop():
    while True:
        for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)
        for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)
```

For more information about the methods used by the AngularServo class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_output.html#angularservo

In the above experiment, you can see that the servo jitter obviously when working.
Note: To reduce servo jitter, use the pigpio pin driver rather than the default RPi.GPIO driver (pigpio uses DMA sampling for much more precise edge timing).
You can refer to the code sweep2.py for more details.

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

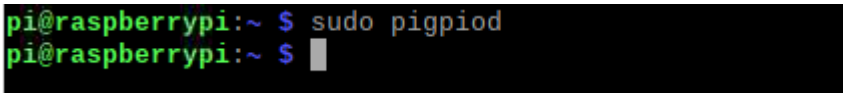
```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15.1.1_Sweep
```

2. Use python command to execute code "Sweep.py".

```
python Sweep2.py
```

After the program executes, the servo will rotate from 0 degrees to 180 degrees, then reverse the direction so it rotates from 180 degrees to 0 degrees, and repeat these actions in an infinite loop. At this time, the steering gear can hardly feel the vibration.
This code is based on pigpio. In the latest Raspberry Pi OS, "pigpio" library has been installed. You only need to run the command to enable it.

```
sudo pigpiod
```



If the "pigpio" library has not yet been installed, please follow the steps to install it.
Run the command to install "pigpio" library.

```
sudo apt-get update
sudo apt-get install pigpio python-pigpio python3-pigpio
```

The following is the program code:

```
1  import os
2  os.system("sudo pigpiod")
3  from gpiozero import AngularServo
4  from gpiozero.pins.pigpio import PiGPIOFactory
5  import time
6
7  my_factory = PiGPIOFactory()
8  myGPIO=18
9  SERVO_DELAY_SEC = 0.001
   myCorrection=0.0
10 maxPW=(2.5+myCorrection)/1000
11 minPW=(0.5-myCorrection)/1000
12 servo =  AngularServo(myGPIO,initial_angle=0,min_angle=0,
13 max_angle=180,min_pulse_width=minPW,max_pulse_width=maxPW,pin_factory=my_factory)
14
15 def loop():
16     while True:
17         for angle in range(0, 181, 1):   # make servo rotate from 0 to 180 deg
18             servo.angle = angle
19             time.sleep(SERVO_DELAY_SEC)
```

```
        time.sleep(0.5)
        for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        servo.close()
        os.system("sudo killall pigpiod")
        print("Ending program")
```

The following values, and the corresponding Factory and Pin classes are listed in the table below. Factories are listed in the order that they are tried by default.

Name	Factory class	Pin class
rpigpio	gpiozero.pins.rpigpio.RPiGPIOFactory	gpiozero.pins.rpigpio.RPiGPIOPin
lgpio	gpiozero.pins.lgpio.LGPIOFactory	gpiozero.pins.lgpio.LGPIOPin
rpio	gpiozero.pins.rpio.RPIOFactory	gpiozero.pins.rpio.RPIOPin
pigpio	gpiozero.pins.pigpio.PiGPIOFactory	gpiozero.pins.pigpio.PiGPIOPin
native	gpiozero.pins.native.NativeFactory	gpiozero.pins.native.NativePin

See Changing the pin factory for further information:
https://gpiozero.readthedocs.io/en/stable/api_pins.html#changing-pin-factory

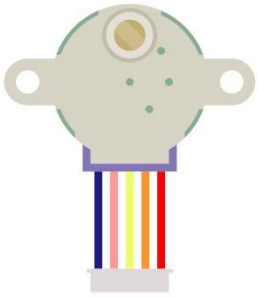
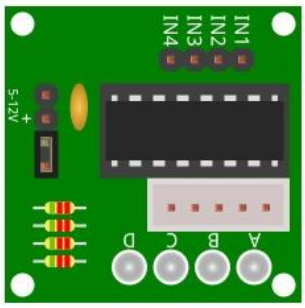
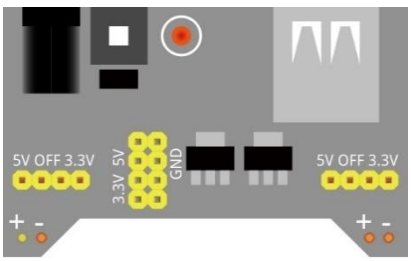
Chapter 16 Stepper Motor

Thus far, we have learned about DC Motors and Servos. A DC motor can rotate constantly in on direction but we cannot control the rotation to a specific angle. On the contrary, a Servo can rotate to a specific angle but cannot rotate constantly in one direction. In this chapter, we will learn about a Stepper Motor which is also a type of motor. A Stepper Motor can rotate constantly and also to a specific angle. Using a Stepper Motor can easily achieve higher accuracies in mechanical motion.

Project 16.1 Stepper Motor

In this project, we will learn how to drive a Stepper Motor, and understand its working principle.

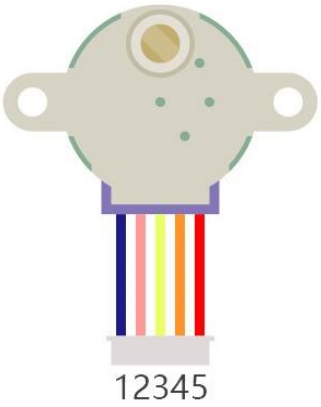
Component List

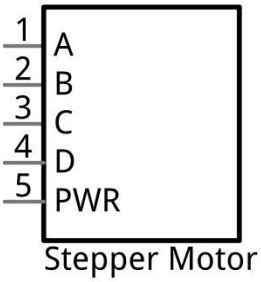
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x12
Stepper Motor x1 	ULN2003 Stepper Motor Driver x1 
9V battery (prepared by yourself) & battery line 	Breadboard Power module x1 

Component knowledge

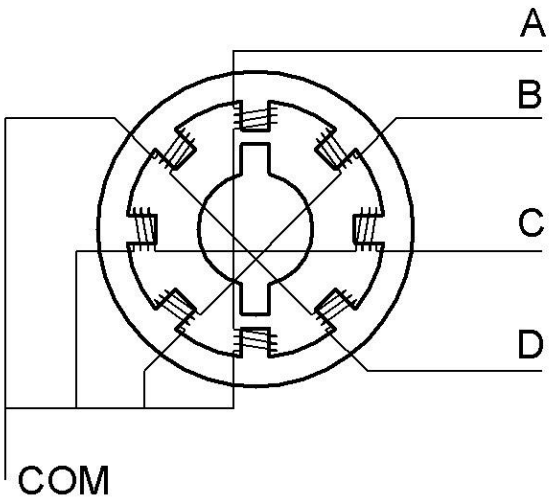
Stepper Motor

Stepper Motors are an open-loop control device, which converts an electronic pulse signal into angular displacement or linear displacement. In a non-overload condition, the speed of the motor and the location of the stops depends only on the pulse signal frequency and number of pulses and is not affected by changes in load as with a DC Motor. A small Four-Phase Deceleration Stepper Motor is shown here:



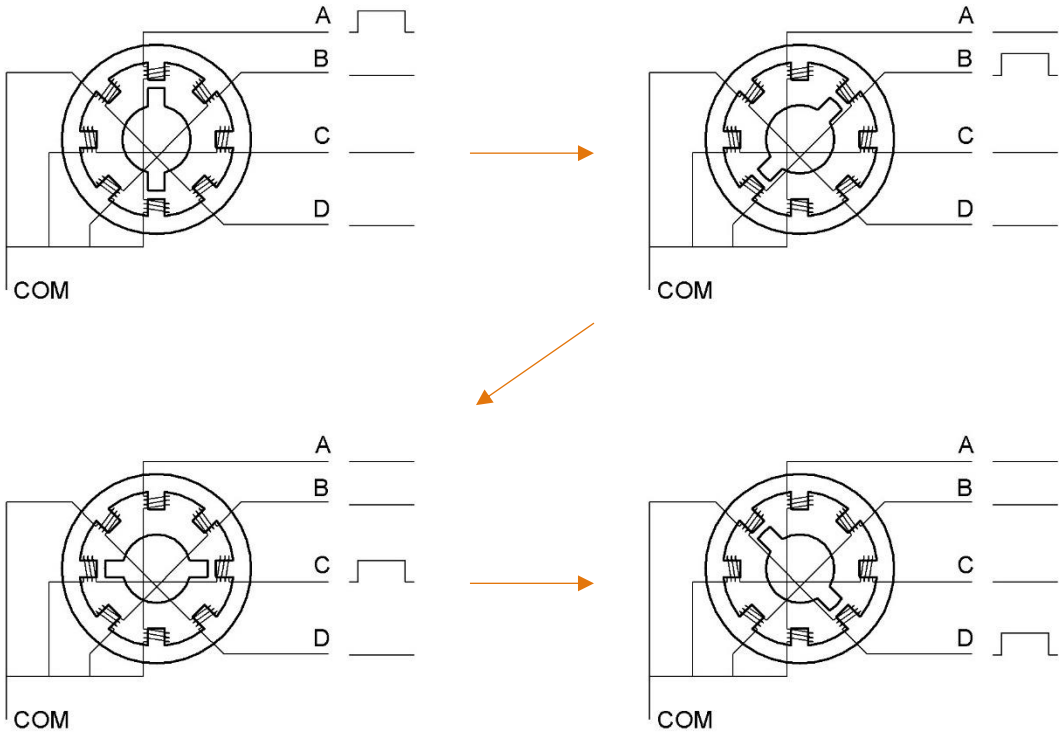


The electronic schematic diagram of a Four-Phase Stepper Motor is shown below:



The outside case or housing of the Stepper Motor is the Stator and inside the Stator is the Rotor. There is a specific number of individual coils, usually an integer multiple of the number of phases the motor has, when the Stator is powered ON, an electromagnetic field will be formed to attract a corresponding convex diagonal groove or indentation in the Rotor's surface. The Rotor is usually made of iron or a permanent magnet. Therefore, the Stepper Motor can be driven by powering the coils on the Stator in an ordered sequence (producing a series of "steps" or stepped movements).

A common driving sequence is shown here:



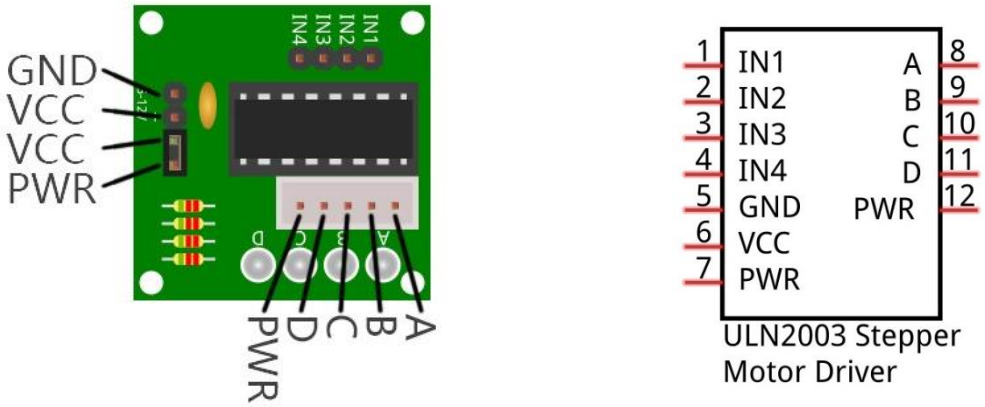
In the sequence above, the Stepper Motor rotates by a certain angle at once, which is called a “step”. By controlling the number of rotational steps, you can then control the Stepper Motor’s rotation angle. By defining the time between two steps, you can control the Stepper Motor’s rotation speed. When rotating clockwise, the order of coil powered on is: A → B → C → D → A →····· . And the rotor will rotate in accordance with this order, step by step, called four-steps, four-part. If the coils is powered ON in the reverse order, D → C → B → A → D →··· , the rotor will rotate in counter-clockwise direction.

There are other methods to control Stepper Motors, such as: connect A phase, then connect A B phase, the stator will be located in the center of A B, which is called a half-step. This method can improve the stability of the Stepper Motor and reduces noise. Tise sequence of powering the coils looks like this: A → AB → B → BC → C → CD → D → DA → A →·····, the rotor will rotate in accordance to this sequence ar, a half-step at a time, called four-steps, eight-part. Conversely, if the coils are powered ON in the reverse order the Stepper Motor will rotate in the opposite direction.

The stator in the Stepper Motor we have supplied has 32 magnetic poles. Therefore, to complete one full revolution requires 32 full steps. The rotor (or output shaft) of the Stepper Motor is connected to a speed reduction set of gears and the reduction ratio is 1:64. Therefore, the final output shaft (exiting the Stepper Motor’s housing) requires 32 X 64 = 2048 steps to make one full revolution.

ULN2003 Stepper Motor driver

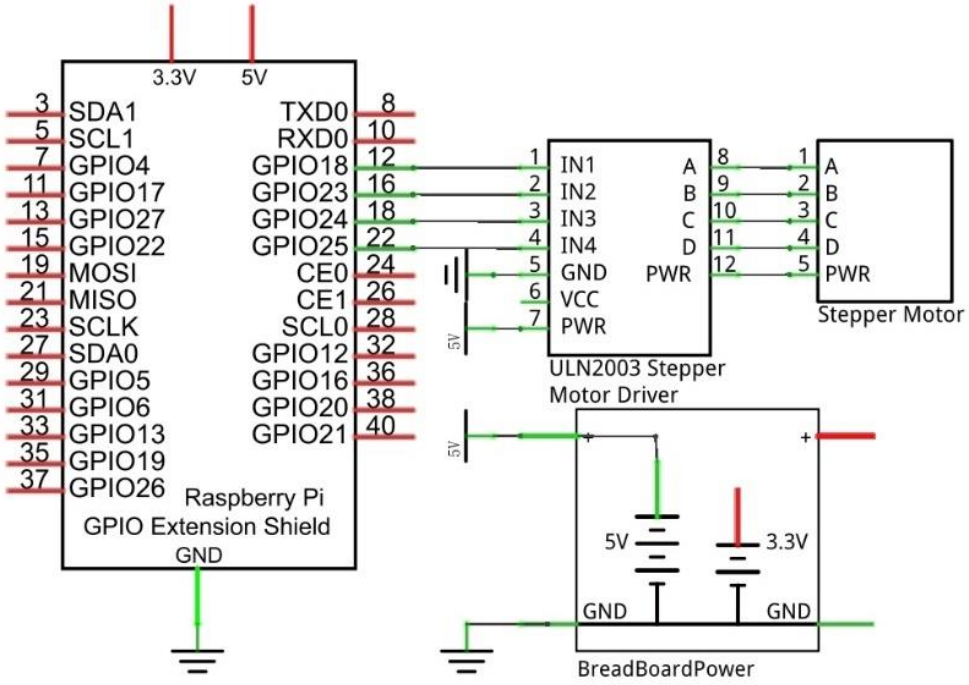
A ULN2003 Stepper Motor Driver is used to convert weak signals into more powerful control signals in order to drive the Stepper Motor. In the illustration below, the input signal IN1-IN4 corresponds to the output signal A-D, and 4 LEDs are integrated into the board to indicate the state of these signals. The PWR interface can be used as a power supply for the Stepper Motor. By default, PWR and VCC are connected.



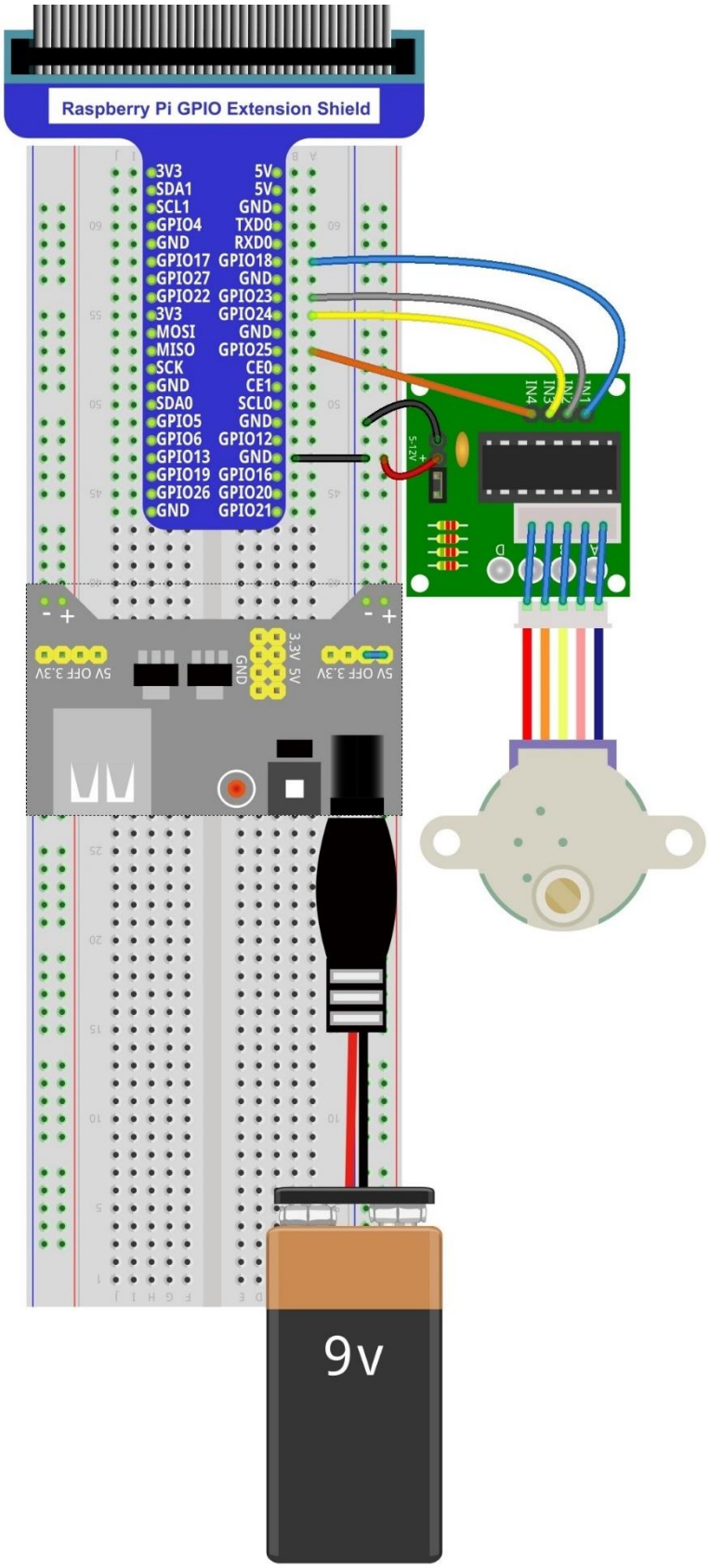
Circuit

When building the circuit, note that rated voltage of the Stepper Motor is 5V, and we need to use the breadboard power supply independently, (**Caution do not use the RPi power supply**). Additionally, the breadboard power supply needs to share Ground with Rpi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/g6R6ldqjQyw

Code

Python Code 16.1.1 SteppingMotor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 16.1.1_SteppingMotor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/16.1.1_SteppingMotor
```

2. Use Python command to execute code "SteppingMotor.py".

```
python SteppingMotor.py
```

After the program is executed, the Stepper Motor will rotate 360° clockwise and then 360° anticlockwise and repeat this action in an endless loop.

The following is the program code:

```
1  from gpiozero import OutputDevice
2  import time
3
4  motorPins = (18, 23, 24, 25) # define pins connected to four phase ABCD of stepper motor
5  # motorPins = ("J8:12", "J8:16", "J8:18", "J8:22") # define pins connected to four phase ABCD
   of stepper motor
6  motors = list(map(lambda pin: OutputDevice(pin), motorPins))
7  CCWStep = (0x01,0x02,0x04,0x08) # define power supply order for rotating anticlockwise
8  CWStep = (0x08,0x04,0x02,0x01)  # define power supply order for rotating clockwise
9
10 # as for four phase stepping motor, four steps is a cycle. the function is used to drive the
   stepping motor clockwise or anticlockwise to take four steps
11 def moveOnePeriod(direction,ms):
12     for j in range(0,4,1):      # cycle for power supply order
13         for i in range(0,4,1):  # assign to each pin
14             if (direction == 1):# power supply order clockwise
15                 motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
16             else :              # power supply order anticlockwise
17                 motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
18         if(ms<3):       # the delay can not be less than 3ms, otherwise it will exceed speed
   limit of the motor
19             ms = 3
20         time.sleep(ms*0.001)
21
22 # continuous rotation function, the parameter steps specifies the rotation cycles, every four
   steps is a cycle
23 def moveSteps(direction, ms, steps):
24     for i in range(steps):
25         moveOnePeriod(direction, ms)
26
27 # function used to stop motor
```

```
def motorStop():
    for i in range(0,4,1):
        motors.off()

def loop():
    while True:
        moveSteps(0,3,512)  # rotating 360 deg clockwise, a total of 2048 steps in a circle,
512 cycles
        time.sleep(0.5)
        moveSteps(1,3,512)  # rotating 360 deg anticlockwise
        time.sleep(0.5)

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
```

In the code we define the four pins of the Stepper Motor and the order to supply power to the coils for a four-step rotation mode.

```
motorPins = (18, 23, 24, 25) # define pins connected to four phase ABCD of stepper motor
# motorPins = ("J8:12", "J8:16", "J8:18", "J8:22") # define pins connected to four phase ABCD
of stepper motor
motors = list(map(lambda pin: OutputDevice(pin), motorPins))
CCWStep = (0x01,0x02,0x04,0x08) # define power supply order for rotating anticlockwise
CWStep = (0x08,0x04,0x02,0x01)  # define power supply order for rotating clockwise
```

Subfunction **moveOnePeriod** ((int dir, int ms) will drive the Stepper Motor rotating four-step clockwise or anticlockwise, four-step as a cycle. Where parameter "dir" indicates the rotation direction, if "dir" is 1, the servo will rotate clockwise, otherwise it rotates to anticlockwise. Parameter "ms" indicates the time between each two steps. The "ms" of Stepper Motor used in this project is 3ms (the shortest time period), a value of less than 3ms will exceed the limits of the Stepper Motor with a result that it does not rotate.

```
def moveOnePeriod(direction,ms):
    for j in range(0,4,1):      # cycle for power supply order
        for i in range(0,4,1):  # assign to each pin
            if (direction == 1):# power supply order clockwise
                motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
            else :              # power supply order anticlockwise
                motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
        if(ms<3):       # the delay can not be less than 3ms, otherwise it will exceed speed
limit of the motor
            ms = 3
        time.sleep(ms*0.001)
```

Subfunction **moveSteps** (direction, ms, steps) is used to specify the cycle number of Stepper Motor.

```
def moveSteps(direction, ms, steps):
    for i in range(steps):
        moveOnePeriod(direction, ms)
```

Subfunction **motorStop** () is used to stop the Stepper Motor.

```
def motorStop():
    for i in range(0,4,1):
        motors.off()
```

Finally, in the while loop of main function, rotate one revolution clockwise, and then one revolution anticlockwise. According to the previous material covered, the Stepper Motor one revolution requires 2048 steps, that is, 2048/4=512 cycle.

```
    while True:
        moveSteps(0,3,512)  # rotating 360 deg clockwise, a total of 2048 steps in a circle,
512 cycles
        time.sleep(0.5)
        moveSteps(1,3,512)  # rotating 360 deg anticlockwise
        time.sleep(0.5)
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

Chapter 17 74HC595 & Bar Graph LED

We have used LED Bar Graph to make a flowing water light, in which 10 GPIO ports of RPi are occupied. More GPIO ports mean that more peripherals can be connected to RPi, so GPIO resource is very precious. Can we make flowing water light with less GPIO ports? In this chapter, we will learn a component, 74HC595, which can achieve the target.

Project 17.1 Flowing Water Light

Now let us learn how to use the 74HC595 IC Chip to make a flowing water light using less GPIO.

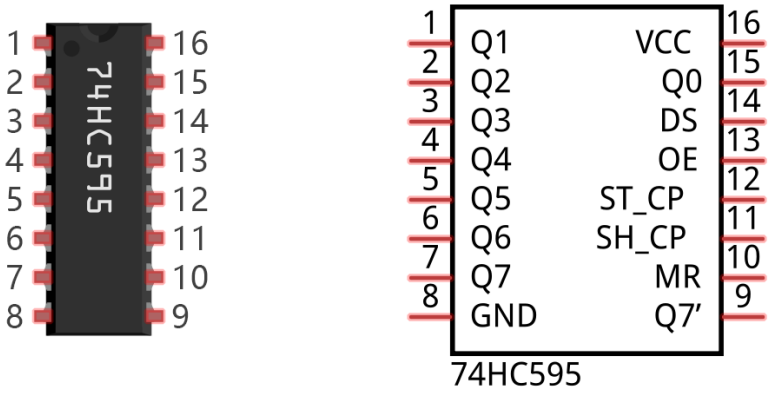
Component List

<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper x17
 </td></tr>
<tr><td>74HC595 x1
 </td><td>Bar Graph LED x1
 </td><td>Resistor 220Ω x8
 </td></tr>
</table>

Component knowledge

74HC595

A 74HC595 chip is used to convert serial data into parallel data. A 74HC595 chip can convert the serial data of one byte into 8 bits, and send its corresponding level to each of the 8 ports correspondingly. With this characteristic, the 74HC595 chip can be used to expand the IO ports of a Raspberry Pi. At least 3 ports on the RPI board are required to control the 8 ports of the 74HC595 chip.



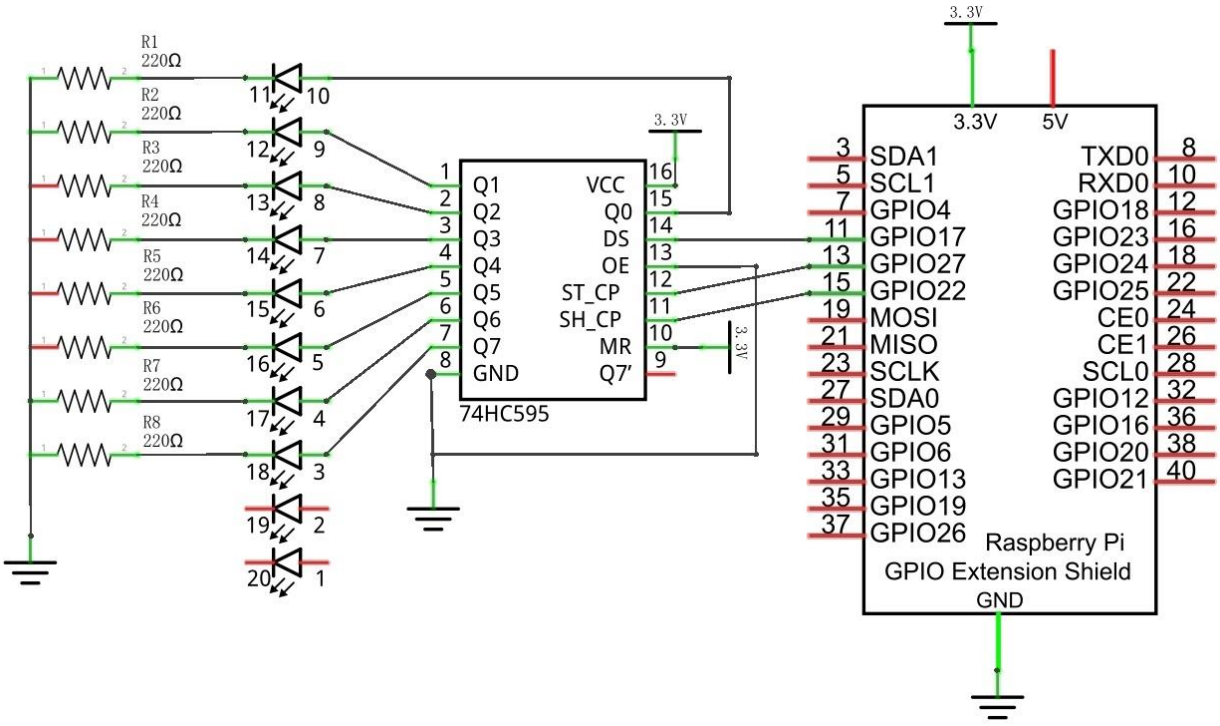
The ports of the 74HC595 chip are described as follows:

Pin name	Pin number	Description
Q0-Q7	15, 1-7	Parallel Data Output
VCC	16	The Positive Electrode of the Power Supply, the Voltage is 2~6V
GND	8	The Negative Electrode of Power Supply
DS	14	Serial Data Input
OE	13	Enable Output, When this pin is in high level, Q0-Q7 is in high resistance state When this pin is in low level, Q0-Q7 is in output mode
ST_CP	12	Parallel Update Output: when its electrical level is rising, it will update the parallel data output.
SH_CP	11	Serial Shift Clock: when its electrical level is rising, serial data input register will do a shift.
MR	10	Remove Shift Register: When this pin is in low level, the content in shift register will be cleared.
Q7'	9	Serial Data Output: it can be connected to more 74HC595 chips in series.

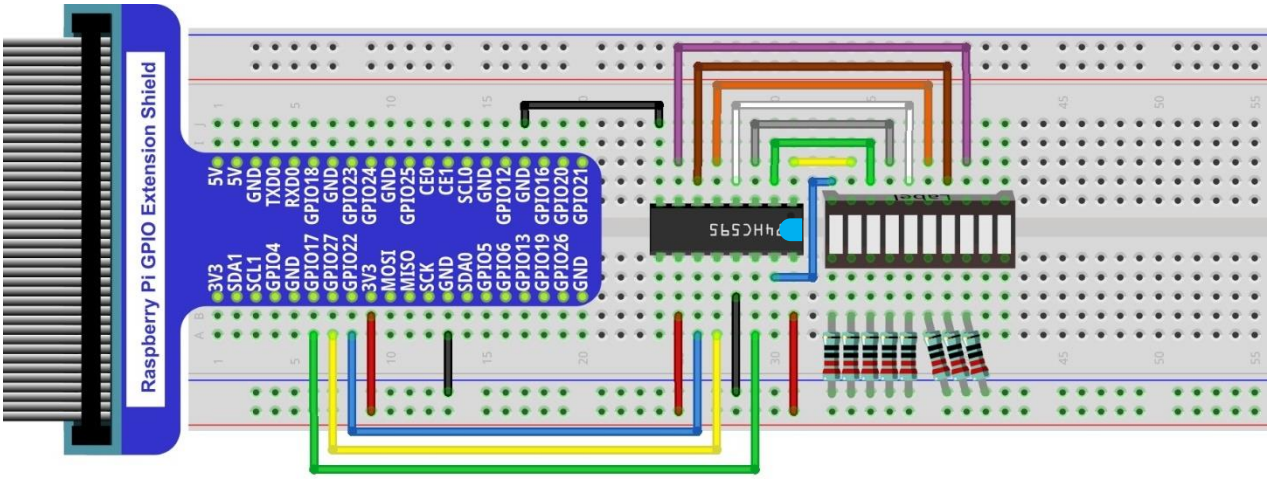
For more details, please refer to the datasheet on the 74HC595 chip.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: **support@freenove.com**



Video: https://www.youtube.com/watch?v=e_SZUGKH2KU

Code

In this project we will make a flowing water light with a 74HC595 chip to learn about its functions.

Python Code 17.1.1 LightWater02

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 17.1.1_LightWater02 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/17.1.1_LightWater02
```

2. Use python command to execute Python code "LightWater02.py".

```
python LightWater02.py
```

After the program is executed, you will see that Bar Graph LED starts with the flowing water pattern flashing from left to right and then back from right to left.

The following is the program code:

```
1  from gpiozero import OutputDevice
2  import time
3  # Defines the data bit that is transmitted preferentially in the shiftOut function.
4  LSBFIRST = 1
5  MSBFIRST = 2
6  # define the pins for 74HC595
7  dataPin   = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
8  latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
9  clockPin  = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
10
11 # shiftOut function, use bit serial transmission.
12 def shiftOut(order,val):
13     for i in range(0,8):
14         clockPin.off()
15         if(order == LSBFIRST):
16             dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
17         elif(order == MSBFIRST):
18             dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
19         clockPin.on()
20
21 def loop():
22     while True:
23         x=0x01
24         for i in range(0,8):
25             latchPin.off()# Output low level to latchPin
26             shiftOut(LSBFIRST,x) # Send serial data to 74HC595
27             latchPin.on()    # Output high level to latchPin, and 74HC595 will update the data
28 to the parallel output port.
29             x<<=1 # make the variable move one bit to left once, then the bright LED move one
30 step to the left once.
```

```
            time.sleep(0.1)
        x=0x80
        for i in range(0,8):
            latchPin.off()
            shiftOut(LSBFIRST,x)
            latchPin.on()
            x>>=1
            time.sleep(0.1)

def destroy():
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the OutputDevice class that controls the 74HC595 chip from the gpiozero library.

```
from gpiozero import OutputDevice
```

Create the OutputDevice class for controlling the 74HC595 chip.

```
dataPin   = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
```

In the code, we define a shiftOut() function, which is used to output values with bits in order, where the dPin for the data pin, cPin for the clock and order for the priority bit flag (high or low). This function conforms to the operational modes of the 74HC595. LSBFIRST and MSBFIRST are two different flow directions.

```
# shiftOut function, use bit serial transmission.
def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()
```

In the loop() function, we use two cycles to achieve the action goal. First, define a variable x=0x01, binary 00000001. When it is transferred to the output port of 74HC595, the low bit outputs high level, then an LED turns ON. Next, x is shifted one bit, when x is transferred to the output port of 74HC595 once again, the LED that turns ON will be shifted. Repeat the operation, over and over and the effect of a flowing water light will be visible. If the direction of the shift operation for x is different, the flowing direction is different.

```
def loop():
    while True:
        x=0x01
        for i in range(0,8):
            latchPin.off()# Output low level to latchPin
            shiftOut(LSBFIRST,x) # Send serial data to 74HC595
            latchPin.on()    # Output high level to latchPin, and 74HC595 will update the data to the parallel output port.
            x<<=1 # make the variable move one bit to left once, then the bright LED move one step to the left once.
            time.sleep(0.1)
        x=0x80
        for i in range(0,8):
            latchPin.off()
            shiftOut(LSBFIRST,x)
            latchPin.on()
            x>>=1
            time.sleep(0.1)
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

Chapter 18 74HC595 & 7-Segment Display

In this chapter, we will introduce the 7-Segment Display.

Project 18.1 7-Segment Display

We will use a 74HC595 IC Chip to control a 7-Segment Display and make it display sixteen decimal characters "0" to "F".

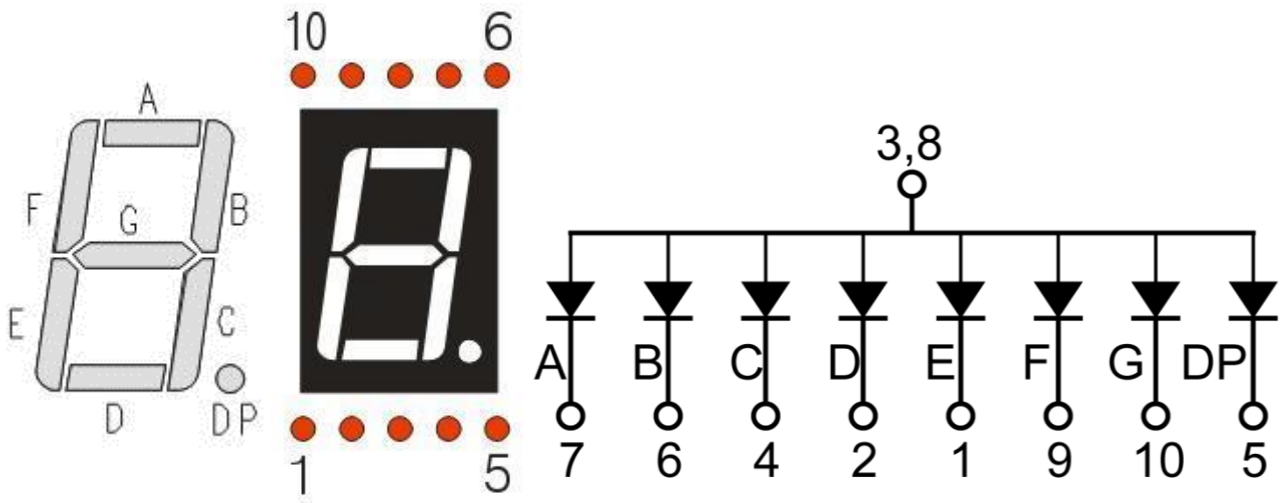
Component List

<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper Wire x18
 </td></tr>
<tr><td>74HC595 x1
 </td><td>7-Segment Display x1
 </td><td>Resistor 220Ω x8
 </td></tr>
</table>

Component knowledge

7-segment display

A 7-Segment Display is a digital electronic display device. There is a figure "8" and a decimal point represented, which consists of 8 LEDs. The LEDs have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:

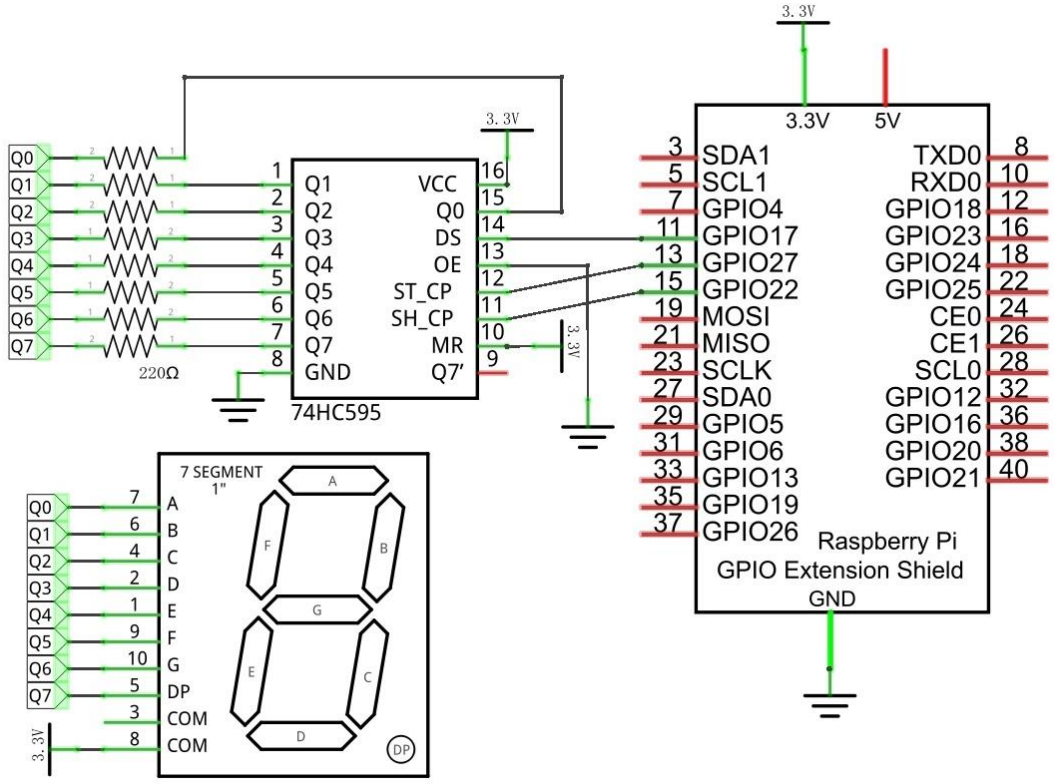


As we can see in the above circuit diagram, we can control the state of each LED separately. Also, by combining LEDs with different states of ON and OFF, we can display different characters (Numbers and Letters). For example, to display a “0”: we need to turn ON LED segments A, B, C, D, E and F, and turn OFF LED segments G and DP.

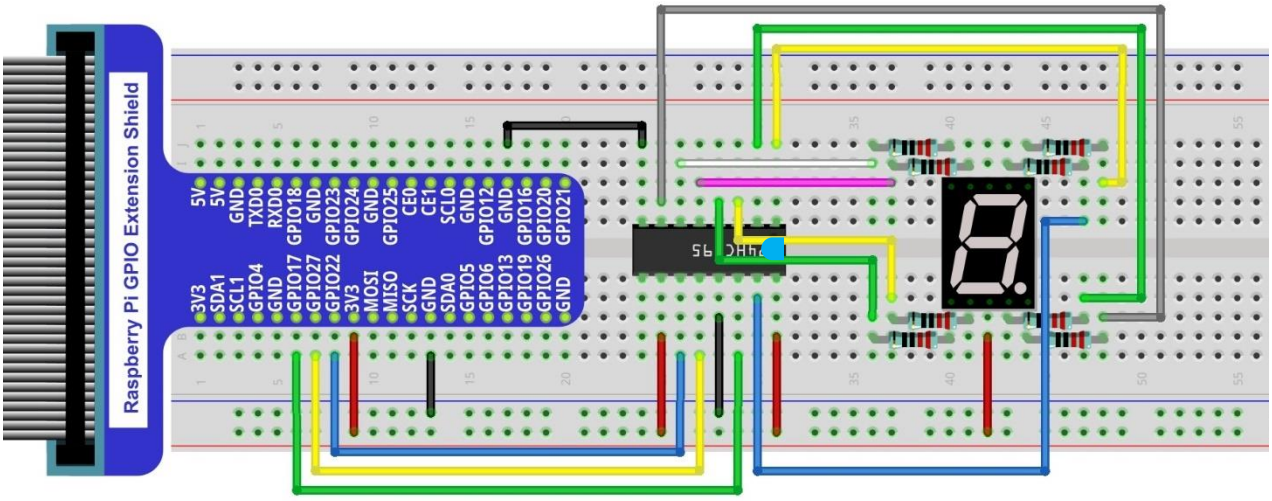
In this project, we will use a 7-Segment Display with a Common Anode. Therefore, when there is an input low level to an LED segment the LED will turn ON. Defining segment “A” as the lowest level and segment “DP” as the highest level, from high to low would look like this: “DP”, “G”, “F”, “E”, “D”, “C”, “B”, “A”. Character "0" corresponds to the code: 1100 0000b=0xc0.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/KSE0LdyuOFM

Code

This code uses a 74HC595 IC Chip to control the 7-Segment Display. The use of the 74HC595 IC Chip is generally the same throughout this Tutorial. We need code to display the characters “0” to “F” one character at a time, and then output to display them with the 74HC595 IC Chip.

Python Code 18.1.1 SevenSegmentDisplay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 18.1.1_SevenSegmentDisplay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/18.1.1_SevenSegmentDisplay
```

2. Use Python command to execute Python code “SevenSegmentDisplay.py”.

```
python SevenSegmentDisplay.py
```

After the program is executed, the 7-Segment Display starts to display the characters “0” to “F” in succession.

The following is the program code:

```
from gpiozero import OutputDevice
import time

LSBFIRST = 1
MSBFIRST = 2
# define the pins for 74HC595
dataPin   = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
# SevenSegmentDisplay display the character "0"- "F" successively
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def loop():
    while True:
        for i in range(0,len(num)):
            latchPin.off()
            shiftOut(MSBFIRST,num[i])  # Send serial data to 74HC595
            latchPin.on()
            time.sleep(0.5)
        for i in range(0,len(num)):
```

```
            latchPin.off()
            shiftOut(MSBFIRST,num[i]&0x7f) # Use "&0x7f" to display the decimal point.
            latchPin.on()
            time.sleep(0.5)

def destroy():
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

First, we need to create encoding for characters "0" to "F" in the array.

```
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

In the "for" loop of loop() function, use the 74HC595 IC Chip to output contents of array "num" successively. SevenSegmentDisplay can then correctly display the corresponding characters. Pay attention to this in regard to shiftOut function, the transmission bit, flag bit amd highest bit will be transmitted preferentially.

```
        for i in range(0,len(num)):
            latchPin.off()
            shiftOut(MSBFIRST,num[i]) #Output the figures and the highest level is transfered
preferentially.
            latchPin.on()
            time.sleep(0.5)
```

If you want to display the decimal point, make the highest bit of each array "0", which can be implemented easily by num[i]&0x7f.

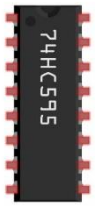
```
            shiftOut(MSBFIRST,num[i]&0x7f) # Use "&0x7f" to display the decimal point.
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

Project 18.2 4-Digit 7-Segment Display

Now, let's try to control more-than-one digit displays by using a Four 7-Segment Display in one project.

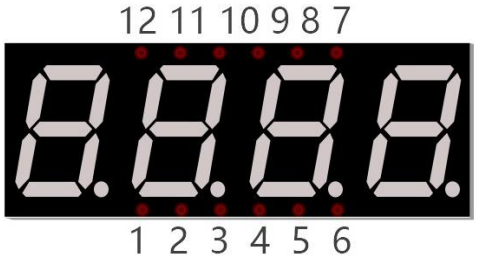
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Wire x1 Breadboard x1			Jumper Wire x30 	
74HC595 x1 	PNP transistor x4 	4-Digit 7-Segment Display x1 	Resistor 220Ω x8 	Resistor 1KΩ x4

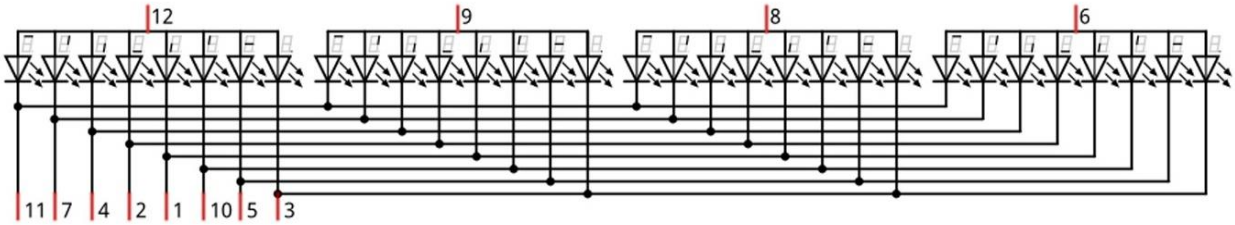
Component knowledge

4 Digit 7-Segment Display

A 4 Digit 7-segment display integrates four 7-Segment Displays into one module, therefore it can display more characters. All of the LEDs contained have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



The internal electronic circuit is shown below, and all 8 LED cathode pins of each 7-Segment Display are connected together.

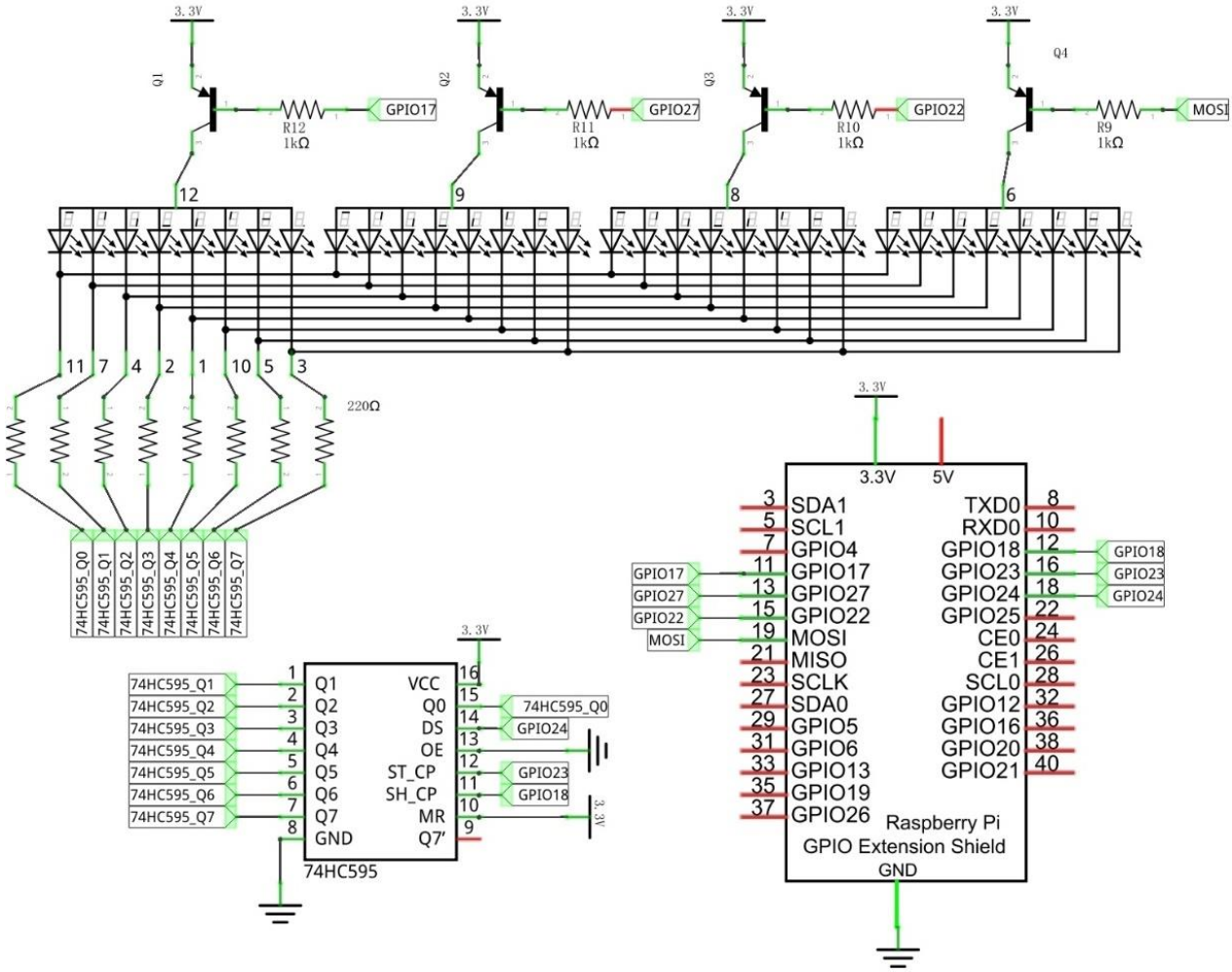


Display method of 4 Digit 7-segment display is similar to 1 Digit 7-segment display. The difference between them is that the 4-Digit displays each Digit is visible in turn, one by one and not together. We need to first send high level to the common end of the first Digit Display, and send low level to the remaining three common ends, and then send content to 8 LED cathode pins of the first Digit Display. At this time, the first 7-Segment Display will show visible content and the remaining three will be OFF.

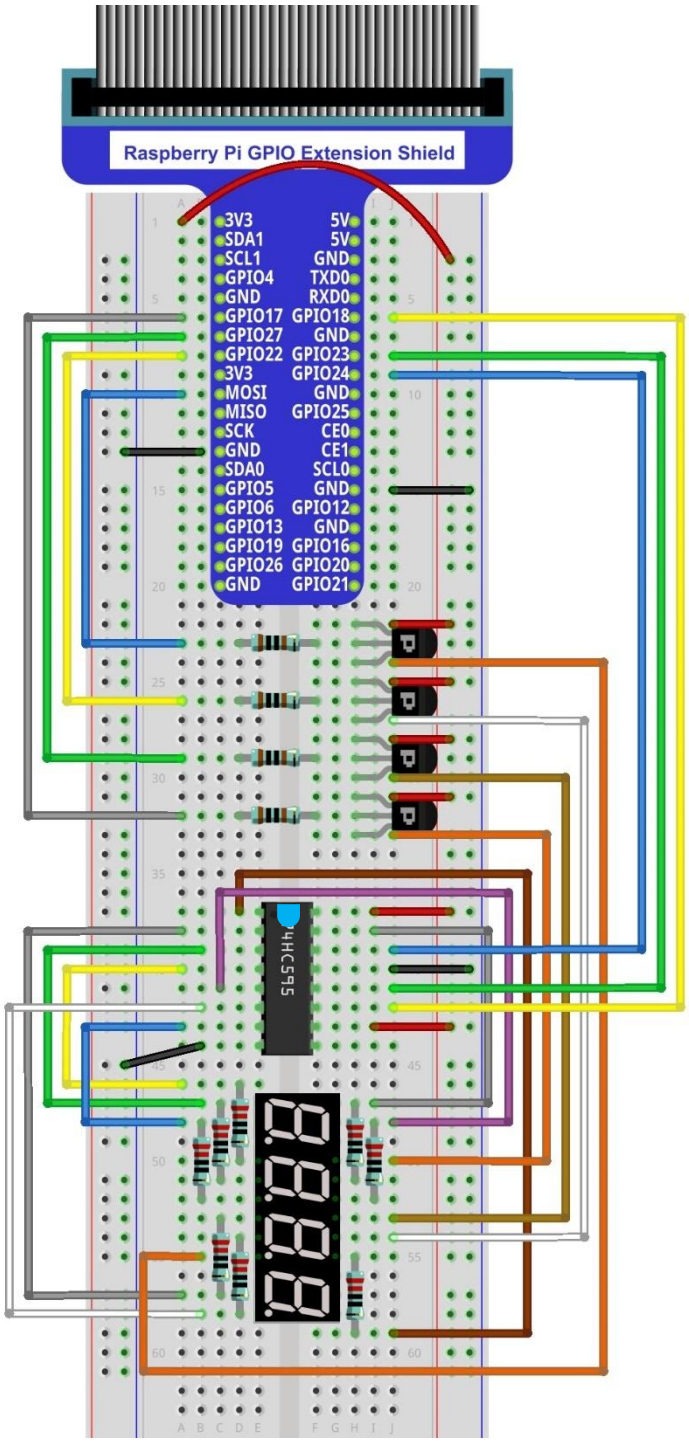
Similarly, the second, third and fourth 7-Segment Displays will show visible content in turn by scanning the display. Although the four number characters are displayed in turn separately, this process is so very fast that it is unperceivable to the naked eye. This is due to the principle of optical afterglow effect and the vision persistence effect in human sight. This is how we can see all 4 number characters at the same time. However, if each number character is displayed for a longer period, you will be able to see that the number characters are displayed separately.

Circuit

Schematic diagram



Hardware connection



Code

In this code, we use the 74HC595 IC Chip to control the 4-Digit 7-Segment Display, and use the dynamic scanning method to show the changing number characters.

Python Code 18.2.1 StopWatch

This code uses the four step four pat mode to drive the Stepper Motor clockwise and reverse direction.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 16.1.1_SteppingMotor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/18.2.1_StopWatch
```

2. Use python command to execute code "StopWatch.py".

```
python StopWatch.py
```

After the program is executed, 4-Digit 7-segment start displaying a four-digit number dynamically, and the will plus 1 in each successive second.

The following is the program code:

```
from gpiozero import OutputDevice
import time
import threading

LSBFIRST = 1
MSBFIRST = 2
# define the pins connect to 74HC595
dataPin   = OutputDevice (24)      # DS Pin of 74HC595
latchPin  = OutputDevice (23)      # ST_CP Pin of 74HC595
clockPin  = OutputDevice (18)      # SH_CP Pin of 74HC595
num = (0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90)
digitPin = (17, 27, 22, 10)     # Define the pin of 7-segment display common end
outputs = list(map(lambda pin: OutputDevice(pin), digitPin))
counter = 0         # Variable counter, the number will be dislayed by 7-segment display
t = 0               # define the Timer object

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def outData(data):      # function used to output data for 74HC595
    latchPin.off()
    shiftOut(MSBFIRST,data)
    latchPin.on()
```

```
30
31  def selectDigit(digit): # Open one of the 7-segment display and close the remaining three, the
32  parameter digit is optional for 1,2,4,8
33      outputs[0].off() if ((digit&0x08) == 0x08) else outputs[0].on()
34      outputs[1].off() if ((digit&0x04) == 0x04) else outputs[1].on()
35      outputs[2].off() if ((digit&0x02) == 0x02) else outputs[2].on()
36      outputs[3].off() if ((digit&0x01) == 0x01) else outputs[3].on()
37
38  def display(dec):   # display function for 7-segment display
39      outData(0xff)   # eliminate residual display
40      selectDigit(0x01)   # Select the first, and display the single digit
41      outData(num[dec%10])
42      time.sleep(0.003)   # display duration
43      outData(0xff)
44      selectDigit(0x02)   # Select the second, and display the tens digit
45      outData(num[dec%100//10])
46      time.sleep(0.003)
47      outData(0xff)
48      selectDigit(0x04)   # Select the third, and display the hundreds digit
49      outData(num[dec%1000//100])
50      time.sleep(0.003)
51      outData(0xff)
52      selectDigit(0x08)   # Select the fourth, and display the thousands digit
53      outData(num[dec%10000//1000])
54      time.sleep(0.003)
55  def timer():
56      global counter
57      global t
58      t = threading.Timer(1.0,timer)      # reset time of timer to 1s
59      t.start()                           # Start timing
60      counter+=1
61      print ("counter : %d"%counter)
62
63  def loop():
64      global t
65      global counter
66      t = threading.Timer(1.0,timer)      # set the timer
67      t.start()                           # Start timing
68      while True:
69          display(counter)                # display the number counter
70
71  def destroy():
72      global t
73      dataPin.close()
```

```
    latchPin.close()
    clockPin.close()
    t.cancel()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...' )
    try:
        loop()
    except KeyboardInterrupt:   # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

First, define the pin of 74HC595 and 7-segment display common end, character encoding and a variable "counter" to be displayed counter.

```
# define the pins connect to 74HC595
dataPin   = OutputDevice (24)      # DS Pin of 74HC595
latchPin  = OutputDevice (23)      # ST_CP Pin of 74HC595
clockPin  = OutputDevice (18)      # SH_CP Pin of 74HC595
num = (0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90)
digitPin = (17,27,22,10)     # Define the pin of 7-segment display common end
outputs = list(map(lambda pin: OutputDevice(pin), digitPin))
counter = 0         # Variable counter, the number will be dislayed by 7-segment display
```

Subfunction **selectDigit** (digit) function is used to open one of the 7-segment display and close the other 7-segment display, where the parameter digit value can be 1,2,4,8. Using "|" can open a number of 7-segment display.

```
def selectDigit(digit): # Open one of the 7-segment display and close the remaining three, the parameter digit is optional for 1,2,4,8
    outputs[0].off() if ((digit&0x08) == 0x08) else outputs[0].on()
    outputs[1].off() if ((digit&0x04) == 0x04) else outputs[1].on()
    outputs[2].off() if ((digit&0x02) == 0x02) else outputs[2].on()
    outputs[3].off() if ((digit&0x01) == 0x01) else outputs[3].on()
```

Subfunction **outData** (data) is used to make the 74HC595 output an 8-bit data immediately.

```
def outData(data):      # function used to output data for 74HC595
    latchPin.off()
    shiftOut(MSBFIRST,data)
    latchPin.on()
```

Subfunction **display** (int dec) is used to make a 4-Digit 7-Segment Display a 4-bit integer. First open the common end of first 7-Segment Display Digit and turn OFF the other three Digits, now it can be used as 1-Digit 7-Segment Display. The first Digit is used for displaying single digits of "dec", the second Digit is for tens, the third for hundreds and fourth for thousands respectively. Each digit will be displayed for a period by using **delay** (). The time in this code is very brief, so you will a mess of Digits. If the time is set long enough, you will see that every digit is displayed independently.

```
def display(dec):   #display function for 7-segment display
    outData(0xff)   #eliminate residual display
    selectDigit(0x01)   #Select the first, and display the single digit
```

```
    outData(num[dec%10])
    time.sleep(0.003)    #display duration
    outData(0xff)
    selectDigit(0x02)    #Select the second, and display the tens digit
    outData(num[dec%100/10])
    time.sleep(0.003)
    outData(0xff)
    selectDigit(0x04)    #Select the third, and display the hundreds digit
    outData(num[dec%1000/100])
    time.sleep(0.003)
    outData(0xff)
    selectDigit(0x08)    #Select the fourth, and display the thousands digit
    outData(num[dec%10000/1000])
    time.sleep(0.003)
```

Subfunction **timer** () is the timer callback function. When the time is up, this function will be executed. Accompanied by the execution, the variable counter will be added 1, and then reset the time of timer to 1s. 1s later, the function will be executed again.

```
def timer():        #timer function
    global counter
    global t
    t = threading.Timer(1.0,timer)      #reset time of timer to 1s
    t.start()                           #Start timing
    counter+=1
    print ("counter : %d"%counter)
```

Subfunction **setup**(), configure all input output modes for the GPIO pin used.

Finally, in loop function, make the digital tube display variable counter value in the while loop. The value will change in function **timer** (), so the content displayed by 7-segment display will change accordingly.

```
def loop():
    global t
    global counter
    t = threading.Timer(1.0,timer)      # set the timer
    t.start()                           #Start timing
    while True:
        display(counter)                #display the number counter
```

After the program is executed, press "Ctrl+C", then subfunction destroy() will be executed, and GPIO resources and timers will be released in this subfunction.

```
def destroy():
    global t
    dataPin.close()
    latchPin.close()
    clockPin.close()
    t.cancel()
```

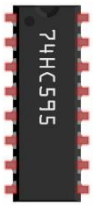
Chapter 19 74HC595 & LED Matrix

Thus far we have learned how to use the 74HC595 IC Chip to control the Bar Graph LED and the 7-Segment Display. We will now use 74HC595 IC Chips to control an LED Matrix.

Project 19.1 LED Matrix

In this project, we will use two 74HC595 IC chips to control a monochrome (one color) (8X8) LED Matrix to make it display both simple graphics and characters.

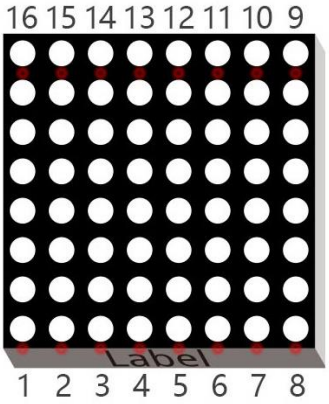
Component List

<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper x36
 </td></tr>
<tr><td>74HC595 x2
  </td><td>8X8 LEDMatrix x1
 </td><td>Resistor 220Ω x8
 </td></tr>
</table>

Component knowledge

LED matrix

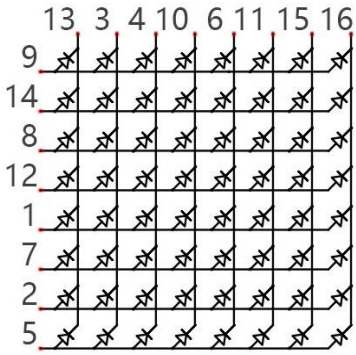
An LED Matrix is a rectangular display module that consists of a uniform grid of LEDs. The following is an 8X8 monochrome (one color) LED Matrix containing 64 LEDs (8 rows by 8 columns).



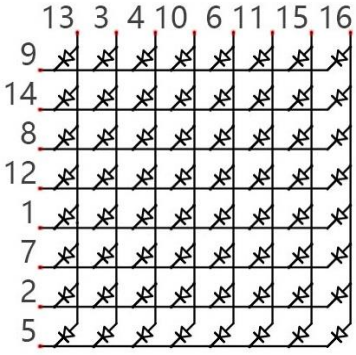
In order to facilitate the operation and reduce the number of ports required to drive this component, the Positive Poles of the LEDs in each row and Negative Poles of the LEDs in each column are respectively connected together inside the LED Matrix module, which is called a Common Anode. There is another arrangement type. Negative Poles of the LEDs in each row and the Positive Poles of the LEDs in each column are respectively connected together, which is called a Common Cathode.

The LED Matrix that we use in this project is a Common Anode LED Matrix.

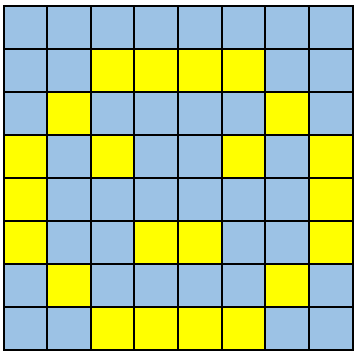
Connection mode of Common Anode



Connection mode of Common Cathode



Here is how a Common Anode LED Matrix works. First, choose 16 ports on RPI board to connect to the 16 ports of LED Matrix. Configure one port in columns for low level, which makes that column the selected port. Then configure the eight port in the row to display content in the selected column. Add a delay value and then select the next column that outputs the corresponding content. This kind of operation by column is called Scan. If you want to display the following image of a smiling face, you can display it in 8 columns, and each column is represented by one byte.



1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	1	0	1
1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

Column	Binary	Hexadecimal
1	0001 1100	0x1c
2	0010 0010	0x22
3	0101 0001	0x51
4	0100 0101	0x45
5	0100 0101	0x45
6	0101 0001	0x51
7	0010 0010	0x22
8	0001 1100	0x1c

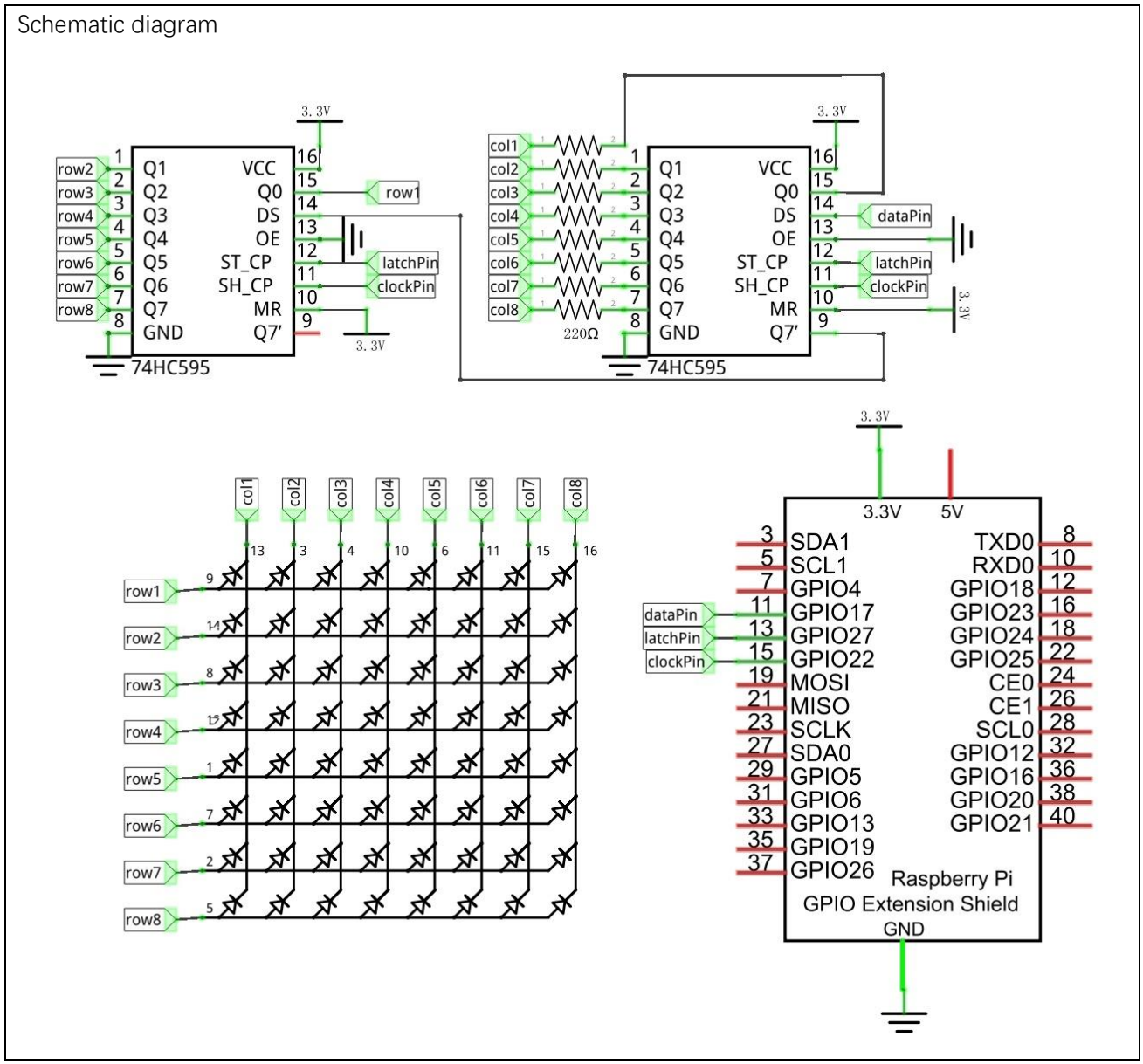
To begin, display the first column, then turn off the first column and display the second column. (and so on) .... turn off the seventh column and display the 8th column, and then start the process over from the first column again like the control of LED Bar Graph project. The whole process will be repeated rapidly in a loop. Due to the principle of optical afterglow effect and the vision persistence effect in human sight, we will see a picture of a smiling face directly rather than individual columns of LEDs turned ON one column at a time (although in fact this is the reality we cannot perceive).

Scanning rows is another option to display on an LED Matrix (dot matrix grid). Whether scanning by row or column, 16 GPIO is required. In order to save GPIO ports of control board, two 74HC595 IC Chips are used in the circuit. Every 74HC595 IC Chip has eight parallel output ports, so two of these have a combined total of 16 ports, which is just enough for our project. The control lines and data lines of the two 74HC595 IC Chips are not all connected to the RPi, but connect to the Q7 pin of first stage 74HC595 IC Chip and to the data pin of second IC Chip. The two 74HC595 IC Chips are connected in series, which is the same as using one "74HC595 IC Chip" with 16 parallel output ports.

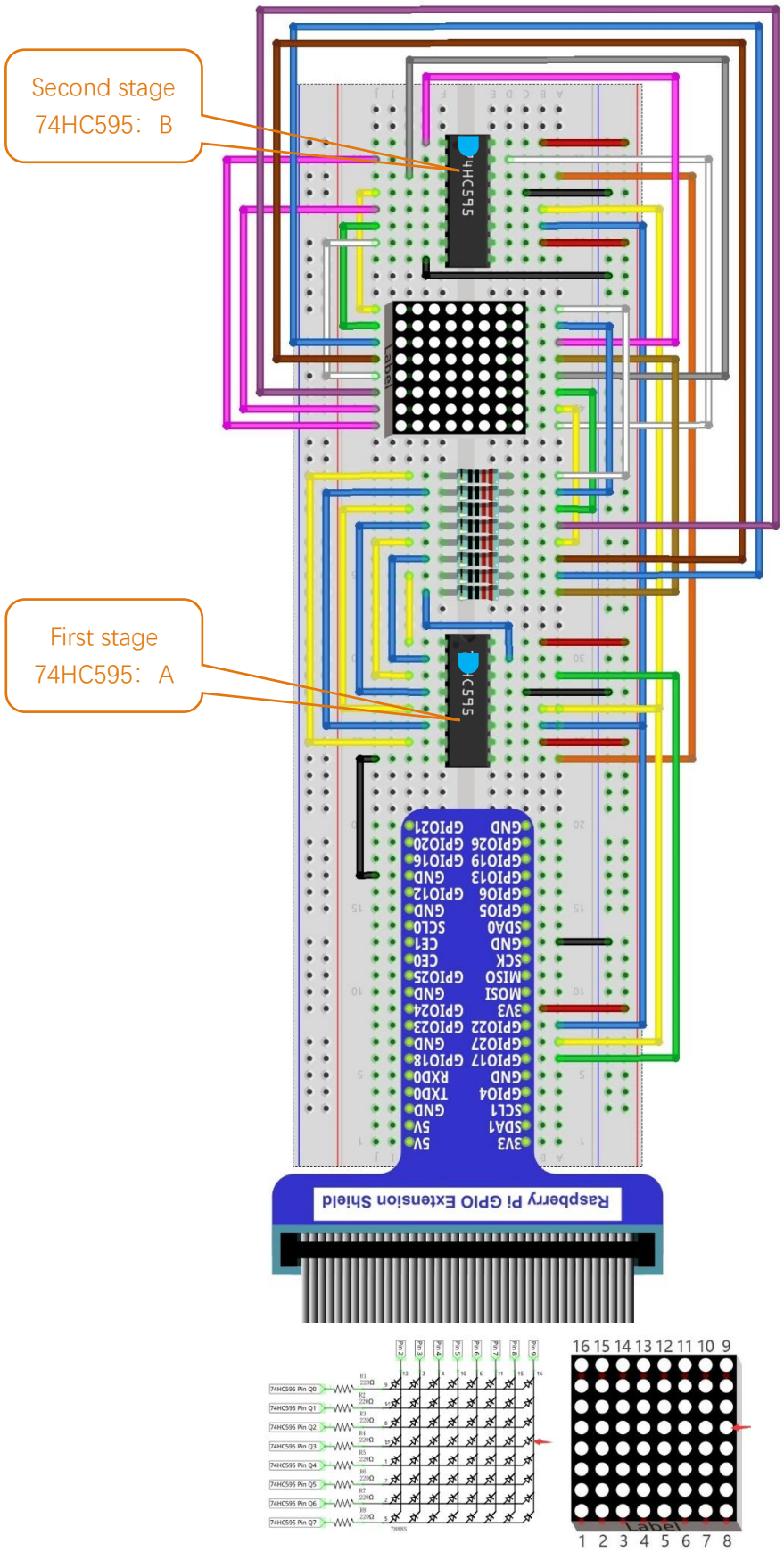
Circuit

In circuit of this project, the power pin of the 74HC595 IC Chip is connected to 3.3V. It can also be connected to 5V to make LED Matrix brighter.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/KSE0LdyuOFM

Code

Two 74HC595 IC Chips are used in this project, one for controlling the LED Matrix's columns and the other for controlling the rows. According to the circuit connection, row data should be sent first, then column data. The following code will make the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

Python Code 19.1.1 LEDMatrix

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 19.1.1_LEDMatrix directory of Python language.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/19.1.1_LEDMatrix
```

2. Use Python command to execute Python code "LEDMatrix.py".

```
python LEDMatrix.py
```

After the program is executed, the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

The following is the program code:

```
from gpiozero import OutputDevice
import time

LSBFIRST = 1
MSBFIRST = 2
dataPin   = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
pic = [0x1c,0x22,0x51,0x45,0x45,0x51,0x22,0x1c]  # data of smiling face
data = [     # data of "0-F"
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x3E, 0x00, 0x00, # "0"
    0x00, 0x00, 0x21, 0x7F, 0x01, 0x00, 0x00, 0x00, # "1"
    0x00, 0x00, 0x23, 0x45, 0x49, 0x31, 0x00, 0x00, # "2"
    0x00, 0x00, 0x22, 0x49, 0x49, 0x36, 0x00, 0x00, # "3"
    0x00, 0x00, 0x0E, 0x32, 0x7F, 0x02, 0x00, 0x00, # "4"
    0x00, 0x00, 0x79, 0x49, 0x49, 0x46, 0x00, 0x00, # "5"
    0x00, 0x00, 0x3E, 0x49, 0x49, 0x26, 0x00, 0x00, # "6"
    0x00, 0x00, 0x60, 0x47, 0x48, 0x70, 0x00, 0x00, # "7"
    0x00, 0x00, 0x36, 0x49, 0x49, 0x36, 0x00, 0x00, # "8"
    0x00, 0x00, 0x32, 0x49, 0x49, 0x3E, 0x00, 0x00, # "9"
    0x00, 0x00, 0x3F, 0x44, 0x44, 0x3F, 0x00, 0x00, # "A"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x36, 0x00, 0x00, # "B"
    0x00, 0x00, 0x3E, 0x41, 0x41, 0x22, 0x00, 0x00, # "C"
    0x00, 0x00, 0x7F, 0x41, 0x41, 0x3E, 0x00, 0x00, # "D"
    0x00, 0x00, 0x7F, 0x49, 0x49, 0x41, 0x00, 0x00, # "E"
    0x00, 0x00, 0x7F, 0x48, 0x48, 0x40, 0x00, 0x00, # "F"
```

```
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
]

def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def loop():
    while True:
        for j in range(0,500): # Repeat enough times to display the smiling face a period of time
            x=0x80
            for i in range(0,8):
                latchPin.off()
                shiftOut(MSBFIRST,pic[i]) #first shift data of line information to first stage 74HC959
                shiftOut(MSBFIRST,~x) #then shift data of column information to second stage 74HC959
                latchPin.on()# Output data of two stage 74HC595 at the same time
                time.sleep(0.001) # display the next column
                x>>=1
        for k in range(0,len(data)-8): #len(data) total number of "0-F" columns
            for j in range(0,20): # times of repeated displaying LEDMatrix in every frame, the bigger the "j", the longer the display time.
                x=0x80      # Set the column information to start from the first column
                for i in range(k,k+8):
                    latchPin.off()
                    shiftOut(MSBFIRST,data[i])
                    shiftOut(MSBFIRST,~x)
                    latchPin.on()
                    time.sleep(0.001)
                    x>>=1
def destroy():
    dataPin.close()
    latchPin.close()
    clockPin.close()
if __name__ == '__main__':  # Program entrance
    print ('Program is starting...' )
    try:
```

```
65          loop()
66      except KeyboardInterrupt:   # Press ctrl-c to end the program.
67          destroy()
68          print("Ending program")
```

The first "for" loop in the "while" loop is used to display a static smile. Displaying column information from left to right, one column at a time with a total of 8 columns. This repeats 500 times to ensure sufficient display time.

```
        for j in range(0,500): # Repeat enough times to display the smiling face a period of time
            x=0x80
            for i in range(0,8):
                latchPin.off()
                shiftOut(MSBFIRST,pic[i]) #first shift data of line information to first stage 74HC959
                shiftOut(MSBFIRST,~x) #then shift data of column information to second stage 74HC959
                latchPin.on()# Output data of two stage 74HC595 at the same time
                time.sleep(0.001) # display the next column
                x>>=1
```

The second "for" loop is used to display scrolling characters "0 to F", for a total of 18 X 8 = 144 columns. Displaying the 0-8 column, then the 1-9 column, then the 2-10 column..... and so on…138-144 column in consecutively to achieve the scrolling effect. The display of each frame is repeated a certain number of times and the more repetitions, the longer the single frame display will be and the slower the scrolling movement.

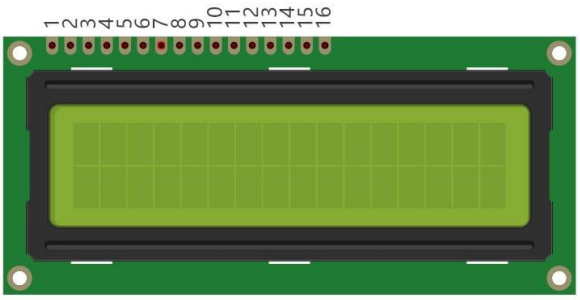
```
        for k in range(0,len(data)-8): #len(data) total number of "0-F" columns
            for j in range(0,20): # times of repeated displaying LEDMatrix in every frame, the bigger the "j", the longer the display time.
                x=0x80      # Set the column information to start from the first column
                for i in range(k,k+8):
                    latchPin.off()
                    shiftOut(MSBFIRST,data[i])
                    shiftOut(MSBFIRST,~x)
                    latchPin.on()
                    time.sleep(0.001)
                    x>>=1
```

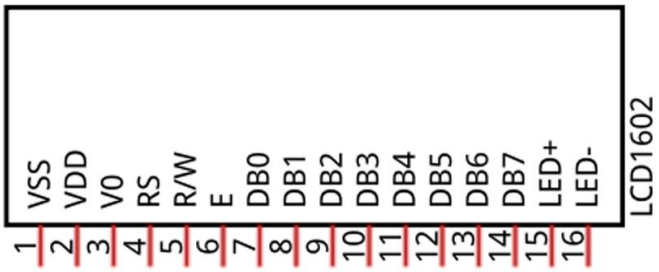
Chapter 20 LCD1602

In this chapter, we will learn about the LCD1602 Display Screen,

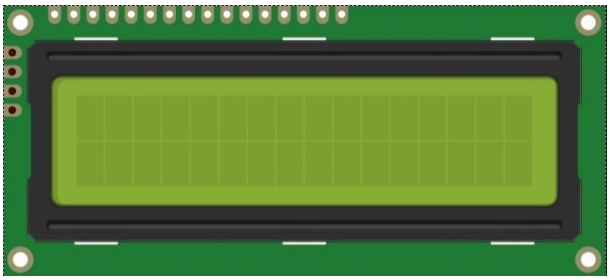
Project 20.1 I2C LCD1602

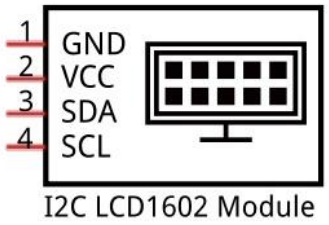
There are LCD1602 display screen and the I2C LCD. We will introduce both of them in this chapter. But what we use in this project is an I2C LCD1602 display screen. The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram





I2C LCD1602 Display Screen integrates a I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to only use 4 lines to operate the LCD1602.

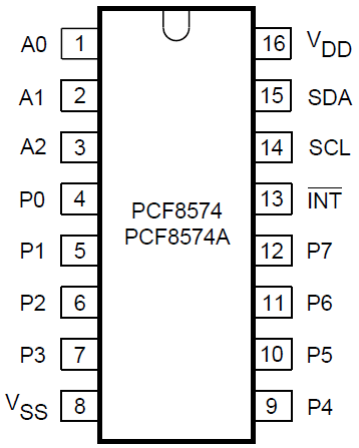




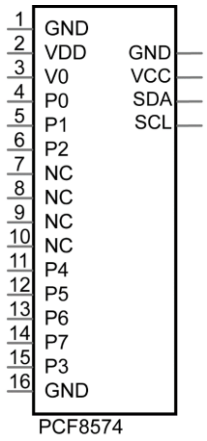
The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F). You can also view the RPI bus on your I2C device address through command "i2cdetect -y 1" (refer to the "configuration I2C" section below).

Below is the PCF8574 chip pin diagram and its module pin diagram:

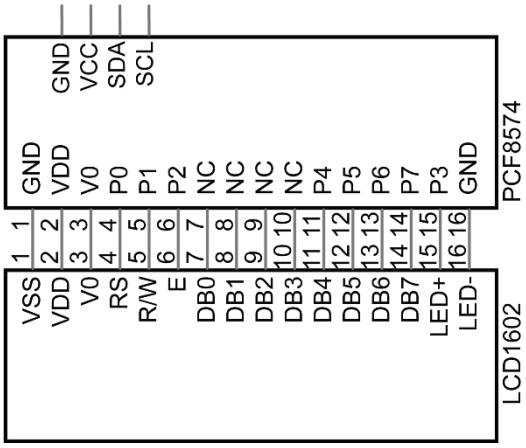
PCF8574 chip pin diagram:



PCF8574 module pin diagram



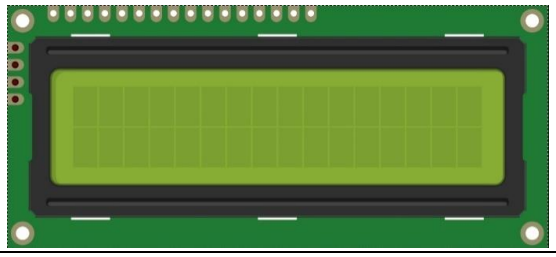
PCF8574 module pins and LCD1602 pins correspond to each other and connected to each other:



Because of this, as stated earlier, we only need 4 pins to control the16 pins of the LCD1602 Display Screen through the I2C interface.

In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

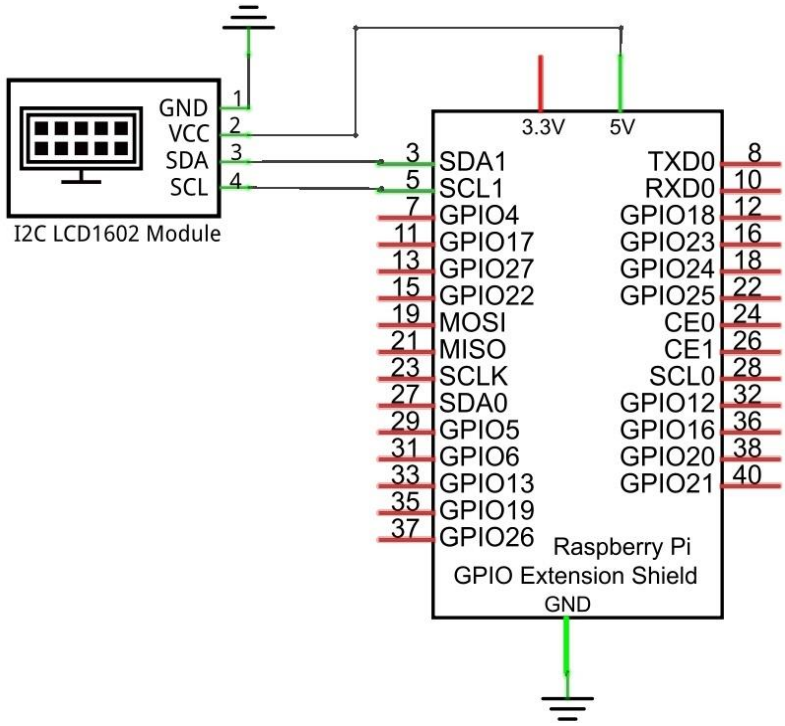
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x4
I2C LCD1602 Module x1 	

Circuit

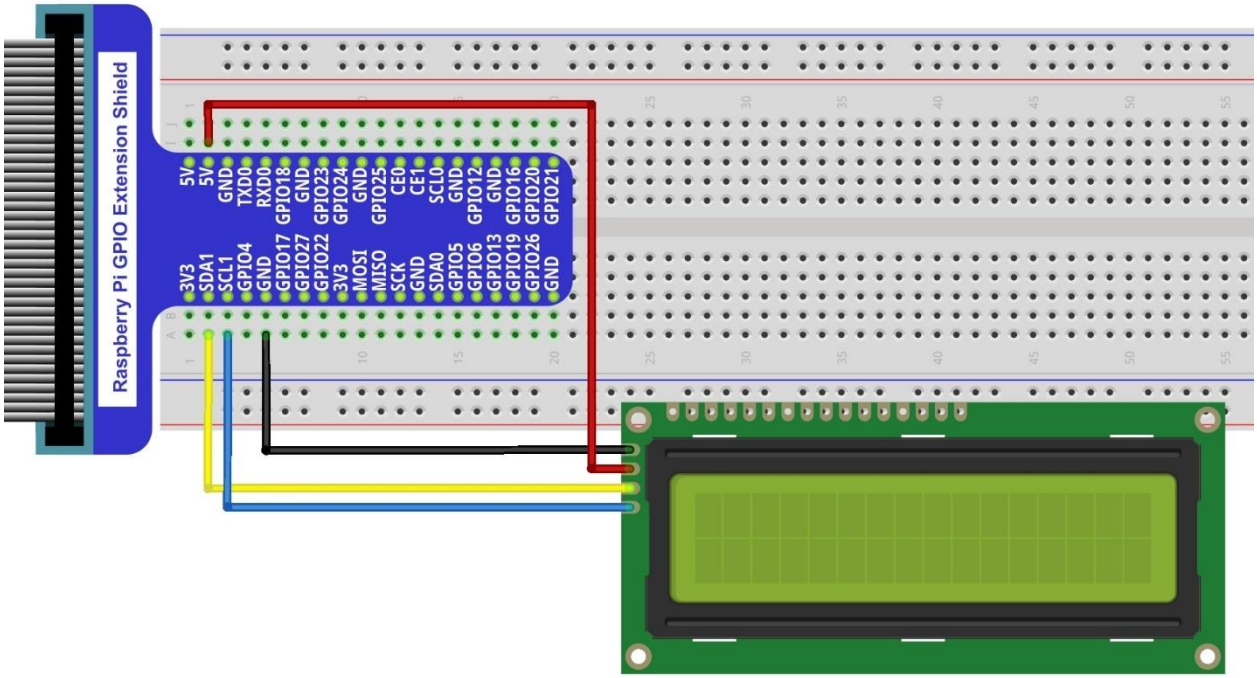
Note that the power supply for I2C LCD1602 in this circuit is 5V.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

NOTE: It is necessary to configure 12C and install Smbus first (see chapter 7 for details)



Video: https://www.youtube.com/watch?v=L1bgX2f4Yio

Code

This code will have your RPi's CPU temperature and System Time Displayed on the LCD1602.

Python Code 20.1.1 I2CLCD1602

If you did not configure I2C and install Smbus, please refer to Chapter 7. If you did, continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 20.1.1_ I2CLCD1602 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/20.1.1_I2CLCD1602
```

2. Use Python command to execute Python code "I2CLCD1602.py".

```
python I2CLCD1602.py
```

After the program is executed, the LCD1602 Screen will display your RPi's CPU Temperature and System Time.

So far, at this writing, we have two types of LCD1602 on sale. One needs to adjust the backlight, and the other does not.

The LCD1602 that does not need to adjust the backlight is shown in the figure below.

The following is the program code:

```
import smbus
from time import sleep, strftime
from datetime import datetime
from LCD1602 import CharLCD1602

lcd1602 = CharLCD1602()
def get_cpu_temp():     # get CPU temperature from file
"/sys/class/thermal/thermal_zone0/temp"
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C '

def get_time_now():     # get system time
    return datetime.now().strftime('    %H:%M:%S')

def loop():
    lcd1602.init_lcd()
    count = 0
```

```
    while(True):
        lcd1602.clear()
        lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
        lcd1602.write(0, 1, get_time_now() )   # display the time
        sleep(1)
def destroy():
    lcd1602.clear()
if __name__ == '__main__':
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```

In a while loop, set the cursor position, and display the CPU temperature and time.

```
    while(True):
        lcd1602.clear()
        lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
        lcd1602.write(0, 1, get_time_now() )   # display the time
        sleep(1)
```

CPU temperature is stored in file "/sys/class/thermal/thermal_zone0/temp". Open the file and read content of the file, and then convert it to Celsius degrees and return. Subfunction used to get CPU temperature is shown below:

```
def get_cpu_temp():     # get CPU temperature and store it into file
  "/sys/class/thermal/thermal_zone0/temp"
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C'
```

Subfunction used to get time:

```
def get_time_now():     # get the time
    return datetime.now().strftime('     %H:%M:%S')
```

Details about LCD1602.py:

Module LCD1602

This module provides the basic operation method of LCD1602, including class CharLCD1602.
Some member functions are described as follows:
def init_lcd(self,addr=None, bl=1) : LDC1602 initializes the setting. When the addr is None, the I2C address of the device will be automatically scanned. You can also specify the I2C address, bl=1 to enable the backlight setting.
def clear(self): clear the screen
def send_command(self,comm): set the cursor position
def i2c_scan(self): scan the device I2C address
def write(self,x, y, str): display contents
More information can be viewed through opening LCD1602.py.

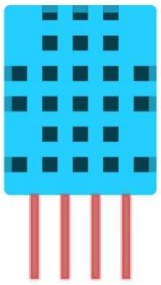
Chapter 21 Hygrothermograph DHT11

In this chapter, we will learn about a commonly used sensor called a Hygrothermograph DHT11.

Project 21.1 Hygrothermograph

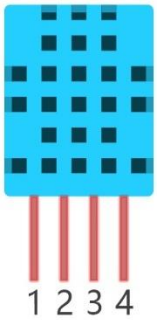
Hygrothermograph is an important tool in our lives to give us data on the temperature and humidity in our environment. In this project, we will use the RPi to read Temperature and Humidity data of the DHT11 Module.

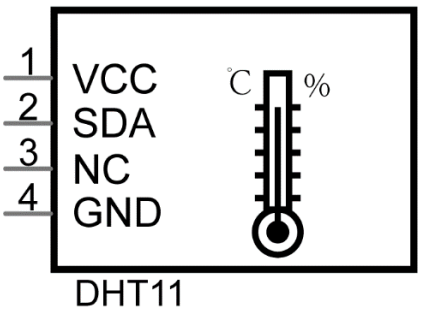
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	DHT11 x1 	Resistor 10kΩ x1
Jumper Wire x4		

Component knowledge

The Temperature & Humidity Sensor DHT11 is a compound temperature & humidity sensor, and the output digital signal has been calibrated by its manufacturer.

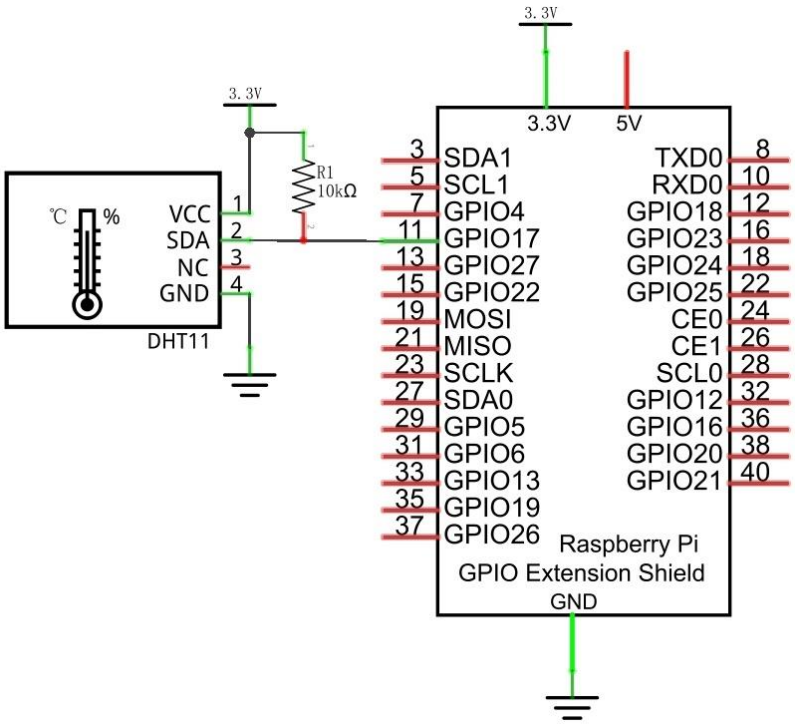




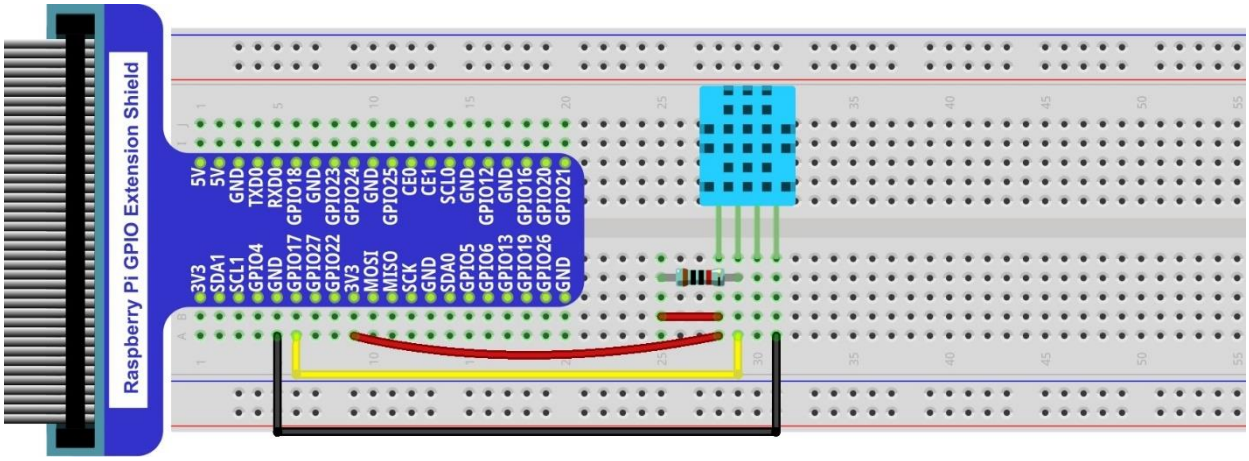
After being powered up, it will initialize in 1 second. Its operating voltage is within the range of 3.3V-5.5V. The SDA pin is a data pin, which is used to communicate with other devices.
The NC pin (Not Connected Pin) are a type of pin found on various integrated circuit packages. Those pins have no functional purpose to the outside circuit (but may have an unknown functionality during manufacture and test). Those pins **should not be connected** to any of the circuit connections.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: **support@freenove.com**



Video: https://www.youtube.com/watch?v=vTeuLWiXUTA

Code

The code is used to read the temperature and humidity data of DHT11, and display them.

Python Code 21.1.1 DHT11

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

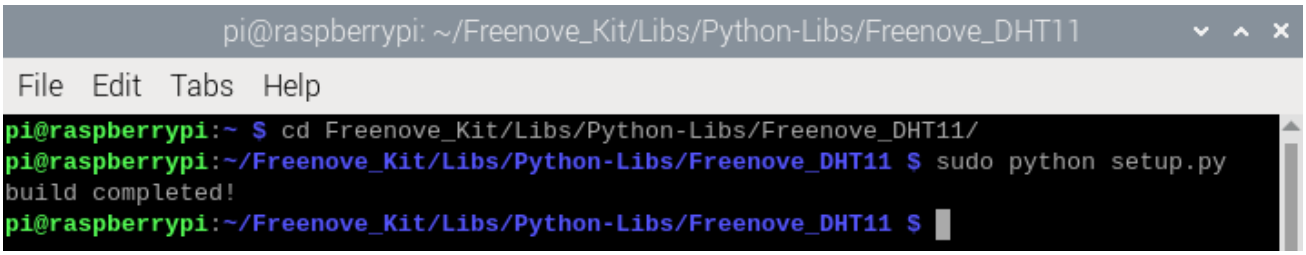
1. Use cd command to enter folder of the ADC Device library.

```
cd ~/Freenove_Kit/Libs/Python-Libs/Freenove_DHT11
```

2. Execute command below to install the library.

```
sudo python setup.py
```

A successful installation, without error prompts, is shown below:



Next, we will execute the code for this project.

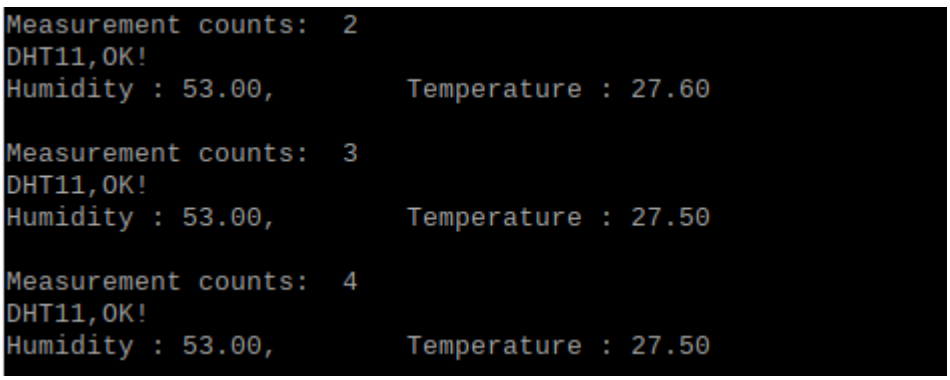
1. Use cd command to enter 21.1.1_DHT11 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/21.1.1_DHT11
```

2. Use Python command to execute code "DHT11.py".

```
python DHT11.py
```

After the program is executed, the Terminal window will display the current total number of read times, the read state, as well as temperature and humidity values as is shown below:



Since gpiozero does not support DHT11 sensors, RPi.GPIO is used here for control.

The following is the program code:

```
1  import time
2  import Freenove_DHT as DHT
3  DHTPin = 17     #define the pin of DHT11
4
5  def loop():
6      dht = DHT.DHT(DHTPin)   #create a DHT class object
7      counts = 0 # Measurement counts
```

```
    while(True):
        counts += 1
        print("Measurement counts: ", counts)
        for i in range(0,15):
            chk = dht.readDHT11()     #read DHT11 and get a return value. Then determine whether data read is normal according to the return value.
            if (chk is dht.DHTLIB_OK):      #read DHT11 and get a return value. Then determine whether data read is normal according to the return value.
                print("DHT11,OK!")
                break
            time.sleep(0.1)
        print("Humidity : %.2f, \t Temperature : %.2f \n"%( dht.getHumidity(), dht.getTemperature()))
        time.sleep(2)

if __name__ == '__main__':
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:
        exit()
```

In this project code, we use a module "**Freenove_DHT**.py", which provides the method of reading the DHT Sensor. It is located in the same directory with program files "**DHT11**.py". By using this library, we can easily read the DHT Sensor. First, we create a DHT class object in the code.

```
    dht = DHT.DHT(DHTPin)    #create a DHT class object
```

Then in the "while" loop, use chk = dht.**readDHT11**() to read the DHT11, and determine whether the data read is normal according to the return value "chk". Then use variable sumCnt to record the number of times read.

```
    while(True):
        counts += 1
        print("Measurement counts: ", counts)
        for i in range(0,15):
            chk = dht.readDHT11()     #read DHT11 and get a return value. Then determine whether data read is normal according to the return value.
            if (chk is dht.DHTLIB_OK):      #read DHT11 and get a return value. Then determine whether data read is normal according to the return value.
                print("DHT11,OK!")
                break
            time.sleep(0.1)
        print("Humidity : %.2f, \t Temperature : %.2f \n"%( dht.getHumidity(), dht.getTemperature()))
        time.sleep(2)
```

Finally display the results:

```
print("Humidity : %.2f, \t Temperature : %.2f \n"%( dht.getHumidity(), dht.getTemperature()))
```

Module "**Freenove_DHT.py**" contains a DHT class. The class function of the def **readDHT11** (pin) is used to read the DHT11 Sensor and store the temperature and humidity data read to member variables humidity and temperature.

Freenove_DHT Module

This is a Python module for reading the temperature and humidity data of the DHT Sensor. Partial functions and variables are described as follows:

getHumidity(): store humidity data read from sensor

getTemperature(): store temperature data read from sensor

readDHT11(): read the temperature and humidity of sensor DHT11, and return values used to determine whether the data is normal.

Chapter 22 Matrix Keypad

Earlier we learned about a single Push Button Switch. In this chapter, we will learn about Matrix Keyboards, which integrates a number of Push Button Switches as Keys for the purposes of Input.

Project 22.1 Matrix Keypad

In this project, we will attempt to get every key code on the Matrix Keypad to work.

Component List

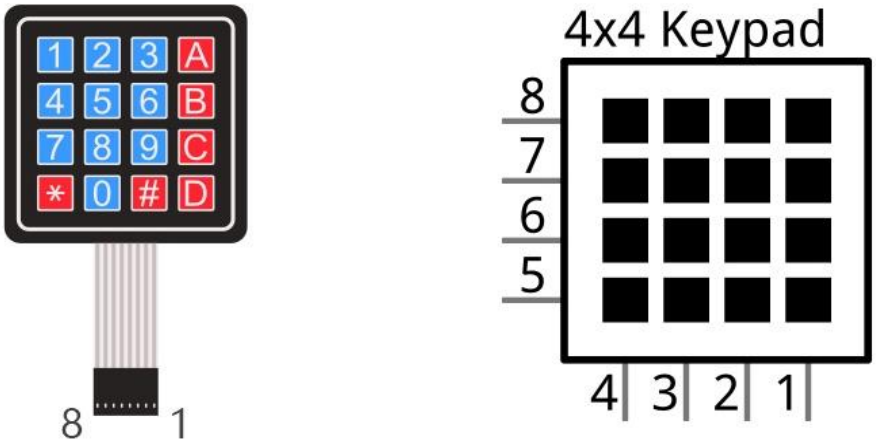
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Wire x1 Breadboard x1	4x4 Matrix Keypad x1
Jumper wire 	
Resistor 10kΩ x4 	

Component knowledge

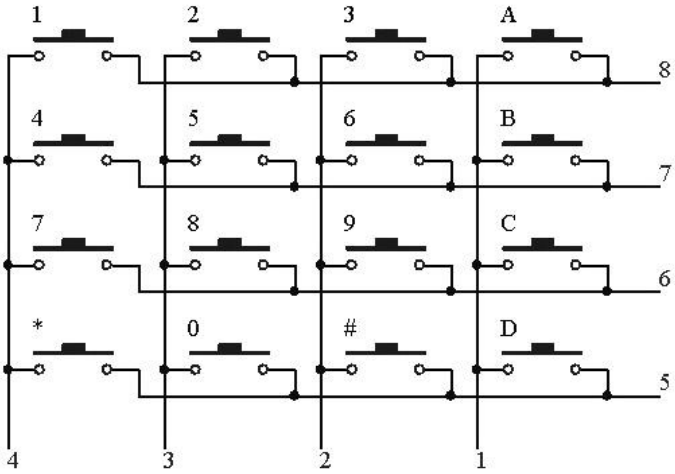
4x4 Matrix Keypad

A Keypad Matrix is a device that integrates a number of keys in one package. As is shown below, a 4x4 Keypad

Matrix integrates 16 keys (think of this as 16 Push Button Switches in one module):



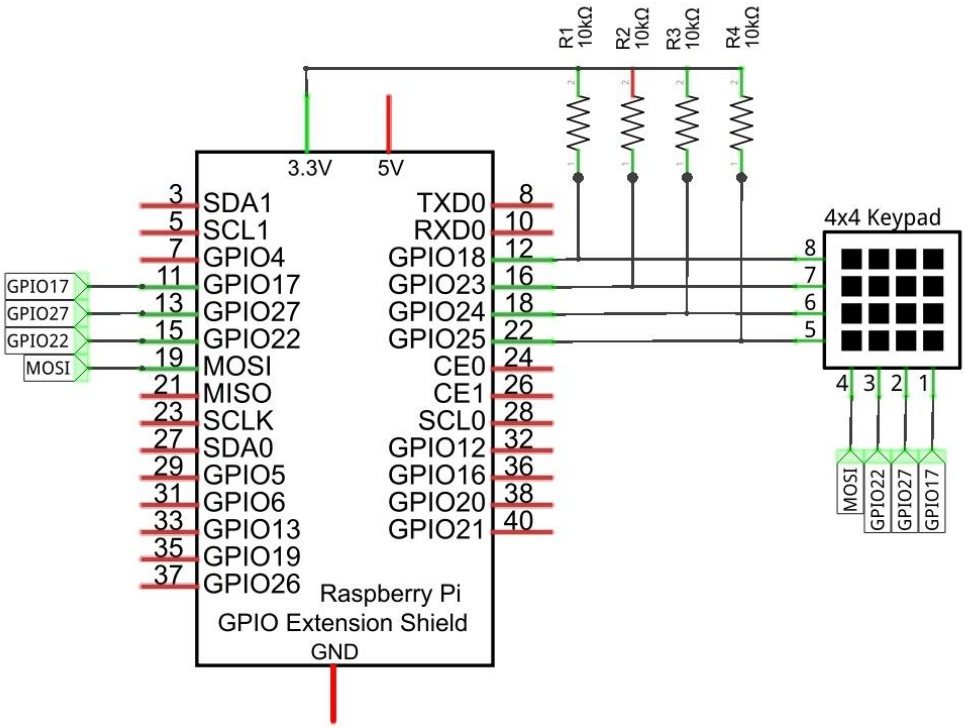
Similar to the integration of an LED Matrix, the 4x4 Keypad Matrix has each row of keys connected with one pin and this is the same for the columns. Such efficient connections reduce the number of processor ports required. The internal circuit of the Keypad Matrix is shown below.



The method of usage is similar to the Matrix LED, by using a row or column scanning method to detect the state of each key's position by column and row. Take column scanning method as an example, send low level to the first 1 column (Pin1), detect level state of row 5, 6, 7, 8 to judge whether the key A, B, C, D are pressed. Then send low level to column 2, 3, 4 in turn to detect whether other keys are pressed. Therefore, you can get the state of all of the keys.

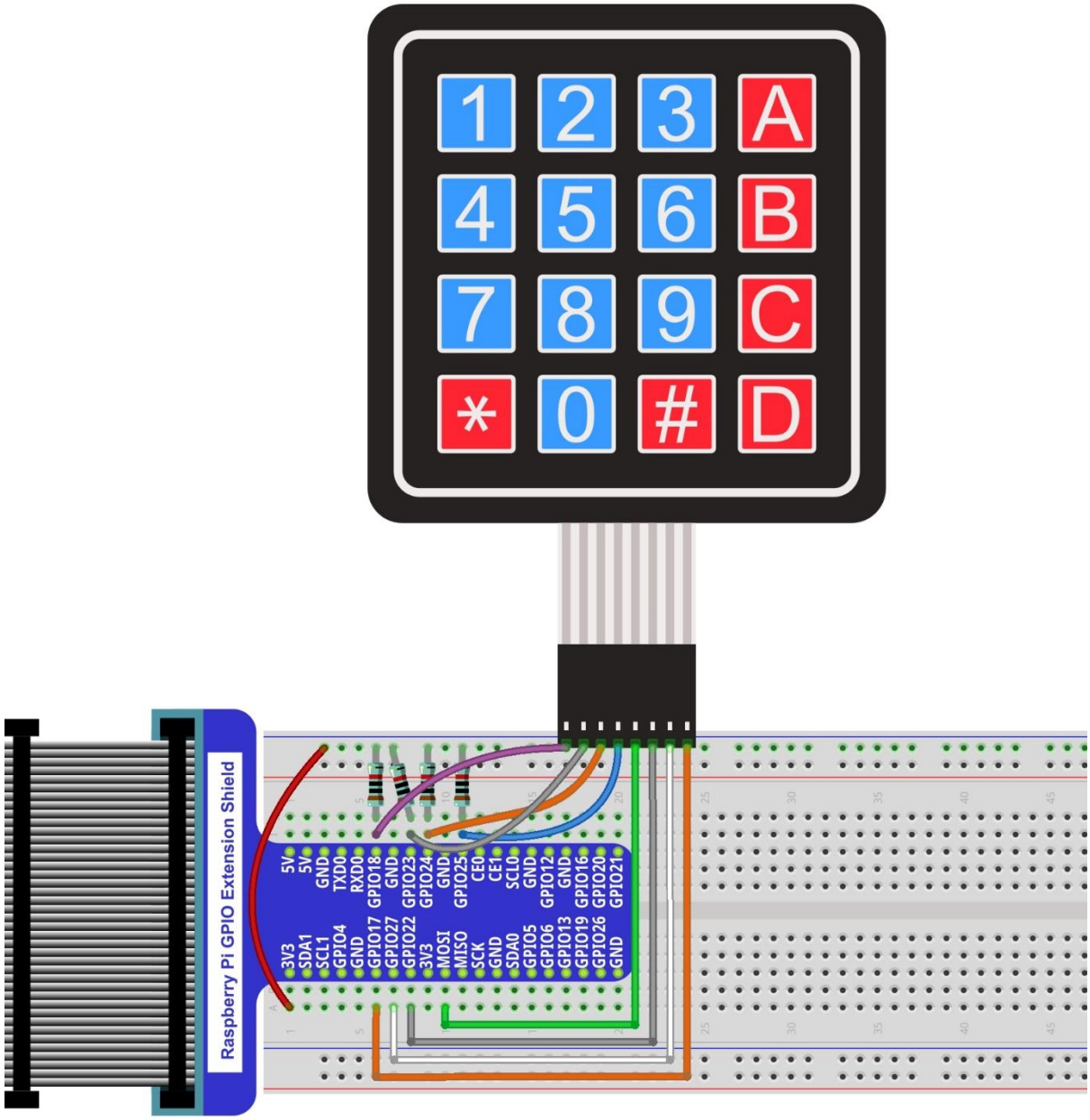
Circuit

Schematic diagram



Video: https://www.youtube.com/watch?v=RPluhXAYhKQ

Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

This code is used to obtain all key codes of the 4x4 Matrix Keypad, when one of the keys is pressed, the key code will be displayed in the terminal window.

Python Code 22.1.1 MatrixKeypad

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

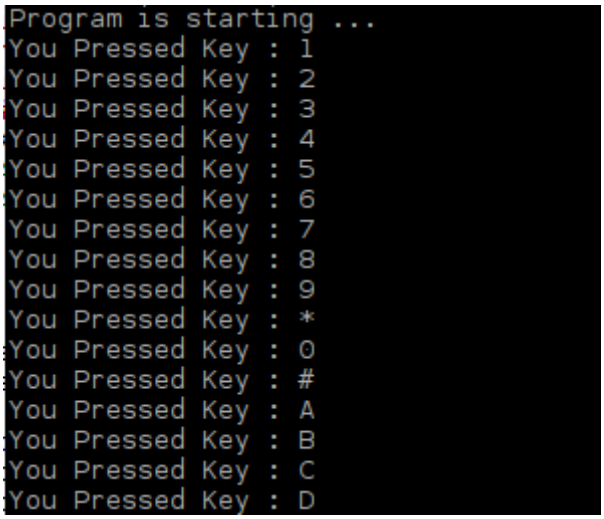
1. Use cd command to enter 22.1.1_MatrixKeypad directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/22.1.1_MatrixKeypad
```

2. Use Python command to execute code "MatrixKeypad.py".

```
python MatrixKeypad.py
```

After the program is executed, pressing any key on the MatrixKeypad, will display the corresponding key code on the Terminal. As is shown below:



The following is the program code:

```
import Keypad   #import module Keypad
ROWS = 4        # number of rows of the Keypad
COLS = 4        #number of columns of the Keypad
keys =  [   '1','2','3','A',     #key code
            '4','5','6','B',
            '7','8','9','C',
            '*','0','#','D'      ]
rowsPins = [18, 23, 24, 25]     #connect to the row pinouts of the keypad
colsPins = [10, 22, 27, 17]     #connect to the column pinouts of the keypad
def loop():
    keypad = Keypad.Keypad(keys,rowsPins,colsPins,ROWS,COLS)     #creat Keypad object
    keypad.setDebounceTime(50)       #set the debounce time
    while(True):
        key = keypad.getKey()        #obtain the state of keys
        if(key != keypad.NULL):      #if there is key pressed, print its key code.
            print ("You Pressed Key : %c "%(key))
```

```
17
18  if __name__ == '__main__':     #Program start from here
19      print ("Program is starting ... ")
20      try:
21          loop()
22      except KeyboardInterrupt:  #When 'Ctrl+C' is pressed, exit the program.
23          print("Ending program")
```

In this project code, we use a custom module "**Keypad**.py", which is located in the same directory with program file "**MatrixKeypad**.py". And this library file, which is transplanted from Arduino function library Keypad, provides a method to read the keyboard. By using this library, we can easily read the matrix keyboard. First, import module Keypad. Then define the information of the matrix keyboard used in this project: the number of rows and columns, code of each key and GPIO pin connected to each column and each row.

```
import Keypad   #import module Keypad
ROWS = 4        # number of rows of the Keypad
COLS = 4        #number of columns of the Keypad
keys =  [   '1','2','3','A',     #key code
            '4','5','6','B',
            '7','8','9','C',
            '*','0','#','D'      ]
rowsPins = [18, 23, 24, 25]     #connect to the row pinouts of the keypad
colsPins = [26, 22, 27, 17]     #connect to the column pinouts of the keypad
```

Then, based on the above information, initiates a Keypad class object to operate the Matrix Keyboard.

```
keypad = Keypad.Keypad(keys,rowsPins,colsPins,ROWS,COLS)
```

Set the debounce time to 50ms, and this value can be set based on the actual characteristics of the keyboard's flexibly, with a default time of 10ms.

```
keypad.setDebounceTime(50)
```

In the "while" loop, use the function key= keypad.**getKey** () to read the keyboard constantly. If there is a key pressed, its key code will be stored in the variable "key", and then be displayed.

```
    while(True):
        key = keypad.getKey()       #get the state of keys
        if(key != keypad.NULL):     # if a key is pressed, print out its key code
            print ("You Pressed Key : %c "%(key))
```

The Keypad Library used for the RPi is “transplanted” from the Arduino Keypad Library. The source files is written by language C++ and translated into Python can be obtained by visiting http://playground.arduino.cc/Code/Keypad. As for the “transplanted” function library, the function and method of all classes, functions, variables, etc. are the same as the original library. Partial contents of the Keypad Library are described below:

class Keypad

```
def __init__(self,usrKeyMap,row_Pins,col_Pins,num_Rows,num_Cols):
```
Constructed function, the parameters are: key code of keyboard, row pin, column pin, the number of rows, the number of columns.

```
def getKey(self):
```
Get a pressed key. If no key is pressed, the return value is keypad NULL.

```
def setDebounceTime(self,ms):
```
Set the debounce time. And the default time is 10ms.

```
def setHoldTime(self,ms):
```
Set the time when the key holds stable state after pressed.

```
def isPressed(keyChar):
```
Judge whether the key with code "keyChar" is pressed.

```
def waitForKey():
```
Wait for a key to be pressed, and return key code of the pressed key.

```
def getState():
```
Get state of the keys.

```
def keyStateChanged():
```
Judge whether there is a change of key state, then return True or False.

For More information about Keypad, please visit: http://playground.arduino.cc/Code/Keypad or through the opening file "Keypad.py".

For more information about the methods used by the InputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#inputdevice

For more information about the methods used by the OutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

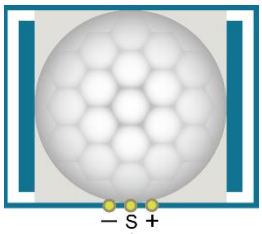
Chapter 23 Infrared Motion Sensor

In this chapter, we will learn a widely used sensor, Infrared Motion Sensor.

Project 23.1 PIR Infrared Motion Detector with LED Indicator

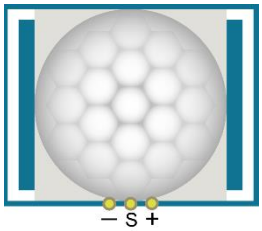
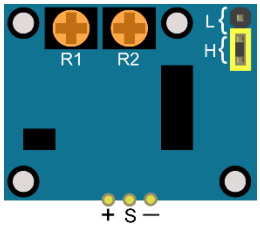
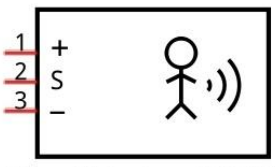
In this project, we will make a Motion Detector, with the human body infrared pyroelectric sensors.
When someone is in close proximity to the Motion Detector, it will automatically light up and when there is no one close by, it will be out.
This Infrared Motion Sensor can detect the infrared spectrum (heat signatures) emitted by living humans and animals.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x5	
HC SR501 x1 	LED x1	Resistor 220Ω x1

Component Knowledge

The following is the diagram of the Infrared Motion Sensor（HC SR-501）a PIR Sensor:

Top	Bottom	Schematic
		 Infrared Motion Sensor(HC SR501)

Description:

1. Working voltage: 5v-20v(DC) Static current: 65uA.
2. Automatic Trigger. When a living body enters into the active area of sensor, the module will output high

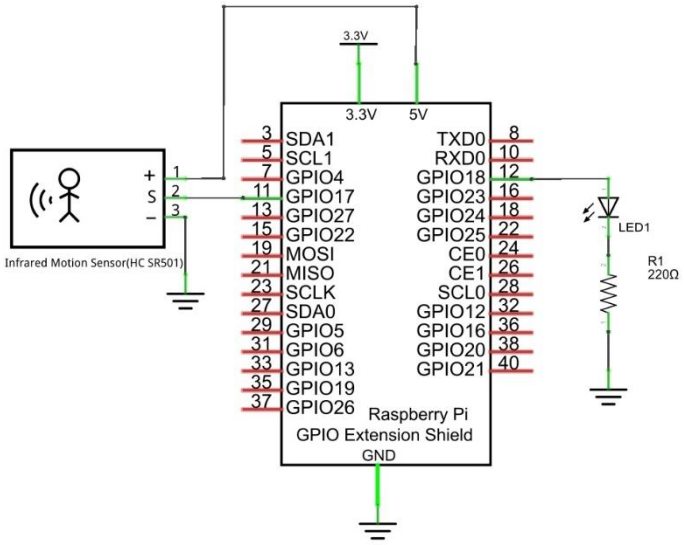
level (3.3V). When the body leaves the sensor's active detection area, it will output high level lasting for time period T, then output low level(0V). Delay time T can be adjusted by the potentiometer R1.

3. Induction block time: the induction will stay in block condition and does not induce external signal at lesser time intervals (less than delay time) after outputting high level or low level
4. Initialization time: the module needs about 1 minute to initialize after being powered ON. During this period, it will alternately output high or low level.
5. One characteristic of this sensor is when a body moves close to or moves away from the sensor's dome edge, the sensor will work at high sensitively. When a body moves close to or moves away from the sensor's dome in a vertical direction (perpendicular to the dome), the sensor cannot detect well (please take note of this deficiency). Actually this makes sense when you consider that this sensor is usually placed on a celling as part of a security product. Note: The Sensing Range (distance before a body is detected) is adjusted by the potentiometer.

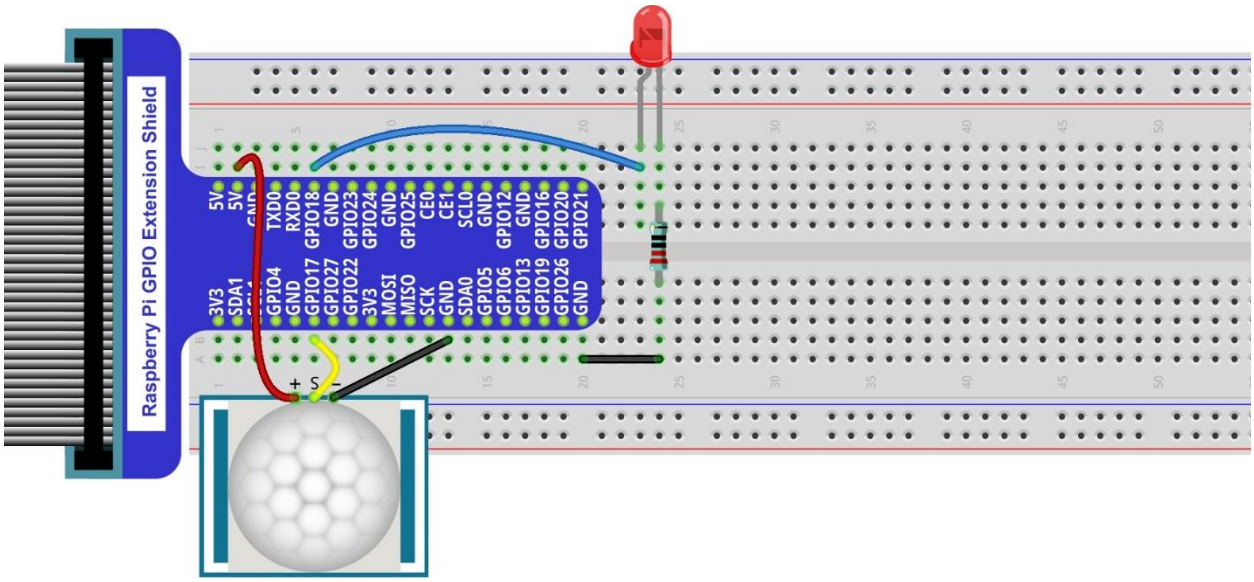
We can regard this sensor as a simple inductive switch when in use.

Circuit

Schematic diagram

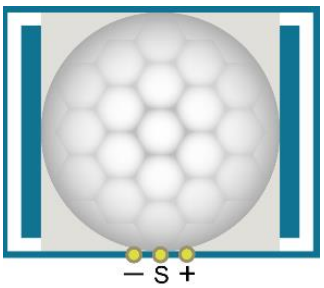


Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

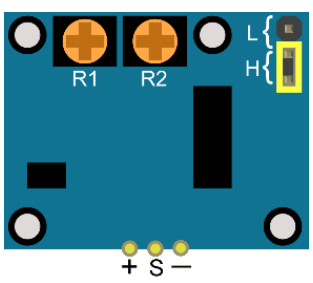


How to use this sensor?

Top



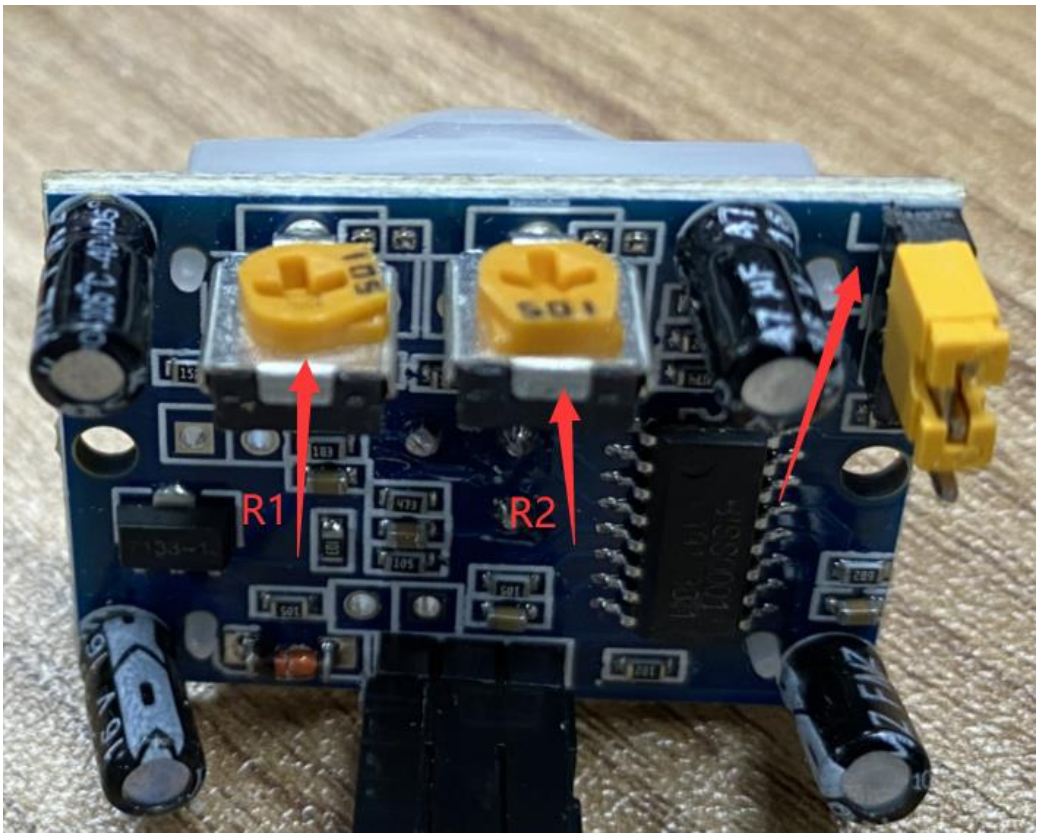
Bottom



Description:

1. You can choose non-repeatable trigger modes or repeatable modes.
   L: non-repeatable trigger mode. The module output high level after sensing a body, then when the delay time is over, the module will output low level. During high level time, the sensor no longer actively senses bodies.
   H: repeatable trigger mode. The distinction from the L mode is that it can sense a body until that body leaves. After this, it starts to time and output low level after delaying T time.
2. R1 is used to adjust HIGH level lasting time when sensor detects human motion, 1.2s-320s.
3. R2 is used to adjust the maxmum distance the sensor can detect, 3~5m.

Here we connect L and adjust R1 and R2 like below to do this project.
Put you hand close and away from the sensor slowly. Obsever the LED in previous circuit.
It need some time between two detections.



Video: https://www.youtube.com/watch?v=0GerHIINURk

Code

In this project, we will use the Infrared Motion Sensor to trigger an LED, essentially making the Infrared Motion sensor act as a Motion Switch. Therefore, the code is very similar to the earlier project "Push Button Switch and LED". The difference is that, when Infrared Motion Sensor detects change, it will output high level; when button is pressed, it will output low level. When the sensor output high level, the LED turns ON, or it will turn OFF.

Python Code 23.1.1 SenseLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 22.1.1_MatrixKeypad directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/23.1.1_SenseLED
```

2. Use Python command to execute code "SenseLED.py".

```
python SenseLED.py
```

After the program is executed, **wait 1 minute for initialization**. Then move away from or move closer to the Infrared Motion Sensor and observe whether the LED turns ON or OFF. The Terminal window will continuously display the state of LED. As is shown below:

The following is the program code:

```
from gpiozero import LED,MotionSensor
import time

ledPin = 18       # define ledPin
sensorPin = 17    # define sensorPin
led    = LED(ledPin)
sensor = MotionSensor(sensorPin)
sensor.wait_for_no_motion()
def loop():
    # Variables to hold the current and last states
    currentstate = False
    previousstate = False
    while True:
        # Read sensor state
        currentstate = sensor.motion_detected
        # If the sensor is triggered
        if currentstate == True and previousstate == False:
            led.on()
            print("Motion detected!led turned on >>>")
            # Record previous state
            previousstate = True
        # If the sensor has returned to ready state
        elif currentstate == False and previousstate == True:
            led.off()
            print("No Motion!led turned off <<")
            previousstate = False
        # Wait for 10 milliseconds
```

```
28          time.sleep(0.01)
29
30  def destroy():
31      led.close()
32      sensor.close()
33
34  if __name__ == '__main__':     # Program entrance
35      print ('Program is starting...')
36      try:
37          loop()
38      except KeyboardInterrupt:  # Press ctrl-c to end the program.
39          destroy()
40          print("Ending program")
```

For more information about the methods used by the MotionSensor class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#motionsensor-d-sun-pir

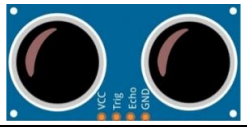
Chapter 24 Ultrasonic Ranging

In this chapter, we learn a module which use ultrasonic to measure distance.

Project 24.1 Ultrasonic Ranging

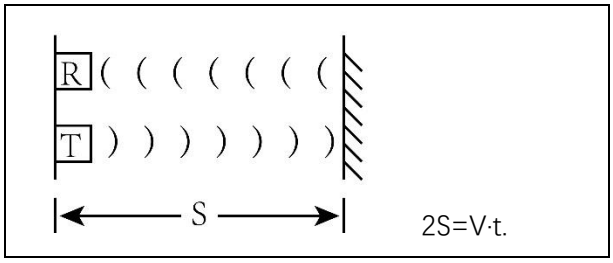
In this project, we use ultrasonic ranging module to measure distance, and print out the data in the terminal.

Component List

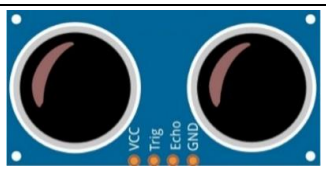
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Ultrasonic Module x1 
Jumper Wire x4 	Resistor 1kΩ x3

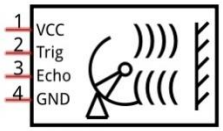
Component Knowledge

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will be reflected when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (delta) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about v=340m/s, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: s=vt/2.



The Ultrasonic Ranging Module integrates a both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the Ultrasonic Ranging Module are shown below:





Pin description:

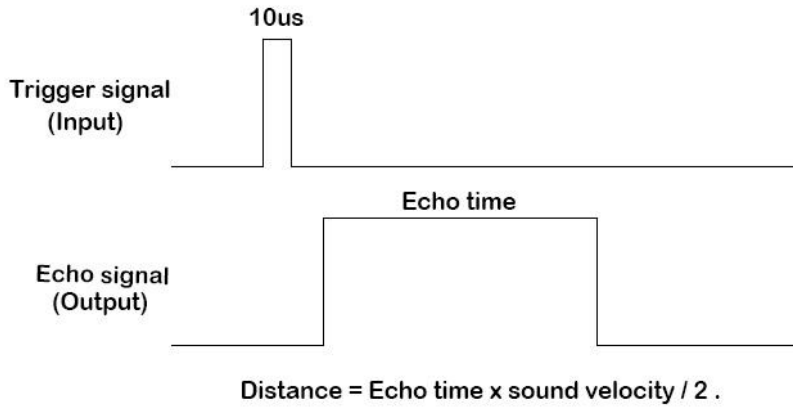
VCC	power supply pin
Trig	trigger pin
Echo	Echo pin
GND	GND

Technical specs:

Working voltage: 5V Working current: 12mA

Minimum measured distance: 2cm Maximum measured distance: 200cm

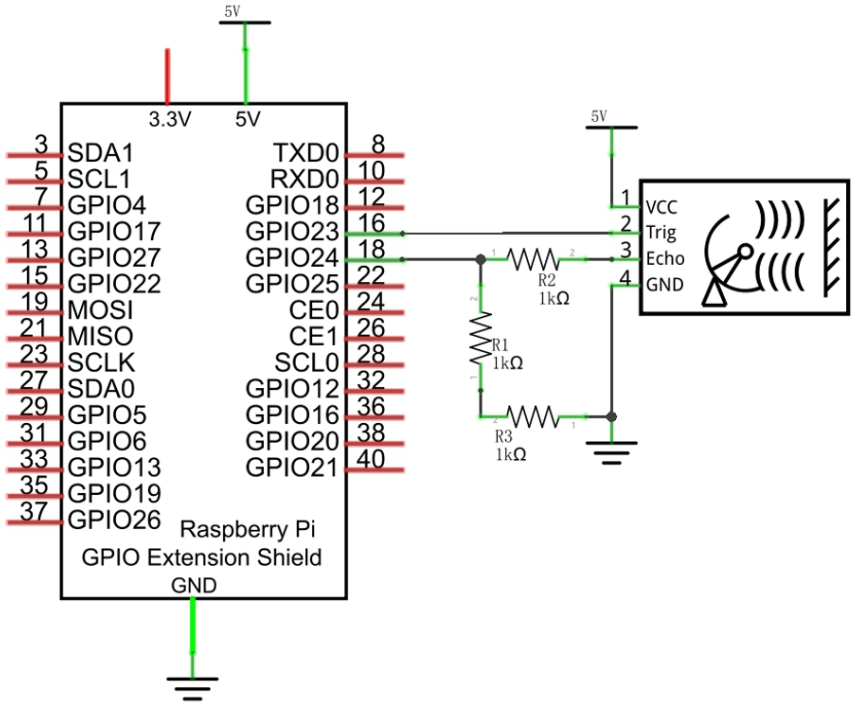
Instructions for Use: output a high-level pulse in Trig pin lasting for least 10uS, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, s=vt/2. This is done constantly.



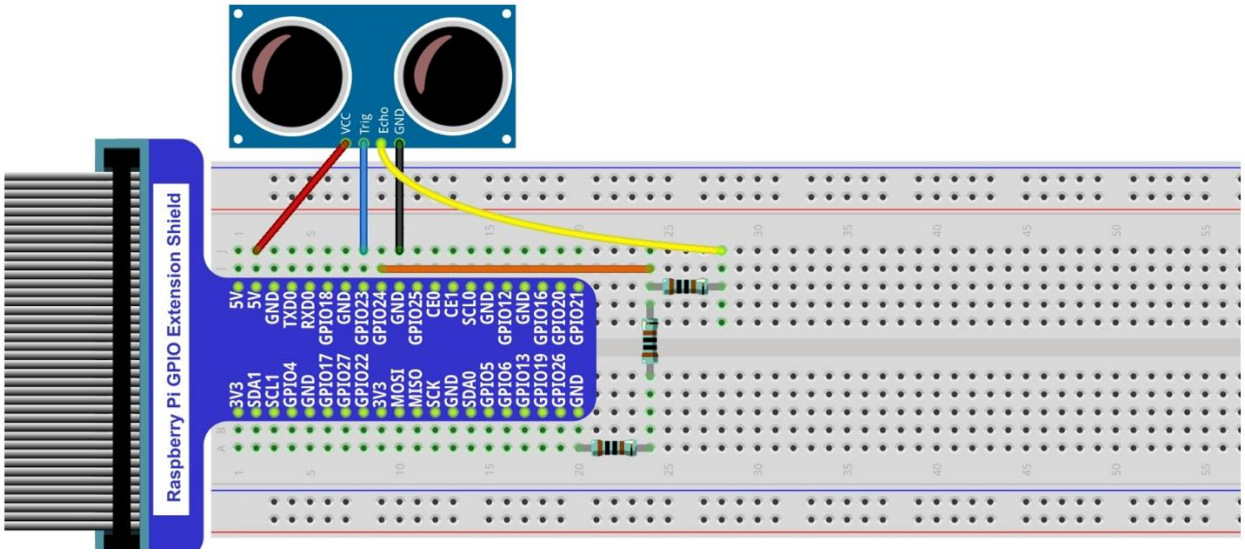
Circuit

Note that the voltage of ultrasonic module is 5V in this circuit.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: **support@freenove.com**



Video: https://www.youtube.com/watch?v=6nffzlN-IUE

Code

Python Code 24.1.1 UltrasonicRanging

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 24.1.1_UltrasonicRanging directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/24.1.1_UltrasonicRanging
```

2. Use Python command to execute code "UltrasonicRanging.py".

```
python UltrasonicRanging.py
```

After the program is executed, aim the Ultrasonic Ranging Module's detectors ("eyes") perpendicular to the surface of an object (try using your hand). The distance between the ultrasonic module and the object will be displayed in the terminal. As is shown below:

```
The distance is : 198.75 cm
The distance is : 199.22 cm
The distance is : 198.42 cm
The distance is : 198.74 cm
The distance is : 198.37 cm
The distance is : 198.47 cm
The distance is : 198.41 cm
```

The following is the program code:

```
1  from gpiozero import DistanceSensor
2  from time import sleep
3
4  trigPin = 23
5  echoPin = 24
6  sensor = DistanceSensor(echo=echoPin, trigger=trigPin ,max_distance=3)
7
8  def loop():
9      while True:
10         print('Distance: ', sensor.distance * 100,'cm')
11         sleep(1)
12
13 if __name__ == '__main__':     # Program entrance
14     print ('Program is starting...')
15     try:
16         loop()
17     except KeyboardInterrupt:  # Press ctrl-c to end the program.
18         sensor.close()
19         print("Ending program")
```

First, define the pins and the maximum measurement distance.

```
trigPin = 23
echoPin = 24
```

```
sensor = DistanceSensor(echo=echoPin, trigger=trigPin ,max_distance=3) # define the maximum measured distance 300cm
```

Finally, in the while loop of main function, get the measurement distance and display it continually.

```
def loop():
    while True:
        print('Distance: ', sensor.distance * 100,'cm')
        sleep(1)
```

For more information about the methods used by the DistanceSensor class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#distancesensor-hc-sr04

In the above experiments, you can see that the measurement data is unstable.
Note: For improved accuracy, use the pigpio pin driver rather than the default RPi.GPIO driver (pigpio uses DMA sampling for much more precise edge timing). This is particularly relevant if you're using Pi 1 or Pi Zero.
You can refer to UltrasonicRanging2.py for detailed code.

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/24.1.1_UltrasonicRanging
```

2. Use python command to execute code "Sweep.py".

```
python UltrasonicRanging2.py
```

After the program is executed, the distance between the ultrasonic module and the measured object will be displayed on the terminal. At this point, the data is more accurate and stable.
This code is based on pigpio libarary. In the latest Raspberry Pi OS, "pigpio" library has been installed. You only need to run the command to enable it.

```
sudo pigpiod
```

```
pi@raspberrypi:~ $ sudo pigpiod
pi@raspberrypi:~ $
```

If the "pigpio" library has not yet been installed, please follow the steps to install it.
Run the command to install "pigpio" library.

```
sudo apt-get update
sudo apt-get install pigpio python-pigpio python3-pigpio
```

The following is the program code:

```
1  import os
2  os.system("sudo pigpiod")
3  from gpiozero import DistanceSensor
4  from gpiozero.pins.pigpio import PiGPIOFactory
5  from time import sleep
6
7  trigPin = 23
8  echoPin = 24
9  my_factory = PiGPIOFactory()
```

```
    sensor = DistanceSensor(echo=echoPin, trigger=trigPin ,max_distance=3,pin_factory=my_factory)
10
11  def loop():
12      while True:
13          print('Distance: ', sensor.distance * 100,'cm')
14          sleep(1)
15
16  if __name__ == '__main__':     # Program entrance
17      print ('Program is starting...')
18      try:
19          loop()
20      except KeyboardInterrupt:  # Press ctrl-c to end the program.
21          sensor.close()
22          os.system("sudo killall pigpiod")
23          print("Ending program")
```

See Changing the pin factory for further information:
https://gpiozero.readthedocs.io/en/stable/api_pins.html#changing-pin-factory

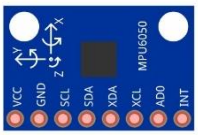
Chapter 25 Attitude Sensor MPU6050

In this chapter, we will learn about a MPU6050 Attitude sensor, which integrates an Accelerometer and Gyroscope.

Project 25.1 Read a MPU6050 Sensor Module

In this project, we will read Acceleration and Gyroscope Data of the MPU6050 Sensor.

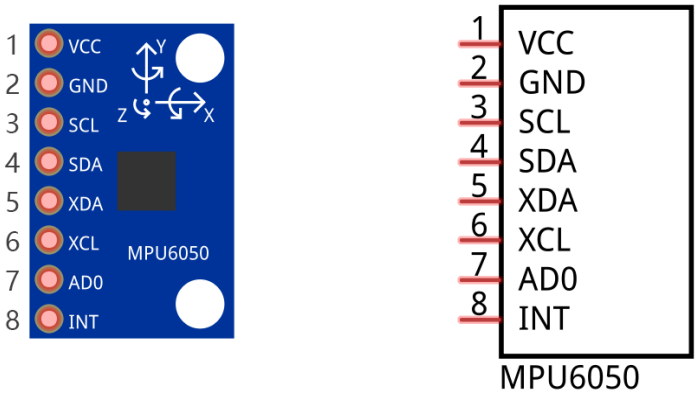
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	MPU6050 x1 
Jumper Wire x4	

Component knowledge

MPU6050

MPU6050 Sensor Module is a complete 6-axis Motion Tracking Device. It combines a 3-axis Gyroscope, a 3-axis Accelerometer and a DMP (Digital Motion Processor) all in a small package. The settings of the Accelerometer and Gyroscope of MPU6050 can be changed. A precision wide range digital temperature sensor is also integrated to compensate data readings for changes in temperature, and temperature values can also be read. The MPU6050 Module follows the I2C communication protocol and the default address is 0x68.



The port description of the MPU6050 Module is as follows:

Pin name	Pin number	Description
VCC	1	Positive pole of power supply with voltage 5V
GND	2	Negative pole of power supply
SCL	3	I2C communication clock pin
SDA	4	I2C communication data pin
XDA	5	I2C host data pin which can be connected to other devices.
XCL	6	I2C host clock pin which can be connected to other devices.
AD0	7	I2C address bit control pin. Low level: the device address is 0x68 High level: the device address is 0x69
INT	8	Output interrupt pin

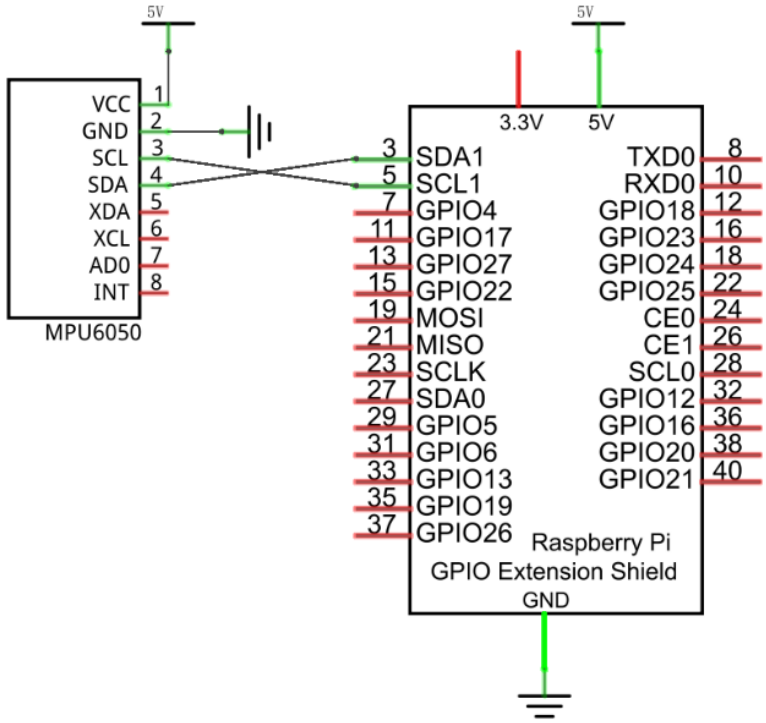
For more detail, please refer to the MPU6050 datasheet.

MPU6050 is widely used to assist with balancing vehicles, robots and aircraft, mobile phones and other products which require stability to control stability and attitude or which need to sense same.

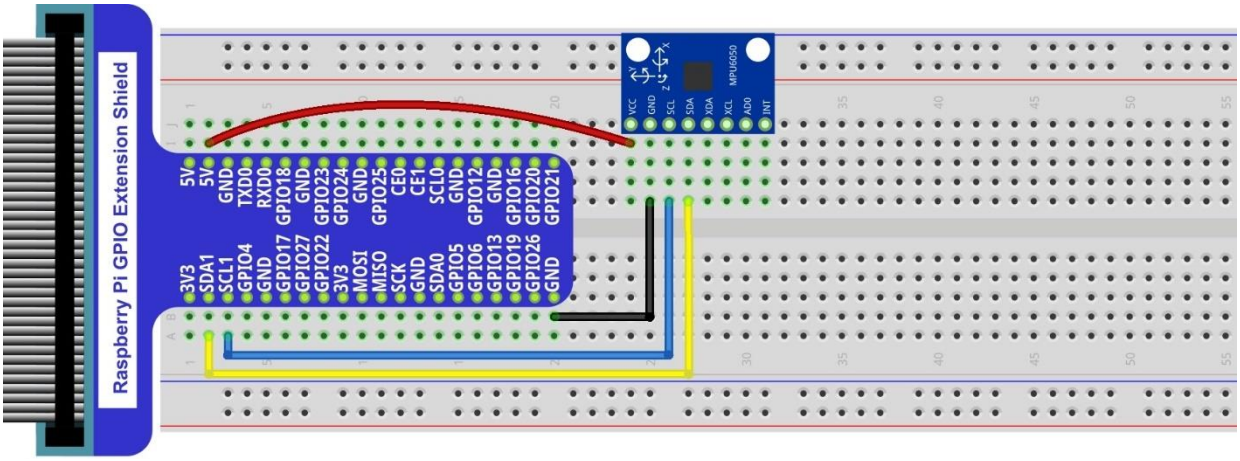
Circuit

Note that the power supply voltage for MPU6050 module is 5V in this circuit.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: **support@freenove.com**



Video: https://www.youtube.com/watch?v=BBuDYBVjJLA

Code

In this project, we will read the acceleration data and gyroscope data of MPU6050, and print them out.

Python Code 25.1.1 MPU6050RAW

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 25.1.1_MPU6050RAW directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/25.1.1_MPU6050
```

2. Use Python command to execute code "MPU6050RAW.py".

```
python MPU6050RAW.py
```

After the program is executed, the Terminal will display active accelerometer and gyroscope data of the MPU6050, as well as the conversion to gravity acceleration and angular velocity as units of data. As shown in the following figure:

```
a/g:1326        -160    16548   -48     -25     -16
a/g:0.08 g      -0.01 g 1.01 g  -0.37 d/s        -0.19 d/s       -0.12 d/s
a/g:1174        -116    15972   -44     -25     -17
a/g:0.07 g      -0.01 g 0.97 g  -0.34 d/s        -0.19 d/s       -0.13 d/s
a/g:1134        -130    16066   -45     -21     -17
a/g:0.07 g      -0.01 g 0.98 g  -0.34 d/s        -0.16 d/s       -0.13 d/s
a/g:1234        -76     15976   -45     -30     -16
a/g:0.08 g      -0.00 g 0.98 g  -0.34 d/s        -0.23 d/s       -0.12 d/s
a/g:996 -88     15748   -45     -22     -16
a/g:0.06 g      -0.01 g 0.96 g  -0.34 d/s        -0.17 d/s       -0.12 d/s
a/g:1196        -174    16182   -46     -25     -15
a/g:0.07 g      -0.01 g 0.99 g  -0.35 d/s        -0.19 d/s       -0.11 d/s
```

The following is the program code:

```
1  import MPU6050
2  import time
3
4  mpu = MPU6050.MPU6050()      #instantiate a MPU6050 class object
5  accel = [0]*3                #store accelerometer data
6  gyro = [0]*3                 #store gyroscope data
7  def setup():
8      mpu.dmp_initialize()     #initialize MPU6050
9
10 def loop():
11     while(True):
12         accel = mpu.get_acceleration()      #get accelerometer data
13         gyro = mpu.get_rotation()           #get gyroscope data
14         print("a/g:%d\t%d\t%d\t%d\t%d\t%d
15 "%(accel[0],accel[1],accel[2],gyro[0],gyro[1],gyro[2]))
16         print("a/g:%.2f g\t%.2f g\t%.2f g\t%.2f d/s\t%.2f d/s\t%.2f
17 d/s"%(accel[0]/16384.0,accel[1]/16384.0,
18             accel[2]/16384.0,gyro[0]/131.0,gyro[1]/131.0,gyro[2]/131.0))
19         time.sleep(0.1)
20
```

```
21  if __name__ == '__main__':
22      print("Program is starting ... ")
23      setup()
24      try:
25          loop()
26      except KeyboardInterrupt:
27          pass
```

A module "**MPU6050**.py" is used in the code. The module includes a class used to operate MPU6050. When used, first initiate an object.

```
    mpu = MPU6050.MPU6050()
```

In the setup function, the MPU6050 is initialized.

```
    def setup():
        mpu.dmp_initialize()
```

In the loop function, read the original data of MPU6050, display them and then convert the original data into the corresponding acceleration and angular velocity values, then display the converted data out.

```
    def loop():
        while(True):
            accel = mpu.get_acceleration()      #get accelerometer data
            gyro = mpu.get_rotation()           #get gyroscope data
            print("a/g:%d\t%d\t%d\t%d\t%d\t%d
    "%(accel[0],accel[1],accel[2],gyro[0],gyro[1],gyro[2]))
            print("a/g:%.2f g\t%.2f g\t%.2f g\t%.2f d/s\t%.2f d/s\t%.2f
    d/s"%(accel[0]/16384.0,accel[1]/16384.0,
                accel[2]/16384.0,gyro[0]/131.0,gyro[1]/131.0,gyro[2]/131.0))
            time.sleep(0.1)
```

About class MPU6050:

Class MPU6050

This is a class library used to operate MPU6050, which can directly read and set MPU6050. Here are some member functions:

```
def __init__(self, a_bus=1, a_address=C.MPU6050_DEFAULT_ADDRESS,
                   a_xAOff=None, a_yAOff=None, a_zAOff=None, a_xGOff=None,
                   a_yGOff=None, a_zGOff=None, a_debug=False):
```

Constructor

```
def dmp_initialize(self):
```

Initialization function, used to wake up MPU6050. Range of accelerometer is ±2g and range of gyroscope is ±250 degrees/sec.

```
def get_acceleration(self):   &   def get_rotation(self):
```

Get the original data of accelerometer and gyroscope.

For details of more relevant member functions, please refer to MPU6050.py in the code folder.

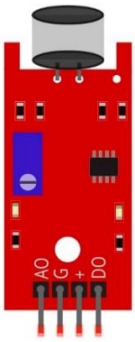
Chapter 26 High-sensitivity microphone sensor

In this chapter, we will learn how to use High-sensitivity microphone sensor.

Project 26.1 High-sensitivity microphone sensor and LED

This project will use a high-sensitivity microphone sensor to make a sound-controlled light.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x6	
High-sensitivity microphone sensor x1 	LED x1	Resistor 220Ω x1

Component knowledge

High-sensitivity microphone sensor

The high-sensitivity microphone sensor module is a component that accepts sound waves and converts them into electrical signals, which can detect the sound intensity in the surrounding environment.

When using it, it should be noted that this sensor can only identify the presence or absence of sound (according to the vibration principle), but cannot identify the size of the sound or the sound of a specific frequency.

This module has 4 pins: digital output (DO), analog output (AO), power supply positive pin and power supply negative pin. AO can output the voltage signal of the microphone in real time. When the ambient sound intensity does not reach the set threshold, the DO outputs a low-level signal, and when the ambient sound intensity exceeds the set threshold, it outputs a high-level signal, and the sensitivity can be adjusted by a potentiometer. When in use, adjust the potentiometer to make the sensitivity to sound reach a more appropriate value, and then read the digital output signal of the module through a pin on the development board. You can speak to the sensor. When the sensor detects a speaking sound, the DO pin outputs a high level; when the sensor does not detect a speaking sound, the DO pin outputs a low level.

Below is the pinout of the high-sensitivity microphone sensor.

Pin description:

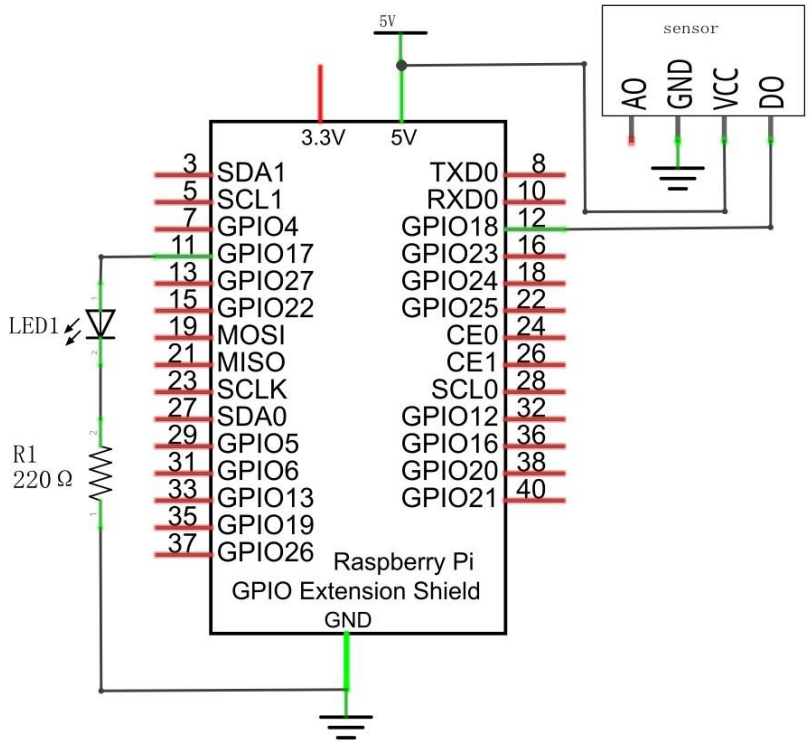
symbol	Function
DO	Digital signal output
VCC	Power supply pin, +3.3V~5.0V
GND	GND
AO	Analog signal output

Since the default sensitivity of the high-sensitivity microphone sensor is high, the two LED lights on the module are lit up after power-on, and the sensitivity should be adjusted to an appropriate value at this time. When the potentiometer is adjusted clockwise, the module identification sensitivity increases; When counterclockwise adjustment potentiometer, module recognition sensitivity decreases. Please adjust the potentiometer before using the module to make its sensitivity reach the appropriate value. Under normal circumstances, you need counterclockwise rotation of the potentiometer, so that the output of the module LED off, when the sensitivity is low can be appropriate clockwise adjustment of the potentiometer, please ensure that your sensor output LED is extinguished when energized, in order to identify the sound.

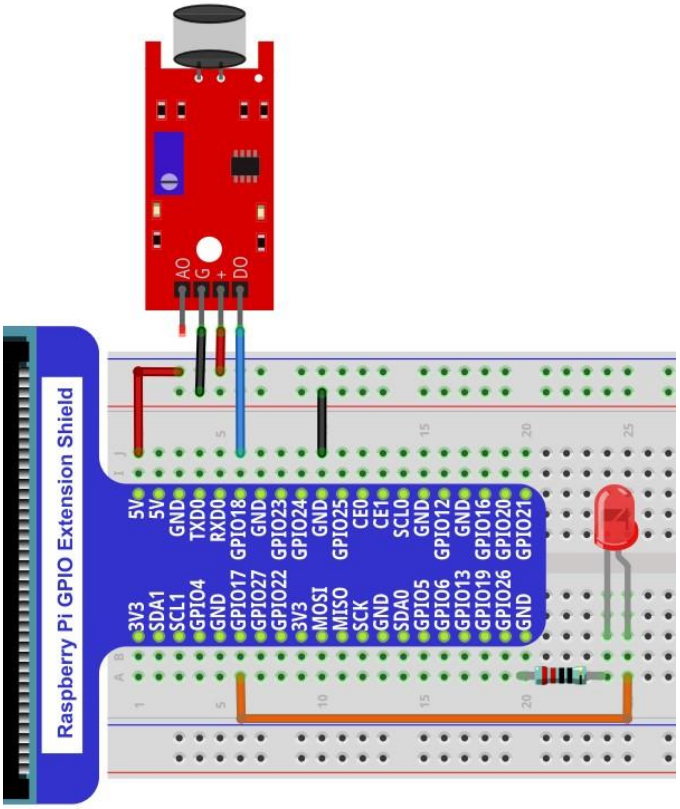
Please do not use voltage beyond the power supply range to avoid damage to the high-sensitivity microphone sensor.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://www.youtube.com/watch?v=W7fi0Sohpi0

Code

Python Code 26.1.1 VoiceLamp

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 26.1 1_VoiceLamp directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/26.1.1_VoiceLamp
```

2. Use python command to execute code "VoiceLamp.py"

```
python VoiceLamp.py
```

After the program is executed, when you speak to the sensor, the LED will turn on for 5 seconds. After 5 seconds, the LED will turn off. When the sensor does not recognize the sound, the LED will turn off.

The following is the program code:

```
1  from gpiozero import LED
2  from sensor import MicrophoneSensor
3  import time
4
5  ledPin = 17       # define ledPin
6  sensorPin = 18    # define sensorPin
```

```
led    = LED(ledPin)
sensor=MicrophoneSensor(sensorPin, pull_up=False)
def loop():
    while True:
        if sensor.is_active:
            led.on()                    # turn on led
            time.sleep(5)
            led.off()                   # turn off led
            print ('led turned on >>>') # print information on termina
        else:
            led.off()
            print ('led turned off >>>')

def destroy():
    led.close()
    sensor.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the MicrophoneSensor class from the sensor module. MicrophoneSensor is similar to the MotionSensor class in the GPIO Zero library in that they both actually use the SmoothedInputDevice class.

```
from sensor import MicrophoneSensor
```

Read the signal pin of the high-sensitivity microphone sensor to determine whether the state of the sensor is high. When the sensor recognizes the sound, it outputs a high level, the variable sensor.is_active is True, and the LED will turn on for 5 seconds and then turn off.

```
def loop():
    while True:
        if sensor.is_active:
            led.on()                    # turn on led
            time.sleep(5)
            led.off()                   # turn off led
            print ('led turned on >>>') # print information on termina
        else:
            led.off()
            print ('led turned off >>>')
```

sensor.py

Import the SmoothedInputDevice class from the GPIO Zero library, create the MicrophoneSensor class, and initialize the parameters.

```
from gpiozero import SmoothedInputDevice

class MicrophoneSensor(SmoothedInputDevice):
    def __init__(self, pin=None, *, pull_up=False, active_state=None,
                 queue_len=5, sample_rate=100, threshold=0.5, partial=False,
                 pin_factory=None):
        super().__init__(
            pin, pull_up=pull_up, active_state=active_state,
            threshold=threshold, queue_len=queue_len,
            sample_wait=1 / sample_rate, partial=partial,
            pin_factory=pin_factory)
        self._queue.start()
    @property
    def value(self):
        return super().value
    @property
    def sound_detected(self):
        return not self.is_active
MicrophoneSensor.when_sound = MicrophoneSensor.when_deactivated
MicrophoneSensor.when_no_sound = MicrophoneSensor.when_activated
MicrophoneSensor.wait_for_sound = MicrophoneSensor.wait_for_inactive
MicrophoneSensor.wait_for_no_sound = MicrophoneSensor.wait_for_active
```

For more information about the methods used by the SmoothedInputDevice class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_input.html#smoothedinputdevice

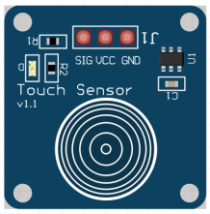
Chapter 27 Touch Sensor

In this chapter, we will learn how to use the touch sensor.

Project 27.1 Touch Sensor and LED

This project will use the touch sensor to control the LED to emit different brightness.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x7	
Touch Sensor x1 	LED x1	Resistor 220Ω x1

Component knowledge

Touch Sensor

The touch sensor module is a capacitive touch switch module based on TTP223 chip. Its use is very simple. This module has 3 pins: signal pin, power positive pin and power negative pin. When the positive and negative pins of the module are connected to a suitable power supply, the module starts to work. At this time, only one pin on the development board is needed to read the output signal of the module. For example, you can touch the front and back of the module. When the touch sensor is touched, the signal pin outputs a high level; when the touch sensor is not touched, it outputs a low level.

Below is the pinout of the touch sensor.

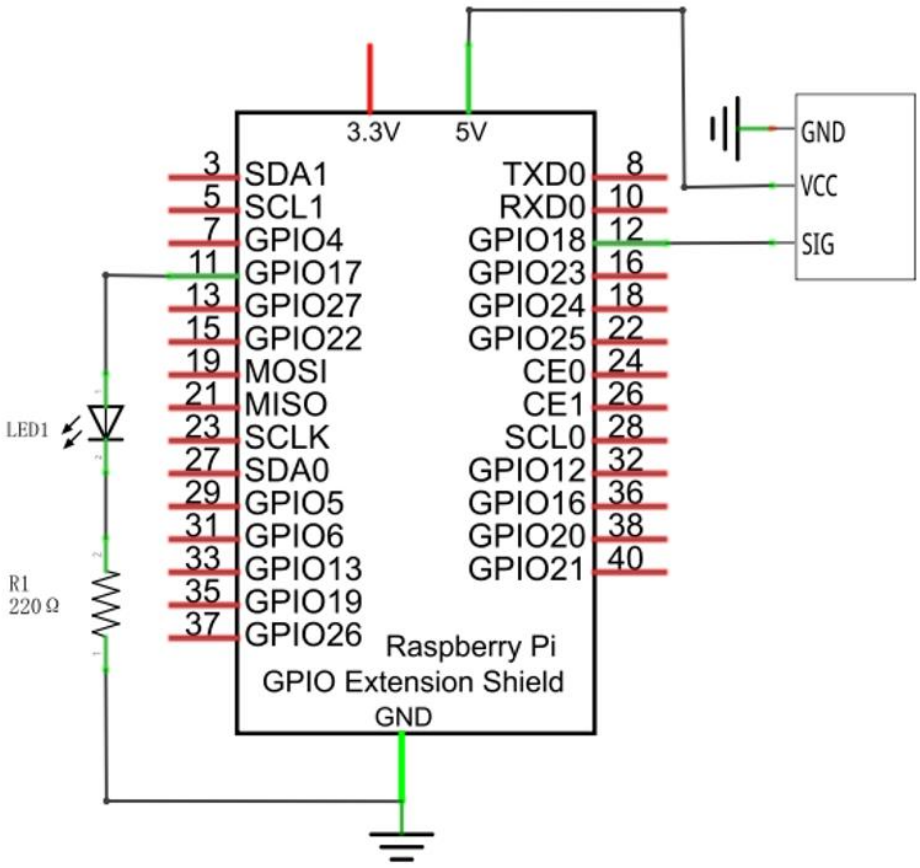
Pin description:

symbol	Function
SIG	Output control signal
VCC	Power supply pin, +2.0V~5.5V
GND	GND

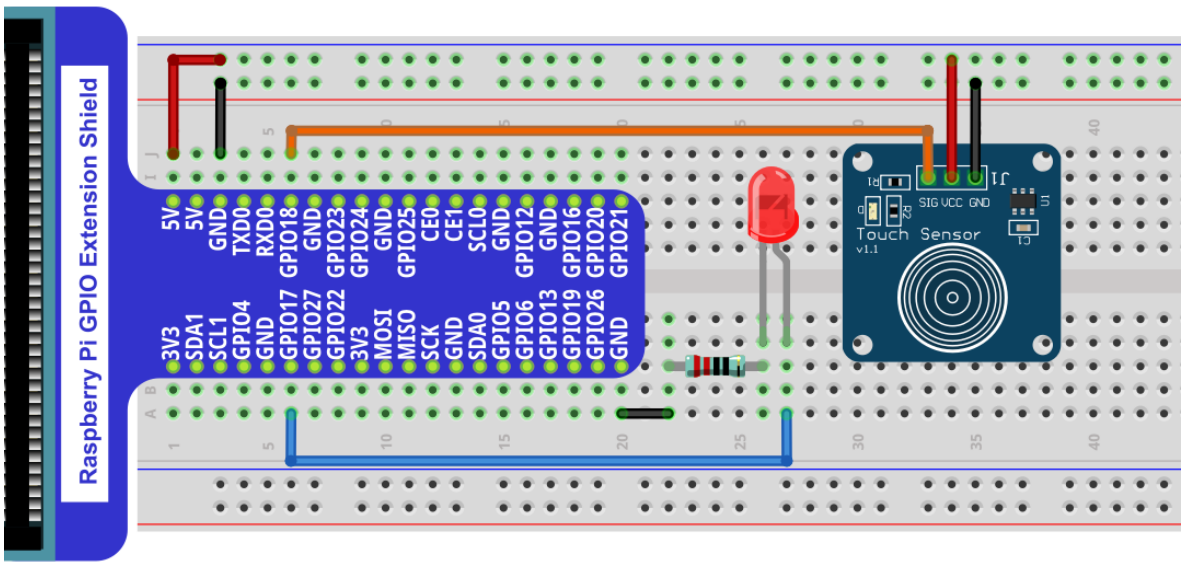
Please do not use voltage beyond the power supply range to avoid damage to the touch sensor.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 27.1.1 TouchSensor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 27.1.1_TouchSensor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/27.1.1_TouchSensor
```

2. Use Python command to execute code " TouchSensor.py".

```
python TouchSensor.py
```

In this code, we use the touch sensor to adjust the brightness of the LED. Every time you touch the sensor with your hand, the brightness of the LED changes.

The following is the program code:

```
from gpiozero import PWMLED
from sensor import TouchSensor
import time

ledPin    = 17     # define ledPin
sensorPin = 18     # define sensorPin
led    = PWMLED(ledPin)
sensor = TouchSensor(sensorPin, pull_up=False)
grade=0

# Define the functions that will be called when the line is
def SensorEvent():
    global  grade
    grade=grade+1
    print("Sensor is pressed!")
    if(grade > 3):
        grade=0

def loop():
    sensor.when_touch = SensorEvent
    while True:
        global grade
        if grade==1:
            dc=35
            led.value = dc / 100.0     # set dc value as the duty cycle
        elif grade==2:
            dc=65
            led.value = dc / 100.0     # set dc value as the duty cycle
        elif grade==3:
```

```
            dc=100
            led.value = dc / 100.0      # set dc value as the duty cycle
        else:
            dc=0
            led.value = dc / 100.0      # set dc value as the duty cycle

def destroy():
    led.close()
    sensor.close()

if __name__ == '__main__':      # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the TouchSensor class from the sensor module. TouchSensor is similar to the MotionSensor class in the GPIO Zero library in that they both actually use the SmoothedInputDevice class.

```
from sensor import TouchSensor
```

sensor.when_touch associates the touch sensor pin with SensorEven(). When SensorPin detects a high level, the SensorEven() function is called and executed. That is, every time the sensor is touched, a variable called grade is used to record the number of touches.

```
# Define the functions that will be called when the line is
def SensorEvent():
    global  grade
    grade=grade+1
    print("Sensor is pressed!")
    if(grade > 3):
        grade=0


def loop():
    sensor.when_touch = SensorEvent
```

Set the LED light to emit different brightness according to the value of grade.

```
        if grade==1:
            dc=35
            led.value = dc / 100.0      # set dc value as the duty cycle
        elif grade==2:
            dc=65
            led.value = dc / 100.0      # set dc value as the duty cycle
        elif grade==3:
            dc=100
            led.value = dc / 100.0      # set dc value as the duty cycle
        else:
```

```
            dc=0
            led.value = dc / 100.0      # set dc value as the duty cycle
```

sensor.py

Import the SmoothedInputDevice class from the GPIO Zero library, create the TouchSensor class, and initialize the parameters

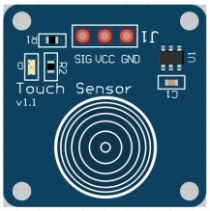
```
from gpiozero import SmoothedInputDevice

class TouchSensor(SmoothedInputDevice):
    def __init__(self, pin=None, *, pull_up=False, active_state=None,
                 queue_len=5, sample_rate=100, threshold=0.5, partial=False,
                 pin_factory=None):
        super().__init__(
            pin, pull_up=pull_up, active_state=active_state,
            threshold=threshold, queue_len=queue_len,
            sample_wait=1 / sample_rate, partial=partial,
            pin_factory=pin_factory)
        self._queue.start()
    @property
    def value(self):
        return super().value
    @property
    def touch_detected(self):
        return not self.is_active
TouchSensor.when_touch = TouchSensor.when_deactivated
TouchSensor.when_no_touch = TouchSensor.when_activated
TouchSensor.wait_for_touch = TouchSensor.wait_for_inactive
TouchSensor.wait_for_no_touch = TouchSensor.wait_for_active
```

For more information about the methods used by the SmoothedInputDevice class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_input.html#smoothedinputdevice

Project 27.2 Touch Sensor and RGB LED

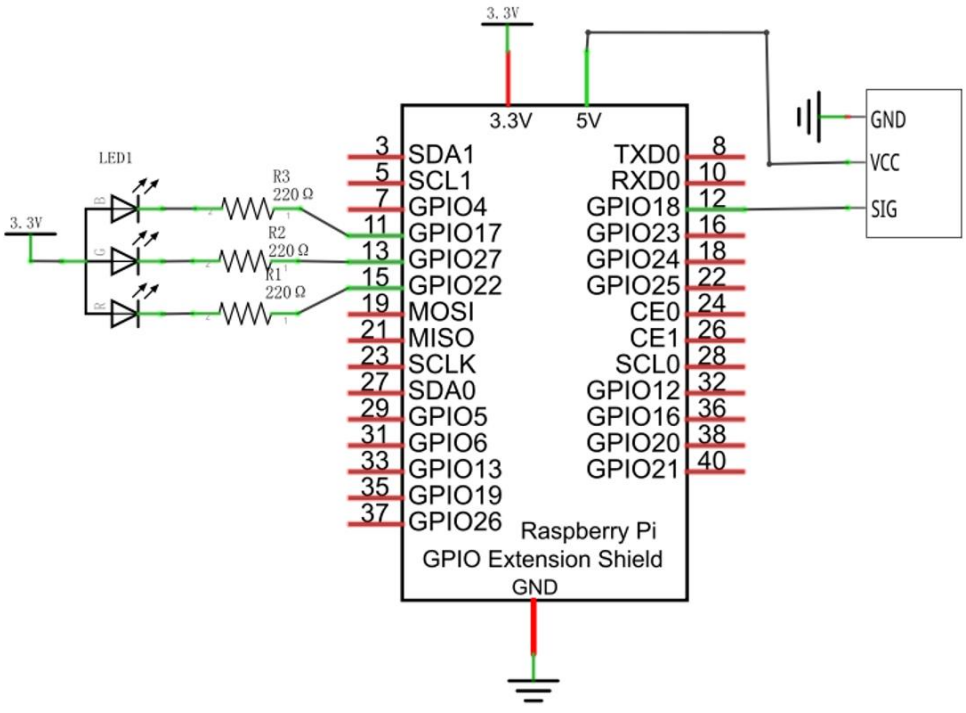
This project uses a touch sensor to control RGB LED to emit different colors.

Component List

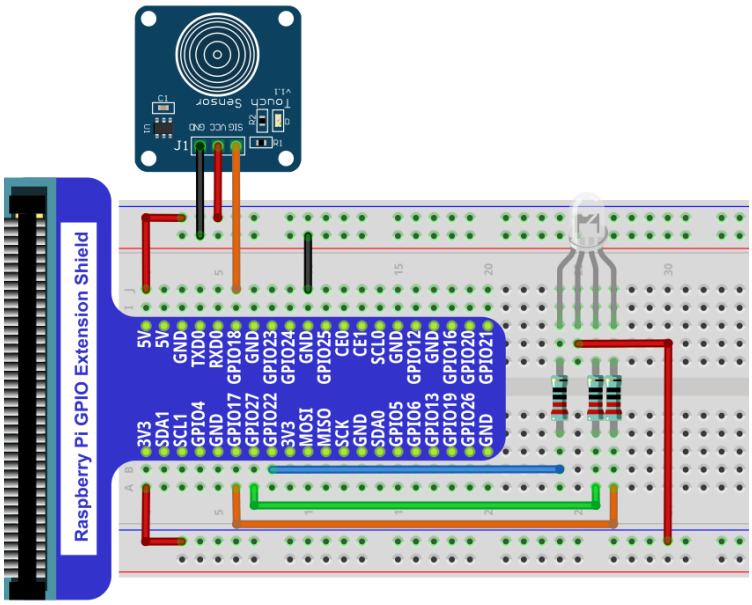
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x10	
Touch Sensor x1 	RGB LED x1	Resistor 220Ω x3

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 27.2.1 Discolor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 27.2.1_Discolor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/27.2.1_Discolor
```

2. Use Python command to execute code " Discolor.py ".

```
python Discolor.py
```

After the program is executed, we can use the touch sensor to adjust the color change of the RGB LED. Every time the sensor is touched, the color of the RGB LED changes. Color changes from red to green to blue.

The following is the program code:

```
1  from gpiozero import RGBLED
2  from sensor import TouchSensor
3  import time
4
5  sensorPin = 18     # define sensorPin
6  sensor = TouchSensor(sensorPin, pull_up=False)
7  led = RGBLED(red=22, green=27, blue=17,active_high=False) # define the pins for
8  R:GPIO22,G:GPIO27,B:GPIO17
9  grade=0
10
11 # Define the functions that will be called when the line is
```

```
def SensorEvent(): # When Sensor is pressed, this function will be executed
    global  grade
    grade=grade+1
    print("Sensor is pressed!")
    if(grade > 3):
        grade=0

def loop():
    sensor.when_no_line = SensorEvent
    while True:
        global grade
        if grade==1:
            led.color = (1, 0, 0)
            print ('The current color is red')
        elif grade==2:
            led.color = (0, 1, 0)
            print ('The current color is green')
        elif grade==3:
            led.color = (0, 0, 1)
            print ('The current color is blue')
        else:
            led.off()
            print ('Close the RGBLED')
        time.sleep(0.001)

def destroy():
    led.close()
    sensor.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

GPIO.add_event_detect() associates the touch sensor pin with SensorEven(). When SensorPin detects a high level, the SensorEven() function is called and executed.

```
def SensorEvent(channel): # When Sensor is pressed, this function will be executed
    global  grade
    grade=grade+1
    print("Sensor is pressed!")
    if(grade > 3):
        grade=0
```

Determine the number of times the sensor is pressed, and control the pin R, G, and B to output different PWM values, thereby controlling the color of the RGB LED.

```
        if grade==1:
            led.color = (1, 0, 0)
            print ('The current color is red')
        elif grade==2:
            led.color = (0, 1, 0)
            print ('The current color is green')
        elif grade==3:
            led.color = (0, 0, 1)
            print ('The current color is blue')
        else:
            led.off()
            print ('Close the RGBLED')
```

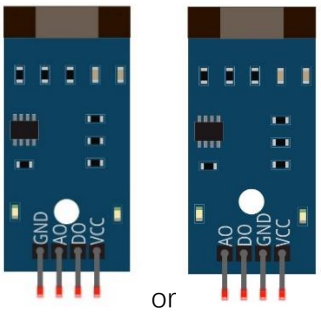
Chapter 28 U-shaped photoelectric sensor

In this chapter, we will learn how to use a U-shaped photoelectric sensor.

Project 28.1 U-shaped photoelectric sensor and LED

This project uses a U-shaped photoelectric sensor to control the state of the LED.

Component List

<table>
<tr><td>Raspberry Pi (with 40 GPIO) x1
GPIO Expansion Board & Ribbon Cable x1
Breadboard x1</td><td colspan="2">Jumper x6
 </td></tr>
<tr><td>U-shaped photoelectric sensor x1
 
or</td><td>LED x1
 </td><td>Resistor 220Ω x1
 </td></tr>
</table>

Component knowledge

U-shaped photoelectric sensor

The U-shaped photoelectric sensor is a through-beam photoelectric sensor, which consists of a transmitting end and a receiving end. Its working principle is blocking and conducting the infrared emission light will change the current induced by the infrared receiving tube.

This module has 4 pins: digital output (DO), analog output (AO), power supply positive pin and power supply negative pin. When the positive and negative pins of the module are connected to a suitable power supply, the module starts to work. Only one pin on the development board is needed to read the digital output (DO) signal of the module. When the photoelectric sensor is blocked, the digital signal pin outputs a low level. If the photoelectric sensor is not blocked, it outputs a high level.
Below is the pinout of the touch sensor.

Pin description:

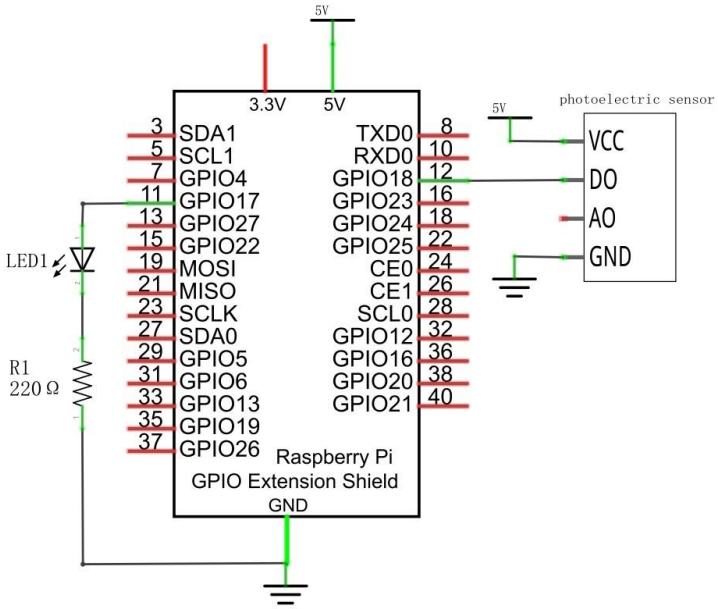
symbol	Function
VCC	Power supply pin, +3.3V~5.5V
DO	Output control signal(High or low level)
AO	Output invalid
GND	GND

Please do not use the voltage beyond the power supply range to avoid damage to the U-shaped photoelectric sensor.

The difference between the above two U-type photoelectric sensors is that in addition to the different pin sequence, the output signal is opposite when you block it, that is, there are modules that output high level and low level U-type photoelectric sensor. The specific module is subject to the one in your hand. Please check the pin sequence of your U-shaped photoelectric sensor and replace the appropriate wiring to avoid permanent damage to your raspberry PI.

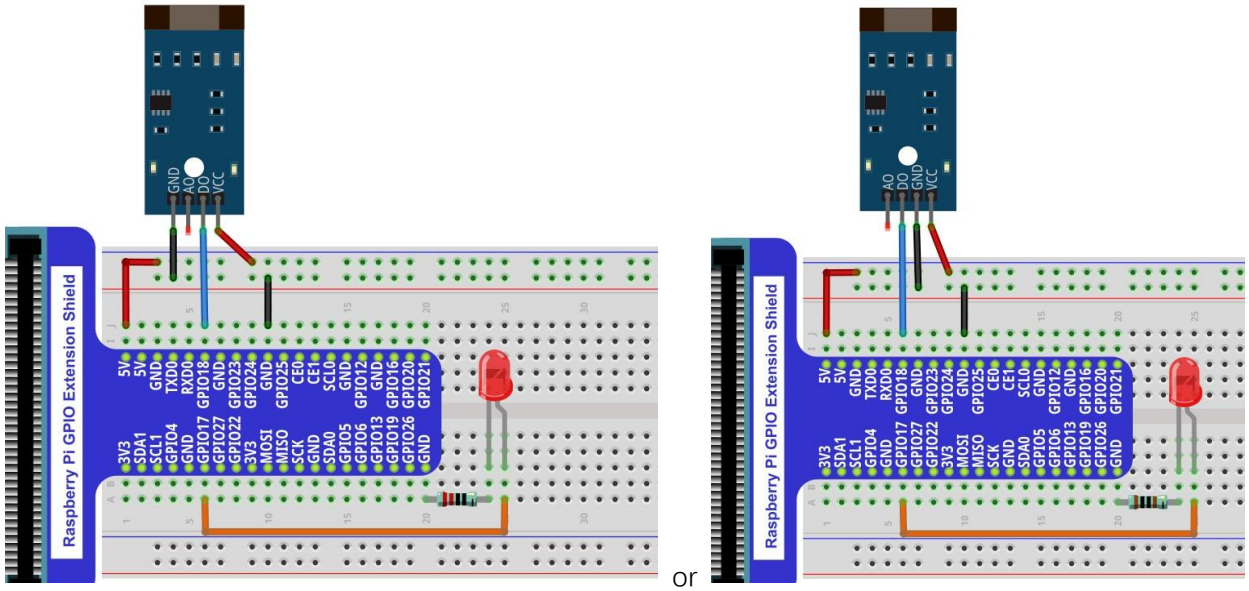
Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

Please check the sequence of your U-shaped photoelectric sensor and select the appropriate wiring to avoid permanent damage to your raspberry PI.



Code

Python Code 28.1.1 PhotoSensor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 28.1.1 1_Photosensor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/28.1.1_PhotoSensor
```

2. Use python command to execute code " PhotoSensor.py"

```
python PhotoSensor.py
```

When you use the module whose output signal is low level, after the program is executed, when the photoelectric sensor is blocked, the LED lights up, when the photoelectric sensor is not blocked, the LED turns off.

When you use the module with high output signal, the LED will be off when the photoelectric sensor is blocked and on when the photoelectric sensor is not blocked after the program is executed.

The phenomena are opposite, depending on the module in your hand.

The following is the program code:

```
from gpiozero import LED
from sensor import PhotoSensor
import time

ledPin    = 17
sensorPin = 18     # define sensorPin
sensor = PhotoSensor(sensorPin, pull_up=False)
led = LED(ledPin, initial_value=True)

# Define the functions that will be called when the line is
def SensorEvent1(channel): # The sensor is blocked
    led.on()
    print ('When the sensor is not blocked, led turned on >>>')     # print information on
terminal
def SensorEvent2(channel): # The sensor is blocked
    led.off()
    print (' When the sensor is blocked, led turned off <<<')

def loop():
    sensor.when_occlusion = SensorEvent1
    sensor.when_no_occlusion = SensorEvent2
    while True:
        time.sleep(1)

def destroy():
    led.close()
```

```
    sensor.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the PhotoSensor class from the sensor module. PhotoSensor is similar to the MotionSensor class in the GPIO Zero library in that they both actually use the SmoothedInputDevice class.

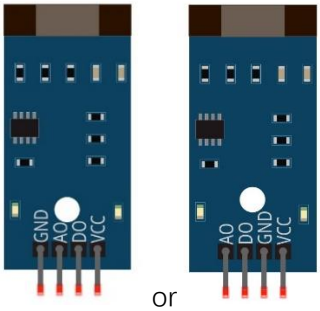
```
from sensor import PhotoSensor
```

For more information about the methods used by the SmoothedInputDevice class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_input.html#smoothedinputdevice

Project 28.2 U-shaped photoelectric sensor and buzzer

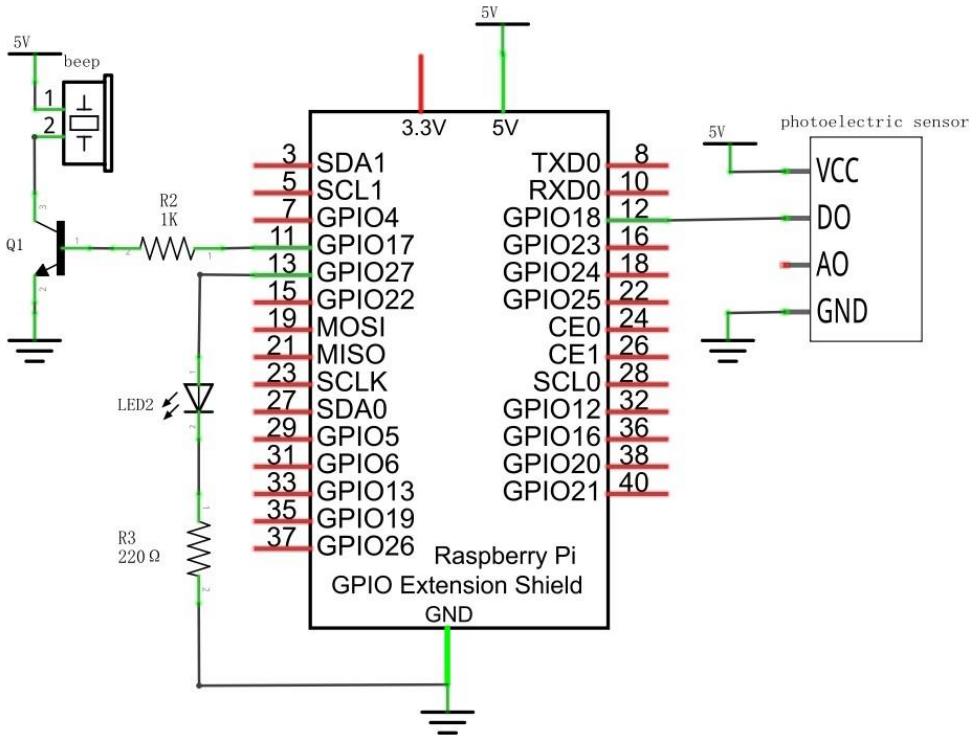
This project uses U-shaped photoelectric sensor to make a simple sound and light alarm.

Component List

<table>
<tr><td>Raspberry Pi (with 40 GPIO) x1
GPIO Expansion Board & Ribbon Cable x1
Breadboard x1</td><td colspan="2">Jumper x1
 </td></tr>
<tr><td>Active buzzer x1
 </td><td>NPN transistorx1
(S8050)
 </td><td>Resistor 1kΩx1
 </td></tr>
<tr><td>U-shaped photoelectric sensor x1
 
or</td><td>LED x1
 </td><td>Resistor 220Ωx1
 </td></tr>
</table>

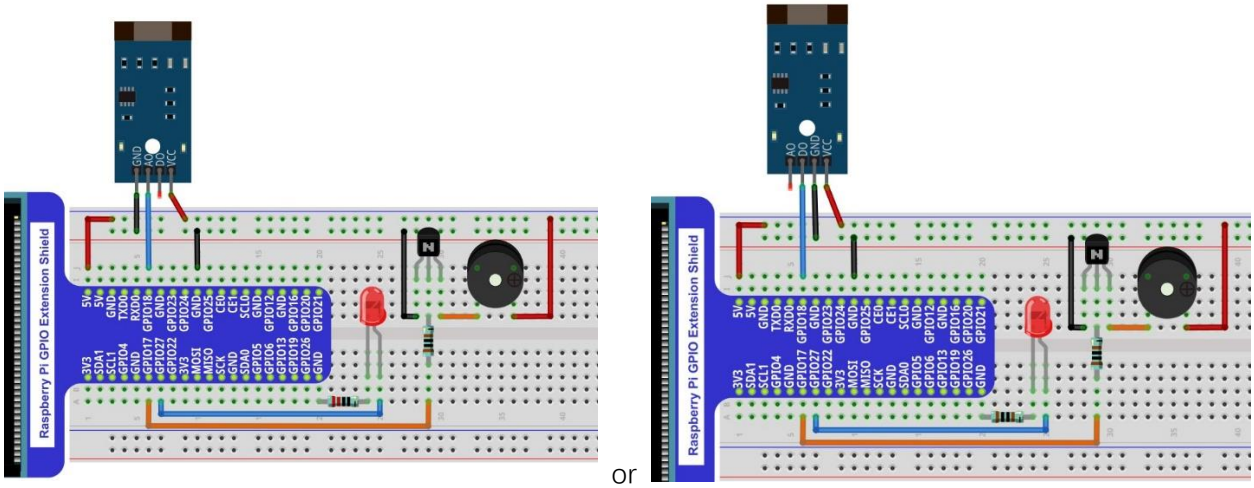
Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

Please check the sequence of your U-shaped photoelectric sensor and select the appropriate wiring to avoid permanent damage to your raspberry PI.



Code

Python Code 28.2.1 Alertor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 28.2.1_Alertor directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/28.2.1_Alertor
```

2. Use python command to execute code " Alertor.py"

```
python Alertor.py
```

After the program is executed, every time the U-shaped photoelectric sensor is blocked by hand, the buzzer will sound an alarm, and the LED will flash to remind.

The following is the program code:

```
1  from gpiozero import LED, Buzzer
2  from sensor import PhotoSensor
3  import time
4
5  ledPin    = 27
6  sensorPin = 18     # define sensorPin
7  BuzzerPin = 17
8  sensor = PhotoSensor(sensorPin, pull_up=True)
9  led    = LED(ledPin, initial_value=False)
10 buzzer = Buzzer(BuzzerPin)
11
12 def alarm():
13     times=3
14     while times:
15         buzzer.on() # turn on buzzer
16         led.on()    # turn on led
17         time.sleep(0.05)
18         buzzer.off()  # turn off buzzer
19         led.off()     # turn on led
20         time.sleep(0.05)
21         times-=1
22
23 # When sensor is blocked, this function will be executed
24 def SensorEvent(channel): # The sensor is blocked
25     alarm()
26
27 def loop():
28     sensor.when_occlusion = SensorEvent
29     sensor.when_no_occlusion = SensorEvent
30     while True:
```

```
        time.sleep(1)

def destroy():
    led.close()
    sensor.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

The sensor.when_occlusion and sensor.when_no_occlusion functions associate the sensor pin with sensorEven().Because there are high level triggering module and low level triggering module in the U-type photoelectric sensor, we use the double-edge detection method here to make the program compatible with the module in your hand. When sensorPin detects low level or high level, it will call and execute the sensorEven() function.

```
# When sensor is blocked, this function will be executed
def SensorEvent(channel): # The sensor is blocked
    alarm()

def loop():
    sensor.when_occlusion = SensorEvent
    sensor.when_no_occlusion = SensorEvent
```

The function alarm() is used to control the active buzzer to emit an alarm sound and control the LED to flash at the same time. Use variable times to pass in the number of times the alarm sounds and the LED blinks.

```
def alarm():
    times=3
    while times:
        buzzer.on() # turn on buzzer
        led.on()    # turn on led
        time.sleep(0.05)
        buzzer.off()  # turn off buzzer
        led.off()     # turn on led
        time.sleep(0.05)
        times-=1
```

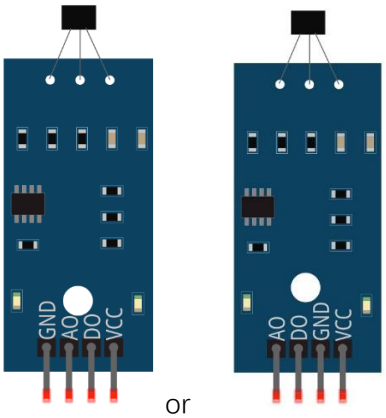
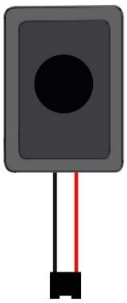
Chapter 29 Hall sensor

In this chapter, we will learn how to use Hall sensor.

Project 29.1 Hall sensor and LED

This project uses hall sensor to control the state of LED.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x6		
Hall sensor x1  or	LED x1	Resistor 220Ω x1	Speaker x1 

Component knowledge

Hall sensor

Hall sensor is a magnetic field sensor made according to the Hall effect. Based on the Hall effect, Hall voltage varies with magnetic field strength. Hall sensors can be divided into linear (analog) Hall sensors and switching Hall sensors. Linear Hall sensors have two outputs: analog output (AO), digital output (DO). Switching Hall sensors only have digital output (DO) and the analog output (AO) output has no effect. In this project, the switch Hall sensor is used. Its use is very simple.

This module has 4 pins: digital output (DO), analog output (AO), power supply positive pin and power supply negative pin. When the positive and negative pins of the module are connected to a suitable power supply, the module starts to work. Only one pin on the development board is needed to read the digital output (DO) signal of the module. In the project, the speaker is used as the magnetic field source, when you put the sensor close to the magnetic field (speaker), the DO outputs low level, and if the sensor does not sense the magnetic field (speaker), the DO outputs high level.

Below is the pinout of the Hall sensor.

Pin description:

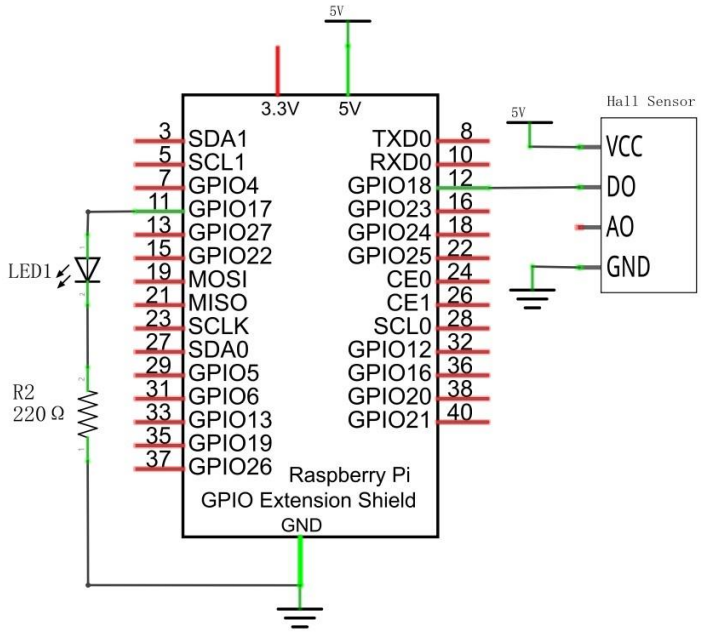
symbol	Function
VCC	Power supply pin, +3.3V~5.5V
DO	Output control signal
AO	Output invalid
GND	GND

Please do not use voltage beyond the power supply range to avoid damage to the Hall sensor.

For the above two hall sensors, their difference is only the pin sequence is different, please get the Hall sensor, check its sequence, change the corresponding wiring, so as not to cause permanent damage to your raspberry PI.

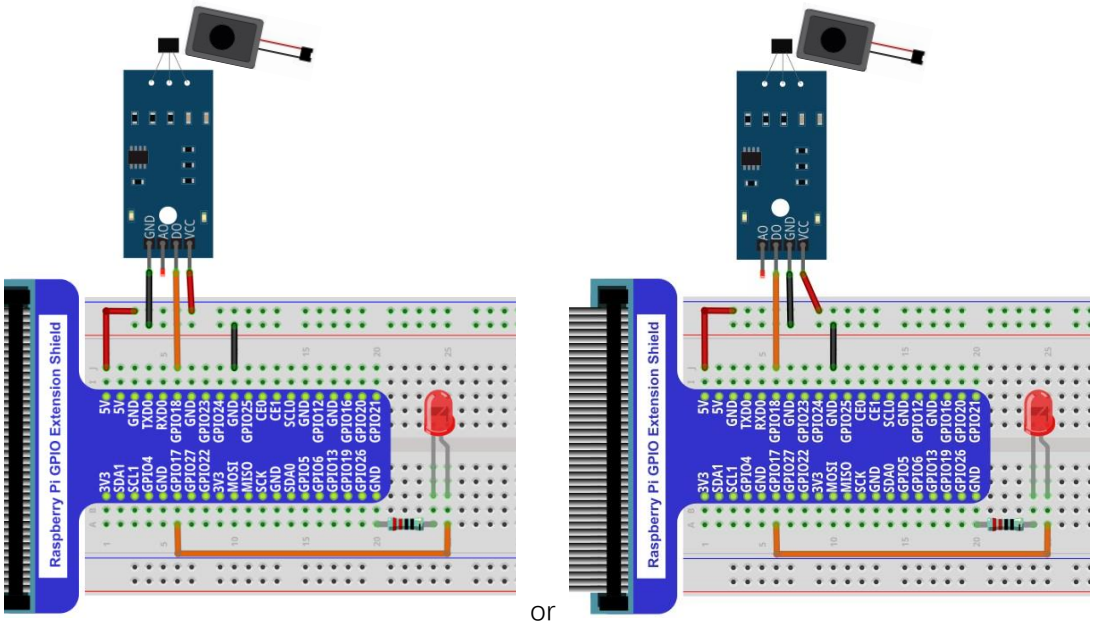
Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

Please check the sequence of your Hall sensor and select the appropriate wiring to avoid permanent damage to your raspberry PI.



Code

Python Code 29.1.1 HallSensor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 29.1.1_HallSensor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/29.1.1_HallSensor
```

2. Use python command to execute code "HallSensor.py"

```
python  HallSensor.py
```

After the program is executed, when the sensor is close to the magnetic field (horn) by hand, the LED will turn on, and if the sensor does not sense the magnetic field (horn), the LED will turn off.

The following is the program code:

```
1  from gpiozero import LED
2  from sensor import HallSensor
3  import time
4
5  ledPin    = 17
6  sensorPin = 18     # define sensorPin
7  sensor = HallSensor(sensorPin, pull_up=False)
8  led = LED(ledPin, initial_value=False)
9
10 # Define the functions that will be called when the line is
11 def SensorEvent1(channel): # The sensor is blocked
12     led.on()
13     print ('led turned on >>>')     # print information on terminal
14 def SensorEvent2(channel): # The sensor is blocked
15     led.off()
16     print ('led turned off >>>')     # print information on terminal
17
18 def loop():
19     sensor.when_magnetic_field = SensorEvent1
20     sensor.when_no_magnetic_field = SensorEvent2
21     while True:
22         time.sleep(1)
23
24 def destroy():
25     led.close()
26     sensor.close()
27
28 if __name__ == '__main__':     # Program entrance
29     print ('Program is starting...')
```

```
30      try:
31          loop()
32      except KeyboardInterrupt:  # Press ctrl-c to end the program.
33          destroy()
34          print("Ending program")
```

Import the HallSensor class from the sensor module. HallSensor is similar to the MotionSensor class in the GPIO Zero library in that they both actually use the SmoothedInputDevice class.

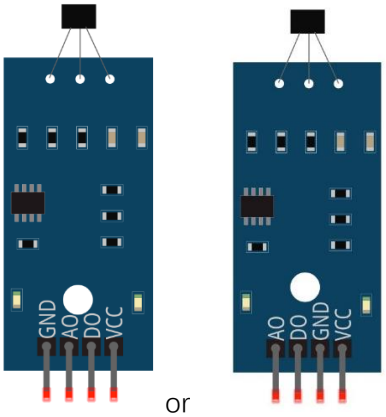
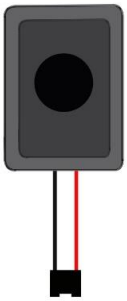
```
    from sensor import HallSensor
```

For more information about the methods used by the SmoothedInputDevice class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_input.html#smoothedinputdevice

Project 29.2 Hall Sensor and Buzzer

This project uses Hall sensor to make a simple magnetic field detection sound and light alarm.

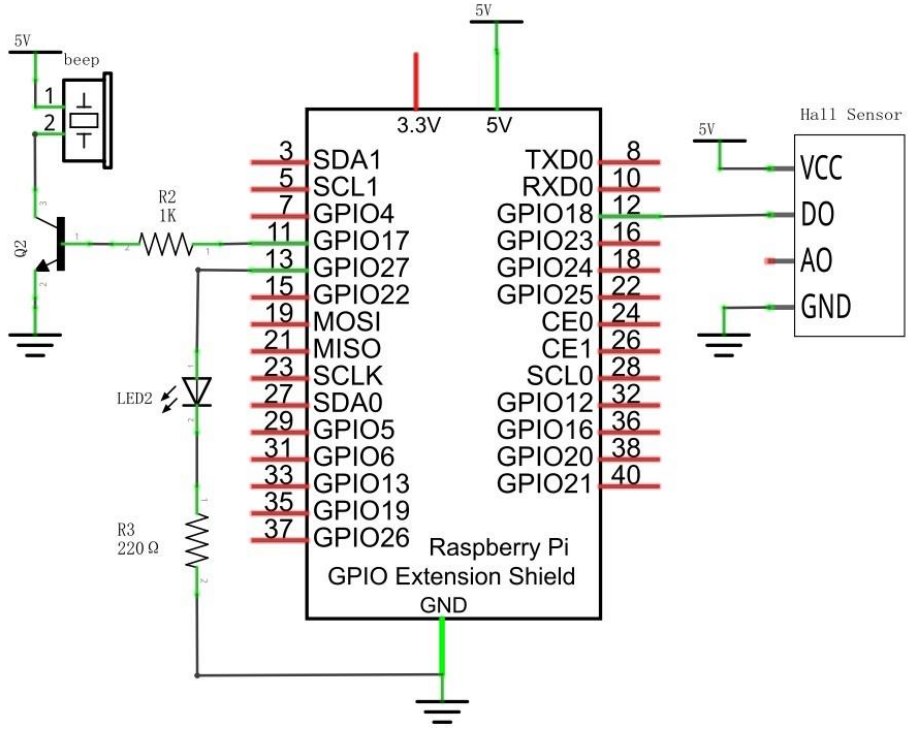
Component List

<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Expansion Board & Ribbon Cable x1
Breadboard x1</td><td colspan="2">Jumper x10
 </td></tr>
<tr><td colspan="2">Active buzzer x1
 </td><td>NPN transistorx1
(S8050)
 </td><td>LED x1
 </td></tr>
<tr><td colspan="2">Hall sensor x1
  </td><td>Resistor 220Ω x1
 </td><td>Resistor 1kΩ x1
 </td><td>Speaker x1
  </td></tr>
</table>

For the above two hall sensors, their difference is only the pin sequence is different, please get the Hall sensor, check its sequence, change the corresponding wiring, so as not to cause permanent damage to your raspberry PI.

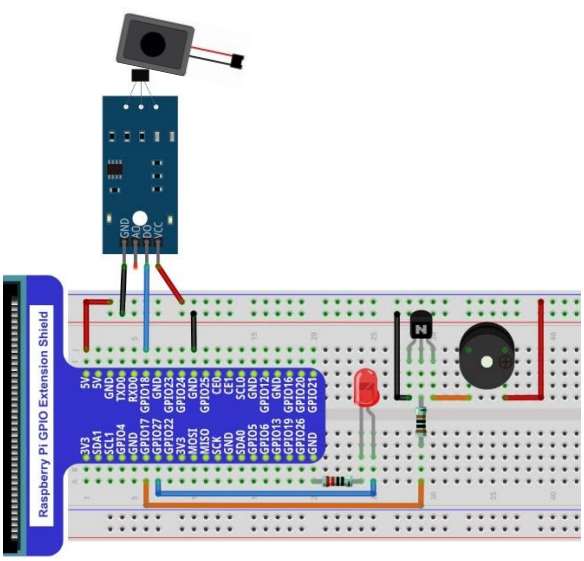
Circuit

Schematic diagram

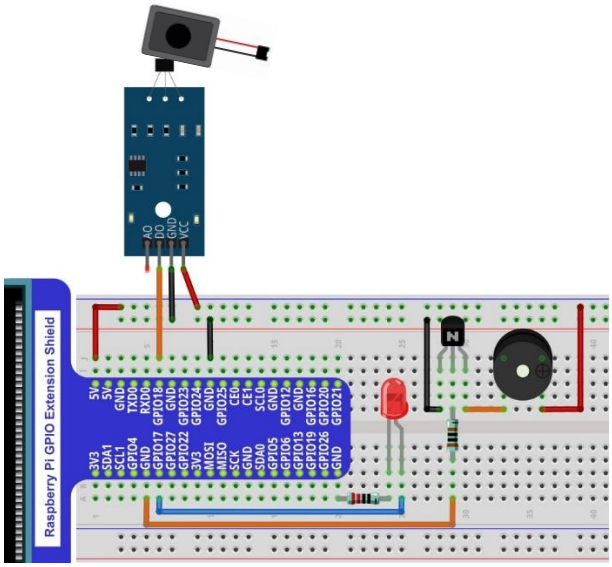


Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

Please check the sequence of your Hall sensor and select the appropriate wiring to avoid permanent damage to your raspberry PI.



or



Code

Python Code 29.2.1 Alertor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 29.2.1_Alertor directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/29.2.1_Alertor
```

2. Use python command to execute code " Alertor.py"

```
python Alertor.py
```

After the program is executed, every time the sensor is close to the magnetic field (speaker) by hand, the buzzer will sound an alarm, and the LED will flash to remind.

The following is the program code:

```
from gpiozero import LED, Buzzer
from sensor import HallSensor
import time

ledPin    = 27
sensorPin = 18     # define sensorPin
BuzzerPin = 17
sensor = HallSensor(sensorPin, pull_up=False)
led    = LED(ledPin, initial_value=False)
buzzer = Buzzer(BuzzerPin)

def alarm():
    times=3
    while times:
        buzzer.on() # turn on buzzer
        led.on()    # turn on led
        time.sleep(0.05)
        buzzer.off()  # turn off buzzer
        led.off()     # turn on led
        time.sleep(0.05)
        times-=1

# When sensor is blocked, this function will be executed
def SensorEvent(channel): # The sensor is blocked
    alarm()

def loop():
    sensor.when_magnetic_field = SensorEvent
    while True:
```

```
30          time.sleep(1)
31
32  def destroy():
33      led.close()
34      sensor.close()
35
36  if __name__ == '__main__':     # Program entrance
37      print ('Program is starting...')
38      try:
39          loop()
40      except KeyboardInterrupt:  # Press ctrl-c to end the program.
41          destroy()
42          print("Ending program")
```

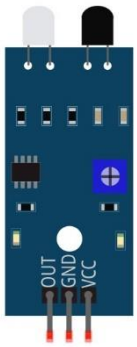
Chapter 30 Infrared Obstacle Avoidance Sensor

In this chapter, we will learn how to use infrared obstacle avoidance sensor.

Project 30.1 Infrared obstacle avoidance sensor and LED

This project uses infrared obstacle avoidance sensor to change the state of LED.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x6	
Infrared obstacle avoidance sensor x1 	LED x1	Resistor 220Ω x1

Component knowledge

Infrared obstacle avoidance sensor

The infrared obstacle avoidance sensor module is a distance adjustable obstacle avoidance sensor. The sensor has strong adaptability to ambient light and high precision. It has a pair of infrared emitting and receiving tubes. The transmitting tube emits infrared rays of a certain frequency. When the detection direction encounters an obstacle (reflecting surface), the infrared rays are reflected back and are received by the receiving tube. The indicator light is on at this time. After the circuit processing, the signal output pin outputs digital signal. The detection distance can be adjusted by the potentiometer knob, the effective distance is 2 ~ 30cm, and the detection angle is 35°.

This module has 3 pins: signal pin, power positive pin and power negative pin. When the positive and negative pins of the module are connected to a suitable power supply, the module starts to work. At this time, only one pin on the development board is needed to read the output signal of the module. When the module is in use, the detection distance can be adjusted to a suitable value through the potentiometer knob. You can use your hand to block the module at a certain distance. When the infrared obstacle avoidance sensor is blocked, the signal pin outputs a low level; if the infrared obstacle avoidance sensor is not blocked, it outputs a high level.

Below is the pinout of infrared obstacle avoidance sensor.

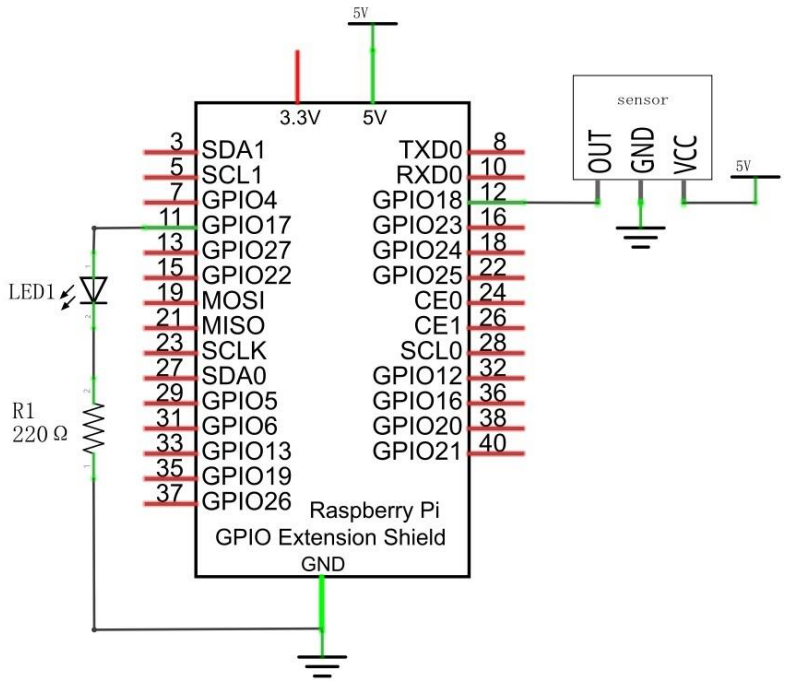
Pin description:

symbol	Function
OUT	Output control signal
VCC	Power supply pin, +3.3V~5.0V
GND	GND

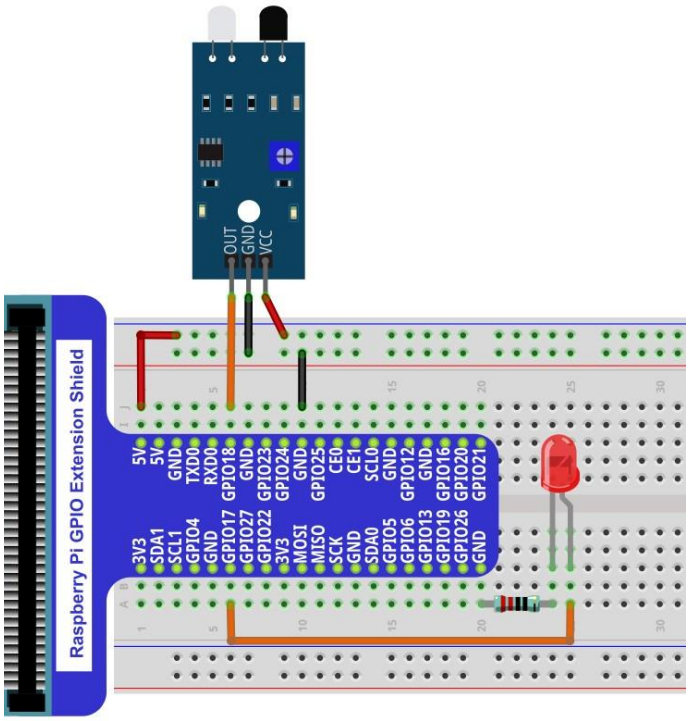
Please do not use the voltage beyond the power supply range to avoid damage to the infrared obstacle avoidance sensor.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 30.1.1 InfraredSensor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 30.1.1_InfraredSensor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/30.1.1_InfraredSensor
```

2. Use python command to execute code "InfraredSensor.py"

```
python InfraredSensor.py
```

After the program is executed, when you block the sensor with your hand at a certain distance or the sensor encounters an obstacle, the LED will turn on, and when the sensor is not blocked or the sensor does not encounter an obstacle, the LED will turn off.

The following is the program code:

```
1  from gpiozero import LED
2  from sensor import InfraredSensor
3  import time
4
5  ledPin    = 17
6  sensorPin = 18     # define sensorPin
7  sensor = InfraredSensor(sensorPin, pull_up=True)
8  led = LED(ledPin, initial_value=False)
```

```
9
10  # Define the functions that will be called when the line is
11  def SensorEvent1(channel): # The sensor is blocked
12      led.on()
13      print ('led turned on >>>')     # print information on terminal
14  def SensorEvent2(channel): # The sensor is blocked
15      led.off()
16      print ('led turned off >>>')     # print information on terminal
17
18  def loop():
19      sensor.when_reflect = SensorEvent1
20      sensor.when_no_reflect = SensorEvent2
21      while True:
22          time.sleep(1)
23
24  def destroy():
25      led.close()
26      sensor.close()
27
28  if __name__ == '__main__':     # Program entrance
29      print ('Program is starting...')
30      try:
31          loop()
32      except KeyboardInterrupt:  # Press ctrl-c to end the program.
33          destroy()
34          print("Ending program")
```

Import the InfraredSensor class from the sensor module. InfraredSensor is similar to the MotionSensor class in the GPIO Zero library in that they both actually use the SmoothedInputDevice class.

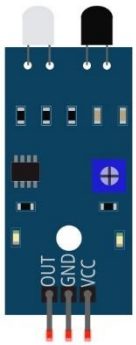
```
    from sensor import InfraredSensor
```

For more information about the methods used by the SmoothedInputDevice class in the GPIO Zero library,please refer to:https://gpiozero.readthedocs.io/en/stable/api_input.html#smoothedinputdevice

Project 30.2 Infrared obstacle avoidance sensor and buzzer

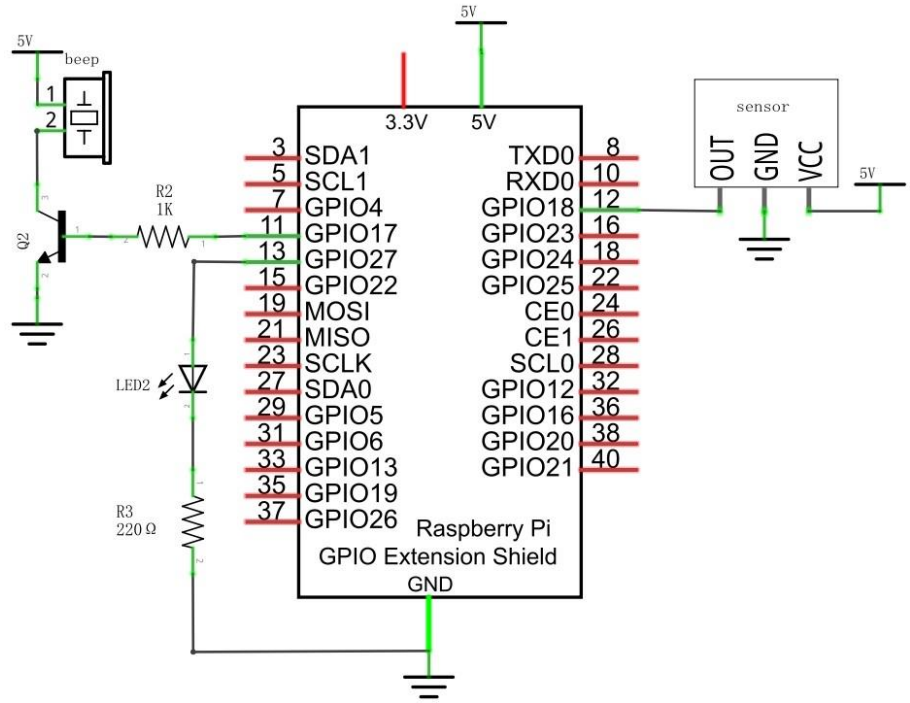
This project uses an infrared obstacle avoidance sensor to make a simple reminder.

Component List

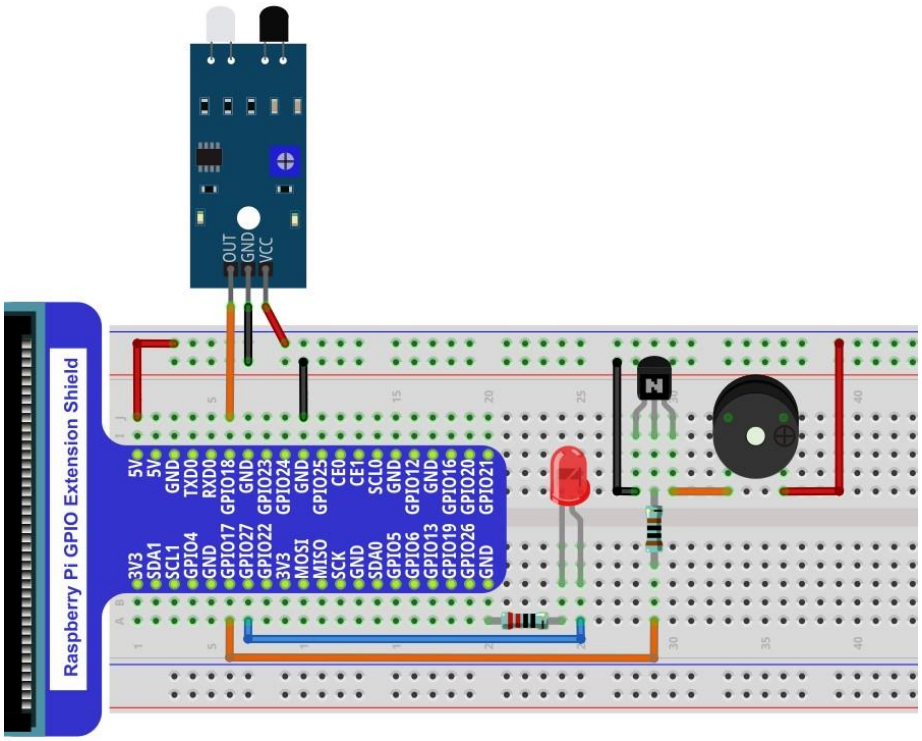
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x10 	
Active buzzer x1 	NPN transistorx1 (S8050) 	Resistor 1kΩx1
Infrared obstacle avoidance sensor x1 	LED x1 	Resistor 220Ωx1

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: **support@freenove.com**



Code

Python Code 30.2.1 Alertor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 30.2.1_Alertor directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/30.2.1_Alertor
```

2. Use python command to execute code " Alertor.py"

```
python Alertor.py
```

After the program is executed, when you block the sensor with your hand at a certain distance or the sensor encounters an obstacle, the buzzer will sound a reminder, and the LED will flash to remind you.

The following is the program code:

```
from gpiozero import LED, Buzzer
from sensor import InfraredSensor
import time

ledPin    = 27
sensorPin = 18     # define sensorPin
BuzzerPin = 17
sensor = InfraredSensor(sensorPin, pull_up=True)
led    = LED(ledPin, initial_value=False)
buzzer = Buzzer(BuzzerPin)

def alarm():
    times=3
    while times:
        buzzer.on() # turn on buzzer
        led.on()    # turn on led
        time.sleep(0.05)
        buzzer.off()  # turn off buzzer
        led.off()     # turn on led
        time.sleep(0.05)
        times-=1

# When sensor is blocked, this function will be executed
def SensorEvent(channel): # The sensor is blocked
    alarm()

def loop():
    sensor.when_reflect = SensorEvent
    while True:
```

```
30          time.sleep(1)
31
32  def destroy():
33      led.close()
34      sensor.close()
35
36  if __name__ == '__main__':     # Program entrance
37      print ('Program is starting...')
38      try:
39          loop()
40      except KeyboardInterrupt:  # Press ctrl-c to end the program.
41          destroy()
42          print("Ending program")
```

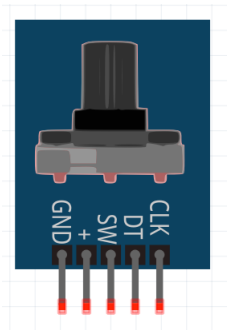
Chapter 31 Rotary Encoder

In this chapter, we will learn how to use rotary encoder.

Project 31.1 Rotary Encoder

This project uses a rotary encoder to make a simple counter.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x7
Rotary encoder x1 	

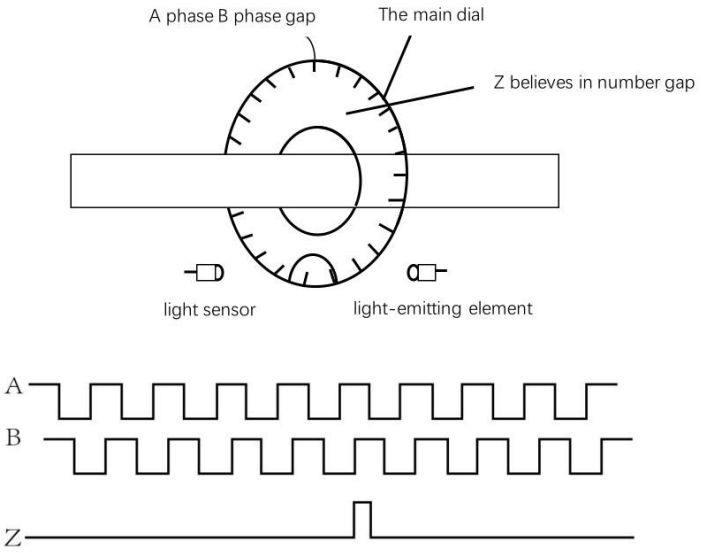
Component knowledge

Rotary Encoder

A rotary encoder is a rotary sensor that converts the rotational displacement into a periodic electrical signal, and then converts the electrical signal into a count pulse, and uses the number of pulses to indicate the magnitude of the rotational displacement. It drives the internal grating disc to rotate through the rotation of the knob. Many slits are preset on the grating disc. The rotation of the grating disc causes the light passing through the slits to produce pulse changes. After the signal is processed by the subsequent circuit, it is output as a pulse signal. Finally, The rotary displacement of the knob can be obtained through the output of the signal pin. The encoder generates pulse signals through the configuration of the internal light source and photosensitive element.

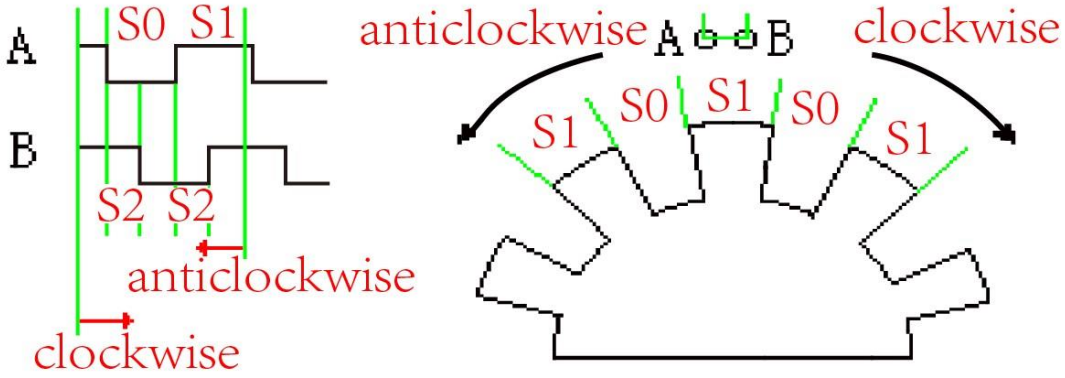
The working principle of the encoder and the schematic diagram of the output waveform are as follows:

clockwise motion	counterclockwise motion
AB	AB
11	11
01	10
00	00
10	01



A rotary encoder gives two-phase square waves, which are 90° out of phase, commonly referred to as A channel and B channel. One of the channels gives the information related to the rotation speed. The information of the rotation direction is obtained by sequentially comparing the signals of the two channels. There is also a special signal called Z or zero channel, which gives the absolute zero position of the encoder. This signal is a square wave that coincides with the center line of the A channel square wave.

The following is the internal cross-sectional structure diagram of the encoder:



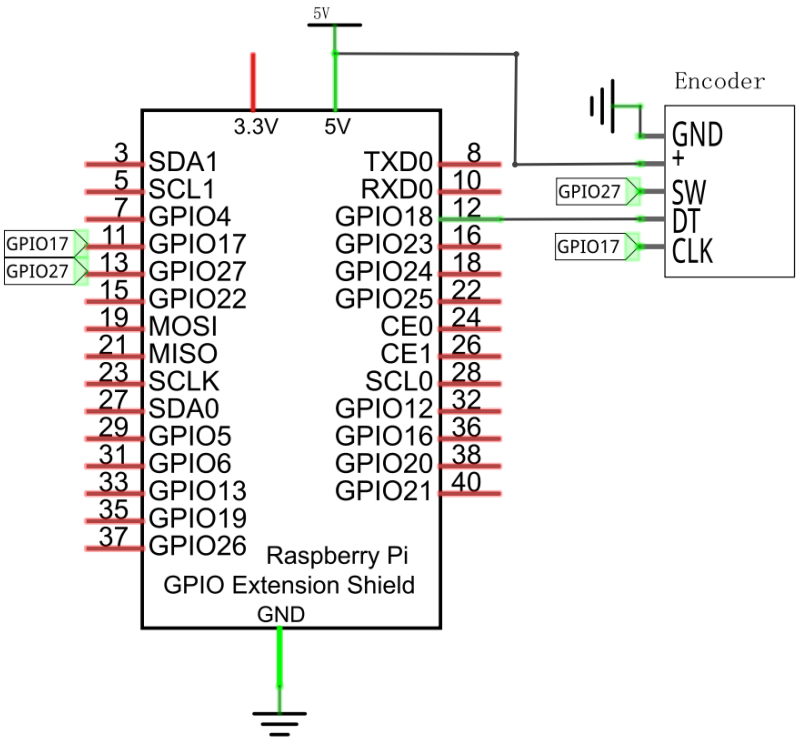
The number of pulses per revolution of the rotary encoder in this project is 20. Its working voltage is 5V. The sensor has 5 pins: SW, CLK, DT, power supply positive pin and power supply negative pin. Among them, the SW pin is the input signal pin, the rotary encoder sensor itself is also a button, when the button is pressed, the SW pin will jump from high level to low level. The CLK pin is a rotation signal pin. When not rotating, this pin outputs a high level, and when rotating, it outputs a low level. The DT pin is used to determine the direction of rotation. If this pin is high when it is not rotating, and becomes low when it is rotating, it means that clockwise rotation has occurred. When this pin is high when it is rotating, it means that counterclockwise rotation has occurred. When the positive and negative pins of the module are connected to a suitable power supply, the module starts to work. At this time, three pins on the development board need to be used to read the SW, CLK and DT of the module respectively. Then according to the above principle, the state of the rotary encoder can be determine.

For example, when you turn the encoder clockwise by hand, the CLK outputs low level, and the DT changes from high level to low level. When you rotate the encoder counterclockwise by hand, the CLK outputs low

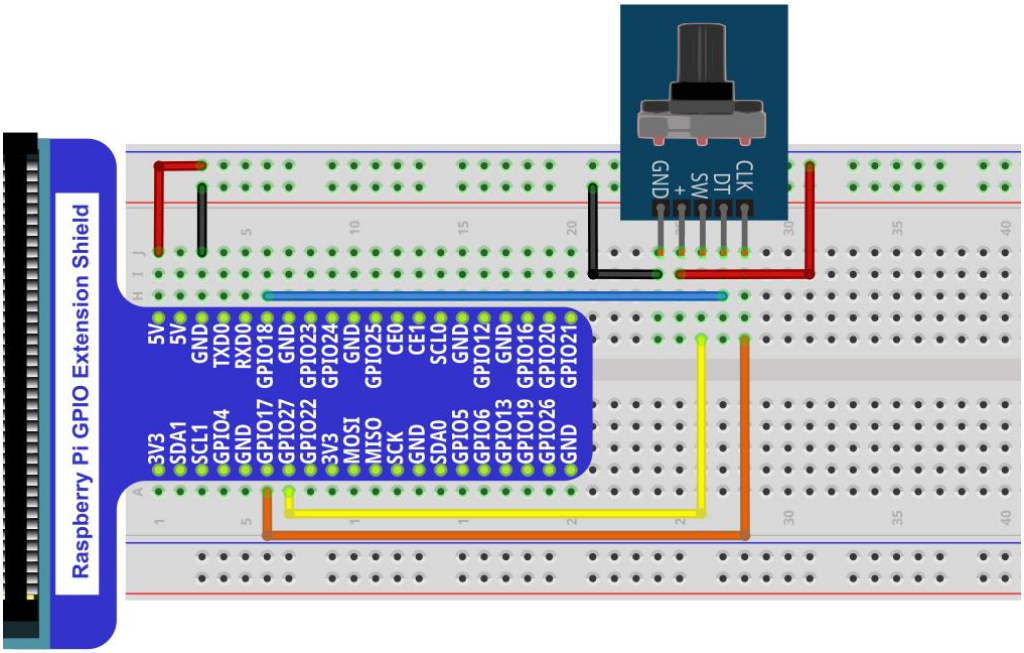
level, and the DT changes from low level to high level. When you press the rotary encoder by hand, the SW signal outputs low level.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 31.1.1 RotaryEncoder

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 31.1.1_RotaryEncoder directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/31.1.1_RotaryEncoder
```

2. Use python command to execute code "RotaryEncoder.py"

```
python  RotaryEncoder.py
```

After the program is executed, the rotary encoder can count the number of output pulses during the forward and reverse rotation. When the rotary encoder is rotated clockwise by hand, the count value will increase. When the rotary encoder is rotated counterclockwise by hand , the count value will decrease. When the rotary encoder is pressed by hand, it can be reset to the initial state, that is, the count starts from 0.

The following is the program code:

```
from gpiozero import DigitalInputDevice,Button
import time

clkPin = 17     # define the clkPin
dtPin  = 18     # define the dtPin
swPin  = 27     # define the swPin

previousCounterValue = 0
symbol = 0
lastDTStatus = 0
currentDTStatus = 0

clk = DigitalInputDevice(clkPin)
dt  = DigitalInputDevice(dtPin)
sw  = Button(swPin,pull_up=True)

def rotaryDeal():
    global symbol
    global lastDTStatus
    global currentDTStatus
    global previousCounterValue
    lastDTStatus = dt.value
    while(not clk.value):     # clk.value is 1 when not rotated and becomes 0 when rotated
        currentDTStatus = dt.value  # Record the current value of the rotation
        symbol = 1
    if symbol == 1:
        symbol = 0
```

```
28          if (lastDTStatus == 1) and (currentDTStatus == 0): # Rotate clockwise, the angular
    displacement increases, and the count value increases
29              previousCounterValue = previousCounterValue + 1
30          if (lastDTStatus == 0) and (currentDTStatus == 1): # Counterclockwise rotation,
    angular displacement reduction, count value reduction
31              previousCounterValue = previousCounterValue - 1
32
33  def sensorEvent():
34      global previousCounterValue
35      previousCounterValue = 0
36
37  def loop():
38      global previousCounterValue
39      currentCounterValue = 0
40      sw.when_pressed = sensorEvent
41      while True:
42          rotaryDeal()
43
44          if currentCounterValue != previousCounterValue:
45              print('Counter = ' ,previousCounterValue)
46              currentCounterValue = previousCounterValue
47
48  def destroy():
49      clk.close()
50      dt.close()
51      sw.close()
52
53  if __name__ == '__main__':     # Program start from here
54      print ('Program is starting ... ')
55      try:
56          loop()
57      except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
    executed.
58          destroy()
59          print("Ending program")
```

Function rotaryDeal() is used to determine whether the rotary encoder is rotating, and when there is rotation, determine whether it is rotating clockwise or counterclockwise. Use variable previousCounterValue to record the number of rotations. When rotating clockwise, the variable increases, and when rotating counterclockwise, the variable decreases.

```
def rotaryDeal():
    global symbol
    global lastDTStatus
    global currentDTStatus
```

```
    global previousCounterValue
    lastDTStatus = dt.value
    while(not clk.value):     # clk.value is 1 when not rotated and becomes 0 when rotated
        currentDTStatus = dt.value  # Record the current value of the rotation
        symbol = 1
    if symbol == 1:
        symbol = 0
        if (lastDTStatus == 1) and (currentDTStatus == 0): # Rotate clockwise, the angular
displacement increases, and the count value increases
            previousCounterValue = previousCounterValue + 1
        if (lastDTStatus == 0) and (currentDTStatus == 1): # Counterclockwise rotation,
angular displacement reduction, count value reduction
            previousCounterValue = previousCounterValue - 1
```

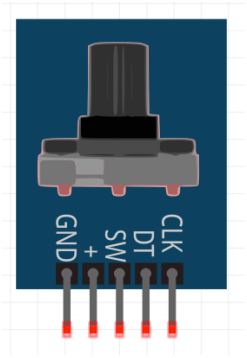
sw.when_pressed associates the SW signal pin of the rotary encoder with sensorEvent(). When swPin detects a low level signal, the sensorEvent() function is called and executed. That is, each time the rotary encoder is pressed, the variable previousCounterValue is cleared.

```
def loop():
    global previousCounterValue
    currentCounterValue = 0
    sw.when_pressed = sensorEvent
def sensorEvent(channel):
    global previousCounterValue
    previousCounterValue = 0
```

Project 31.2 Rotary Encoder and LED

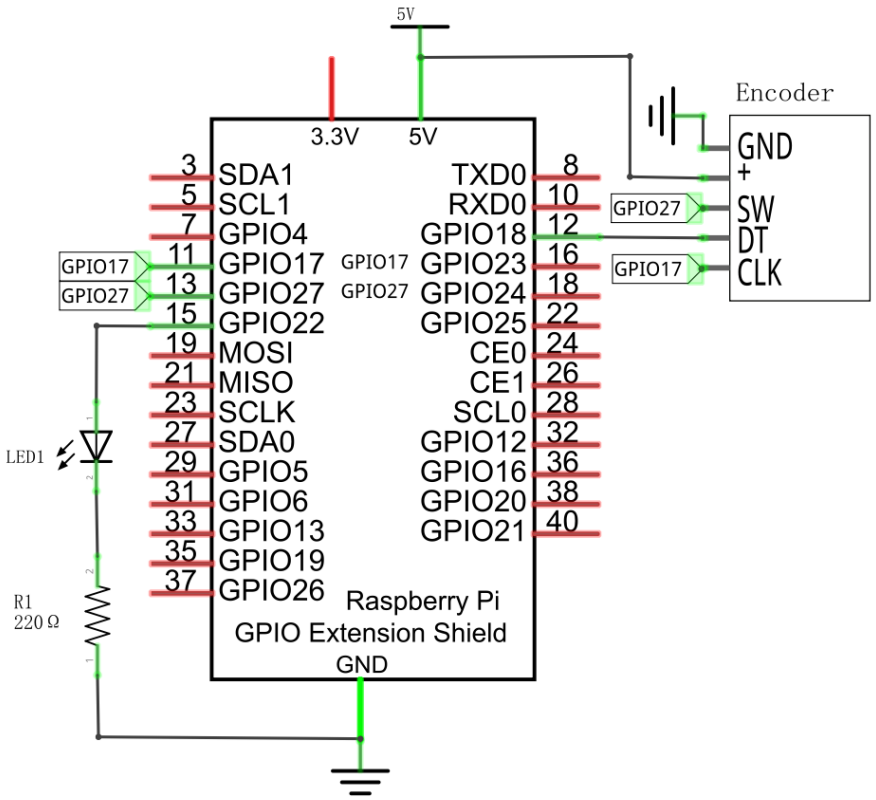
This project uses a rotary encoder to adjust the LEDs to emit different brightness.

Component List

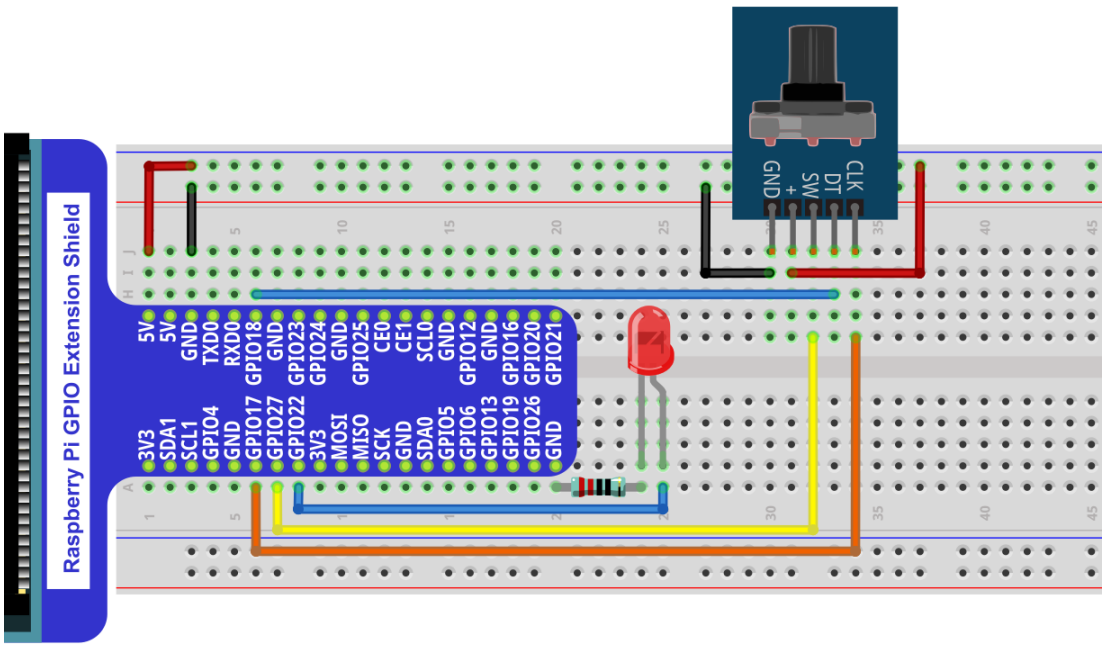
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x8 	
Rotary encoder x1 	LED x1 	Resistor 220Ω x1

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 31.2.1 Dimmable

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 31.2.1_Dimmable directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/31.2.1_Dimmable
```

2. Use python command to execute code "Dimmable.py"

```
python Dimmable.py
```

After the program is executed, the rotary encoder can count the number of output pulses during forward and reverse rotation. Adjust the LED to emit different brightness by the number of rotations. When the rotary encoder is rotated clockwise by hand, the brightness of the led will increase, when the rotary encoder is rotated counterclockwise by hand, the brightness of the led will decrease, and when the rotary encoder is pressed by hand, the led will turn off.

The following is the program code:

```
1  from gpiozero import DigitalInputDevice,Button,PWMLED
2  import time
3
4  clkPin = 17     # define the clkPin
5  dtPin  = 18     # define the dtPin
6  swPin  = 27     # define the swPin
7
8  previousCounterValue = 0
9  symbol = 0
10 lastDTStatus = 0
11 currentDTStatus = 0
12
13 clk = DigitalInputDevice(clkPin)
14 dt  = DigitalInputDevice(dtPin)
15 sw  = Button(swPin,pull_up=True)
16 led = PWMLED(22)
17
18 def rotaryDeal():
19     global symbol
20     global lastDTStatus
21     global currentDTStatus
22     global previousCounterValue
23     lastDTStatus = dt.value
24     while(not clk.value):      # When not rotating, the value of clk.value is 1, and it will
   become 0 when rotating.
25         currentDTStatus = dt.value  # Record the current value of the rotation
26         symbol = 1
```

```
    if symbol == 1:
        symbol = 0
        if (lastDTStatus == 1) and (currentDTStatus == 0): #When rotate clockwise, the angular displacement increases, and the count value increases.
            previousCounterValue = previousCounterValue + 1
        if (lastDTStatus == 0) and (currentDTStatus == 1): #When rotate counterclockwise, the angular displacement decreases, and the count value decreases.
            previousCounterValue = previousCounterValue - 1

def sensorEvent(channel):
    global previousCounterValue
    previousCounterValue = 0

def loop():
    global previousCounterValue
    currentCounterValue = 0
    sw.when_pressed = sensorEvent
    while True:
        rotaryDeal()
        if previousCounterValue>=100:
            previousCounterValue=100
        if previousCounterValue<=0:
            previousCounterValue=0
        if currentCounterValue != previousCounterValue:
            print('Counter = %d' % previousCounterValue)
            currentCounterValue = previousCounterValue
        led.value = previousCounterValue / 100.0     # set dc value as the duty cycle

def destroy():
    clk.close()
    dt.close()
    sw.close()

if __name__ == '__main__':     # Program start from here
    print ('Program is starting ... ')
#     setup()
    try:
        loop()
    except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be executed.
        destroy()
        print("Ending program")
```

Function rotaryDeal() is used to determine whether the rotary encoder is rotating, and when there is rotation,

determined whether it is rotating clockwise or counterclockwise. Use variable previousCounterValue to record the number of rotations. When rotating clockwise, the variable increases, and when rotating counterclockwise, the variable decreases.

```
def rotaryDeal():
    global symbol
    global lastDTStatus
    global currentDTStatus
    global previousCounterValue
    lastDTStatus = dt.value
    while(not clk.value):      #When not rotating, the value of clk.value is 1, and it will become 0 when rotating.
        currentDTStatus = dt.value  # Record the current value while rotating
        symbol = 1
    if symbol == 1:
        symbol = 0
        if (lastDTStatus == 1) and (currentDTStatus == 0): #When rotate clockwise, the angular displacement increases, and the count value increases.
            previousCounterValue = previousCounterValue + 1
        if (lastDTStatus == 0) and (currentDTStatus == 1): #When rotate counterclockwise, the angular displacement decreases, and the count value decreases.
            previousCounterValue = previousCounterValue - 1
```

sw.when_pressed associates the SW signal pin of the rotary encoder with sensorEvent(). When swPin detects a low level signal, the sensorEvent() function is called and executed. That is, each time the rotary encoder is pressed, the variable previousCounterValue is cleared.

```
def loop():
    global previousCounterValue
    currentCounterValue = 0
    sw.when_pressed = sensorEvent
def sensorEvent(channel):
    global previousCounterValue
    previousCounterValue = 0
```

The variable previousCounterValue is limited here, and the PWM duty cycle of ledPin is set to previousCounterValue. The duty cycle of PWM should be less than or equal to 100.

```
        if previousCounterValue>=100:
            previousCounterValue=100
        if previousCounterValue<=0:
            previousCounterValue=0
        if currentCounterValue != previousCounterValue:
            print('Counter = %d' % previousCounterValue)
            currentCounterValue = previousCounterValue
        led.value = previousCounterValue / 100.0     # set dc value as the duty cycle
```

Chapter 32 LEDpixel

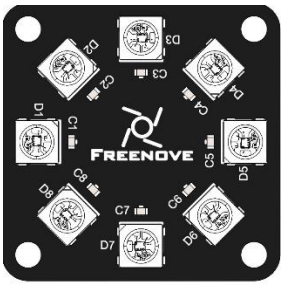
In this chapter, we will learn Freenove 8 RGB LED Module.

Note: if your Raspberry PI is Raspberry Pi5, skip 32.1 and 32.2 and look at 32.3 instead. Because 32.1 and 32.2 use the rpi_ws2812 library, they are not compatible with Raspberry Pi5.

Project 32.1 LEDpixel

This project will achieve an Freenove 8 RGB LED Module flowing water.

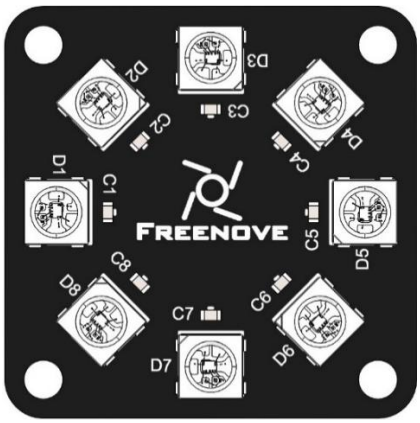
Component List

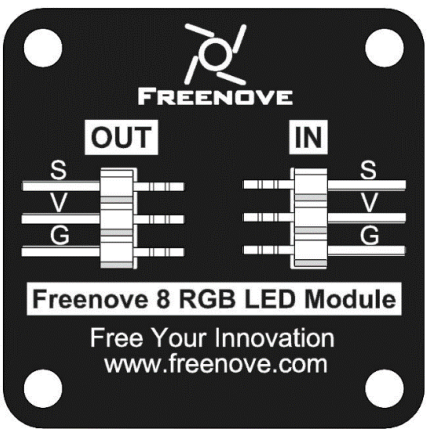
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x4
Freenove 8 RGB LED Module x1 	

Component knowledge

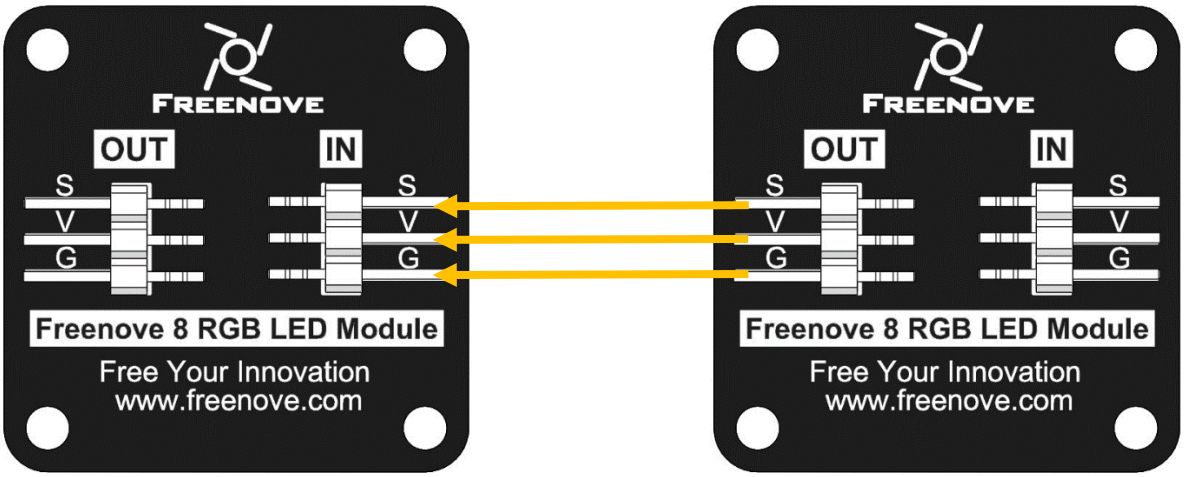
Freenove 8 RGB LED Module

The Freenove 8 RGB LED Module is as below. You can use only one data pin to control the eight LEDs on the module. As shown below:





And you can also control many modules at the same time. Just connect OUT pin of one module to IN pin of another module. In such way, you can use one data pin to control 8, 16, 32 ··· LEDs.

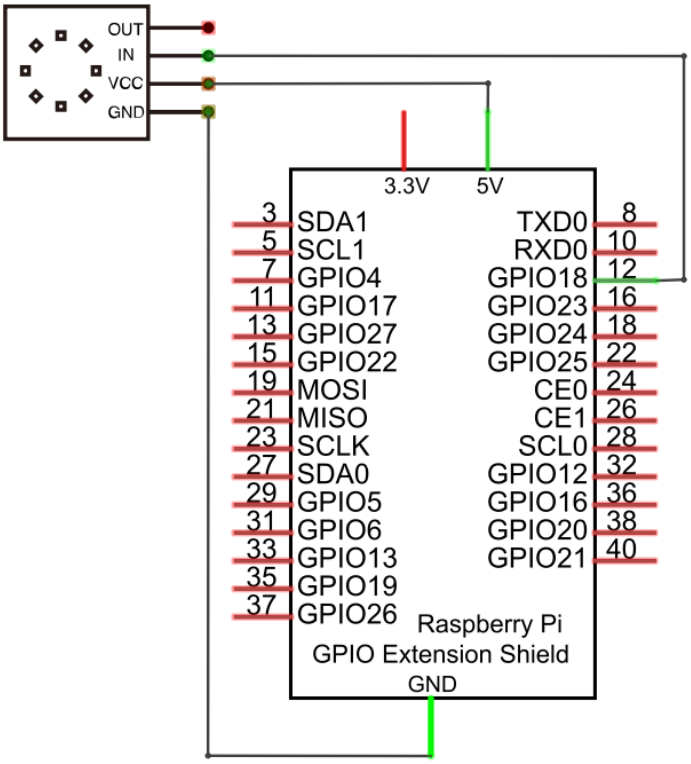


Pin description:

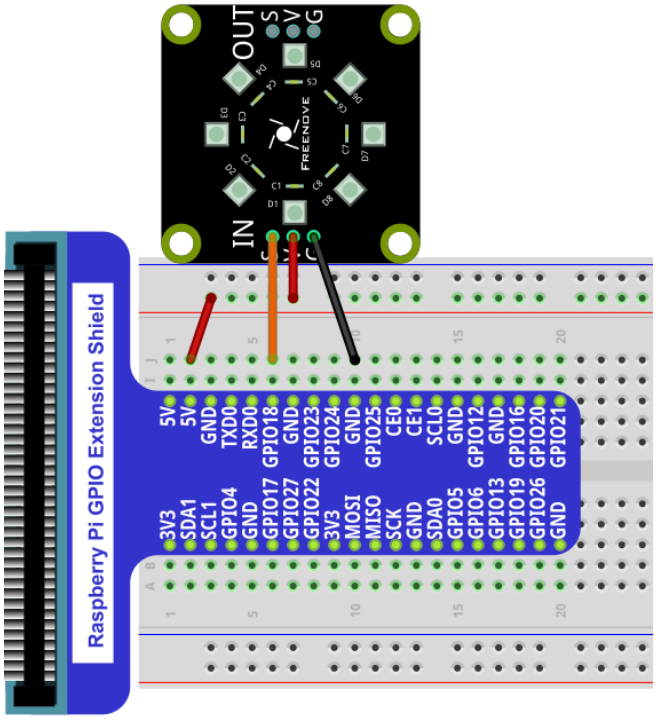
(IN)		(OUT)	
symbol	Function	symbol	Function
S	Input control signal	S	Output control signal
V	Power supply pin, +3.5V~5.5V	V	Power supply pin, +3.5V~5.5V
G	GND	G	GND

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Additional supplement

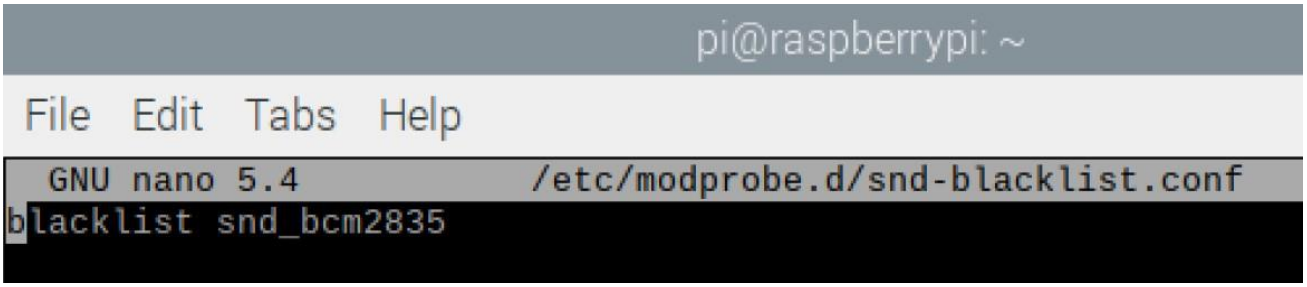
Raspbery Pi, other than 4B and 400, nees to disable the audio module, othewise the LED will not work properly.

1. Create a new snd-blacklist.conf and open it for editing

```
sudo nano /etc/modprobe.d/snd-blacklist.conf
```

Add following content: After adding the contents, you need to press Ctrl+O, Enter, Ctrl+Z.

```
blacklist snd_bcm2835
```



2. We also need to edit config file.

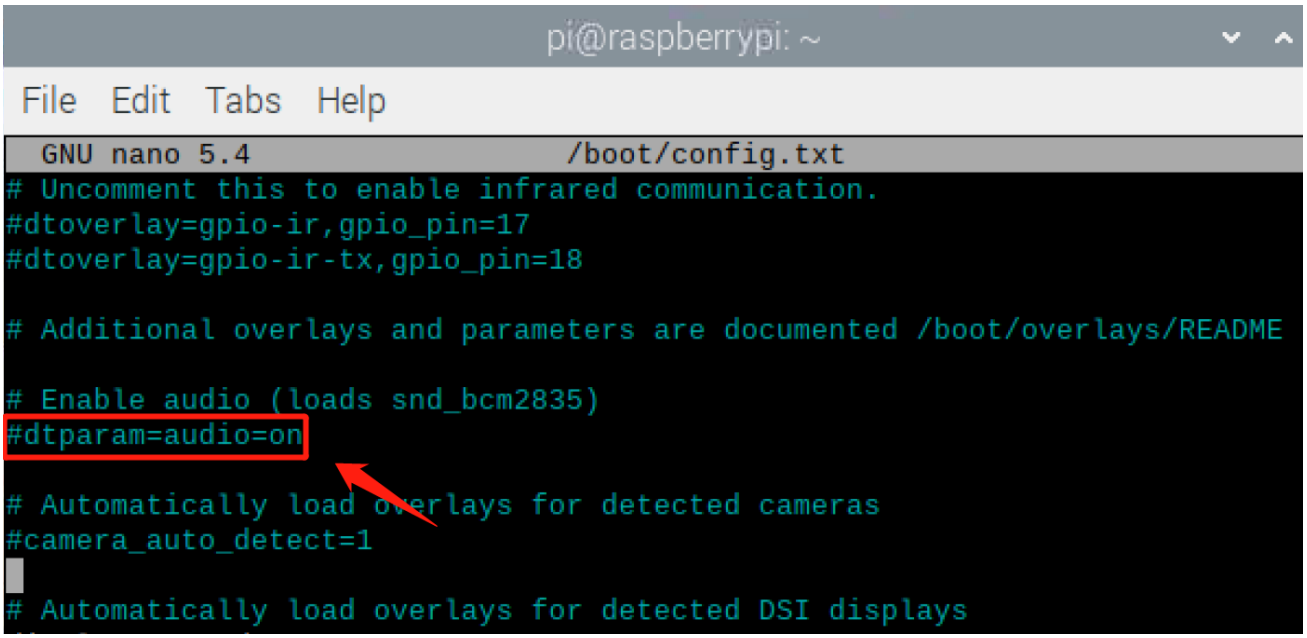
```
sudo nano /boot/config.txt
```

Find the contents of the following two lines (with Ctrl + W you can search):

```
# Enable audio (loads snd_bcm2835)
dtparam=audio=on
```

Add # to comment out the second line. Press Ctrl+O, Enter, Ctrl+X.

```
# Enable audio (loads snd_bcm2835)
# dtparam=audio=on
```



It will take effect after restarting, restart your RPi.

If you want to restart the audio module, just restore the content modified in the above two steps.

Code

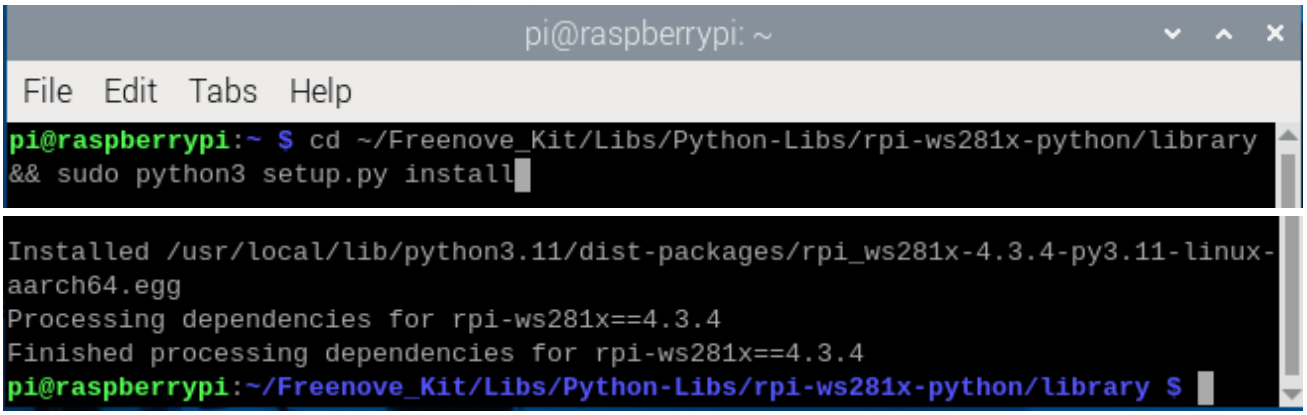
Python Code 32.1.1 Ledpixel

Before running python code, please install WS281X library first.

1. Enter the following command to install.

```
cd ~/Freenove_Kit/Libs/Python-Libs/rpi-ws281x-python/library && sudo python3 setup.py install
```

The installation is completed as shown in the figure below.



First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

1. Use cd command to enter 32.1.1_Ledpixel directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/32.1.1_Ledpixel
```

2. Use python command to execute code "Ledpixel.py".

```
sudo python Ledpixel.py
```

After the program runs, the LEDpixel will emit red, green and blue colors in turn like flowing water. If your Freenove 8 RGB LED Module doesn't work, you can try additional supplements to solve.

The following is the program code:

```
1  import time
2  from rpi_ws281x import *
3  # LED strip configuration:
4  LED_COUNT      = 8      # Number of LED pixels.
5  LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
6  LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
7  LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
8  LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
9  LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
10 LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
11 # Define functions which animate LEDs in various ways.
12 class Led:
13     def __init__(self):
```

```
        #Control the sending order of color data
        self.ORDER = "RGB"
        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
LED_BRIGHTNESS, LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()
        #self.strip.setPixelColor(i, color)
        #self.strip.show()
led=Led()
# Main program logic follows:
if __name__ == '__main__':
    print ('Program is starting ... ')
    col=[Color(255,0,0),Color(0,255,0),Color(0,0,255)]
    try:
        while True:
            for c in range(3):
                for i in range(8):
                    led.strip.setPixelColor(i,col[c])
                    time.sleep(0.1)
                    led.strip.show()
    except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
executed.
        for i in range(8):
            led.strip.setPixelColor(i, Color(0,0,0))
        led.strip.show()
```

Import rpi_ws281x modile. Set the number, pins and brightness of the LED.

```
from rpi_ws281x import *
# LED strip configuration:
LED_COUNT      = 8      # Number of LED pixels.
LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
```

Define LED class.

```
class Led:
    def __init__(self):
        #Control the sending order of color data
        self.ORDER = "RGB"
        # Create NeoPixel object with appropriate configuration.
```

```
        self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
LED_BRIGHTNESS, LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()
        #self.strip.setPixelColor(i, color)
        #self.strip.show()
```

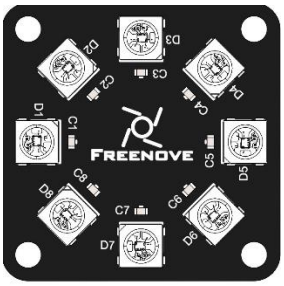
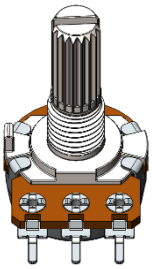
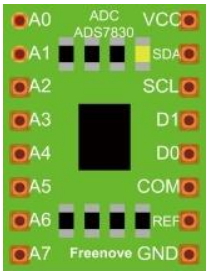
Light up the eight LEDs in red, green and blue in turn.

```
    col=[Color(255,0,0),Color(0,255,0),Color(0,0,255)]
    try:
        while True:
            for c in range(3):
                for i in range(8):
                    led.strip.setPixelColor(i,col[c])
                    time.sleep(0.1)
                    led.strip.show()
    except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
executed.
        for i in range(8):
            led.strip.setPixelColor(i, Color(0,0,0))
        led.strip.show()
```

Project 32.2 RainbowLight

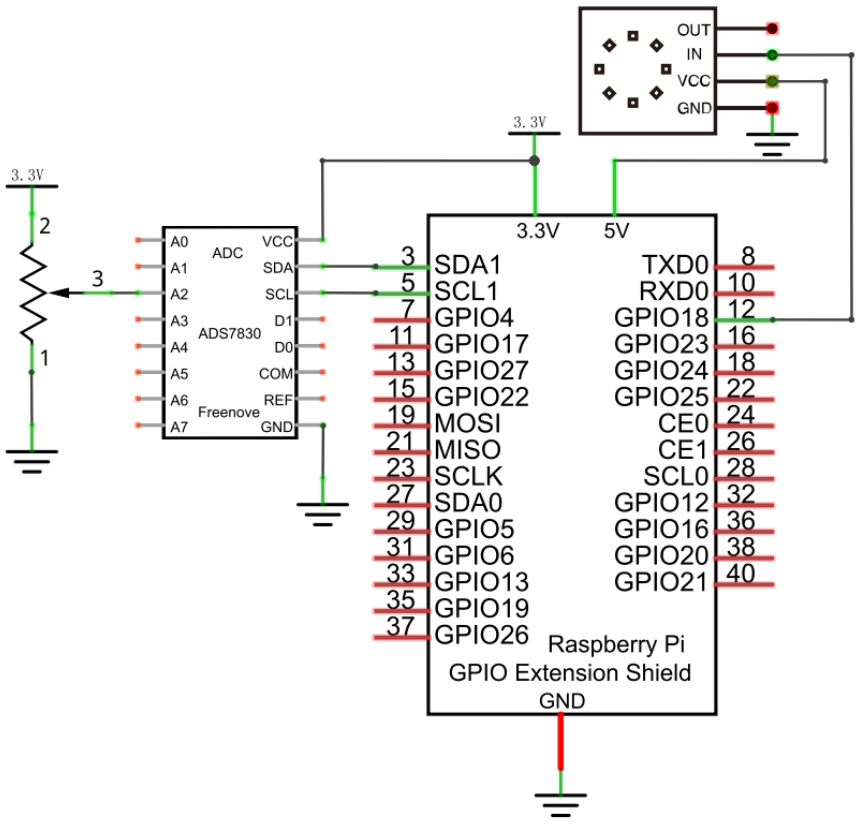
In this project, we will learn to control the LED module with a potentiometer.

Component List

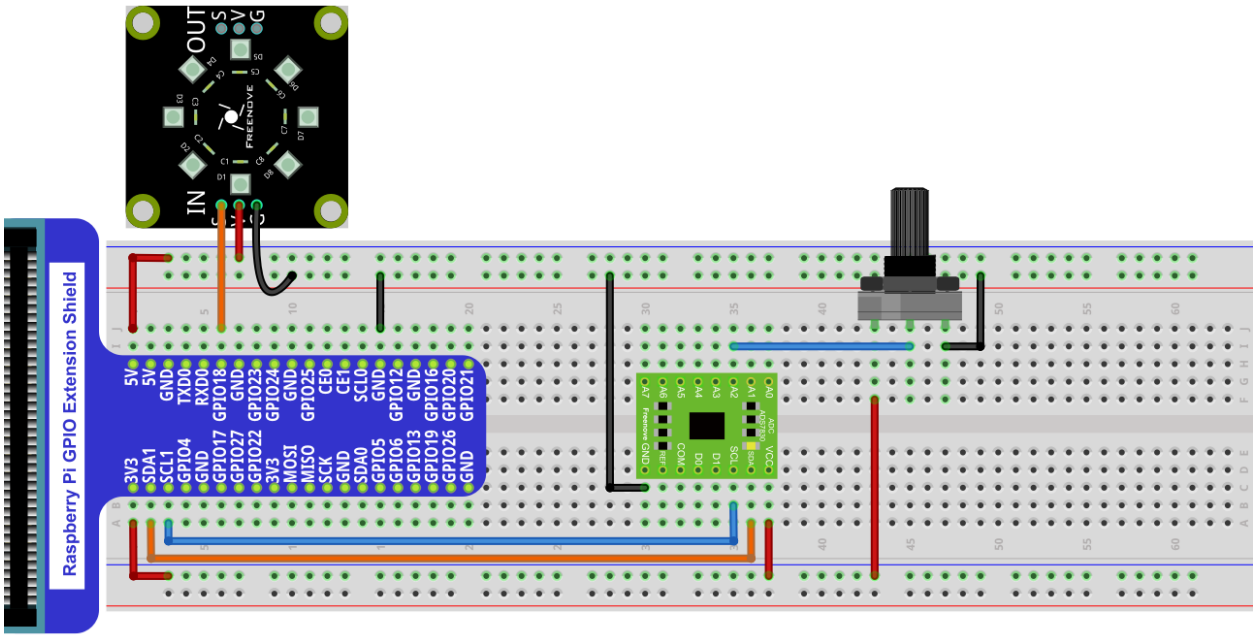
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x13	
Freenove 8 RGB LED Module x1 	Rotary Potentiometer x1 	ADC Module x1 

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

In this project, we will make a Servo rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

Python Code 32.2.1 RainbowLight

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.

For Python code, ADCDevice requires a custom module which needs to be installed. If you have installed it in Chapter 7.Please skip the installation.

1. Use cd command to enter folder of ADCDevice.

```
cd ~/Freenove_Kit/Libs/Python-Libs/
```

2. Open the unzipped folder.

```
cd ADCDevice-1.0.4
```

3. Install library for python2 and python3.

```
sudo python2 setup.py install
sudo python3 setup.py install
```

First observe the project result, and then learn about the code in detail.

If you have any concerns, please send an email to: support@freenove.com

4. Use cd command to enter 32.2.1_Rainbow Light directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/32.2.1_RainbowLight
```

5. Use python command to execute code " RainbowLight.py ".

```
sudo python RainbowLight.py
```

After running the program, you can change the color of the LED module by rotating the potentiometer.

The following is the program code:

```
import time
from rpi_ws281x import *
from ADCDevice import *
# LED strip configuration:
LED_COUNT      = 8      # Number of LED pixels.
LED_PIN        = 18      # GPIO pin connected to the pixels (18 uses PWM!).
LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53
# Define functions which animate LEDs in various ways.
class Led:
    def __init__(self):
        #Control the sending order of color data
        self.ORDER = "RGB"
        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(LED_COUNT, LED_PIN, LED_FREQ_HZ, LED_DMA, LED_INVERT,
LED_BRIGHTNESS, LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()

        self.adc = ADCDevice(0x4b) # Define an ADCDevice class object
        if(self.adc.detectI2C(0x4b)):
            self.adc = ADS7830(0x4b)
        elif(adc.detectI2C(0x48)):  # Detect the pcf8591.
            self.adc = PCF8591()
        else:
            print("No correct I2C address found, \n"
            "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
            "Program Exit. \n");
            exit(-1)

    def HSL_RGB(self,degree):
        degree=degree/360*255
        if degree < 85:
            red = 255 - degree * 3
            green = degree * 3
            blue = 0
        elif degree < 170:
            degree = degree - 85
            red = 0
            green = 255 - degree * 3
```

```
43              blue = degree * 3
44          else:
45              degree = degree - 170
46              red = degree * 3
47              green = 0
48              blue = 255 - degree * 3
49          return int(red),int(green),int(blue)
50  led=Led()
51  # Main program logic follows:
52  if __name__ == '__main__':
53      print ('Program is starting ... ')
54      try:
55          while True:
56              for i in range(8):
57                  value = round(led.adc.analogRead(2) / 255.0 * 360+i*45)     # read the ADC
    value of channel 2
58                  if value > 360 :
59                      value = value-360
60                  red,green,blue=led.HSL_RGB(value)
61                  led.strip.setPixelColor(i, Color(red,green,blue))
62              time.sleep(0.1)
63              led.strip.show()
64      except KeyboardInterrupt:  # When 'Ctrl+C' is pressed, the child program destroy() will be
    executed.
65          led.adc.close()
66          for i in range(8):
67              led.strip.setPixelColor(i, Color(0,0,0))
68          led.strip.show()
```

This function converts HSL colors to RGB colors.

```
    def HSL_RGB(self,degree):
        degree=degree/360*255
        if degree < 85:
            red = 255 - degree * 3
            green = degree * 3
            blue = 0
        elif degree < 170:
            degree = degree - 85
            red = 0
            green = 255 - degree * 3
            blue = degree * 3
        else:
            degree = degree - 170
            red = degree * 3
            green = 0
```

```
            blue = 255 - degree * 3
        return int(red),int(green),int(blue)
```

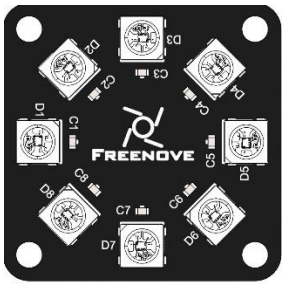
Read the ADC value of channel 2 in an infinite loop. Let the color of the eight LEDs change according to the value of the ADC.

```
        while True:
                for i in range(8):
                    value = round(led.adc.analogRead(2) / 255.0 * 360+i*45)     # read the ADC
value of channel 2
                    if value > 360 :
                        value = value-360
                    red,green,blue=led.HSL_RGB(value)
                    led.strip.setPixelColor(i, Color(red,green,blue))
                time.sleep(0.1)
                led.strip.show()
```

Project 32.3 SPI_LEDpixel

This project will achieve an Freenove 8 RGB LED Module flowing water.

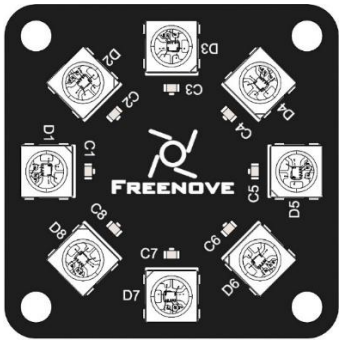
Component List

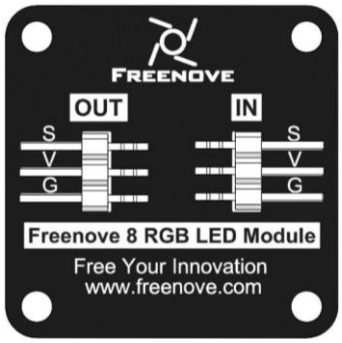
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x4
Freenove 8 RGB LED Module x1 	

Component knowledge

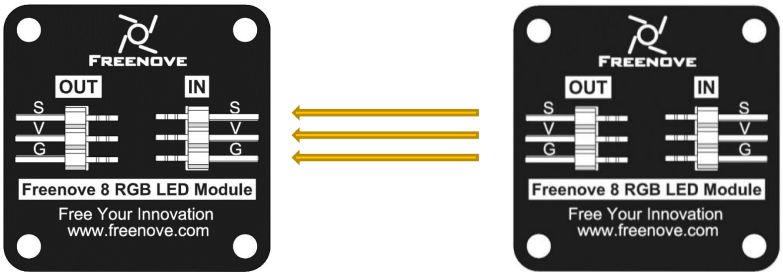
Freenove 8 RGB LED Module

The Freenove 8 RGB LED Module is as below. You can use only one data pin to control the eight LEDs on the module. As shown below:





And you can also control many modules at the same time. Just connect OUT pin of one module to IN pin of another module. In such way, you can use one data pin to control 8, 16, 32 ··· LEDs.

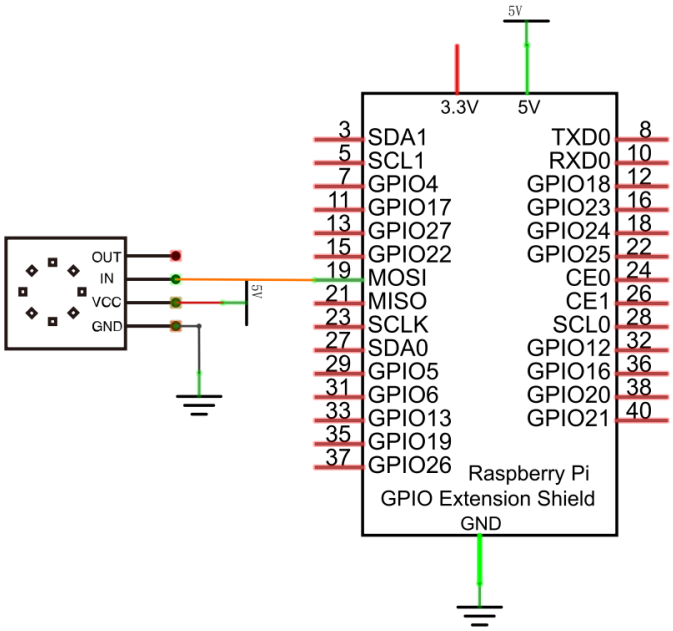


Pin description:

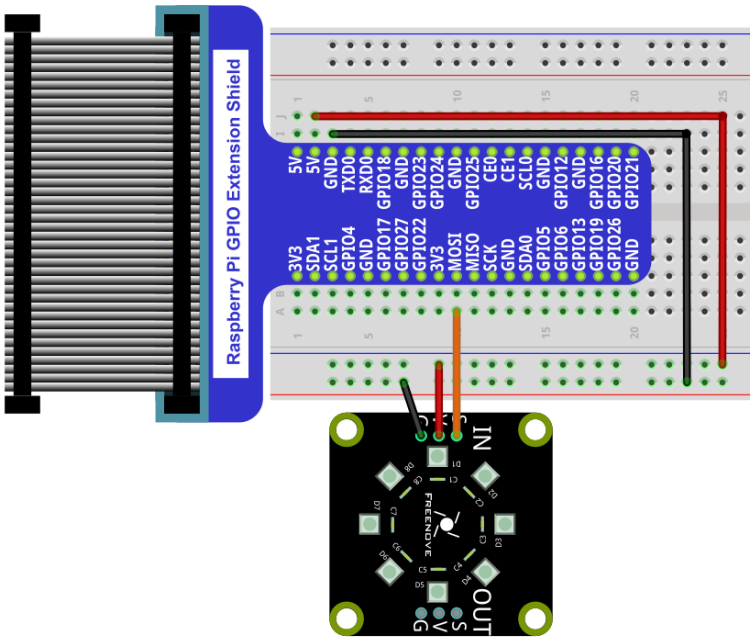
(IN)		(OUT)	
symbol	Function	symbol	Function
S	Input control signal	S	Output control signal
V	Power supply pin, +3.5V~5.5V	V	Power supply pin, +3.5V~5.5V
G	GND	G	GND

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

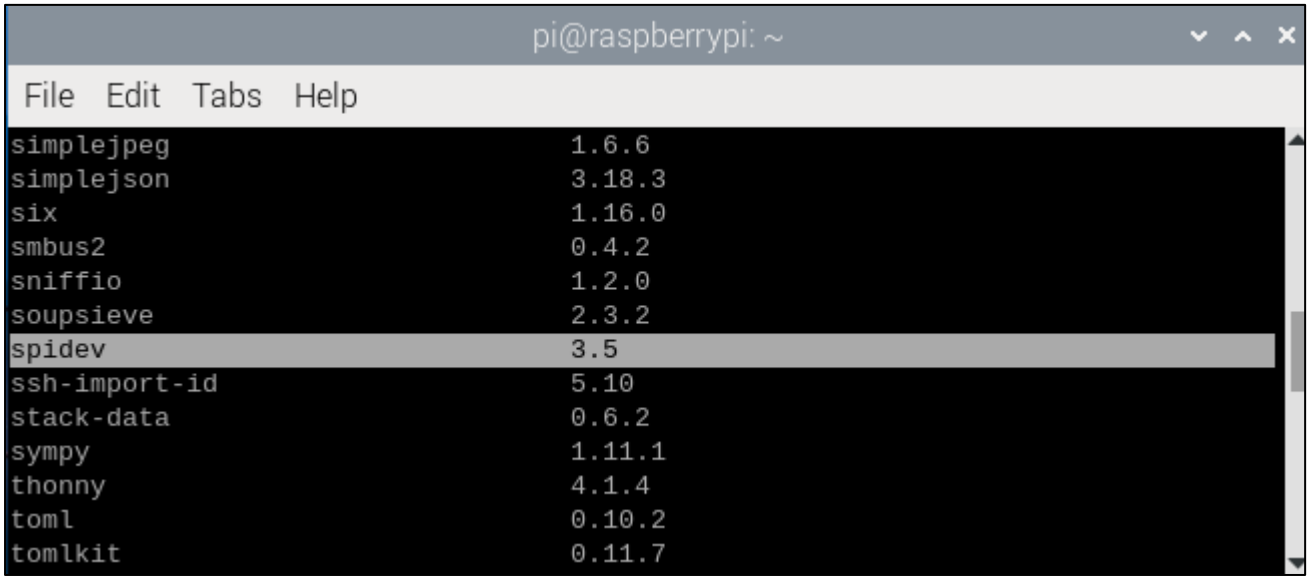
Python Code 32.3.1 Ledpixel

Before you run your python code, check that the spidev library exists.

Enter the following command to install.

```
pip list
```

The spidev is installed on Raspberry PI by default. As shown in the figure below.

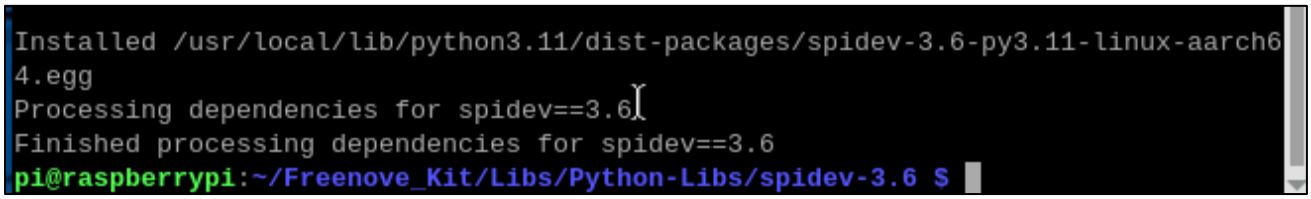


If your Raspberry PI system does not have this library, you can find **spidev-3.6.tar.gz** in **Freenove_Kit/Libs/Python-Libs**.

Enter the following instructions to install spidev.

```
cd Freenove_Kit/Libs/Python-Libs
tar –zxvf spidev-3.6.tar.gz
cd spidev-3.6
sudo python setup.py install
```

The installation is complete as shown in the following figure.



If you have any concerns, please send an email to: support@freenove.com

Additional supplement

Note that the frequency of the SPI changes as the CPU frequency self-regulates, so we need to fix the cpu frequency before we start using the code. Please refer to the following operations.

1. Open the config.txt file and prepare to edit it.

```
sudo nano /boot/firmware/config.txt
```

2, If your Raspberry PI is Raspberry PI 4 or Raspberry PI 5, please add at the bottom:

```
force_turbo=1
```

If your Raspberry PI is Raspberry PI 3, add it at the bottom:

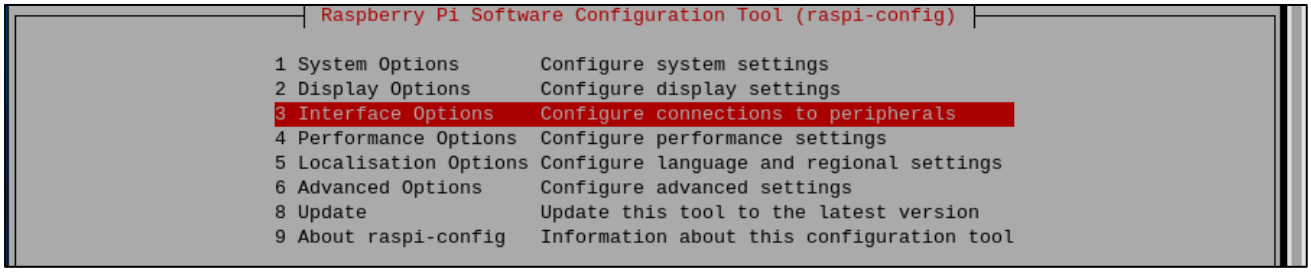
```
core_freq=250
```

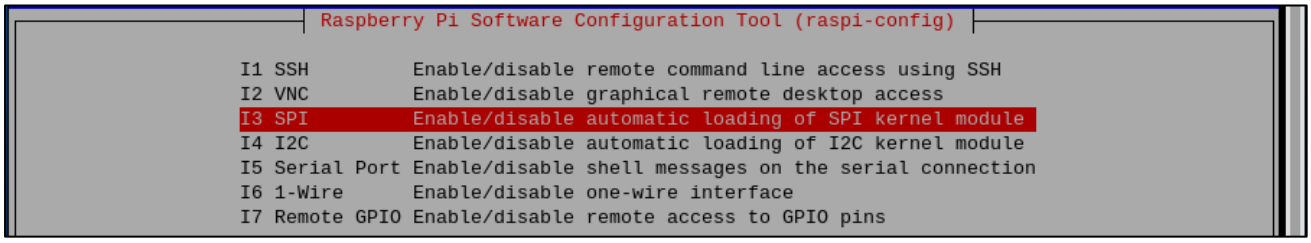
3, Save the file.

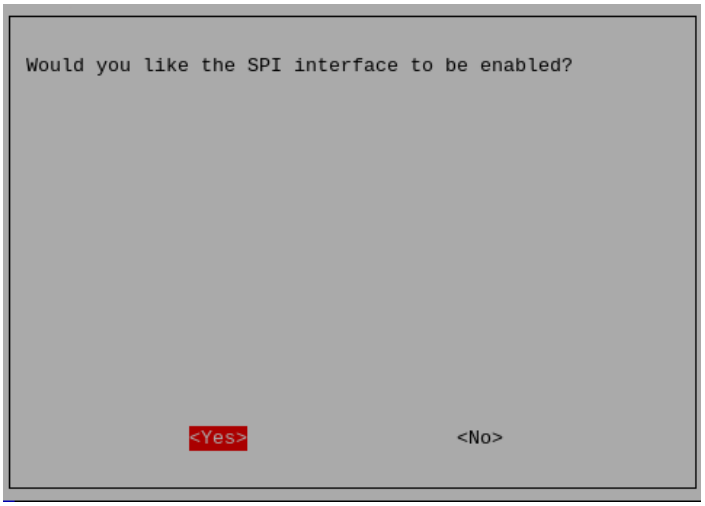
4, Turn on the spi feature of the Raspberry PI.

```
sudo raspi-config
```

5, **Select Interface Options, then SPI, and turn it on.**







6, Select Finish.

7, Reboot the Raspberry PI.

```
sudo reboot
```

Use cd command to enter 32.3.1_Ledpixel directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/32.3.1_Ledpixel
```

Use python command to execute code "Ledpixel.py".

```
python Ledpixel.py
```

After the program runs, ledpixel emits red, green, and blue three colors in turn. And then the colors of the rainbow.

The following is the program code:

```
from SPI_Ledpixel import Freenove_SPI_LedPixel
import time

# Main program logic follows:
if __name__ == '__main__':
    # Freenove_SPI_LedPixel(led_count, led_brightness, led_transmission_sequence, spidev_bus)
    led = Freenove_SPI_LedPixel(8, 255, 'GRB', 0)

    try:
        if led.check_spi_state() != 0:
            led.set_led_count(8)                        # Set the number of lights.
            led.set_led_brightness(20)                  # Set the brightness of lights.
            color = [[255,0,0],[0,255,0],[0,0,255],[0,0,0]]  # Set the color of the lights
            for j in range(4):
                for i in range(8):
                    # Set the color of the lights, but it does not take effect
                    led.set_led_rgb_data(i, color[j])
                    # Send the color data and make the color data effective
                    led.show()
                    time.sleep(0.1)

            while True:
                for j in range(255):
                    for i in range(led.led_count):
                        #Converts values ranging from 0 to 255 to color data.
                        led.set_led_rgb_data(i, led.wheel((round(i * 255 / led.led_count) + j)%256))
                    led.show()
                    time.sleep(0.002)
        else:
            led.led_close()
    except KeyboardInterrupt:
        led.led_close()
```

Call the light library and the time library.

```
from SPI_Ledpixel import Freenove_SPI_LedPixel
import time
```

Apply for a light object, set the number of lights to 8, brightness to 255, light type to "GRB", use the mosi pin of spi0 to control the lights.

```
# Freenove_SPI_LedPixel(led_count, led_brightness, led_transmission_sequence, spidev_bus)
led = Freenove_SPI_LedPixel(8, 255, 'GRB', 0)
```

Check whether the SPI is configured successfully.

```
if led.check_spi_state() != 0:
```

Reconfigure the number and brightness of ledpixel.

```
led.set_led_count(8)                    # Set the number of lights.
led.set_led_brightness(20)              # Set the brightness of lights.
```

Let ledpixel display red, green and blue one by one, then turn off.

```
color = [[255,0,0],[0,255,0],[0,0,255],[0,0,0]]    # Set the color of the lights
for j in range(4):
    for i in range(8):
        # Set the color of the lights, but it does not take effect
        led.set_led_rgb_data(i, color[j])
        # Send the color data and make the color data effective
        led.show()
        time.sleep(0.1)
```

Let ledpixel cycle rainbow colors.

```
while True:
    for j in range(255):
        for i in range(led.led_count):
            #Converts values ranging from 0 to 255 to color data.
            led.set_led_rgb_data(i, led.wheel((round(i * 255 / led.led_count) + j)%256))
        led.show()
        time.sleep(0.002)
```

For more details, see Freenove_Kit/Code/Python_GPIOZero_Code/SPI_Ledpixel.py.

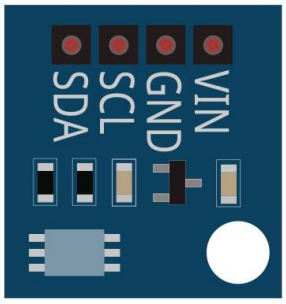
Chapter 33 BMP180 Barometric Pressure Sensor

In this chapter, we will learn how to use BMP180 barometric pressure sensor.

Project 33.1 Barometer

This project uses the BMP180 barometric pressure sensor to display real-time barometric pressure, temperature conditions and current altitude.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x6
BMP180 barometric pressure sensor x1 	

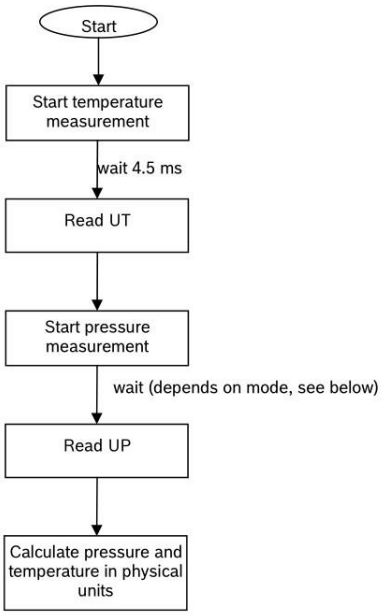
Component knowledge

BMP180 barometric pressure sensor

The BMP180 barometric pressure sensor module is a high-precision barometric pressure sensor, which can detect temperature and barometric pressure, and through these two data, the current altitude can be calculated. Its pressure measurement range is from 300 to 1100 hPa (altitude 9000m to -500m), and temperature from -40°C to 85°C with an accuracy of ±1.0°C. Using the BMP180 module to get altitude and barometric pressure requires math.

The BMP180 is designed to connect directly to the microcontroller via the IIC bus. The pressure and temperature data must be compensated by the BMP180's E2PROM calibration data. The BMP180 consists of a piezoresistive sensor, an analog-to-digital converter and a control unit with E2PROM and serial IIC interface. The BMP180 provides uncompensated pressure and temperature values. The E2PROM stores 176 bits of individual calibration data, which is used to compensate sensor for offset, temperature dependence and other parameters.

The general process of BMP180 reading data and the calculation of each data are as follows:



Among them, UT is temperature data (16 bits), and UP is pressure data (16 to 19 bits).
Calculate temperature:

calculate true temperature
$X1 = (UT - AC6) * AC5 / 2^{15}$
$X2 = MC * 2^{11} / (X1 + MD)$
$B5 = X1 + X2$
$T = (B5 + 8) / 2^{4}$

Calculate pressure:

calculate true pressure
$B6 = B5 - 4000$
$X1 = (B2 * (B6 * B6 / 2^{12})) / 2^{11}$
$X2 = AC2 * B6 / 2^{11}$
$X3 = X1 + X2$
$B3 = ((AC1*4+X3) << oss + 2) / 4$
$X1 = AC3 * B6 / 2^{13}$
$X2 = (B1 * (B6 * B6 / 2^{12})) / 2^{16}$
$X3 = ((X1 + X2) + 2) / 2^{2}$
$B4 = AC4 * (unsigend\ long)(X3 + 32768) / 2^{15}$
$B7 = ((unsigned\ long)UP - B3) * (50000 >> oss)$
if (B7 < 0x80000000) { p = (B7 * 2) / B4 }
else { p = (B7 / B4) * 2 }
$X1 = (p / 2^{8}) * (p / 2^{8})$
$X1 = (X1 * 3038) / 2^{16}$
$X2 = (-7357 * p) / 2^{16}$
$p = p + (X1 + X2 + 3791) / 2^{4}$

Calculate absolute altitude. According to the measured air pressure p and sea level air pressure po (for example, 1013.25hPa), the altitude in meters can be calculated via the following international pressure formula:

$$altitude = 44330 * \left(1 - \left(\frac{p}{p_0}\right)^{\frac{1}{5.255}}\right)$$

Below is the pinout of BMP180 barometric pressure sensor.

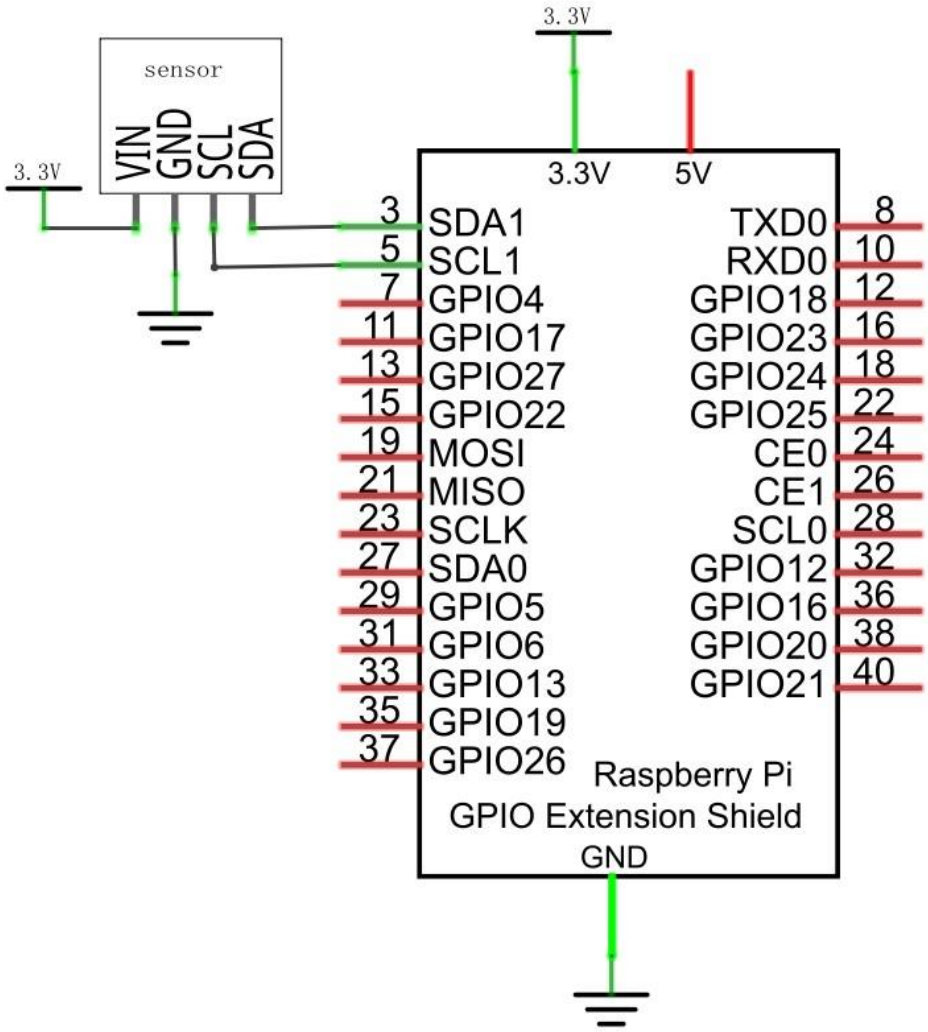
Pin description:

symbol	Function
VIN	Power supply pin, +1.6V~3.6V
GND	GND
SCL	Serial clock pin of I2C interface
SDA	Serial data pin of I2C interface

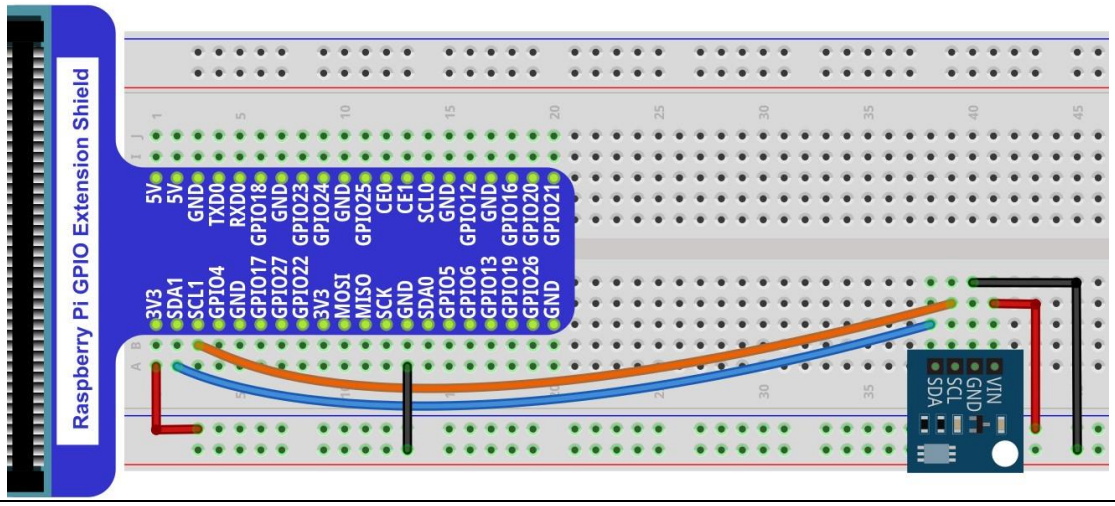
Please do not use voltage beyond the power supply range to avoid damage to the BMP180 barometric pressure sensor.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Python Code 33.1.1 Barometer

If you did not configure I2C and Install Smbus, please refer to Chapter 7. If you did, please move on.
First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Make sure you have set python3 as default python. Then use cd command to enter 33.1.1_Barometer directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/33.1.1_Barometer
```

2. Use python command to execute code "Barometer.py"

```
python Barometer.py
```

After the program is executed, the terminal will display the current air pressure value, temperature value and altitude.

The following is the program code:

```
1  import time
2  import smbus
3
4  # BMP180 default address.
5  BMP180_I2CADDR = 0x77
6
7  # Operating Modes
8  BMP180_ULTRALOWPOWER = 0
9  BMP180_STANDARD = 1
10 BMP180_HIGHRES = 2
11 BMP180_ULTRAHIGHRES = 3
12
13 # BMP180 Registers
14 BMP180_AC1 = 0xAA  #   Calibration data (16 bits)
15 BMP180_AC2 = 0xAC  #   Calibration data (16 bits)
16 BMP180_AC3 = 0xAE  #   Calibration data (16 bits)
17 BMP180_AC4 = 0xB0  #   Calibration data (16 bits)
18 BMP180_AC5 = 0xB2  #   Calibration data (16 bits)
19 BMP180_AC6 = 0xB4  #   Calibration data (16 bits)
20 BMP180_B1 = 0xB6   #   Calibration data (16 bits)
21 BMP180_B2 = 0xB8   #   Calibration data (16 bits)
22 BMP180_MB = 0xBA   #   Calibration data (16 bits)
23 BMP180_MC = 0xBC   #   Calibration data (16 bits)
24 BMP180_MD = 0xBE   #   Calibration data (16 bits)
25 BMP180_CONTROL = 0xF4
26 BMP180_TEMPDATA = 0xF6
27 BMP180_PRESSUREDATA = 0xF6
28
```

```
# Commands
BMP180_READTEMPCMD = 0x2E
BMP180_READPRESSURECMD = 0x34

class BMP180(object):
    def __init__(self, address=BMP180_I2CADDR, mode=BMP180_ULTRAHIGHRES):
        self._mode = mode
        self._address = address
        self._bus = smbus.SMBus(1)
        # Load calibration values.
        self._load_calibration()
        # Kalman Filter
        self._x_last = 0
        self._p_last = 0
    def _read_byte(self, cmd):
        return self._bus.read_byte_data(self._address, cmd)
    def _read_u16(self, cmd):
        MSB = self._bus.read_byte_data(self._address, cmd)
        LSB = self._bus.read_byte_data(self._address, cmd + 1)
        return (MSB << 8) + LSB
    def _read_s16(self, cmd):
        result = self._read_u16(cmd)
        if result > 32767: result -= 65536
        return result
    def _write_byte(self, cmd, val):
        self._bus.write_byte_data(self._address, cmd, val)
    def _load_calibration(self):
        "load calibration"
        self.AC1 = self._read_s16(BMP180_AC1)  # INT16
        self.AC2 = self._read_s16(BMP180_AC2)  # INT16
        self.AC3 = self._read_s16(BMP180_AC3)  # INT16
        self.AC4 = self._read_u16(BMP180_AC4)  # UINT16
        self.AC5 = self._read_u16(BMP180_AC5)  # UINT16
        self.AC6 = self._read_u16(BMP180_AC6)  # UINT16
        self.B1 = self._read_s16(BMP180_B1)  # INT16
        self.B2 = self._read_s16(BMP180_B2)  # INT16
        self.MB = self._read_s16(BMP180_MB)  # INT16
        self.MC = self._read_s16(BMP180_MC)  # INT16
        self.MD = self._read_s16(BMP180_MD)  # INT16
    def read_raw_temp(self):
        """Reads the raw (uncompensated) temperature from the sensor."""
        self._write_byte(BMP180_CONTROL, BMP180_READTEMPCMD)
        time.sleep(0.005)  # Wait 5ms
        raw = self._read_u16(BMP180_TEMPDATA)
```

```
73          return raw
74      def read_raw_pressure(self):
75          """Reads the raw (uncompensated) pressure level from the sensor."""
76          self._write_byte(BMP180_CONTROL, BMP180_READPRESSURECMD + (self._mode << 6))
77          if self._mode == BMP180_ULTRALOWPOWER:
78              time.sleep(0.005)
79          elif self._mode == BMP180_HIGHRES:
80              time.sleep(0.014)
81          elif self._mode == BMP180_ULTRAHIGHRES:
82              time.sleep(0.026)
83          else:
84              time.sleep(0.008)
85          MSB = self._read_byte(BMP180_PRESSUREDATA)
86          LSB = self._read_byte(BMP180_PRESSUREDATA + 1)
87          XLSB = self._read_byte(BMP180_PRESSUREDATA + 2)
88          raw = ((MSB << 16) + (LSB << 8) + XLSB) >> (8 - self._mode)
89          return raw
90      def read_temperature(self):
91          UT = self.read_raw_temp()
92          X1 = ((UT - self.AC6) * self.AC5) >> 15
93          X2 = (self.MC << 11) // (X1 + self.MD)
94          B5 = X1 + X2
95          temp = ((B5 + 8) >> 4) / 10.0
96          return temp
97      def read_pressure(self):
98          UT = self.read_raw_temp()
99          UP = self.read_raw_pressure()
100         X1 = ((UT - self.AC6) * self.AC5) >> 15
101         X2 = (self.MC << 11) // (X1 + self.MD)
102         B5 = X1 + X2
103         # Pressure Calculations
104         B6 = B5 - 4000
105         X1 = (self.B2 * (B6 * B6) >> 12) >> 11
106         X2 = (self.AC2 * B6) >> 11
107         X3 = X1 + X2
108         B3 = (((self.AC1 * 4 + X3) << self._mode) + 2) // 4
109         X1 = (self.AC3 * B6) >> 13
110         X2 = (self.B1 * ((B6 * B6) >> 12)) >> 16
111         X3 = ((X1 + X2) + 2) >> 2
112         B4 = (self.AC4 * (X3 + 32768)) >> 15
113         B7 = (UP - B3) * (50000 >> self._mode)
114         if B7 < 0x80000000:
115             p = (B7 * 2) // B4
116         else:
```

```
117             p = (B7 // B4) * 2
118         X1 = (p >> 8) * (p >> 8)
119         X1 = (X1 * 3038) >> 16
120         X2 = (-7357 * p) >> 16
121         p = p + ((X1 + X2 + 3791) >> 4)
112         return p
113     def read_altitude(self, local_pa=101325.0, sealevel_pa=101325.0):
114         pressure = float(local_pa)
115         altitude = 44330.0 * (1.0 - pow(pressure / sealevel_pa, (1.0 / 5.255)))
116         return altitude
117     def read_sealevel_pressure(self, local_pa=101325.0, altitude_m=0.0):
118         pressure = float(local_pa)
119         p0 = pressure / pow(1.0 - altitude_m / 44330.0, 5.255)
120         return p0
121 if __name__ == '__main__':
122     bmp = BMP180()
123     while (True):
124             temp = bmp.read_temperature()
125             pressure=bmp.read_pressure()
126             altitude = bmp.read_altitude(pressure)
127             print("Temperature:%.1f ℃" %temp)
128             print("Pressure:%.2fhPa" %(pressure / 100.0))
129             print("Altitude:%.2fm" %altitude)
130             print()
131             time.sleep(2)
```

The function bmp180_GetTemperature() is used to calculate the current temperature value.

```
    def read_temperature(self):
        UT = self.read_raw_temp()
        X1 = ((UT - self.AC6) * self.AC5) >> 15
        X2 = (self.MC << 11) // (X1 + self.MD)
        B5 = X1 + X2
        temp = ((B5 + 8) >> 4) / 10.0
        return temp
```

The function bmp180_GetPressure() is used to calculate the current pressure value.

```
    def read_pressure(self):
        UT = self.read_raw_temp()
        UP = self.read_raw_pressure()
        X1 = ((UT - self.AC6) * self.AC5) >> 15
        X2 = (self.MC << 11) // (X1 + self.MD)
        B5 = X1 + X2
        # Pressure Calculations
        B6 = B5 - 4000
```

```
        X1 = (self.B2 * (B6 * B6) >> 12) >> 11
        X2 = (self.AC2 * B6) >> 11
        X3 = X1 + X2
        B3 = (((self.AC1 * 4 + X3) << self._mode) + 2) // 4
        X1 = (self.AC3 * B6) >> 13
        X2 = (self.B1 * ((B6 * B6) >> 12)) >> 16
        X3 = ((X1 + X2) + 2) >> 2
        B4 = (self.AC4 * (X3 + 32768)) >> 15
        B7 = (UP - B3) * (50000 >> self._mode)
        if B7 < 0x80000000:
            p = (B7 * 2) // B4
        else:
            p = (B7 // B4) * 2
        X1 = (p >> 8) * (p >> 8)
        X1 = (X1 * 3038) >> 16
        X2 = (-7357 * p) >> 16
        p = p + ((X1 + X2 + 3791) >> 4)
        return p
```

The function bmp180_Altitude() is used to calculate the current altitude.

```
def read_altitude(self, local_pa=101325.0, sealevel_pa=101325.0):
        # Calculation taken straight from section 3.6 of the datasheet.
        pressure = float(local_pa)
        altitude = 44330.0 * (1.0 - pow(pressure / sealevel_pa, (1.0 / 5.255)))
        return altitude
```

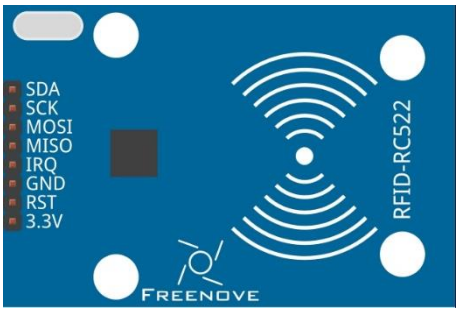
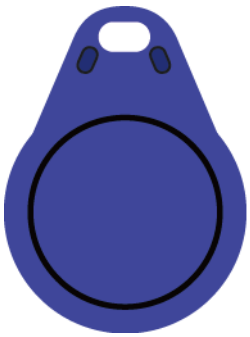
Chapter 34 RFID

In this chapter, we will learn how to use RFID.

Project 34.1 RFID

In this project, we will use RC522 RFID card reader to read and write the M1-S50 card.

Component List

Raspberry Pi x1 GPIO Extension Board & Wire x1 Breadboard x1	RC522 module x1 
Jemper M/F x7 	
Mifare1 S50 Standard card x1 	Mifare1 S50 Non-standard card x1 

Component Knowledge

RFID

RFID（Radio Frequency Identification）is a form of wireless communication technology. A complete RFID system is generally composed of a transponder and a reader. Generally, the transponder may be known as a tag, and each tag has a unique code, which is attached to an object to identify the target object. The reader is a device that reads (or writes) information in the tag.

Products derived from RFID technology can be divided into three categories: passive RFID products, active RFID products and semi active RFID products, among which, Passive RFID products are the earliest, the most mature and most widely used products in the market. It can be seen everywhere in our daily life such as, the bus card, dining card, bank card, hotel access cards, etc., and all of them are classified as close-range contact recognition. The main operating frequency of Passive RFID products are: 125KHZ (low frequency), 13.56MHZ (high frequency), 433MHZ (ultrahigh frequency), 915MHZ (ultrahigh frequency). Active and semi active RFID products work at higher frequencies.

The RFID module we use is a passive RFID product with the operating frequency of 13.56MHz.

MFRC522

The MFRC522 is a highly integrated reader/writer IC for contactless communication at 13.56MHz.

The MFRC522's internal transmitter is able to drive a reader/writer antenna designed to communicate with ISO/IEC 14443 A/MIFARE cards and transponders without additional active circuitry. The receiver module provides a robust and efficient implementation for demodulating and decoding signals from ISO/IEC 14443 A/MIFARE compatible cards and transponders. The digital module manages the complete ISO/IEC 14443A framing and error detection (parity and CRC) functionality

This RFID Module uses MFRC522 as the control chip, and SPI (Peripheral Interface Serial) as the reserved interface.

Technical specs:

Operating Voltage	13-26mA(DC)\3.3V
Idle current	10-13mA(DC)\3.3V
Sleep current in the	<80uA
Peak current	<30mA
Operating frequency	13.56MHz
Supported card type	Mifare1 S50、Mifare1 S70、Mifare Ultralight、Mifare Pro、Mifare Desfire
Size	40mmX60mm
Operation temperature	20-80 degrees(Celsius)
Storage temperature	40-85 degrees (Celsius)
Operation humidity	5%-95%(Relative humidity)

Mifare1 S50 Card

Mifare S50 is often called Mifare Standard with the capacity of 1K bytes. And each card has a 4-bytes global unique identifier number (USN/UID), which can be rewritten 100 thousand times and read infinite times. Its storage period can last for 10 years.

The Mifare S50 capacity (1K byte) is divided into 16 sectors (Sector0-Sector15). Each sector contains 4 data block (Block0-Block3. 64 blocks of 16 sectors will be numbered according absolute address, from 0 to 63).

And each block contains 16 bytes (Byte0-Byte15), 64*16=1024. As is shown in the following table:

Sector No.	Block No.	Storage area	Block type	Absolute block No.
sector 0	block 0	vendor code	vendor block	0
	block 1		data block	1
	block 2		data block	2
	block 3	Password A-access control-password B	control block	3
sector 1	block 0		data block	4
	block 1		data block	5
	block 2		data block	6
	block 3	Password A-access control-password B	control block	7
……	……	……	……	
sector 15	block 0		data block	60
	block 1		data block	61
	block 2		data block	62
	block 3	Password A-access control-password B	control block	63

Each sector has a set of independent password and access control put in its last block, that is, Block 3, which is also known as sector trailer. Sector 0, block 0 (namely absolute address 0) of S50 is used to store the card serial number and vendor code, which has been solidified and can't be changed. Except the manufacturer and the control block, the rest of the cards are data blocks, which can be used to store data. Data block can be used for two kinds of applications:

(1) used as general data storage and can be operated for reading and writing data.

(2) used as data value, and can be operated for initializing, adding, subtracting and reading the value.

The sector trailer block in each sector is the control block, including a 6-byte password A, a 4-byte access control and a 6-byte password B. For example, the control block of a brand new card is as follows:

A0 A1 A2 A3 A4 A5	FF 07 80 69	B0 B1 B2 B3 B4 B5
password A	access control	password B

The default password of a brand new card is generally 0A1A2A3A4A5 for password A and B0B1B2B3B4B5 for password B, or both the password A and password B are 6 FF. Access control is used to set the access conditions for each block (including the control block itself) in a sector.

Blocks of S50 are divided into data blocks and control blocks. There are four operations, "read", "write", "add value", "subtract value (including transmission and storage)" for data blocks, and there are two operations, "read" and "write" for control blocks.

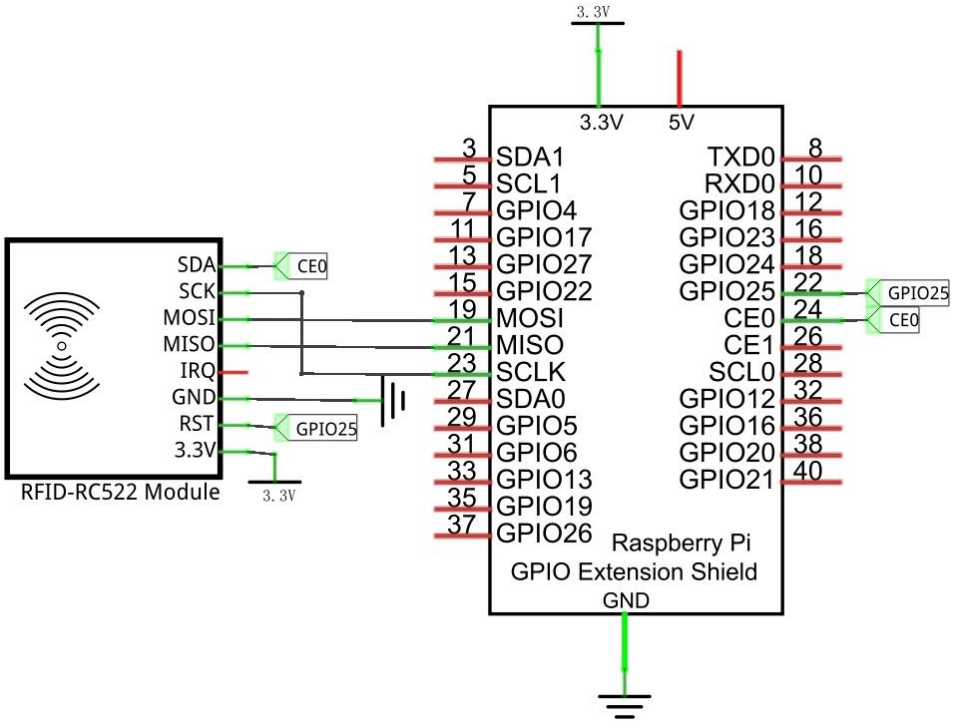
For more details about how to set data blocks and control blocks, please refer to Datasheet.

By default, after verifying password A or password B, we can do reading or writing operation to data blocks. And after verifying password A, we can do reading or writing operation to control blocks. But password A can never be read, so if you choose to verify password A but forget the password A, the block will never be able to read again. **It is highly recommended that beginners should not try to change the contents of control blocks.**

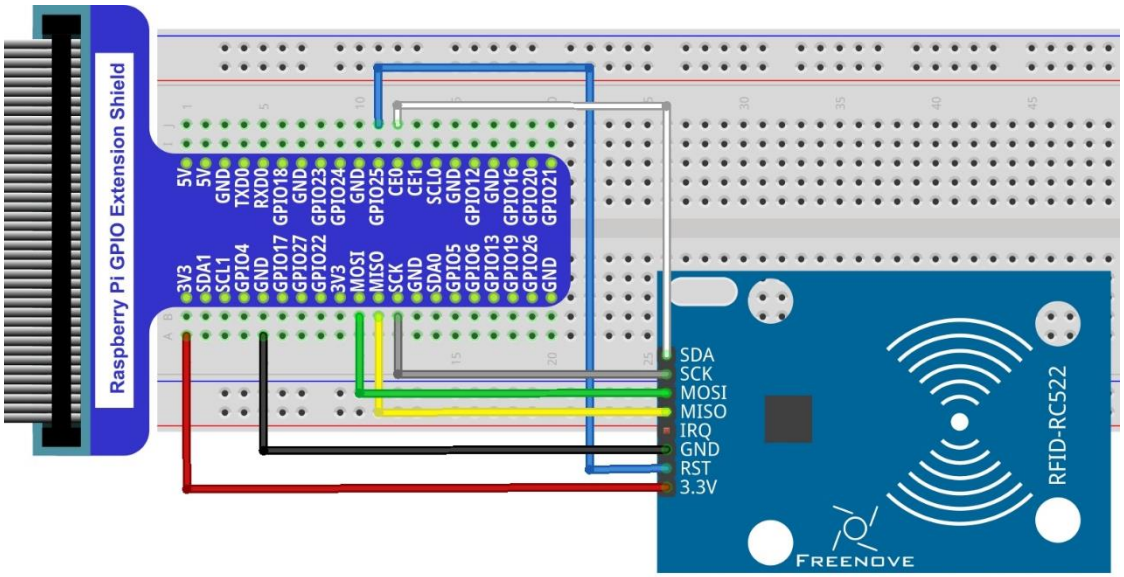
For Mifare1 S50 card equipped in Freenove RFID Kit, the default password A and B are both FFFFFFFFFFFF.

Circuit

Schematic diagram:



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Configure SPI

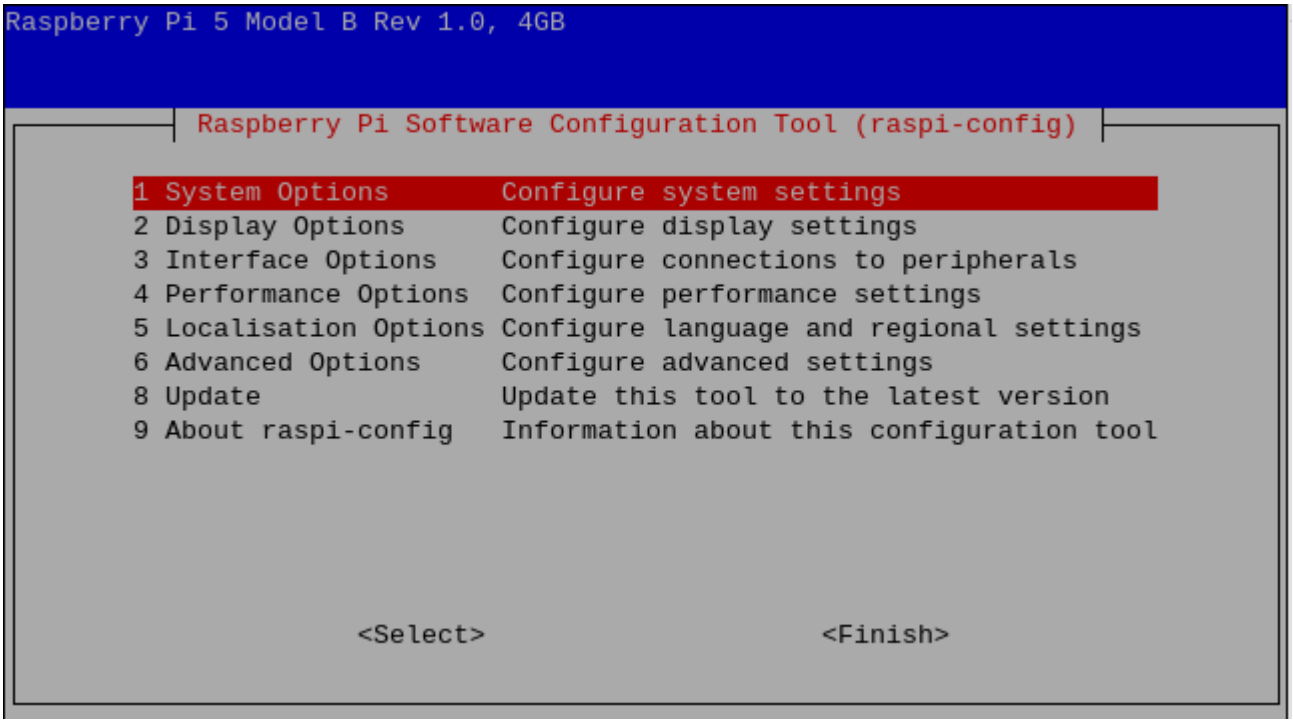
Enable SPI

The SPI interface of raspberry pi is closed by default. You need to open it manually. You can enable the SPI interface in the following way.

Type the following command in the terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose “3 Interfacing Options”→“I4 SPI”→“Yes”→“Finish” in order and then restart your RPi. Then the SPI module is started.

Type the following command to check whether the module SPI is loaded successfully:

```
ls /dev/sp*
```

The following result indicates that the module SPI has been loaded successfully:

```
pi@raspberrypi:~ $ ls /dev/sp*
/dev/spidev0.0  /dev/spidev0.1
```

Code

The project code uses human-computer interaction command line mode to read and write the M1-S50 card.

Python Code 34.1.1 RFID

There are two code files for this project. They are respectively under Python2 folder and Python3 folder. **Their functions are the same, but they are not compatible.** Code under Python2 folder can only run on Python2. And code under Python3 folder can only run on Python3.

First observe the project result, and then learn about the code in detail.

If you need any support, please contact us via: support@freenove.com

1. Use cd command to enter RFID directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/34.1.1_RFID
```

2. Use python command to execute code "RFID.py".

```
python RFID.py
```

After the program is executed, the following contents will be displayed in the terminal:

```
pi@raspberrypi:~ $ cd ~/Freenove_Kit/Code/Python_Code/24.1.1_RFID/Python3
pi@raspberrypi:~/Freenove_Kit/Code/Python_Code/24.1.1_RFID/Python3 $ python RFID.py
Program is starting ...
Press Ctrl-C to exit.
RC522>
```

Here, type the command “quit” to exit the program.

Type command "scan", then the program begins to detect whether there is a card close to the sensing area of MFRC522 reader. Place a M1-S50 card in the sensing area. The following results indicate that the M1-S50 card has been detected, the UID of which is E6CF5C8EFB (HEX).

```
RC522> scan
scan
Scanning ...
Card detected
Card UID: ['0xe6', '0xcf', '0x5c', '0x8e', '0xfb']
Size: 8
RC522> E6CF5C8EFB>
```

When the Card is placed in the sensing area, you can read and write the card with the following command.

```
Usage:
        read <blockstart>
        dump
        halt
        clean <blockaddr>
        write <blockaddr> <data>
```

In the command read<blockstart>, the parameter blockstart is the address of the data block, and the range is 0-63. As is shown below:

In the command read<blockstart>, the parameter blockstart is the address of the data block, and the range is 0-63. This command is used to read the data of data block with address “blockstart”. For example, using command “read 0” can display the content of data block 0. Using the command “read 1” can display the content of data block 1. As is shown below:

```
RC522> E6CF5C8EFB> read 0
['read', '0']
Sector 0 :  e6 cf 5c 8e fb 8 4 0 62 63 64 65 66 67 68 69  | ??bcdefghi
RC522> E6CF5C8EFB> read 1
['read', '1']
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
RC522> E6CF5C8EFB>
```

Command “dump” is used to display the content of all data blocks in all sectors.

Command <address> <data> is used to write “data" to data block with address “address”, where the address range is 0-63 and the data length is 0-16. In the process of writing data to the data block, both the contents of data block before written and after written will be displayed. For example, if you want to write the string "Freenove" to the data block with address “1”, you can type the following command.

```
write 1 Freenove
```

```
RC522> E6CF5C8EFB> write 1 Freenove
['write', '1', 'Freenove']
Before writing , The data in block 1  is:
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
4 backdata &0x0F == 0x0A 10
Data written
After written , The data in block 1  is:
Sector 1 :  46 72 65 65 6e 6f 76 65 0 0 0 0 0 0 0 0  | Freenove
```

Command “clean <address>” is used remove the contents of the data block with address "address". For example, if you want to clear the contents of the data block 1 that has just been written, you can type the following command.

```
clean 1
```

```
RC522> E6CF5C8EFB> clean 1
['clean', '1']
Before cleaning , The data in block 1  is:
Sector 1 :  46 72 65 65 6e 6f 76 65 0 0 0 0 0 0 0 0  | Freenove
4 backdata &0x0F == 0x0A 10
Data written
After cleaned , The data in block 1  is:
Sector 1 :  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  |
```

Command “halt” is used to quit the selection state of the card.

```
RC522> E6CF5C8EFB> halt
['halt']
RC522>
```

The following is the program code :

```
1  from gpiozero import OutputDevice
2  import MFRC522
3  import sys
4  import os
5
6  # Create an object of the class MFRC522
7  mfrc = MFRC522.MFRC522()
8
9
10 def dis_ConmandLine():
11     print ("RC522>",end="")
```

```
def dis_CardID(cardID):
    print ("%2X%2X%2X%2X%2X>"%(cardID[0],cardID[1],cardID[2],cardID[3],cardID[4]),end="")
def setup():
    print ("Program is starting ... "  )
    print ("Press Ctrl-C to exit.")
    pass

def loop():
    global mfrc3s
    while(True):
        dis_ConmandLine()
        inCmd = input()
        print (inCmd)
        if (inCmd == "scan"):
            print ("Scanning ... ")
            mfrc = MFRC522.MFRC522()
            isScan = True
            while isScan:
                # Scan for cards
                 (status,TagType) = mfrc.MFRC522_Request(mfrc.PICC_REQIDL)
                # If a card is found
                if status == mfrc.MI_OK:
                    print ("Card detected")
                # Get the UID of the card
                 (status,uid) = mfrc.MFRC522_Anticoll()
                # If we have the UID, continue
                if status == mfrc.MI_OK:
                    print ("Card UID: "+ str(map(hex,uid)))
                    # Select the scanned tag
                    if mfrc.MFRC522_SelectTag(uid) == 0:
                        print ("MFRC522_SelectTag Failed!")
                    if cmdloop(uid) < 1 :
                        isScan = False

        elif inCmd == "quit":
            destroy()
            exit(0)
        else :
            print ("\tUnknown command\n"+"\tscan:scan card and dump\n"+"\tquit:exit
program\n")

def cmdloop(cardID):
    pass
    while(True):
```

```
        dis_ConmandLine()
        dis_CardID(cardID)
        inCmd = input()
        cmd = inCmd.split(" ")
        print (cmd)
        if(cmd[0] == "read"):
            blockAddr = int(cmd[1])
            if((blockAddr<0) or (blockAddr>63)):
                print ("Invalid Address!")
            # This is the default key for authentication
            key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
            # Authenticate
            status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
            # Check if authenticated
            if status == mfrc.MI_OK:
                mfrc.MFRC522_Readstr(blockAddr)
            else:
                print ("Authentication error")
                return 0

        elif cmd[0] == "dump":
            # This is the default key for authentication
            key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
            mfrc.MFRC522_Dump_Str(key,cardID)

        elif cmd[0] == "write":
            blockAddr = int(cmd[1])
            if((blockAddr<0) or (blockAddr>63)):
                print ("Invalid Address!")
            data = [0]*16
            if(len(cmd)<2):
                data = [0]*16
            else:
                data = cmd[2][0:17]
                data = map(ord,data)
                data = list(data)
                lenData = len(list(data))
            if lenData<16:
                    data+=[0]*(16-lenData)
            # This is the default key for authentication
            key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
            # Authenticate
            status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
            # Check if authenticated
```

```
99              if status == mfrc.MI_OK:
100                 print ("Before writing , The data in block %d  is: "%(blockAddr))
101                 mfrc.MFRC522_Readstr(blockAddr)
102                 mfrc.MFRC522_Write(blockAddr, data)
103                 print ("After written , The data in block %d  is: "%(blockAddr))
104                 mfrc.MFRC522_Readstr(blockAddr)
105             else:
106                 print ("Authentication error")
107                 return 0
108
109         elif cmd[0] == "clean":
110             blockAddr = int(cmd[1])
111             if((blockAddr<0) or (blockAddr>63)):
112                 print ("Invalid Address!")
113             data = [0]*16
114             # This is the default key for authentication
115             key = [0xFF,0xFF,0xFF,0xFF,0xFF,0xFF]
116             # Authenticate
117             status = mfrc.MFRC522_Auth(mfrc.PICC_AUTHENT1A, blockAddr, key, cardID)
118             # Check if authenticated
119             if status == mfrc.MI_OK:
120                 print ("Before cleaning , The data in block %d  is: "%(blockAddr))
121                 mfrc.MFRC522_Readstr(blockAddr)
122                 mfrc.MFRC522_Write(blockAddr, data)
123                 print ("After cleaned , The data in block %d  is: "%(blockAddr))
                    mfrc.MFRC522_Readstr(blockAddr)
124             else:
125                 print ("Authentication error")
126                 return 0
127         elif cmd[0] == "halt":
128             return 0
129         else :
130             print ("Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n" "\tclean
    <blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n")
131
132 def destroy():
133     print("Ending program")
134
135 if __name__ == "__main__":
136     setup()
137     try:
138         loop()
139     except KeyboardInterrupt:  # Ctrl+C captured, exit
140         destroy()
```

In the code, first create an MFRC522 class object.

```
mfrc = MFRC522.MFRC522()
```

In the function loop, wait for the command input. If command "scan" is received, the function will begin to detect whether there is a card close to the sensing area. If a card is detected, the card will be selected and card UID will be acquired. Then enter the function scan_loop (). If command "quit" or "exit" is received, the program will exit.

```
        if (inCmd == "scan"):
            print "Scanning ... "
            isScan = True
            while isScan:
                ...........
                    if cmdloop(uid) < 1 :
                        isScan = False
        elif inCmd == "quit":
            destroy()
            exit(0)
        else :
            print "\tUnknown command\n"+"\tscan:scan card and dump\n"+"\tquit:exit program\n"
```

The function cmdloop() will detect command read, write, clean, halt, dump and do the corresponding processing to each command. The functions of each command and the method have been introduced before.

```
def cmdloop(cardID):
    pass
    while(True):
        dis_ConmandLine()
        dis_CardID(cardID)
        inCmd = raw_input()
        cmd = inCmd.split(" ")
        print cmd
        if(cmd[0] == "read"):
            ...........
        elif cmd[0] == "dump":
            ...........
        elif cmd[0] == "write":
            ...........
        elif cmd[0] == "clean":
            ...........
        elif cmd[0] == "halt":
            return 0
        else :
            print "Usage:\r\n" "\tread <blockstart>\r\n" "\tdump\r\n" "\thalt\r\n" "\tclean
<blockaddr>\r\n" "\twrite <blockaddr> <data>\r\n"
```

The file "MFRC522.py" contains the associated operation method for the MFRC522. You can open the file to view all the definitions and functions.

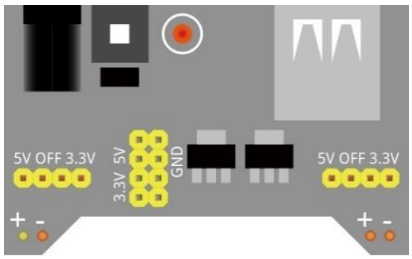
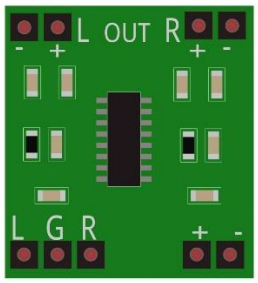
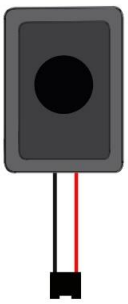
Chapter 35 Speaker amplifier module PAM8403

In this chapter, we will learn how to use the speaker amplifier module PAM8403.
Please note: This chapter applies to Raspberry PI 2-4.

Project 35.1 Play local music

This project uses speakers to play local music.

Component List

<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper Wires
 </td></tr>
<tr><td colspan="2">Breadboard Power Module x1
 
</td><td>9V Battery (you provide) & 9V Battery Cable
</td></tr>
<tr><td>Speaker amplifier module PAM8403 x1
 
</td><td>Speaker x2
  </td><td>Audio cable x1
 </td></tr>
</table>

Component knowledge

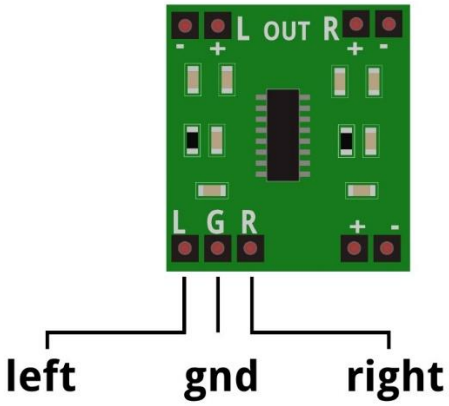
Speaker

Raspberry Pi can route sound via HDMI to the screen's built-in speakers or to the analog headphone jack. If your monitor has speakers, the sound is output via HDMI by default. If not, it is output via the headphone jack. This may not be the desired output setting, or the automatic detection is inaccurate, in which case you can switch the output manually. In following project, the analog headphone jack is used for output.

PAM8403 amplifier module

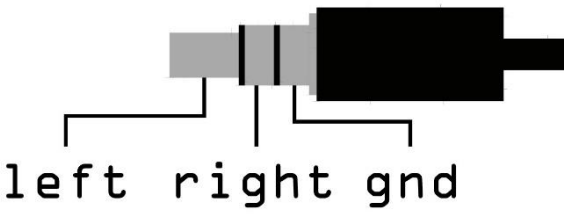
PAM8403 is a class D audio power amplifier IC with output power of 3 watts. It has the characteristics of low harmonic distortion and small noise crosstalk, so that it can reproduce the sound with better sound quality. It adopts a new structure without coupling output and without low-pass filter circuit, so that it can directly drive the speaker.

Its working voltage range is 2.5-5V. Two speakers (loads) should be connected before the PAM8403 module is powered up. Don't use voltages that exceed the power supply range to avoid damaging the Hall sensors. The audio input interface of PAM8403 amplifier module is shown in the figure below:

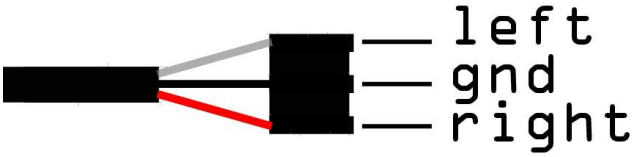


3.5mm 3 conductor audio cable to DuPont cable

3.5mm headphone plugs are divided into 3 conductor plugs and 4 conductor plugs. 3 conductor plug has three wires, namely: left channel, right channel and ground wire. 4 conductor plug has four wires, namely: left channel, right channel, ground wire and MIC microphone level. In the kit, a 3.5mm 3 conductor plug is used. As shown in the figure below, it is divided into left channel, right channel and ground wire from left to right.



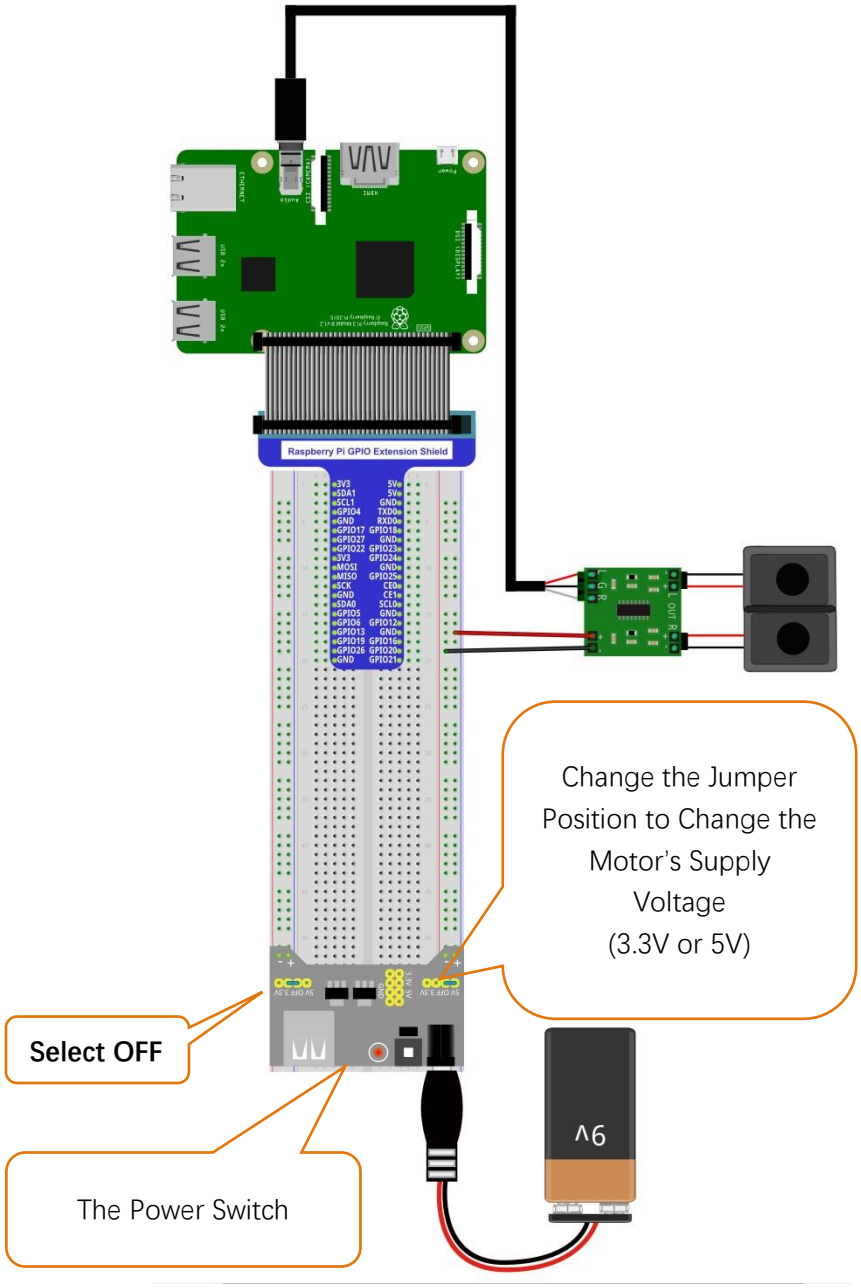
The audio cable to DuPont cable interface is shown in the figure below:



Circuit

Be careful when connecting this circuit. Do not use the RPi to power the speaker amplifier module PAM8403 directly, as this may cause permanent damage to your RPi!

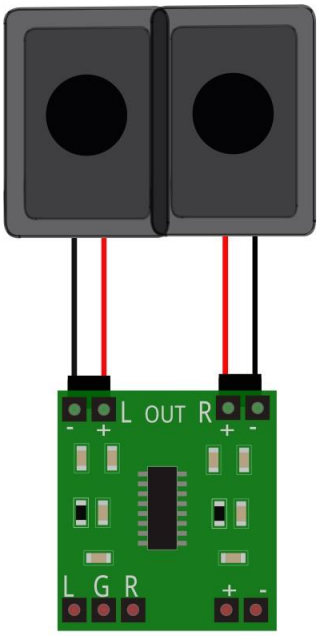
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



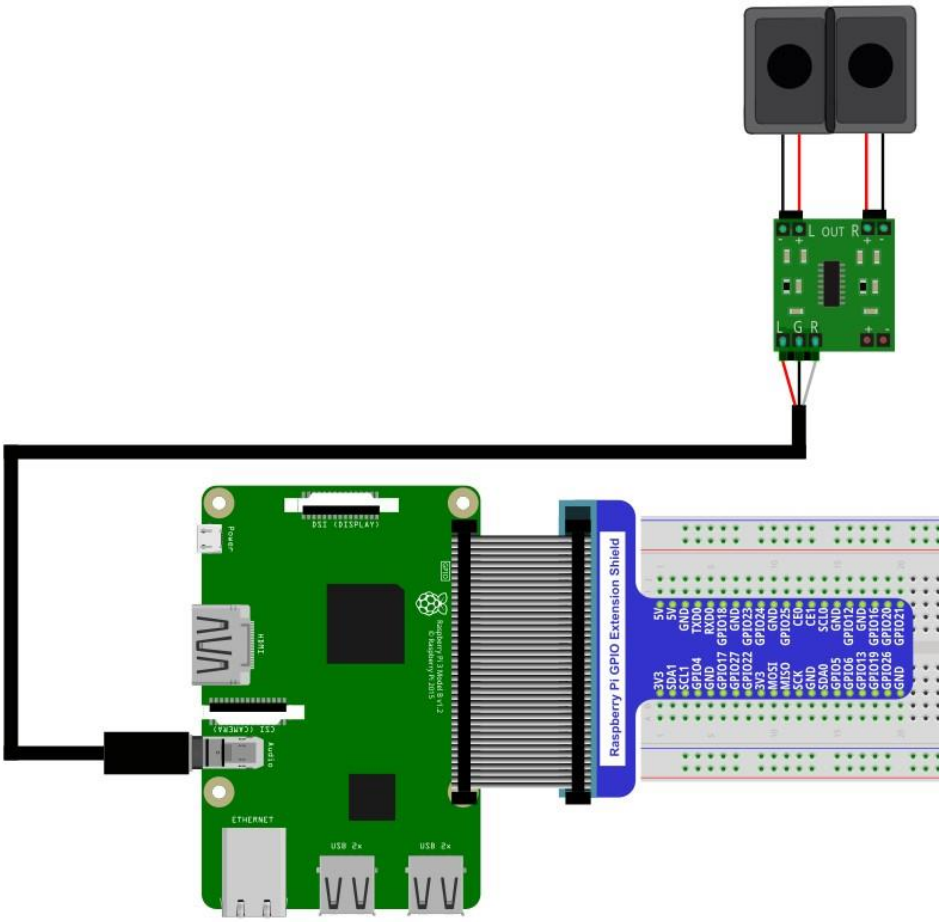
Next connect the peaker amplifier module PAM8403 cable.

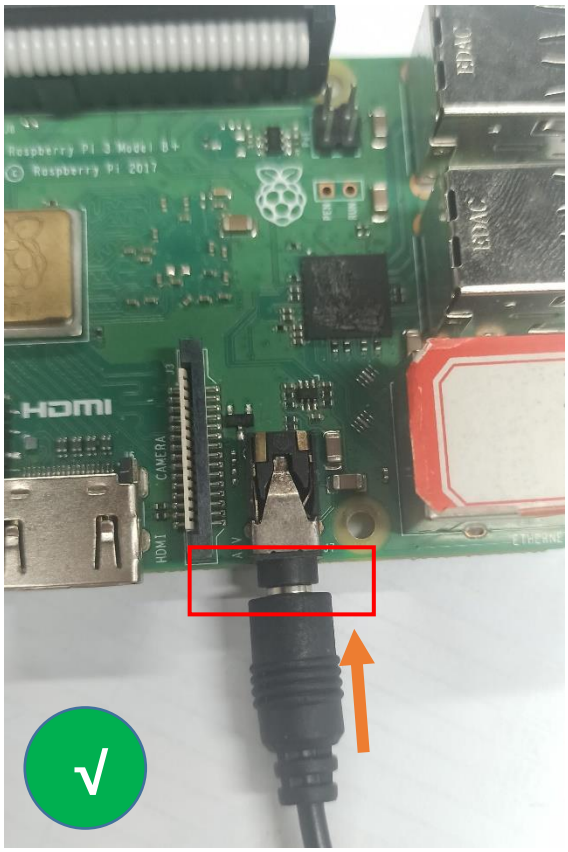
The specific connection steps are as follows:

Step 1

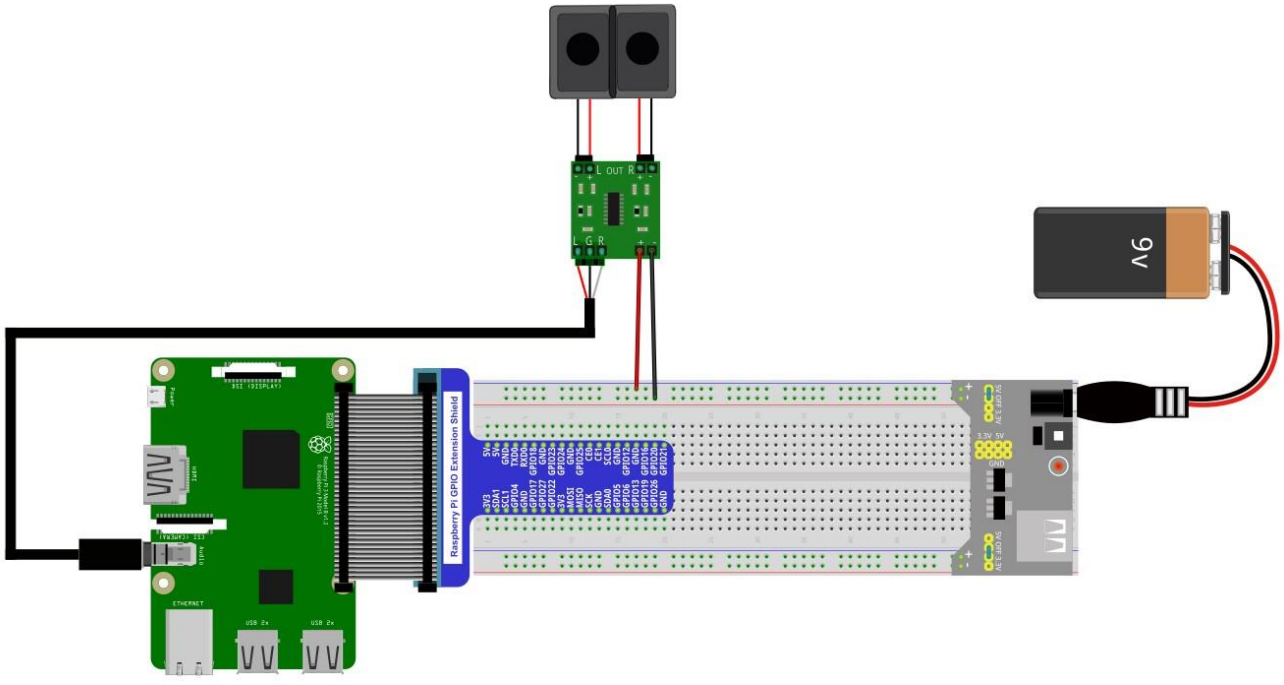


Step 2





Step 3



Additional supplement

Raspbery Pi, other than 4B and 400, nees to restart the audio module. Otherwise the speaker amplifier module

PAM8403 cannot be recognized.

1. Create a new snd-blacklist.conf and open it for editing

```
sudo nano /etc/modprobe.d/snd-blacklist.conf
```

Find the contents of the following one lines (with Ctrl + W you can search):

```
blacklist snd_bcm2835
```

Add # to comment out the first line. Press Ctrl+O, Enter, Ctrl+X.

```
#blacklist snd_bcm2835
```

2. We also need to edit config file.

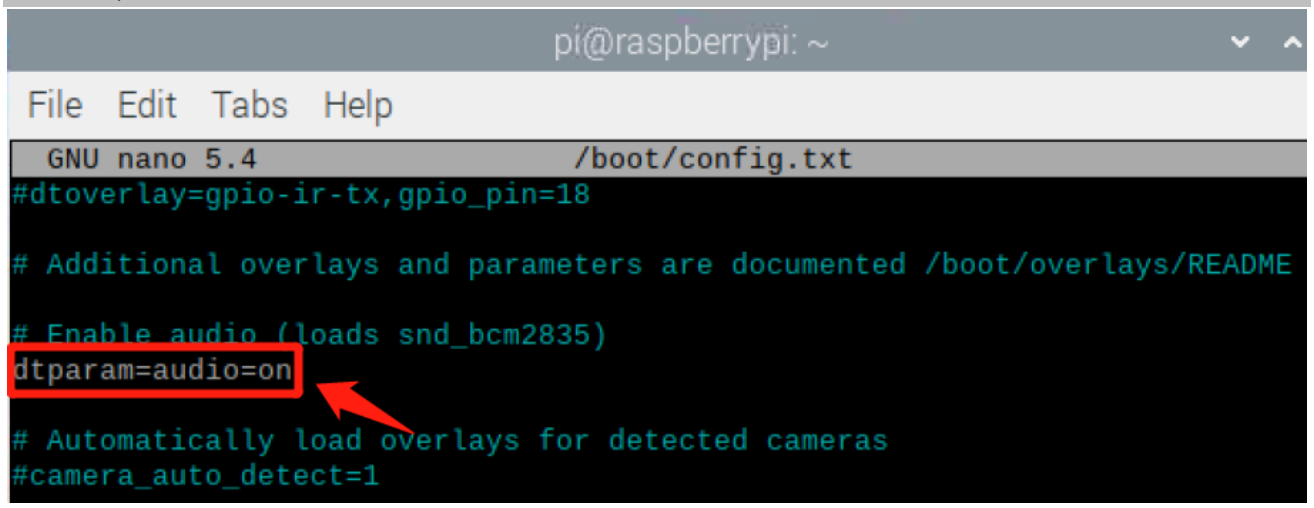
```
sudo nano /boot/firmware/config.txt
```

Find the contents of the following two lines (with Ctrl + W you can search):

```
# Enable audio (loads snd_bcm2835)
# dtparam=audio=on
```

Add # to comment out the second line. Press Ctrl+O, Enter, Ctrl+X.

```
# Enable audio (loads snd_bcm2835)
  dtparam=audio=on
```

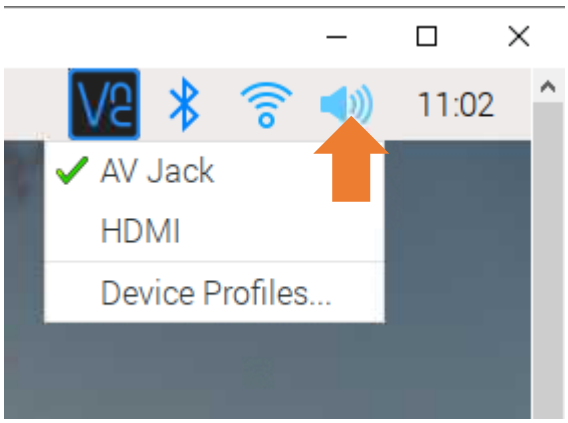


It will take effect after restarting, restart your RPi.

Configure audio output method and volume

Configure audio output method

When your Pi doesn't have a desktop, you can right-click the speaker icon in the upper right corner to choose whether your Pi uses HDMI or Analog connection to handle sound. As shown below:

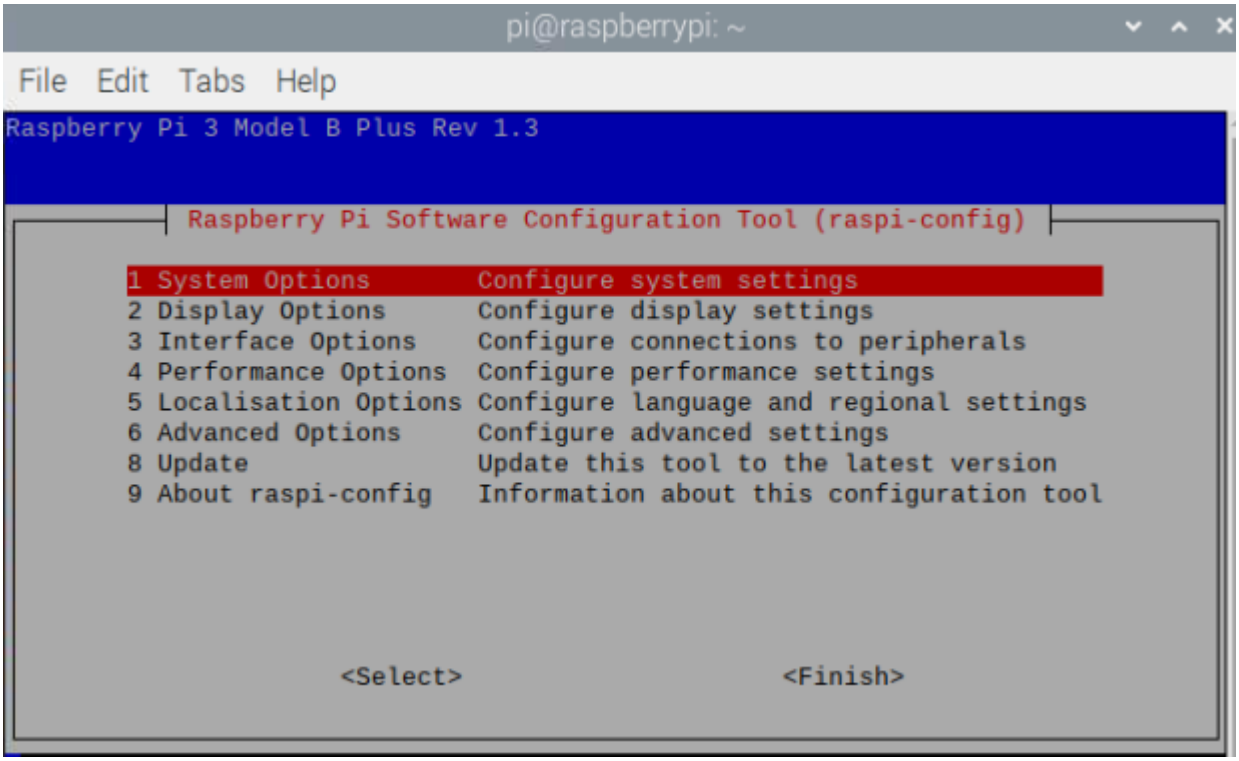


When your Pi doesn't have a desktop, you can choose how the sound is handled through the terminal command line.

Open the terminal and enter the following command.

```
sudo raspi-config
```

Then open the following dialog box:



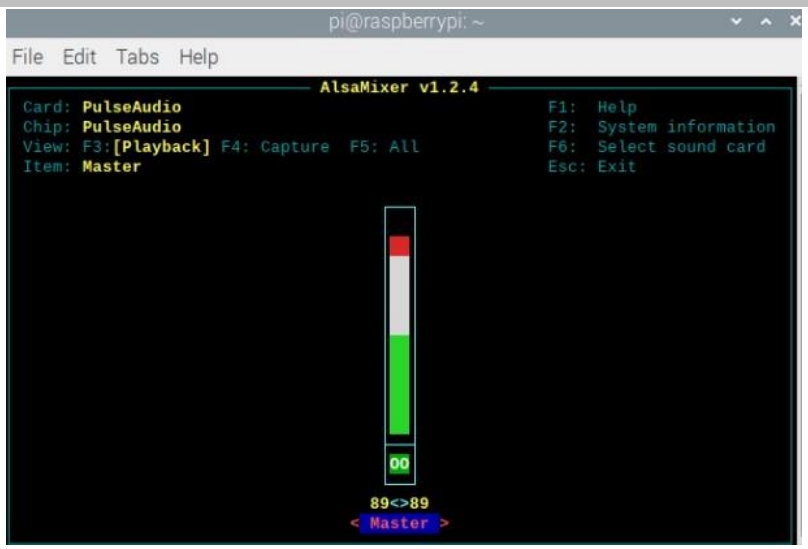
Choose “1 System Options” ->“S2 Audio” ->“3.5mm jack” ->“Yes” ->“Finish” in this order.

Configure volume

When your Pi has desktop, you left-click the speaker icon and move the slider up or down with the keyboard to adjust the volume. As shown below:

When your Pi doesn't have desktop, you can adjust the sound volume through the terminal command line. Type command in the Terminal:

```
alsamixer
```



Determine if the sound card device is accessible and not muted (press the keyboard letter m to toggle). You can adjust the volume through the up and down arrows on the keyboard. After confirming that there is no problem, press Esc to exit. Finally, reboot your RPi.

Ffplay tool

ffplay

FFmpeg is a complete cross-platform audio and video solution, which can be used to handle audio and video transcoding, recording, streaming and other application scenarios. FFmpeg has three major tools, namely ffmpeg, ffprobe, and ffplay. Here we mainly introduce ffplay, which is a sub-tool of ffmpeg, which has powerful audio and video decoding and playback capabilities. This tool is already installed on your Raspberry Pi by default. You can use this tool from the command line. If you need to see more, please visit the official link: http://ffmpeg.org/.

You can see the common commands of ffmpeg with the following commands

```
ffplay --help
```

The following are some advanced commands of ffplay. As shown in the table below:

Command	Description
-autoexit	Exit when video is done playing.
-exitonkeydown	Exit if any key is pressed.
-exitonmousedown	Exit if any mouse button is pressed.
-acodec codec_name	Force a specific audio decoder.
-vcodec codec_name	Force a specific audio decoder.
-scodec codec_name	Force a specific audio decoder.
-autorotate	Automatically rotate the video according to file metadata. Enabled by default, use to disable it. -noautorotate

Code

Python Code 35.1.1 Music

Run program

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 35.1.1_Music directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/35.1.1_Music
```

2. Use command to execute

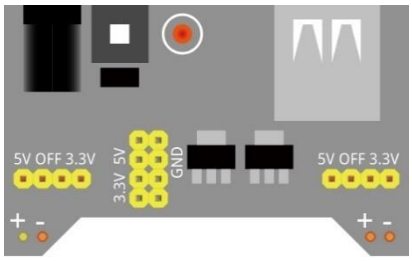
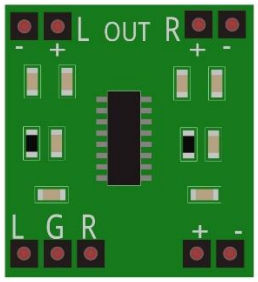
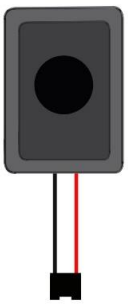
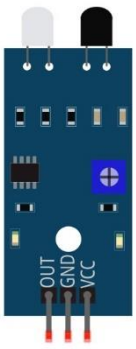
```
ffplay test.mp3
```

After executing the command, you can hear the test music playing. Here you can also play other music in this way. You can exit playback by pressing Esc on the keyboard.

Project 35.2 TTS reminder

This project uses the speaker amplifier module PAM8403 to make a TTS reminder.

Component List

<table>
<tr><td>Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="2">Jumper Wires
 </td></tr>
<tr><td>Breadboard Power Module x1
  </td><td colspan="2">9V Battery (you provide) & 9V Battery Cable
 </td></tr>
<tr><td>Speaker amplifier module PAM8403 x1
  </td><td>Speaker x2
  </td><td>Audio cable x1
 </td></tr>
<tr><td colspan="3">Infrared obstacle avoidance sensor x1
  </td></tr>
</table>

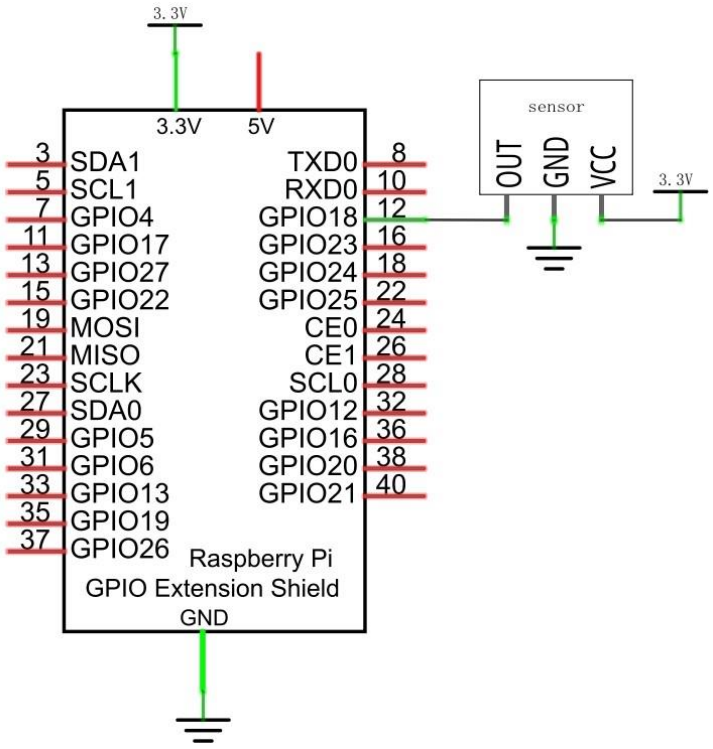
Component knowledge

TTS

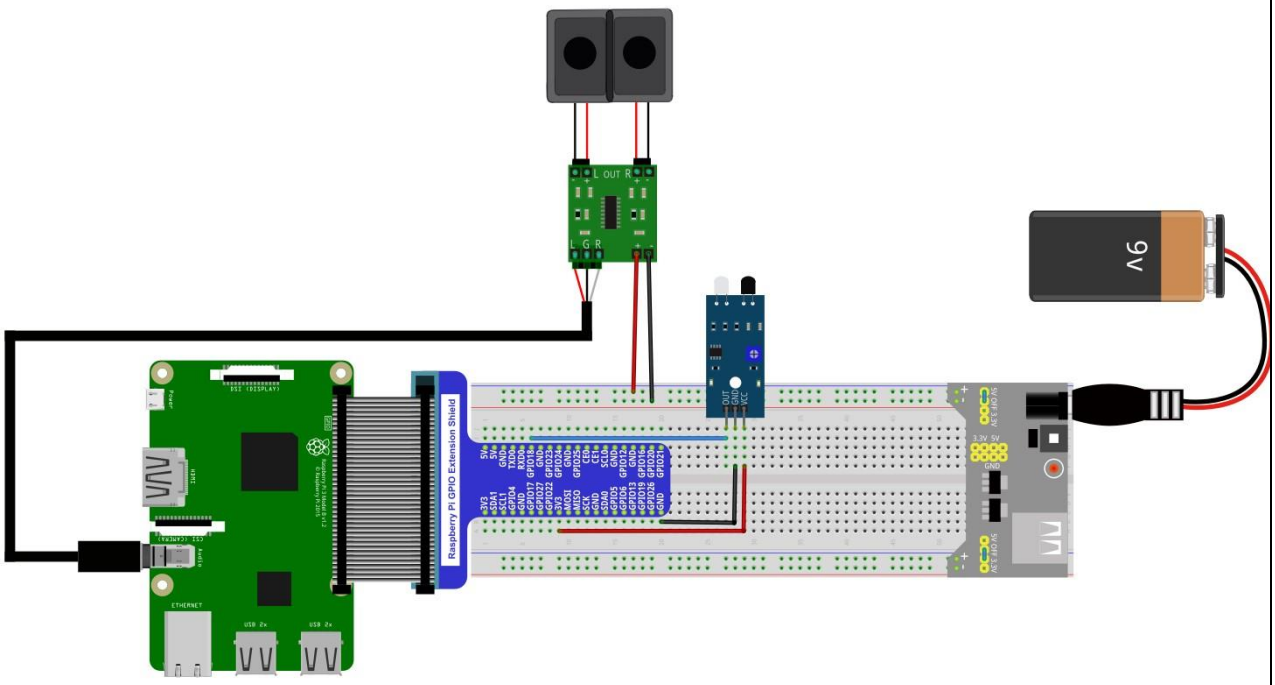
TTS, short for Text To Speech, is part of the human-machine dialogue that enables machines to speak. TTS can intelligently convert text into natural speech stream. TTS technology converts text files in real time, and the conversion time is fast. In theproject, TTS was used to broadcast the text "Hello, please stay away".

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



For the connection steps of the speaker amplifier module PAM8403, please refer to Project 35.1 play local music.

Code

Python Code 35.2.1 TTS

Install espeak before run python code.

1. Enter the following command to install.

```
sudo apt-get install espeak
```

Enter 'y' to continu espeak library installation.

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

2. Use cd command to enter 35.2.1_TTS directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/35.2.1_TTS
```

3. Use python command to execute code "TTS.py"

```
python TTS.py
```

After the program is executed, when you touch the infrared detection sensor, the speaker will broadcast "Hello, please stay away".

The following is the program code:

```
1  from sensor import InfraredSensor
2  import os
3  import time
4
5  sensorPin = 18    # define sensorPin
```

```
6   sensor = InfraredSensor(sensorPin, pull_up=True)
7
8   def SensorEvent(): # The sensor is blocked
        print("Hello, please  stay away")
9       os.system("espeak 'Hello, please  stay away'")
10
11  def loop():
12      sensor.when_reflect = SensorEvent
13      while True:
14          time.sleep(1)
15
16  def destroy():
17      sensor.close()
18
191 if __name__ == '__main__':     # Program entrance
20      print ('Program is starting...')
21      try:
22          loop()
23      except KeyboardInterrupt:  # Press ctrl-c to end the program.
24          destroy()
25          print("Ending program")
```

Read the signal pin of the infrared obstacle avoidance sensor, and determine whether the sensor is touched by human beings. If the sensor is touched by human beings, the speaker will broadcast a reminder.

```
    def SensorEvent(): # The sensor is blocked
        print("Hello, please  stay away")
        os.system("espeak 'Hello, please  stay away'")
    def loop():
        sensor.when_reflect = SensorEvent
```

Chapter 36 Camera

In this chapter, we will learn how to use camera.

Project 36.1 Photograph

This project uses a camera to take a picture and save it in the corresponding location.

Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x3 	
Camera x1 	Resistor 10kΩ x2 	Push Button Switch x1

Component knowledge

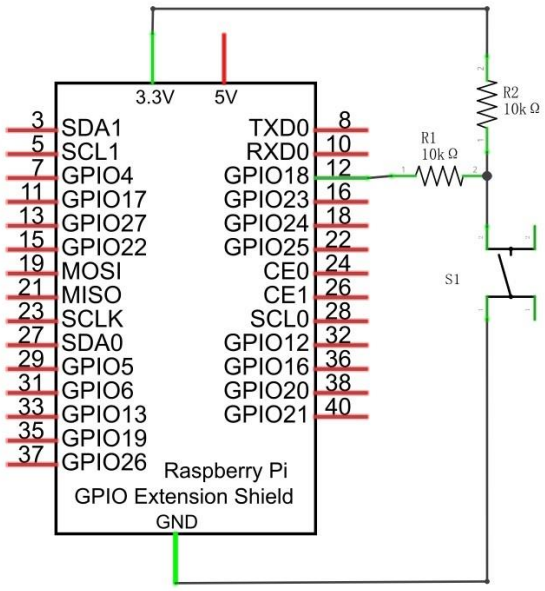
Camera

The camera is connected to the Raspberry Pi through a 15-pin cable. There are only two sockets to connect.

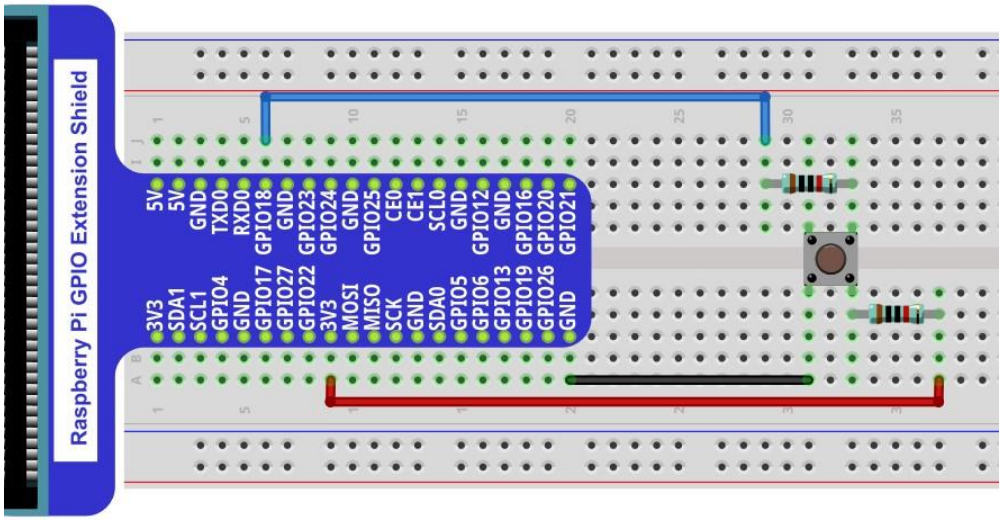
Next connect the camera to the Raspberry Pi. First turn off the power switch of the Raspberry Pi and disconnect the power cord. Then connect the CSI camera with Raspberry Pi camera port. **(The CSI camera must be connected or disconnected without power and the Raspberry Pi is turned off, otherwise it may burn the camera.)** It needs to be connected correctly, otherwise the camera will not work.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

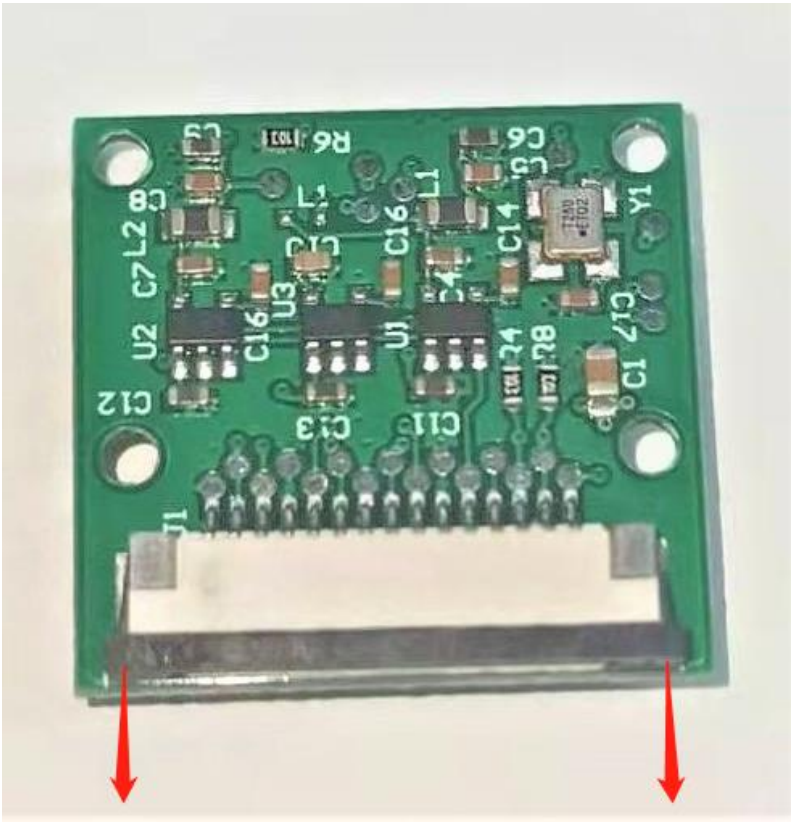


Next connect the camera cable.

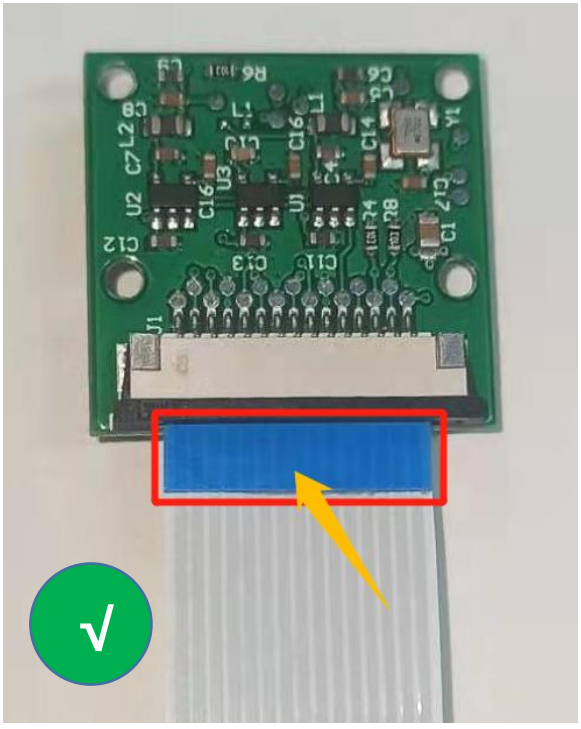
You need power off Raspberry Pi when wire camera.

After connected, it should be as follows

Step 1



Step 2

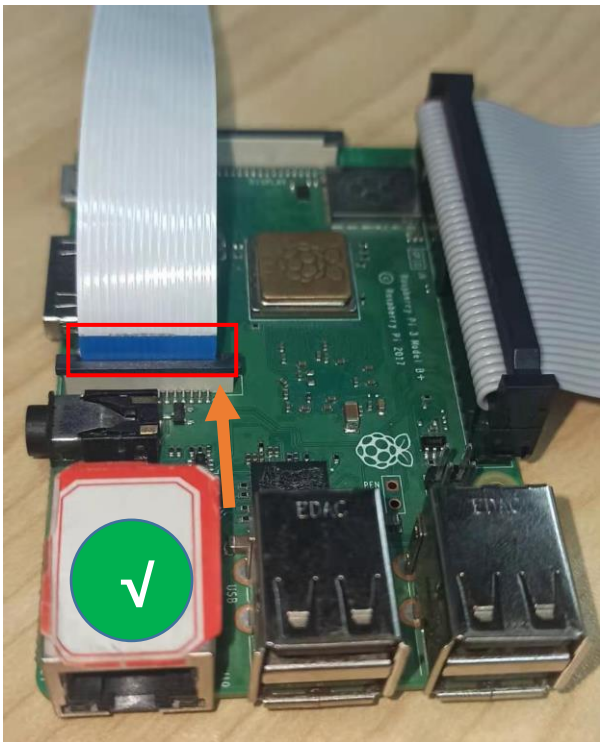


The Blue side of cable should be toward to Servo.

Connect one end of cable to camera. Please note the front and back of the cable.

Step 3

Step 4



The Blue side of cable should be toward to RPi USB port.

Connect another end of cable to raspberry pi. Please note the front and back of the cable.

The CSI camera must be connected or disconnected under no power and when Raspberry Pi is shut down, or the camera may be burned.

Code

Detect camera:

```
rpicam-hello --list -camera
```

```
pi@raspberrypi:~ $ rpicam-hello --list -camera
Available cameras
-----------------
0 : ov5647 [2592x1944 10-bit GBRG] (/base/axi/pcie@1000120000/rp1/i2c@88000/ov5647@36)
    Modes: 'SGBRG10_CSI2P' : 640x480 [58.92 fps - (16, 0)/2560x1920 crop]
                             1296x972 [46.34 fps - (0, 0)/2592x1944 crop]
                             1920x1080 [32.81 fps - (348, 434)/1928x1080 crop]
                             2592x1944 [15.63 fps - (0, 0)/2592x1944 crop]
```

If you do not get the result above, please check whether the camera wire is connected correctly.

(Note: When plugging in or pulling out camera wire, please make sure Raspberry Pi is turned OFF. Otherwise, it may burn out the camera.)

Python Code 36.1.1 Camera

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 36.1.1_Camera directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/36.1.1_Camera
```

2. Use python command to execute code "Camera.py"

```
python Camera.py
```

After the program is executed, you can take a photo by pressing the button. When the button is pressed, you will see image.jpg in the corresponding directory. As shown below:

The following is the program code:

```
1  import time
2  from picamera2 import Picamera2, Preview
3  from gpiozero import Button
4
5  buttonPin = 18    # define buttonPin
```

```
button = Button(buttonPin,pull_up=True) # define Button pin according to BCM Numbering

def loop():
    while True:
        if button.is_pressed:  # if button is pressed
            picam2 = Picamera2()
            preview_config = picam2.create_preview_configuration(main={"size": (800, 600)})
            picam2.configure(preview_config)
            picam2.start_preview(Preview.QTGL)
            picam2.start()
            time.sleep(2)
            metadata = picam2.capture_file("image.jpg")
            print ('Hello.a photo has been to taken successfully')   # print information on
terminal
            picam2.close()
            print ('Please preess the button take a photo')
def destroy():
    button.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    print ('Please preess the button take a photo')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Read the signal pin of the button, and determine whether the button is pressed, if the button is pressed, take a photo and save it in the corresponding directory.

```
        if button.is_pressed:  # if button is pressed
            picam2 = Picamera2()
            preview_config = picam2.create_preview_configuration(main={"size": (800, 600)})
            picam2.configure(preview_config)
            picam2.start_preview(Preview.QTGL)
            picam2.start()
            time.sleep(2)
            metadata = picam2.capture_file("image.jpg")
            print ('Hello.a photo has been to taken successfully')   # print information on
terminal
            picam2.close()
            print ('Please preess the button take a photo')
```

Project 36.2 Video Recording

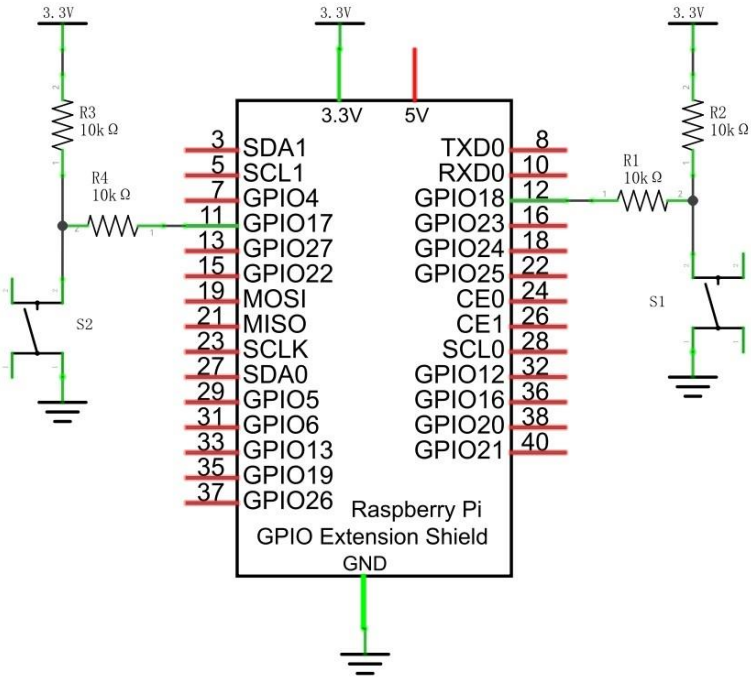
This project uses a camera to shoot a video and play it back.

Component List

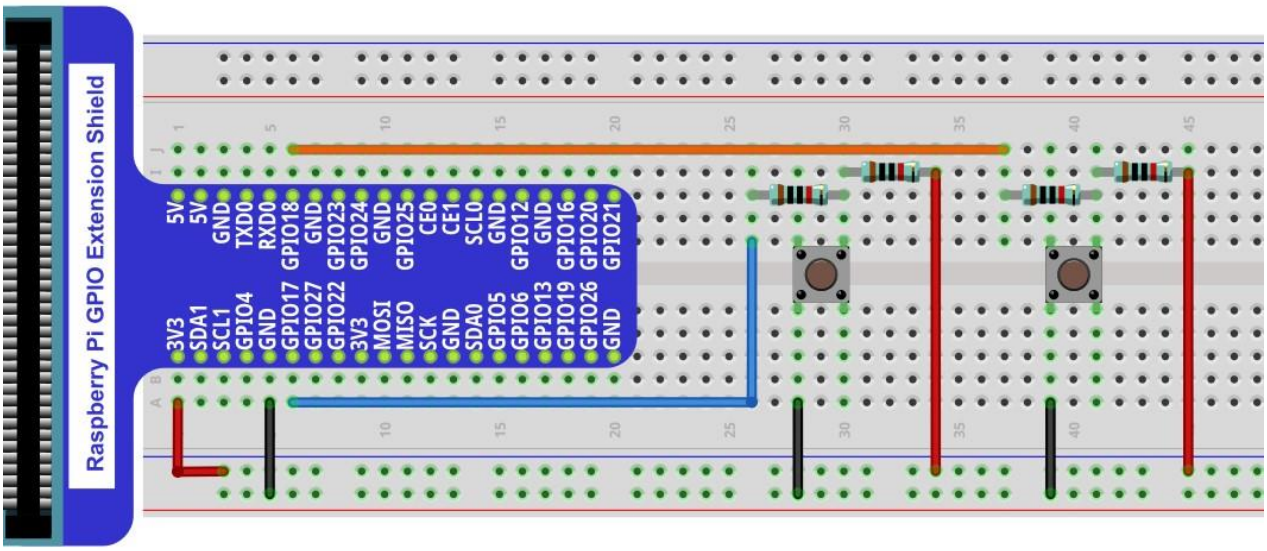
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper x8	
Camera x1	Resistor 10kΩ x4	Push Button Switch x2

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

Detect camera:

Detect camera:

```
rpicam-hello --list -camera
```

```
pi@raspberrypi:~ $ rpicam-hello --list -camera
Available cameras
-----------------
0 : ov5647 [2592x1944 10-bit GBRG] (/base/axi/pcie@1000120000/rp1/i2c@88000/ov5647@36)
    Modes: 'SGBRG10_CSI2P' : 640x480 [58.92 fps - (16, 0)/2560x1920 crop]
                             1296x972 [46.34 fps - (0, 0)/2592x1944 crop]
                             1920x1080 [32.81 fps - (348, 434)/1928x1080 crop]
                             2592x1944 [15.63 fps - (0, 0)/2592x1944 crop]
```

If you do not get the result above, please check whether the camera wire is connected correctly.

(Note: When plugging in or pulling out camera wire, please make sure Raspberry Pi is turned OFF. Otherwise, it may burn out the camera.)

Python Code 36.2.1 Video

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

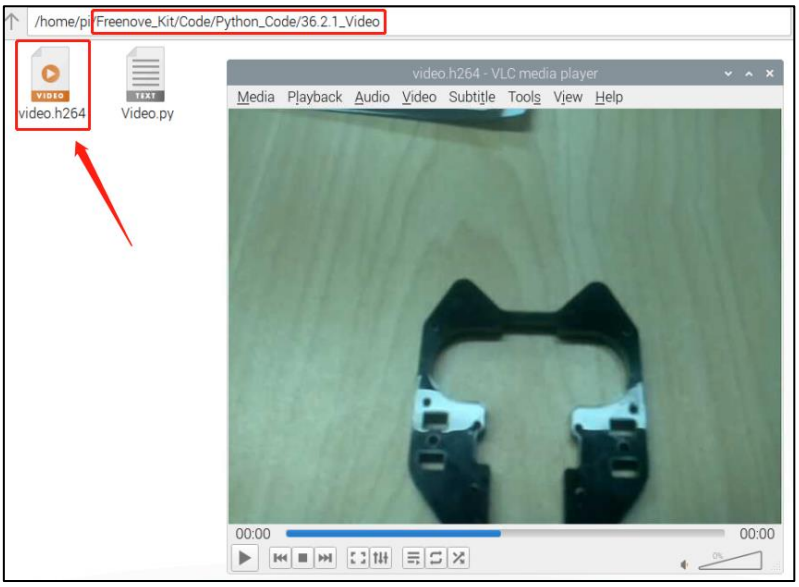
1. Use cd command to enter 36.2.1_Camera directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/36.2.1_Video
```

2. Use python command to execute code "Video.py"

```
python Video.py
```

After the program is executed, you can shoot a video by pressing the button. When the shooting button is pressed, you will see Video.h264 in the corresponding directory. When the play button is pressed, the corresponding captured video will be played, as shown in the image below. You can repeat the above operation to capture other videos, but it will overwrited the previous video.



The following is the program code:

```
import time
from picamera2.encoders import H264Encoder, Quality
from picamera2 import Picamera2
from gpiozero import Button
import os

buttonPin = 17    # define buttonPin
button2Pin =18    # define button2Pin
shootbutton = Button(buttonPin,pull_up=True) # define Button pin according to BCM Numbering
openbutton = Button(button2Pin,pull_up=True) # define Button pin according to BCM Numbering

def loop():
    while True:
        if shootbutton.is_pressed:  # if button is pressed
            picam2 = Picamera2()
            video_config = picam2.create_video_configuration(main={"size": (640, 480)})
            picam2.configure(video_config)
            encoder = H264Encoder()
            picam2.start_recording(encoder, 'video.h264', quality=Quality.HIGH)
            time.sleep(2)
            picam2.stop_recording()
            picam2.close()
            print ('Hello,a video has been taken successfully')   # print information on
terminal

        elif openbutton.is_pressed:  # if button is pressed
            print ('The video has been opened')  # print information on terminal
            os.system('ffplay -x 640 -y 480 -autoexit  video.h264  -vf setpts=PTS/4 ')
```

```
            print ('The video has been closed')  # print information on terminal

def destroy():
    shootbutton.close()
    openbutton.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    print ('Please preess the button to shoot a video')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Read the signal pin of the shooting button, and determine whether the button is pressed, if the button is pressed, shoot a video and save it in the corresponding directory.

```
        if shootbutton.is_pressed:  # if button is pressed
            picam2 = Picamera2()
            video_config = picam2.create_video_configuration(main={"size": (640, 480)})
            picam2.configure(video_config)
            encoder = H264Encoder()
            picam2.start_recording(encoder, 'video.h264', quality=Quality.HIGH)
            time.sleep(2)
            picam2.stop_recording()
            picam2.close()
            print ('Hello,a video has been taken successfully')   # print information on
terminal
```

Read the signal pin of the play button, and determine whether the button is pressed. If the button is pressed, use the ffplay tool to play the corresponding video. When the video finishes playing, the playback interface will be closed automatically.

```
        elif openbutton.is_pressed:  # if button is pressed
            print ('The video has been opened')  # print information on terminal
            os.system('ffplay -x 640 -y 480 -autoexit  video.h264  -vf setpts=PTS/4 ')
            print ('The video has been closed')  # print information on terminal
```

Chapter 37 Web IoT

In this chapter, we will learn how to use GPIO to control the RPi remotely via a network and how to build a WebIO service on the RPi.
This concept is known as “IoT” or Internet of Things. The development of IoT will greatly change our habits and make our lives more convenient and efficient

Project 37.1 Remote LED

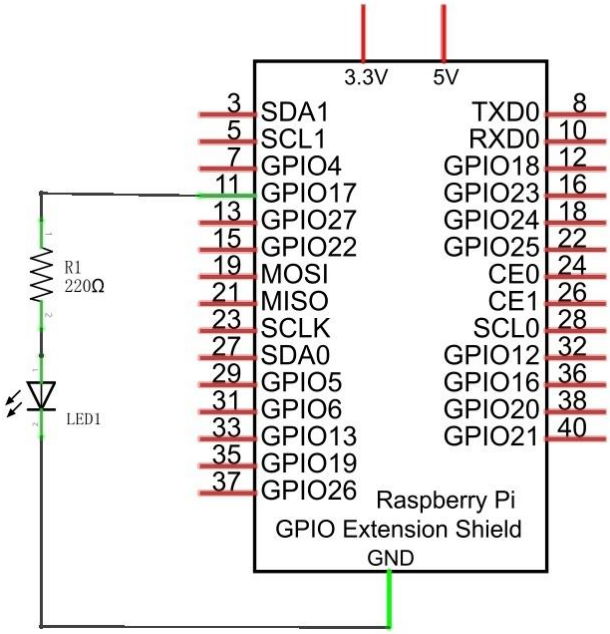
In this project, we need to build a WebIOPi service, and then use the RPi GPIO to control an LED through the web browser of phone or PC.

Component List

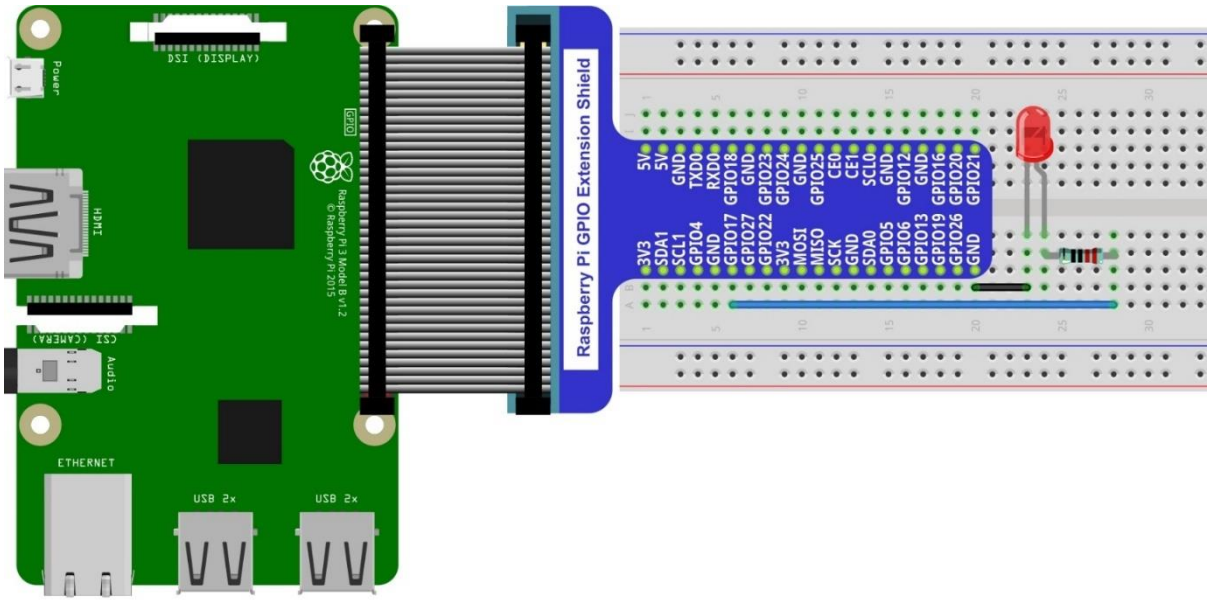
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	LED x1	Resistor 220Ω x1
Jumper M/M x2		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Solution from E-Tinkers

Here is a solution fromblog E-Tinkers, author Henry Cheung. For more details, please refer to link below:
https://www.e-tinkers.com/2018/04/how-to-control-raspberry-pi-gpio-via-http-web-server/

1, Make sure you have set python3 as default python. Then run following command in terminal to install http.server in your Raspberry Pi.

```
sudo apt-get install http.server
```

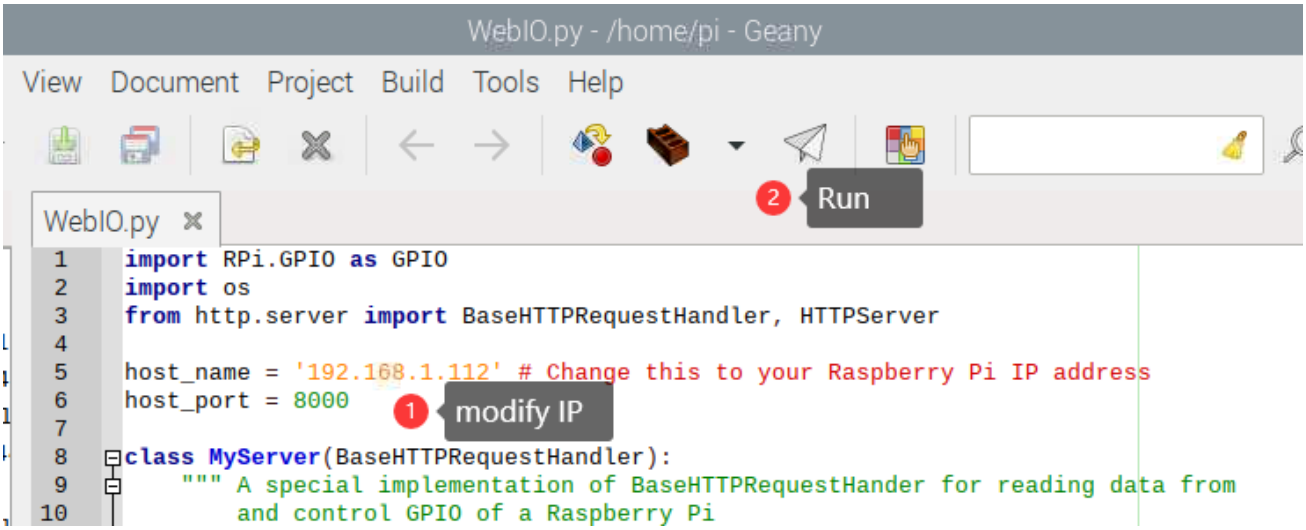
2, Open WebIO.py

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/37.1.1_WebIO
geany WebIO.py
```

3, Change the host_name into your Raspberry Pi IP address.

```
host_name = '192.168.1.112'     # Change this to your Raspberry Pi IP address
```

Then run the code WebIO.py



3, Visit http://192.168.1.112:8000/ in web brower on compter under local area networks. **Change IP to your Raspberry Pi IP address.**

WebIOPi Service Framework

Note: **If you have a Raspberry Pi 4B, you may have some trouble. The reason for changing the file in the configuration process is that the newer generation models of the RPi CPUs are different form the older ones and you may not be able to access the GPIO Header at the end of this tutorial.** A solution to this is given in an online tutorial by from E-Tinkers blogger Henry Cheung. For more details, please refer to previouse section.

The following is the key part of this chapter. The installation steps refer to WebIOPi official. And you also can directly refer to the official installation steps. The latest version (in 2016-6-27) of WebIOPi is 0.7.1. So, you may encounter some issues in using it. We will explain these issues and provide the solution in the following installation steps.

Here are the steps to build a WebIOPi:

Installation

1. Get the installation package. You can use the following command to obtain.

```
wget https://github.com/Freenove/WebIOPi/archive/master.zip -O WebIOPi.zip
```

2. Extract the package and generate a folder named "WebIOPi-master". Then enter the folder.

```
unzip WebIOPi.zip
cd WebIOPi-master/WebIOPi-0.7.1
```

3. Patch for Raspberry Pi B+, 2B, 3B, 3B+.

```
patch -p1 -i webiopi-pi2bplus.patch
```

4. Run setup.sh to start the installation, the process takes a while and you will need to be patient.

```
sudo ./setup.sh
```

5. If setup.sh does not have permission to execute, execute the following command

```
sudo sh ./setup.sh
```

Run

After the installation is completed, you can use the webiopi command to start running.

```
$ sudo webiopi [-h] [-c config] [-l log] [-s script] [-d] [port]

    Options:
      -h, --help            Display this help
      -c, --config  file    Load config from file
      -l, --log     file    Log to file
      -s, --script  file    Load script from file
      -d, --debug           Enable DEBUG

    Arguments:
      port                  Port to bind the HTTP Server
```

Run webiopi with verbose output and the default config file:

```
sudo webiopi -d -c /etc/webiopi/config
```

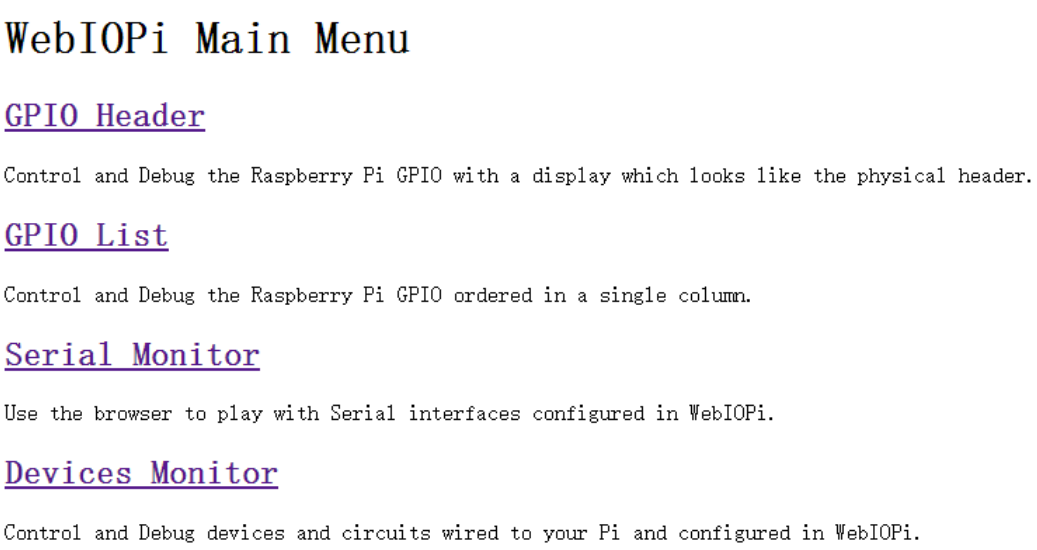
The Port is 8000 in default. Now WebIOPi has been launched. Keep it running.

Access WebIOPi over local network

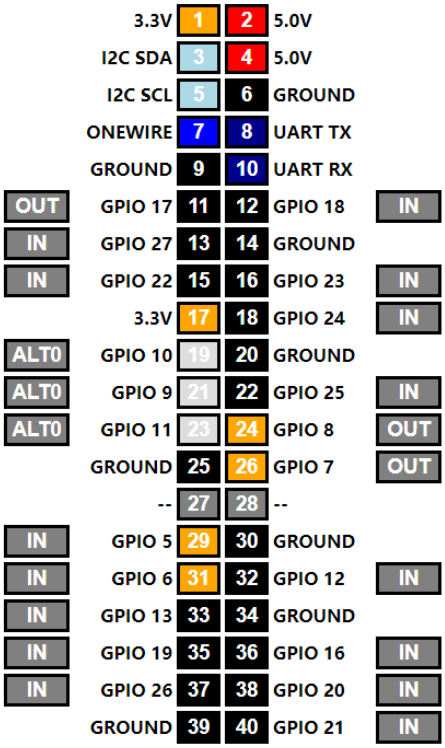
Under the same network, use a mobile phone or PC browser to open your RPi IP address, and add a port number like 8000. For example, my personal Raspberry Pi IP address is 192.168.1.109. Then, in the browser, I then should input: http://192.168.1.109:8000/

Default user is "webiopi" and password is "raspberry"。

Then, enter the main control interface:



Click on GPIO Header to enter the GPIO control interface.



Control methods:

- Click/Tap the OUT/IN button to change GPIO direction.
- Click/Tap pins to change the GPIO output state.

Completed

According to the circuit we build, set GPIO17 to OUT, then click Header11 to control the LED.
You can end the webioPi in the terminal by "Ctr+C".

Chapter 38 Soldering a Circuit Board

From previous chapters, we have learned about electronic circuits and components and have built a variety of circuits using a Breadboard device, which is not designed to be used permanently. We now will take a further step to make permanent projects using a Perfboard (a type of Prototype Circuit Board). Note: Perfboard is a stiff, thin sheet of insulated material with holes bored on a grid. The grid is usually a squared off shape with a spacing of 0.1 inches. Square copper pads cover these holes to make soldering electronic components easier.

To finish this chapter, you need to prepare the necessary soldering equipment, including an electric soldering iron (or soldering pencil) and solder. We have already prepared the Perfboard for you.

CAUTION: Please use extreme caution and attention to safety when you operate soldering tools used in these projects.

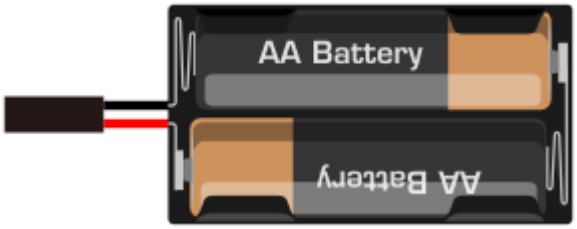
Project 38.1 Soldering a Buzzer

You should be familiar with the Buzzer from our previous project. We will solder a permanent circuit that when a Push Button Switch is pressed a Buzzer sounds

Note: This circuit **does not** require programming and will work when it is powered ON. When the button is not pressed and the Buzzer is not in use, there is no power consumption.

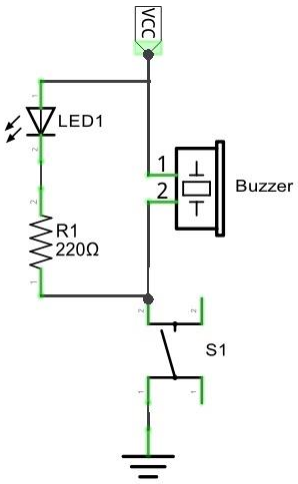
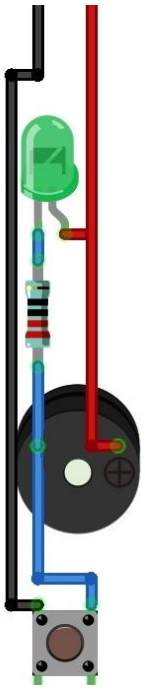
You can install it on your bicycle, your bedroom door or any other place where you want a Buzzer.

Component list

Female Pin Header x2	LED x1	Resistor 220Ω x1	Active buzzer x1	Push button x1
AA Battery Holder x1 and AA Batteries x2 				

Circuit

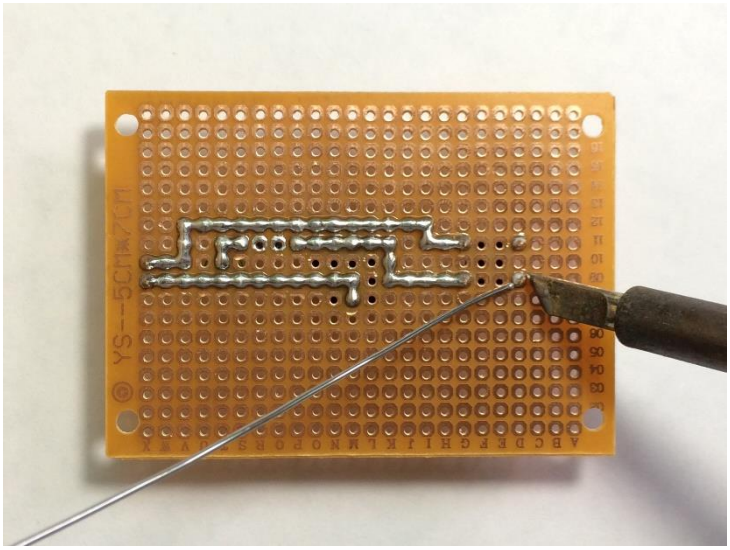
We will solder the following circuit on the Perfboard.

Schematic diagram	Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com
	

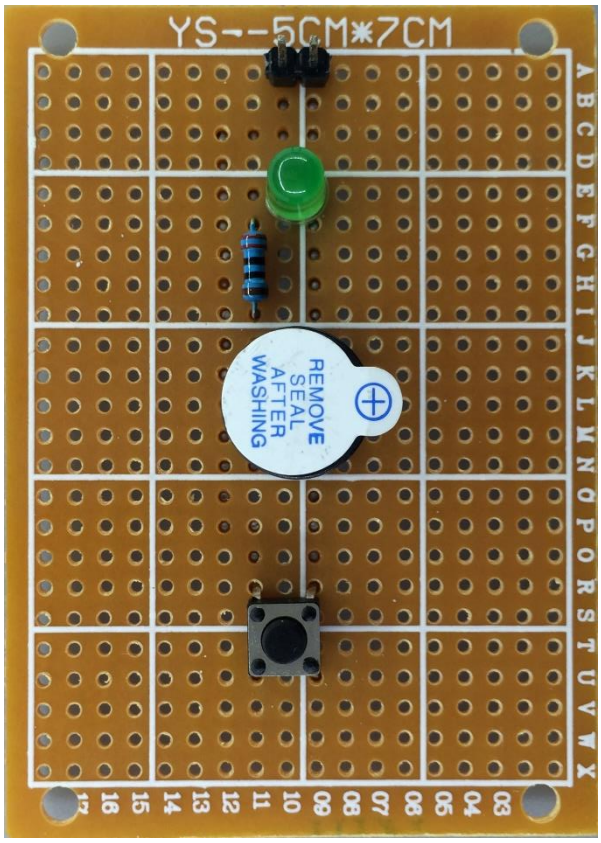
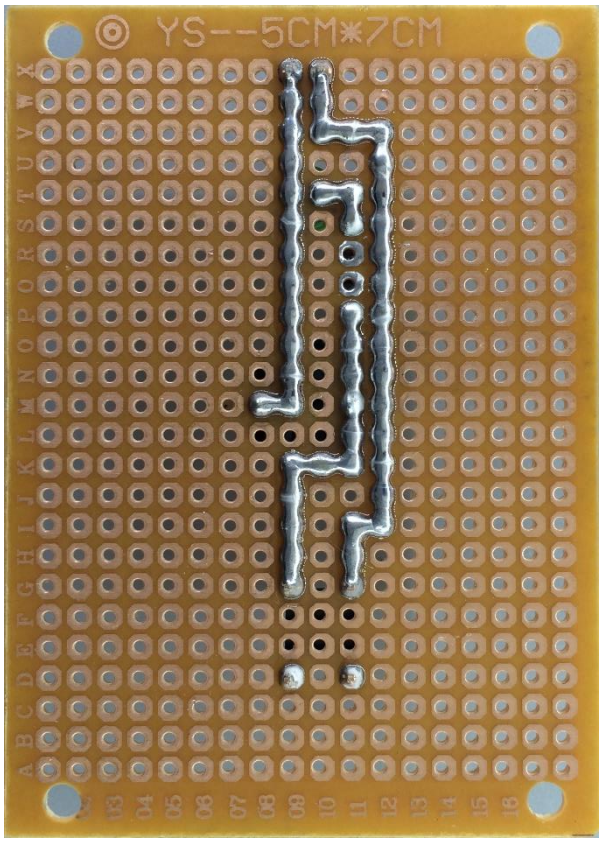
Note: If you are new to soldering electronic components on any type of circuit board we strongly recommend that you watch some instructional How-To videos by doing an Internet search and practice your soldering technique before attempting to solder the following projects. Some components can be damaged by exposure to excessive heat for prolonged times and there are various techniques you can learn that will help with making neater solder joints.

Solder the Circuit

Insert the components in the Perfboard following the Hardware Connection image as a general visual guide. Insert the pins of the components (all from the same side) so that you have only the components on one side of the Perfboard and the pins on the other. Then from the side with the pins carefully solder the circuit on the backside without having excess solder shorting out any portions of the circuit.

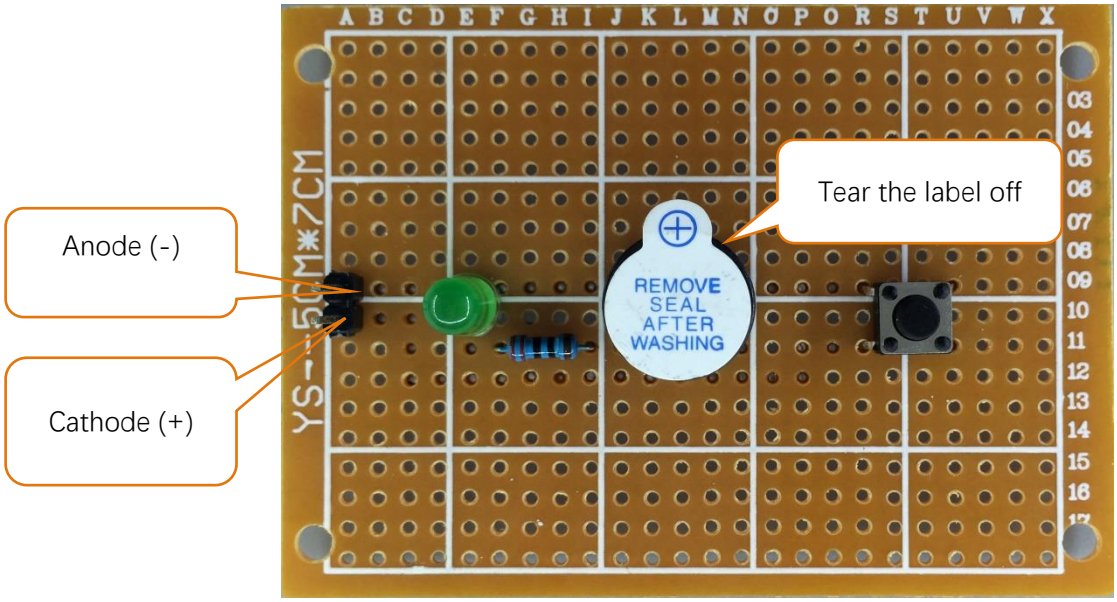


Here is a diagram after soldering from both sides of the Perfboard:

Front	Back
	

Test the Circuit

Connect the circuit board to a power supply (3~5V). You can use Raspberry Pi board or your 2 AA Cell Battery Box as the power supply.

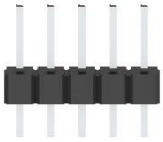
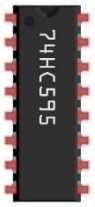


Press the Push Button Switch after connecting the power and then the buzzer will sound.

Project 38.2 Soldering a Flowing Water Light

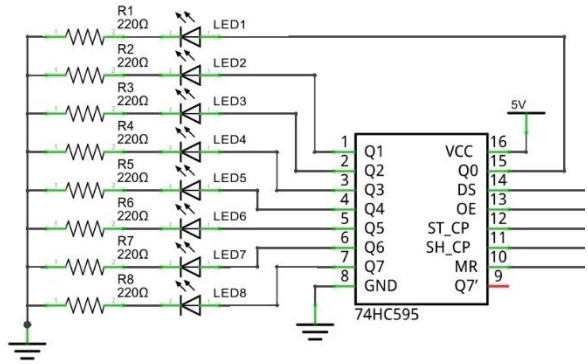
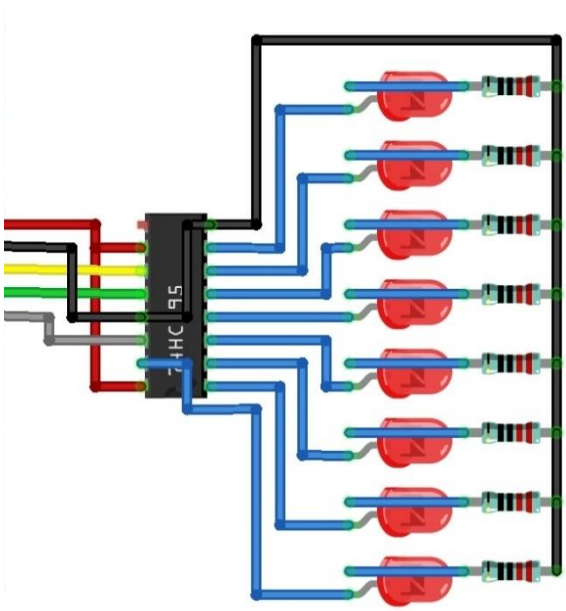
You should be familiar with the Flowing Water Light from our previous project. We will solder a permanent circuit using improved code to make a more interesting Flowing Water Light.

Component List

Female Pin Header x5 	Resistor 220Ω x8	LED x8	74HC595 x1 

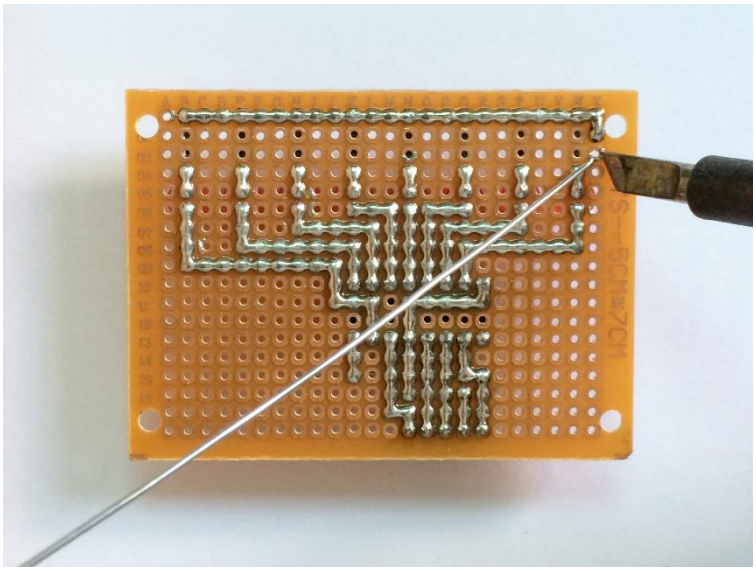
Circuit

Solder the following circuit on the Perfboard.

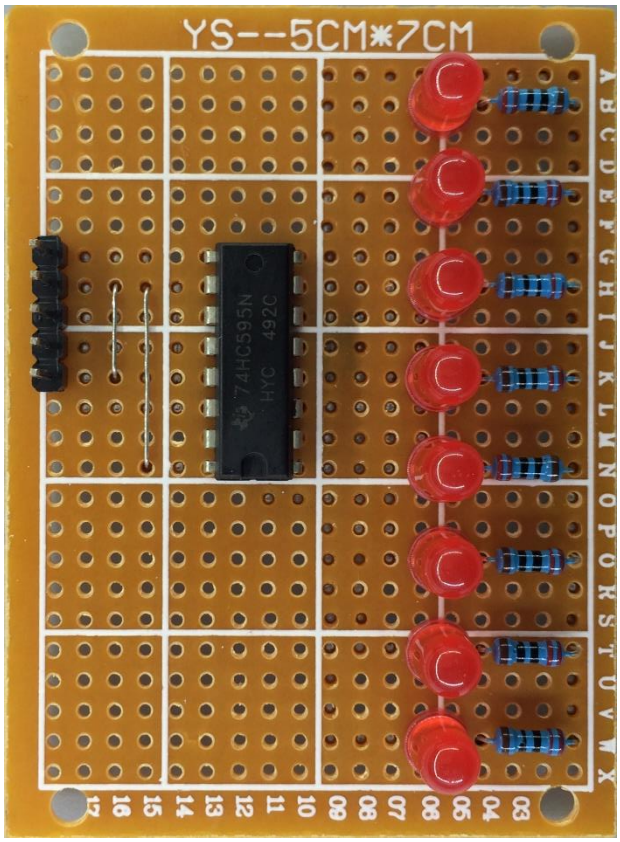
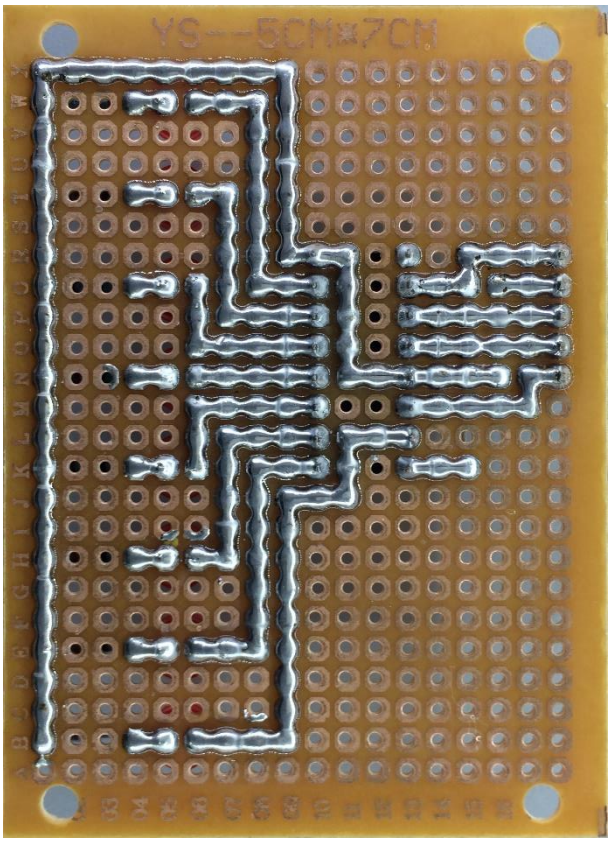
Schematic diagram	Hardware connection
	

Soldering the Circuit

Insert the components in the Perfboard, and solder the circuit on the back per earlier instructions.

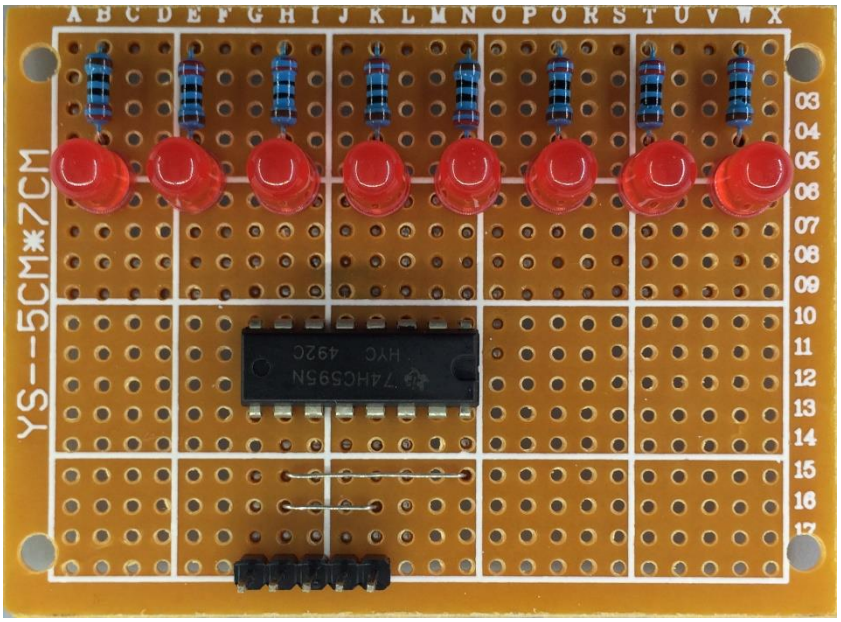


Here is a diagram after soldering from both sides of the Perfboard:

Front	Back
	

Connecting the Circuit

Connect the board to Raspberry Pi with jumper wire in the following way.


YS--5CM*7CM
74HC595N
DS —GPIO.17
ST_CP —GPIO27
SH_CP—GPIO22
GND —GND
VCC —3.3V/5V

Code

This now will be the third time we have made the Flowing Water Light. In this project, we will solder a completely new circuit for Flowing Water Light. Additionally, the program is also different from the previous ones we have used. When this light flows, it will have a long “tail”.

Python Code 38.2.1 LightWater03

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 38.2.1_LightWater03 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/38.2.1_LightWater03
```

2. Use Python command to execute Python code “LightWater03.py”.

```
python LightWater03.py
```

After the program is executed, the LEDs will light up in the form of flowing water with a long “tail”.

The following is the program code:

```
from gpiozero import OutputDevice
import time

LSBFIRST = 1
MSBFIRST = 2

# define the pins for 74HC595
dataPin   = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
latchPin  = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
clockPin  = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)

# Define an array to store the pulse width of LED
pluseWidth = [0,0,0,0,0,0,0,0,64,32,16,8,4,2,1,0,0,0,0,0,0,0,0,0]

# shiftOut function, use bit serial transmission.
def shiftOut(order,val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

def outData(data):
    latchPin.off()
    shiftOut(LSBFIRST,data)
    latchPin.on()

```

```
def loop():
    moveSpeed = 0.1 # moveSpeed works like a relay, the larger, the slower
    index = 0       # array index starts from 0
    lastMove = time.time()      # record the start time
    while True:
        if(time.time() - lastMove > moveSpeed): # control speed
            lastMove = time.time()      # Record the time point of the move
            index +=1           # move to next
            if(index > 15):     # index to 0
                index = 0

        for i in range(0,64):   # The cycle of PWM is 64 cycles
            data = 0
            for j in range(0,8):    #Calculate the output state of this loop
                if(i < pluseWidth[j+index]):    #Calculate the LED state according to the
pulse width
                    data |= 1<<j    # Calculate the data
            outData(data)           # Send the data to 74HC595

def destroy():
    dataPin.close()
    latchPin.close()
    clockPin.close()

if __name__ == '__main__': # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:   # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

We can see that this program is different from the previous one that we had used. We define an array to modulate different PWM pulse widths for LEDs, in doing so different LEDs can emit varied brightness. Starting from the array index 0, take an array of 8 adjacent numbers as the LED duty cycle and output it one at a time. Increasing the starting index number in turn, then it will create a flowing effect.

```
pluseWidth = [0, 0, 0, 0, 0, 0, 0, 0, 64, 32, 16, 8, 4, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

By recording the moving time point to control the speed of the movement of index number, controls the speed of the Flowing Water Light. Variable moveSpeed saves the time interval of each move, and the greater the value is, the slower the rate of the flowing movement (the reverse creates faster flowing movement).

```
        if(time.time() - lastMove > moveSpeed): #speed control
            lastMove = time.time()      #Record the time point of the move
            index +=1           #move to next
```

```
            if(index > 15):     #index to 0
                index = 0
```

Finally, in a “for” loop with i=64, modulate the output pulse width of the PWM square wave. The process, from the beginning of implementing the for loop to the end, is a PWM cycle. In the loop, there is another for loop with j=8 and in this loop, it compares the cycle number “i” to the value of the array to determine output high or low level. Then, the data will be sent to the 74HC595 IC Chip.

```
        for i in range(0,64):   #The cycle of PWM is 64 cycles
            data = 0            #This loop of output data
            for j in range(0,8):    #Calculate the output state of this loop
                if(i < pluseWidth[j+index]):    #Calculate the LED state according to the
pulse width
                    data |= 1<<j    #Calculate the data
            outData(data)           #Send the data to 74HC595
```

Other Components

This kit also includes other common components that can help your ideas come true. Now we will introduce components not mentioned in the previous section.

Component Knowledge

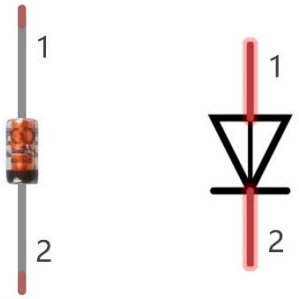
Toggle switch

Like push button switch, toggle switch is also a kind of switching devices. The difference is that toggle switch is suitable for long-time open or close circuits.

When the lever is moved to the left, pin 1, 2 get conducted, and pin 2, 3 are disconnected from each other;
When the lever is moved to the right, pin 2,3 get conducted, and pin 1,2 are disconnected from each other;

Switch diode

There are several types of diodes. We have used 1N4001 before, which is a common rectifier diode and commonly used in ac rectifier.
Here is 1N4148, which is a kind of high-speed switching diodes and is characterized by a relatively rapid switching.



For the switching diode, the changing time from conduction to cut off or from cut off to conduction is shorter than general diode and it is mainly used in electronic computer, pulse and switching circuits.

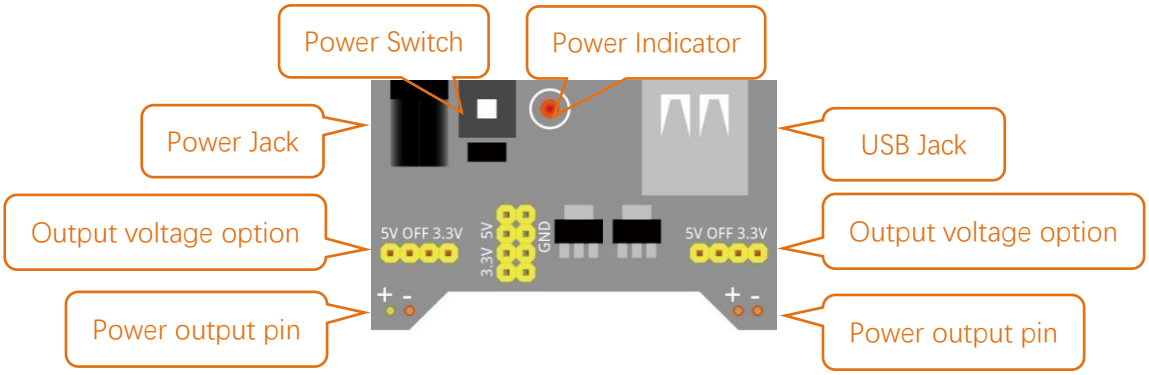
9V battery cable

A 9V battery cable can connect a 9 V battery, which can supply power for control board.

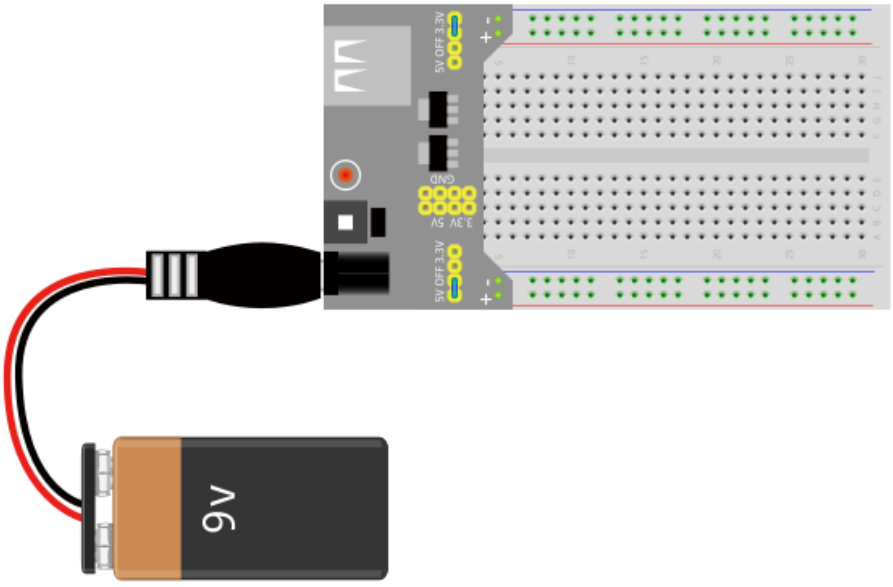
The installation of 9V battery cable is as follows:

Power supply module for breadboard

The following is the power supply module for breadboard. This module can provide the breadboard with two-channel power supply separately, and each can be configured to 3.3 V or 5 V separately through a jumper.



We can build a circuit conveniently by using this module. You only need to provide power supply for this module, and then insert it on the breadboard.



What's Next?

THANK YOU for participating in this learning experience! If you have completed all of the projects successfully you can consider yourself a Raspberry Pi Master.

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us: **support@freenove.com**
We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you are interesting in processing, you can study the Processing.pdf in the unzipped folder.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.
http://www.freenove.com/

Thank you again for choosing Freenove products.